INTRODUCTION · EXAM STRATEGY - ALL DOMAINS

Start here: the KCNA course

What the exam is, how this course works, and the route that gets you ready fastest.

EXAM KCNA RUNS 8 hours 33 minutes MODULES 35 plus this one

admin@sthithapragna.com

KCNA tests whether you can **choose correctly** between options that all sound right. This course trains exactly that.

The exam at a glance

ITEM	DETAIL
Certification	Kubernetes and Cloud Native Associate, from the CNCF and the Linux Foundation
Format	Online, remotely proctored, multiple choice
Time and questions	90 minutes; about 60 questions, as widely reported
Pass mark	75%
Price and attempts	US$250, including one retake
Validity	Two years. No prerequisites

REMEMBER

Check the Linux Foundation's exam page the week you book: details can change.

The four domains, and where they are taught

DOMAIN	WEIGHT	MODULES	WORKED SCENARIOS
Kubernetes Fundamentals	44%	1 to 12	56
Container Orchestration	28%	13 to 21	44
Cloud Native Application Delivery	16%	22 to 25	28
Cloud Native Architecture	12%	26 to 28	16

ON THE EXAM

Teaching time follows the weights. Nearly three quarters of the exam is the first two domains.

Who this course is for

- **For** anyone new to Kubernetes: developers, operations, testers, managers, students.
- **No Kubernetes experience needed.** Module 1 starts from what a container actually is.
- **Helpful but not required:** basic command-line comfort, and a rough idea of what a server is.
- **Not a hands-on lab course.** For CKA or CKAD you will need a keyboard as well.

> REMEMBER
>
> KCNA is conceptual. You will read commands and manifests, but never type them in the exam.

Read this first: a lot of study material is out of date

- **The curriculum changed on 24 November 2025:** five domains became four, and Delivery doubled.
- **Much older material is now wrong:** PodSecurityPolicy, SMI, OpenTracing, the Ingress NGINX controller.
- **This course teaches what is true today,** and flags every change on a 'Currency check' slide.
- **Module 29** gathers every change in one place, with a strategy for dated questions.

> ON THE EXAM
>
> If you studied from older notes or question banks, watch for the currency-check slides.

How this course was built

- **Mapped to the current curriculum,** topic by topic, with teaching time following the weights.
- **Every fact checked** against kubernetes.io, CNCF announcements and project documentation, September 2026.
- **Every scenario is original.** Nothing here is a reproduced question bank.
- **One running example:** Quayside, a ferry-booking company, from its first container to GitOps.

> REMEMBER
>
> Where sources disagree or cannot be confirmed, the course says so rather than guessing.

Meet Quayside: the running example

PART OF THE SYSTEM	RUNS AS	EXPLAINED IN
Web front end	Deployment and Service	Modules 7 and 14
Booking API	Deployment, with an autoscaler	Modules 7 and 27
Payment worker	Deployment reading a queue	Module 8
Ticket validator	A tiny distroless image	Module 2
Bookings database	StatefulSet with its own volumes	Modules 8 and 20
Log agent and nightly export	DaemonSet and CronJob	Module 8

REMEMBER

By module 24, every part is deployed from Git. You will know why each one is the kind of object it is.

How to read the slides

- **The top-left label** shows the domain, so you always know which part of the exam you are in.
- **'On the exam'** boxes hold the fact most likely to be tested.
- **'The tiebreaker'** boxes give the one fact that separates two look-alike options.
- **'Currency check'** slides mark something that changed recently.
- **'In five lines'** closes every teaching module: rewatch those for fast revision.

> REMEMBER
>
> If you only have ten minutes, the five-line summaries and tiebreakers are the highest-value revision.

Using the worked scenarios

- **Pause and commit.** Choose your answer before the walkthrough. Watching without answering teaches almost nothing.
- **Read every explanation,** including why each wrong option fails.
- **Right answer, wrong reason, is a miss.** Count it as one.
- **After a miss, ask:** did I not know it, or did I misread the question?

> REMEMBER
>
> Knowledge errors need a rewatch. Reading errors need slower reading. They have different fixes.

Using the practice exams

- **Four papers of twenty,** weighted exactly like the real exam across all eighty questions.
- **Take practice exam 1 only after the teaching modules.** Earlier, it measures nothing.
- **Score by domain,** not just in total. The end of each paper shows how.
- **Leave days between papers,** and aim for a steady 85% before you book.

> REMEMBER
>
> Taking all four papers in one sitting wastes them. Each one is a measurement; space them out.

The study route

01

Watch in order

Modules 1 to 31. Later modules build on earlier ones.

02

Exam 1

Score it by domain.

03

Fix the weakest

Rewatch only that domain's scenarios.

04

Exams 2 to 4

Spaced days apart; repeat the fix each time.

05

Night before

Modules 29 and 30: what changed, and the confusion pairs.

REMEMBER

Revising what you already know feels productive, because the scores stay high. It moves nothing.

The course map

PART	MODULES	SCENARIOS OR QUESTIONS	RUNTIME
Domain 1: Kubernetes Fundamentals	1 to 12	56 scenarios	2h 57m
Domain 2: Container Orchestration	13 to 21	44 scenarios	2h 05m
Domain 3: Application Delivery	22 to 25	28 scenarios	1h 03m
Domain 4: Cloud Native Architecture	26 to 28	16 scenarios	0h 47m
Exam skills	29 to 31	16 checks	0h 34m
Practice exams	32 to 35	80 questions	1h 04m

REMEMBER

8 hours 33 minutes after this introduction. 160 worked scenarios, plus 80 practice questions.

Module 1 starts with the thing everything else rests on: **what a container actually is.**

MODULE 01 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Containers from first principles

What a container really is, and why Kubernetes needs one.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 1 of 35

admin@sthithapragna.com

A container is not a small virtual machine. It is an ordinary process with a restricted view.

The problem containers solve

Meet Quayside, the ferry-booking app we will follow all course.

- **It works on my laptop.** The booking API passes every test, then fails on the server.
- **Dependency drift.** The server has a different Python version and an older system library.
- **Slow, heavy environments.** A new virtual machine per service takes minutes and gigabytes.
- **The fix.** Ship the app together with everything it depends on, as one sealed unit.

> ON THE EXAM
>
> Containers package the app with its dependencies, so it runs the same everywhere.

How we got here

01

Physical servers

One app per box. Wasted capacity, slow to buy.

02

Virtual machines

Many guest operating systems on one host. Better use, still heavy.

03

Containers

Many isolated processes on one kernel. Light and fast.

04

Orchestration

Software that runs thousands of containers across many machines.

ON THE EXAM

Each step fixed the last one's waste. Orchestration fixes the problem containers created: there are now too many to manage by hand.

Containers versus virtual machines

VIRTUAL MACHINE

Its own kernel

- Hypervisor emulates hardware
- Full guest OS in every VM
- Boots in tens of seconds or more
- Strong isolation boundary

CONTAINER

Shares the host kernel

- Kernel features isolate processes
- No guest OS, just app files
- Starts in about a second
- Weaker boundary: one shared kernel

THE TIEBREAKER

The shared kernel is the whole difference. It makes containers fast and dense, and it makes their isolation weaker.

Why the shipping container is the right picture

- **A standard outside.** Every box has the same size and fittings, whatever is inside.
- **The crane does not care.** Ports move boxes without opening them.
- **For software:** the image is the standard box, and any compliant runtime can run it.
- The standard that makes this true is the OCI, covered in module 2.

> ON THE EXAM
>
> Standardise the package, and the machinery that moves it can be generic.

Two kernel features do all the work

- **Namespaces** control what a process can **see**: its own processes, network, hostname, files.
- **Control groups (cgroups)** control what a process can **use**: CPU, memory, process count, I/O.
- A container runtime simply starts a process with both applied, plus an image as its filesystem.

> REMEMBER
>
> Namespaces isolate. Cgroups limit. If an option mixes them up, it is wrong.

The eight Linux namespaces

NAMESPACE	GIVES THE PROCESS ITS OWN...
PID	process IDs, so the app sees itself as PID 1
Network	interfaces, IP addresses, ports and routes
Mount	view of the filesystem and mount points
UTS	hostname
IPC	shared memory and message queues
User	user and group IDs, so root inside need not be root outside
Cgroup	view of its own cgroup tree
Time	boot and monotonic clocks, added in Linux 5.6

ON THE EXAM

In Kubernetes, containers in one Pod share the network namespace. That is why they reach each other on localhost.

One process, two points of view

Illustrative output: the same web server seen from the host and from inside.

```
1 # On the host
2 $ ps -eo pid,comm | grep nginx
3   48213 nginx
4
5 # Inside the container
6 $ ps -eo pid,comm
7       1 nginx
8 $ hostname
9   7f3c9a1e2b4d
```

3 On the host it is just one more process, number 48213.

7 The PID namespace makes it believe it is process 1.

9 The UTS namespace gives it its own hostname: the container ID.

THE TAKEAWAY

Nothing was virtualised. The kernel simply shows the process a different picture.

Control groups: the limits

- **CPU.** A share of processor time, and optionally a hard quota.
- **Memory.** A hard ceiling. Cross it and the kernel kills the process.
- **Process count and I/O.** Stop a fork bomb or a noisy disk user starving neighbours.
- **In Kubernetes**, the requests and limits you set on a container become cgroup settings.

> ON THE EXAM
>
> Kubernetes does not enforce limits itself. The kubelet asks the runtime, which programs the kernel's cgroups.

Hitting a CPU limit versus a memory limit

OVER THE CPU LIMIT

Throttled

- Kernel pauses the process each period
- App gets slower, latency rises
- Container keeps running
- No restart, no error event

OVER THE MEMORY LIMIT

Killed

- Kernel's OOM killer ends the process
- Status shows OOMKilled
- Kubernetes restarts the container
- Repeated kills lead to CrashLoopBackOff

THE TIEBREAKER

CPU is compressible: you can give less and the app just slows. Memory is not: you cannot take back memory in use.

Currency check: cgroup version 2

- **cgroup v2** is the current Linux interface: one unified hierarchy, better memory control.
- **Kubernetes 1.35 (December 2025):** the kubelet refuses to start on a cgroup v1 node by default.
- An operator can still override it with a kubelet setting, but v1 is deprecated and slated for removal.
- Older study material presents v1 and v2 as equal choices. They no longer are.

> ON THE EXAM
>
> If a question asks which cgroup version current Kubernetes expects, the answer is version 2.

An image is **read-only and shared**. A container is that image plus **one thin writable layer**.

An image is a stack of layers

01
Base
Minimal OS files, for example a slim Debian or Alpine.

02
Runtime
The language runtime: Python, Java, Node.

03
Dependencies
Libraries the app installs.

04
App code
Your source or compiled binary.

05
Metadata
Which command to run, ports, user, environment.

ON THE EXAM

Layers are cached and shared. Put what changes least at the bottom, and rebuilds only redo the top.

Quayside's booking API, as a Dockerfile

```
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 USER 10001
7 EXPOSE 8080
8 CMD ["python", "-m", "booking"]
```

1 The base layer. A slim image keeps size and attack surface down.

3-4 Dependencies first, so the cached layer survives code changes.

5 Code last: it changes most often.

8 The process that becomes PID 1 in the container.

THE TAKEAWAY

A Dockerfile is a text file of build instructions. Order the instructions for the cache.

The writable layer, and why data vanishes

- **Copy-on-write.** A container that changes a file gets a private copy in its writable layer.
- **Cheap to run many.** Ten containers from one image share every read-only layer.
- **Gone on replacement.** Delete or replace the container and its writable layer goes with it.
- **Data that must survive** belongs in a volume, mounted from outside the container.

> ON THE EXAM
>
> Treat containers as disposable. Anything worth keeping lives outside them.

Reading an image reference

PART	EXAMPLE	MEANING
Registry	registry.example.com	Where to pull from. Docker Hub if omitted.
Repository	quayside/booking	The image's name.
Tag	:2.4.1	A movable label. Omitted means "latest".
Digest	@sha256:9f86…	A hash of the content. Can never point elsewhere.

> ON THE EXAM
>
> Tags can move. Digests cannot. Pin by digest when you must know exactly what runs.

When Kubernetes pulls an image

IMAGEPULLPOLICY	BEHAVIOUR
`Always`	Resolve the tag to a digest at every container start; pull if not cached.
`IfNotPresent`	Pull only if the image is not already on the node.
`Never`	Never pull. Start fails if the image is missing.
`Default`	Always for :latest or no tag; IfNotPresent for any other tag or a digest.

ON THE EXAM

While the image downloads, the Pod is Pending. If the pull fails, you will see ErrImagePull, then ImagePullBackOff.

Where Kubernetes fits

- **Kubernetes does not build images.** A CI pipeline does, and pushes them to a registry.
- **Kubernetes does not start processes itself.** It asks a container runtime on each node.
- **What it does:** decide where containers run, keep the right number running, restart failures, connect and update them.
- The unit it schedules is the Pod, a wrapper around one or more containers.

> ON THE EXAM
>
> Orchestration means scheduling, scaling, healing, networking and rolling updates. Building images is not on that list.

> At the evening rush, Quayside's payment worker takes eight seconds per payment instead of one. It never restarts, and no error events appear.
>
> Monitoring shows the container's CPU usage flat at exactly its configured limit. Memory sits at half its limit.

What is the most likely explanation?

A The kernel's out-of-memory killer is ending the process

B The container has exhausted its PID namespace

C The kernel is throttling the container at its CPU limit

D The image layers are being re-downloaded under load

◆ PAUSE AND DECIDE

What is the most likely explanation?

A The kernel's out-of-memory killer is ending the process

Memory is at half its limit, and an OOM kill would restart the container.

B The container has exhausted its PID namespace

A namespace changes what a process sees. It is not a quota that runs out.

C The kernel is throttling the container at its CPU limit

Slow, never restarted, CPU pinned at the limit: the signature of throttling.

D The image layers are being re-downloaded under load

Images are pulled once at start, not while the container runs.

THE SIGNAL

Slow but alive means CPU throttling. Killed and restarted means memory.

> Quayside wants to run forty small services on three Linux servers. Several services need different, conflicting versions of the same library.
>
> Every service runs on Linux. The team wants new copies to start within seconds when traffic rises, with the least overhead per service.

Which approach best meets these requirements?

A Run each service as a container from its own image

B Give each service its own virtual machine

C Install every library version on the shared hosts

D Buy a dedicated physical server per service

◆ PAUSE AND DECIDE

Which approach best meets these requirements?

A Run each service as a container from its own image

Each image carries its own libraries; shared kernel means fast start and low overhead.

B Give each service its own virtual machine

It isolates dependencies, but forty guest OSes are heavy and slow to start.

C Install every library version on the shared hosts

This is the dependency drift problem, made permanent.

D Buy a dedicated physical server per service

Works, but wastes hardware and takes weeks, not seconds.

THE SIGNAL

All Linux, needs density and fast start: containers. Virtual machines are for different kernels or a stronger boundary.

> A Quayside developer's container saves uploaded ferry manifests to the /data folder inside the container. Everything works for a week.
>
> After the team deploys a new version of the image, every previously uploaded manifest has disappeared. The new container starts cleanly.

Why did the files disappear?

A The new container's mount namespace hides the old files

B The new image version was pulled using a different tag

C The memory cgroup limit truncated the files on disk

D The files were in the old container's writable layer

◆ PAUSE AND DECIDE

Why did the files disappear?

A. The new container's mount namespace hides the old files

Sounds plausible, but the files are gone, not hidden. The old layer was deleted.

B. The new image version was pulled using a different tag

True but irrelevant. Images never contain runtime uploads.

C. The memory cgroup limit truncated the files on disk

Cgroups limit memory use; they do not edit files on disk.

D. The files were in the old container's writable layer

Deploying replaces the container, and its writable layer goes with it.

THE SIGNAL

Data written inside a container dies with the container. Put it in a volume.

> Two copies of Quayside's web front end use the image quayside/web:latest. One was started last Tuesday; the other started this morning.
>
> They behave differently: only this morning's copy shows the new timetable page. Nobody changed any configuration.

What is the most likely cause?

A Containers on different nodes see different kernel namespaces

B The latest tag was moved to a new image in between

C One copy has modified its read-only image layers

D The two containers were given different CPU limits

◆ PAUSE AND DECIDE

What is the most likely cause?

A. Containers on different nodes see different kernel namespaces

Namespaces isolate each container; they do not change which code runs.

B. The latest tag was moved to a new image in between

Someone pushed a new image under latest between Tuesday and today.

C. One copy has modified its read-only image layers

Read-only layers cannot be changed. Changes go to the writable layer.

D. The two containers were given different CPU limits

Limits affect speed, not which page the app renders.

THE SIGNAL

Two start times plus one movable tag equals two versions. Pin a version tag or a digest.

Module 1 in five lines

- A container is **a process with a restricted view**, sharing the host kernel.
- **Namespaces isolate** what it sees; **cgroups limit** what it uses.
- Over the CPU limit it is **throttled**; over the memory limit it is **OOMKilled**.
- An image is **read-only layers**; the container's writable layer dies with it.
- **Tags move, digests do not.** No tag means latest, and latest means pull Always.

Kubernetes never starts a container itself. So **who does**, and what makes them interchangeable?

MODULE 02 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Runtimes and standards

Who actually starts a container, and why they are interchangeable.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 2 of 35

admin@sthithapragna.com

Kubernetes talks to **one interface**, so the runtime underneath can change without Kubernetes noticing.

admin@sthithapragna.com

From kubelet to a running process

01

kubelet

The node agent. Decides a container should run.

02

CRI

A gRPC API. The kubelet is the client.

03

High-level runtime

containerd or CRI-O. Pulls images, manages containers.

04

OCI runtime

runc, by default. Creates the process from a bundle.

05

Linux kernel

Applies namespaces and cgroups.

ON THE EXAM

Two standards sit on this chain: the CRI above the high-level runtime, and the OCI runtime spec below it.

High-level versus low-level runtime

HIGH-LEVEL RUNTIME

containerd, CRI-O

- Implements the CRI for the kubelet
- Pulls and stores images
- Asks CNI plugins to set up Pod networking
- Runs as a long-lived daemon

LOW-LEVEL RUNTIME

runc, crun

- Implements the OCI runtime spec
- Takes an unpacked filesystem bundle
- Creates namespaces and cgroups
- Starts the process, then steps away

THE TIEBREAKER

If it pulls images or speaks CRI, it is high-level. If it only turns a bundle into a process, it is low-level.

The Open Container Initiative: three specifications

SPECIFICATION	WHAT IT STANDARDISES
Runtime spec	How to run an unpacked filesystem bundle. Reference implementation: runc.
Image spec	The image format: a manifest, a config and a set of layers.
Distribution spec	The API for pushing and pulling images to and from registries.

ON THE EXAM

The OCI is a Linux Foundation project, launched in June 2015 by Docker, CoreOS and others. It is not a CNCF project.

The Container Runtime Interface

- **Defined by Kubernetes** (SIG Node), introduced in 2016. A gRPC API between the kubelet and a runtime.
- **Two services:** RuntimeService for Pods and containers, ImageService for pulling and listing images.
- **Every node needs a CRI runtime.** Without one, the kubelet cannot start a single Pod.
- **crictl** is the CLI for inspecting any CRI runtime on a node.

> REMEMBER
>
> OCI standardises images and low-level runtimes. CRI standardises how the kubelet talks to a runtime. Do not swap them.

Runtimes you should recognise

NAME	WHAT IT IS
containerd	General-purpose CRI runtime. Came from Docker. CNCF graduated, 2019.
CRI-O	Built only for Kubernetes by Red Hat. Versions track Kubernetes. Graduated 2023.
runc	The low-level OCI reference runtime. Used under both of the above.
crun	A fast low-level OCI runtime written in C.
gVisor (runsc)	A sandboxed OCI runtime with a user-space kernel.
Kata Containers	A sandboxed runtime that runs each Pod in a lightweight VM.

ON THE EXAM

containerd and CRI-O are the high-level choices. The rest sit underneath, as OCI runtimes.

The dockershim story

01

Docker only

Early Kubernetes spoke only to Docker Engine.

02

CRI arrives

2016. Any runtime can plug in. Docker Engine never implemented it.

03

dockershim

A shim inside the kubelet translated CRI to Docker.

04

Deprecated

Kubernetes 1.20 announced the removal.

05

Removed

Kubernetes 1.24, May 2022.

ON THE EXAM

cri-dockerd, maintained by Mirantis and Docker, now plays the shim's role for anyone who still wants Docker Engine.

What the removal did, and did not, change

UNCHANGED

Images and laptops

- Images from docker build run everywhere
- They are OCI images
- Developers keep using Docker locally
- Registries work the same

CHANGED

The node runtime

- Nodes need containerd, CRI-O or cri-dockerd
- The kubelet no longer speaks to Docker
- Node tools using the Docker socket break
- Use crictl on nodes instead

THE TIEBREAKER

Kubernetes removed Docker the runtime, not Docker the image format.

Docker Engine versus containerd

DOCKER ENGINE

A developer platform

- CLI, image builds, Compose, networks
- Aimed at people at a keyboard
- Uses containerd underneath
- Does not implement the CRI itself

CONTAINERD

A runtime daemon

- Pulls images, runs containers
- Aimed at other software
- Built-in CRI plugin for the kubelet
- No image builds of its own

THE TIEBREAKER

Docker Engine uses containerd. containerd does not need Docker.

Currency check: runtime versions

- **containerd 1.x:** Kubernetes 1.35 is the last release that supports it.
- **containerd 2.x** is required to upgrade past 1.35. Kubernetes is now at 1.37.
- **CRI-O** versions match Kubernetes minor versions, so it moves in step.
- Old material may still say Docker Engine is the default Kubernetes runtime. Not since 2022.

> ON THE EXAM
>
> If a question asks for the default or recommended runtime today, think containerd or CRI-O, never dockershim.

Sandboxed runtimes: gVisor versus Kata

GVISOR (RUNSC)

A kernel in user space

- Intercepts the app's system calls
- Host kernel sees far fewer calls
- Lighter than a VM
- Some syscalls not supported

KATA CONTAINERS

A VM per Pod

- Each Pod gets its own guest kernel
- Hardware virtualisation boundary
- Stronger isolation
- More memory and start-up cost

THE TIEBREAKER

Both are OCI runtimes, chosen per Pod. Use them for untrusted or multi-tenant code, not by default.

Choosing a runtime per Pod: RuntimeClass

```
 1 apiVersion: node.k8s.io/v1
 2 kind: RuntimeClass
 3 metadata: {name: sandboxed}
 4 handler: runsc
 5 ---
 6 apiVersion: v1
 7 kind: Pod
 8 metadata: {name: partner-fares}
 9 spec:
10   runtimeClassName: sandboxed
11   containers:
12   - name: fares
13     image: partners/fares:3.2.0
```

4 The handler names a runtime already configured on the nodes.

10 Each Pod opts in by name. Pods without it use the default runtime.

Names that are not CRI runtimes

- **systemd.** The Linux init system. It can manage cgroups, but it does not run containers for the kubelet.
- **Podman.** A daemonless, Docker-compatible CLI for people. Not a CRI runtime.
- **kubelet.** The node agent that calls the runtime. It is a client, not a runtime.
- **Buildah, BuildKit.** Image builders. They produce images; they do not run workloads.

> ON THE EXAM
>
> When asked which is not a CRI runtime, look for the init system, the builder, or the human-facing CLI.

A good image is small, pinned, non-root and free of secrets.

admin@sthithapragna.com

The biggest win: a multi-stage build

SINGLE STAGE

Ships the toolchain

```
1 FROM golang:1.25
2 WORKDIR /src
3 COPY . .
4 RUN go build -o /app .
5 CMD ["/app"]
```

Compiler, shell and package manager all ship to production.

MULTI-STAGE

Ships only the binary

```
1 FROM golang:1.25 AS build
2 WORKDIR /src
3 COPY . .
4 RUN CGO_ENABLED=0 go build -o /app .
5
6 FROM gcr.io/distroless/static-debian12
7 COPY --from=build /app /app
8 USER nonroot
9 CMD ["/app"]
```

The build stage is thrown away. The final image holds one file.

THE TIEBREAKER

Build in one stage, copy only the result into a minimal base. Smaller to pull, far less to attack.

Image practices, and why each matters

PRACTICE	WHY
Multi-stage builds	Build tools never reach the final image.
Minimal base: distroless, slim, scratch	Fewer packages, fewer vulnerabilities, faster pulls.
Pin versions, not latest	Every node runs the same thing; rollbacks mean something.
Run as non-root with USER	A breakout starts with fewer privileges.
Use .dockerignore	Keeps .git, .env and junk out of the build context.
Scan images, publish an SBOM	Know what is inside, and find known vulnerabilities.

ON THE EXAM

If an option says fewer layers is the main way to shrink an image, it is overstating a minor effect.

Why a deleted secret is still in the image

```
1 COPY deploy_key /root/.ssh/id_ed25519
2 RUN git clone git@git.example.com:quayside/fares.git
3 RUN rm /root/.ssh/id_ed25519
```

1 The key is now stored in this layer, permanently.

3 The rm only adds a later layer that hides the file. Anyone can extract layer 1.

Use build secrets that never become a layer, or give the key to the running Pod as a Kubernetes Secret.

THE TAKEAWAY

Layers are additive. Deleting a file in a later layer never removes it from an earlier one.

Building without a Docker daemon

- **BuildKit.** Docker's modern build engine, also usable on its own.
- **Buildah and Podman.** Daemonless builds, often used in CI.
- **Cloud Native Buildpacks.** Source code in, OCI image out, no Dockerfile. CNCF graduated, August 2026.
- **kaniko.** Google archived the original in June 2025. Community forks continue.

> ON THE EXAM
>
> Whatever builds it, the output is an OCI image, so any runtime can run it.

> Quayside is moving from an old cluster that used Docker Engine on its nodes to a new cluster running containerd.
>
> Developers ask whether they must rebuild every image with a different tool, and whether they can keep using Docker on their laptops.

Which statement is accurate?

A Images must be rebuilt with Buildah to run on containerd

B Existing images run unchanged, and Docker stays fine locally

C Each node needs the dockershim reinstalled to run the images

D containerd cannot pull images that were pushed to Docker Hub

◆ PAUSE AND DECIDE

Which statement is accurate?

A Images must be rebuilt with Buildah to run on containerd

Docker-built images are OCI images. No rebuild is needed.

B Existing images run unchanged, and Docker stays fine locally

Only the node runtime changed. Images and laptops are unaffected.

C Each node needs the dockershim reinstalled to run the images

The dockershim was removed in 1.24, and containerd needs no shim.

D containerd cannot pull images that were pushed to Docker Hub

Registries follow the OCI distribution spec. containerd pulls from Docker Hub.

THE SIGNAL

Removing Docker as a runtime changed the nodes, not the images.

> Quayside will let partner ferry operators upload their own fare-calculation plugins, which run as containers in the main cluster.
>
> The code is untrusted. Security wants a kernel exploit in a plugin contained, without slowing Quayside's own workloads.

What should the platform team do?

A Put the plugins in their own Kubernetes namespace

B Give each plugin container a strict CPU and memory limit

C Require every plugin to be built on a distroless base image

D Run plugin Pods with a RuntimeClass for gVisor or Kata

◆ PAUSE AND DECIDE

What should the platform team do?

A Put the plugins in their own Kubernetes namespace

A Kubernetes namespace organises objects. The kernel is still shared.

B Give each plugin container a strict CPU and memory limit

Limits stop resource hogging, not kernel exploits.

C Require every plugin to be built on a distroless base image

Good hygiene, but the attacker writes the code. The kernel is still exposed.

D Run plugin Pods with a RuntimeClass for gVisor or Kata

Puts a sandbox or VM boundary around only the untrusted Pods.

THE SIGNAL

Untrusted code plus a kernel-level threat means a sandboxed runtime, applied per Pod.

> An operations script runs docker ps on every node to list running containers. After the move to containerd, it fails on every node.
>
> The team wants a replacement that lists Pods and containers the way the kubelet sees them, on any CRI runtime.

What should the script use?

A crictl, the command-line tool from the cri-tools project

B Install Docker Engine on each node alongside containerd

C runc list, which shows every container runc has created

D kubectl exec into each Pod and list its own processes

◆ PAUSE AND DECIDE

What should the script use?

A crictl, the command-line tool from the cri-tools project

It speaks the CRI, so it sees Pods and containers as the kubelet does.

B Install Docker Engine on each node alongside containerd

Docker would show its own containers, not the kubelet's.

C runc list, which shows every container runc has created

Too low-level: no Pod view, and tied to one OCI runtime.

D kubectl exec into each Pod and list its own processes

Sees inside one container at a time, not what runs on the node.

THE SIGNAL

A question that says 'on any CRI runtime' wants the tool built for the CRI: crictl.

> Quayside's ticket validator image is 1.1 gigabytes. New nodes take minutes to pull it during scale-out.
>
> A scanner reports hundreds of vulnerabilities, almost all in compilers and shell tools the program never uses at run time.

Which single change best addresses both problems?

A Merge every RUN instruction into one, to cut the layer count

B Set imagePullPolicy to Always so nodes pull it more promptly

C Rebuild multi-stage, copying just the binary to a minimal base

D Tag the image latest so every node pulls the newest version

◆ PAUSE AND DECIDE

Which single change best addresses both problems?

A Merge every RUN instruction into one, to cut the layer count

A small saving. The compilers and shells are still in the image.

B Set imagePullPolicy to Always so nodes pull it more promptly

Adds registry checks. The image is just as large and vulnerable.

C Rebuild multi-stage, copying just the binary to a minimal base

Removes the tools: smaller pulls and far fewer vulnerabilities.

D Tag the image latest so every node pulls the newest version

Makes versions unpredictable and forces a pull on every start.

THE SIGNAL

Vulnerabilities in tools the app never uses means those tools should not be in the image.

Module 2 in five lines

- kubelet **calls the CRI**; containerd or CRI-O answer; **runc** creates the process.
- **OCI**: runtime, image and distribution specs. runc is the reference runtime.
- dockershim was **removed in 1.24**. Docker-built images still run everywhere.
- **gVisor and Kata** sandbox untrusted Pods, chosen with a RuntimeClass.
- Good images are **multi-stage, minimal, pinned, non-root and secret-free**.

We have containers and a runtime on every node. Next: **the cluster that tells them what to do**.

admin@sthithapragna.com

MODULE 03 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Cluster architecture and the life of a kubectl apply

Every component, what it does, and how they cooperate without talking to each other.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 3 of 35

admin@sthithapragna.com

Kubernetes is **a database of desired state**, and **a crowd of controllers** working to make reality match it.

The thermostat: a control loop

01

Desired

You set the thermostat to 21 degrees.

02

Observe

The sensor reads 18 degrees.

03

Compare

Three degrees short.

04

Act

Turn on the heating. Then observe again.

ON THE EXAM

A controller never finishes. It loops forever: observe, compare, act. Kubernetes calls this reconciliation.

The shape of a cluster

kubectl and clients
Users, CI, tools
Outside the cluster

etcd
All cluster state
Only the API server uses it

cloud-controller-mgr
Talks to your cloud
Optional

kube-scheduler
Places new Pods
Control plane

kube-apiserver
REST API over HTTPS
Authenticates and authorises
Scales horizontally
The only door to cluster state.

controller-manager
Built-in controllers
Control plane

kube-proxy
Service network rules
Every node, optional

kubelet
Runs Pods on its node
Every node

THE TAKEAWAY

Hub and spoke: components watch the API server for changes. They do not call each other.

Control plane components

COMPONENT	ITS ONE JOB
kube-apiserver	Exposes the Kubernetes API. Front end of the control plane.
etcd	Consistent, highly available key-value store for all cluster data.
kube-scheduler	Watches for new Pods with no node, and picks a node for each.
kube-controller-manager	Runs the built-in controllers in one process.
cloud-controller-manager	Links the cluster to a cloud provider's API.

REMEMBER

Scheduler places Pods. Controllers create them. The API server stores everything via etcd. Keep the verbs straight.

Controllers inside the controller manager

CONTROLLER	WATCHES FOR...	THEN...
`Deployment`	Deployments	Creates and scales ReplicaSets
`ReplicaSet`	ReplicaSets	Creates or deletes Pods to match replicas
`Node`	Node heartbeats	Marks unreachable nodes and evicts their Pods
`Job`	Jobs	Creates Pods and tracks completions
`EndpointSlice`	Services and ready Pods	Lists which Pod IPs back each Service
`ServiceAccount`	New namespaces	Creates the default ServiceAccount

ON THE EXAM

When a question asks what updates a node's Ready condition, or replaces a deleted Pod, the answer is a controller.

Node components

COMPONENT	ITS JOB
kubelet	Agent on every node. Makes sure the containers in its Pods' specs are running and healthy.
Container runtime	containerd or CRI-O. Actually runs the containers, via the CRI.
kube-proxy	Programs network rules so Service traffic reaches the right Pods. Optional with some CNI plugins.
Add-ons	Not core, but near universal: CNI plugin, CoreDNS, metrics-server.

ON THE EXAM

The kubelet only manages containers that Kubernetes created. It ignores anything started by hand on the node.

Control plane node versus worker node

CONTROL PLANE NODE

Decides

- Runs API server, etcd, scheduler, controllers
- Holds and changes cluster state
- Usually tainted so apps do not land there
- Several of them for high availability

WORKER NODE

Runs the apps

- Runs kubelet, kube-proxy, a runtime
- Hosts your Pods
- Reports status to the API server
- Every cluster needs at least one

THE TIEBREAKER

Control plane nodes manage state. Worker nodes run workloads. Managed services hide the control plane from you entirely.

Quayside deploys its booking API

```
$ kubectl apply -f booking-api.yaml

# booking-api.yaml
apiVersion: apps/v1
kind: Deployment
metadata: {name: booking-api}
spec:
  replicas: 3
  selector: {matchLabels: {app: booking-api}}
  template:
    metadata: {labels: {app: booking-api}}
    spec:
      containers:
      - {name: api, image: quayside/booking-api:2.4.1}
```

1 One command. We will follow what happens next, step by step.

8 The desired state: three copies. Nothing here says how or where.

The journey, part 1: from kubectl to Pods that exist

01
kubectl
Sends the object to the API server over HTTPS.

›

02
API server
Authenticates, authorises, runs admission, validates.

›

03
etcd
Stores the Deployment. kubectl reports created.

›

04
Deployment controller
Sees it and creates a ReplicaSet.

›

05
ReplicaSet controller
Creates three Pods, with no node yet.

ON THE EXAM

kubectl returns as soon as the object is stored. Nothing is running yet.

The journey, part 2: from Pods to traffic

01

Scheduler

Sees unassigned Pods. Filters, scores, binds each to a node.

›

02

kubelet

Sees a Pod bound to its node.

›

03

Runtime

Pulls the image, starts the containers via the CRI.

›

04

kubelet reports

Pod status goes back to the API server.

›

05

EndpointSlice

Ready Pods are added; kube-proxy updates rules.

ON THE EXAM

Pending means not yet scheduled or still pulling. Running means the kubelet has started the containers.

No component told another what to do. Each one **watched the API server** and reacted to what it saw.

When a component fails, what do you see?

IF THIS IS DOWN...	YOU SEE...
`kube-apiserver`	Running Pods keep running. No kubectl, no changes, no rescheduling.
`etcd`	The API server cannot read or store state. Lose it without a backup and the cluster is gone.
`kube-scheduler`	New Pods stay Pending with no node. Existing Pods are fine.
`controller-manager`	Nothing is replaced or scaled. Deleted Pods stay deleted.
`kubelet on one node`	That node goes NotReady; its Pods are later evicted and replaced elsewhere.

ON THE EXAM

Match the symptom to the one job. Pending with no node assigned points at the scheduler.

The cloud-controller-manager

- **Node controller.** Asks the cloud whether a silent node's VM still exists, and removes it if not.
- **Route controller.** Sets up routes in the cloud network between nodes.
- **Service controller.** Creates a cloud load balancer for each Service of type LoadBalancer.
- **No cloud integration?** A LoadBalancer Service's external IP stays pending forever.

> ON THE EXAM
>
> On-premises clusters have no cloud-controller-manager. They need something like MetalLB for LoadBalancer Services.

Quayside's system: the running example

PART	KUBERNETES OBJECT	WHY THAT OBJECT
Web front end	Deployment + Service	Stateless, many identical copies
Booking API	Deployment + autoscaler	Stateless, load varies by hour
Payment worker	Deployment	Long-running queue consumer
Ticket validator	Deployment	Go binary, distroless image
Bookings database	StatefulSet + volumes	Needs stable identity and storage
Log agent	DaemonSet	One copy on every node
Nightly manifest export	CronJob	Runs on a schedule, then finishes
Partner fare plugins	Pods + RuntimeClass	Untrusted code, sandboxed

ON THE EXAM

Every module from here uses this system. When a scenario mentions a part, you already know its shape.

> Quayside builds a test cluster with kubeadm on virtual machines in a public cloud. Nobody configures any cloud provider integration.
>
> They create a Service of type LoadBalancer for the web front end. Its external IP shows pending, and stays pending.

Which component's absence explains this?

A kube-proxy, which programs the Service rules on each node

B CoreDNS, which gives the Service its DNS name in the cluster

C kube-scheduler, which places the Service's Pods on nodes

D cloud-controller-manager, which talks to the cloud's API

◆ PAUSE AND DECIDE

Which component's absence explains this?

A kube-proxy, which programs the Service rules on each node

Handles traffic inside the cluster. It never creates cloud load balancers.

B CoreDNS, which gives the Service its DNS name in the cluster

DNS names are internal. The external IP comes from the cloud.

C kube-scheduler, which places the Service's Pods on nodes

Services are not scheduled, and Pod placement is unrelated.

D cloud-controller-manager, which talks to the cloud's API

Its service controller creates the cloud load balancer and fills in the IP.

THE SIGNAL

'No cloud integration' plus 'external IP pending' is the cloud-controller-manager, every time.

> After a control plane restart, Quayside scales the booking API from 3 to 6 replicas. kubectl shows the ReplicaSet now wants 6, and 6 Pods exist.
>
> The 3 new Pods stay Pending. They have no node assigned and no scheduling events. The 3 original Pods serve traffic normally.

Which component is most likely not working?

A kube-controller-manager

B etcd

C kube-scheduler

D The kubelet on every node

◆ PAUSE AND DECIDE

Which component is most likely not working?

A kube-controller-manager

It is working: the ReplicaSet controller created the three new Pods.

B etcd

State is being stored and read, so etcd is fine.

C kube-scheduler

Pods exist but no node was ever chosen: the scheduler's one job.

D The kubelet on every node

Kubelets act only after a Pod is bound to their node. None was.

THE SIGNAL

Pods exist but have no node: scheduler. Pods do not exist at all: controller manager.

> Quayside's security review wants a firewall rule so that only one Kubernetes component can connect to the etcd cluster.
>
> The cluster must keep working normally afterwards.

Which component should be allowed to reach etcd?

A kube-apiserver

B kube-controller-manager

C The kubelet on each node

D kube-scheduler

◆ PAUSE AND DECIDE

Which component should be allowed to reach etcd?

A kube-apiserver

The only component that reads and writes etcd. Everything else goes through it.

B kube-controller-manager

Controllers read and write state through the API server, not etcd.

C The kubelet on each node

Kubelets talk to the API server. Opening etcd to every node would be a big risk.

D kube-scheduler

The scheduler watches and binds through the API server too.

THE SIGNAL

Everything goes through the API server. etcd has exactly one client.

> During a control plane upgrade, Quayside's API server is unreachable for ten minutes. All worker nodes stay healthy.
>
> The on-call engineer asks what is happening to the booking API Pods already running on the workers.

What best describes those Pods during the outage?

A They stop, because the kubelet cannot confirm their spec

B They keep running, but nothing can be changed or replaced

C The scheduler moves them to nodes with a working API

D The kubelet deletes them after missing status updates

◆ PAUSE AND DECIDE

What best describes those Pods during the outage?

A They stop, because the kubelet cannot confirm their spec

The kubelet keeps running what it already has. It does not need permission.

B They keep running, but nothing can be changed or replaced

Workloads carry on; changes, scaling and healing wait for the API.

C The scheduler moves them to nodes with a working API

The scheduler needs the API server too, and never moves running Pods.

D The kubelet deletes them after missing status updates

Nothing deletes healthy Pods because reporting failed.

THE SIGNAL

The control plane decides; nodes run. A control plane outage freezes change, not workloads.

Module 3 in five lines

- Kubernetes is **desired state plus controllers** that reconcile, like a thermostat.
- **Only the API server talks to etcd.** Everything else watches the API server.
- **Scheduler places, controllers create, kubelet runs.** Keep the verbs straight.
- **Pending with no node** means scheduler. No LoadBalancer IP means cloud-controller-manager.
- A control plane outage **freezes change, not running workloads**.

Every object you apply has the same skeleton. Next: **the API and the object model**.

MODULE 04 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

The API and the object model

What an object is, how the API is versioned, and how Kubernetes learns new kinds.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 4 of 35

admin@sthithapragna.com

Everything in Kubernetes is **an object**, read and written through **one REST API**.

The anatomy of an object

Trimmed from kubectl get deployment booking-api -o yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: booking-api
  namespace: quayside
  labels: {app: booking-api}
  uid: 6f1c2e9a-4b7d-4e21-9a0c-2d5e8f3b1a77
spec:
  replicas: 3
  # ...the Pod template...
status:
  replicas: 3
  readyReplicas: 2
```

1-2 Which API group and version, and which kind of object.

4-7 Identity. Name is unique per kind in a namespace; uid is unique forever.

8 spec: the desired state. You write this.

11 status: the observed state. The system writes this.

spec versus status

SPEC

What you want

- Written by you, or your tooling
- Declares desired state
- Changes when you change your mind
- Example: replicas 3

STATUS

What is true

- Written by controllers and the kubelet
- Reports observed state
- Changes as the world changes
- Example: readyReplicas 2

THE TIEBREAKER

You write spec. The system writes status. Reconciliation is the work of closing the gap between them.

Which fields are required?

- **apiVersion, kind and metadata** appear on every object. metadata must carry a name.
- **spec** is required for most kinds, such as Pods, Deployments and Services.
- **Not every kind has a spec.** ConfigMaps and Secrets carry data instead.
- **status** is never something you write in a manifest.

> ON THE EXAM
>
> If asked for the fields every manifest needs, the documented answer is apiVersion, kind, metadata and spec.

API groups you will see in manifests

APIVERSION	KINDS
v1	The core group: Pod, Service, ConfigMap, Secret, Namespace, Node, PVC
apps/v1	Deployment, ReplicaSet, StatefulSet, DaemonSet
batch/v1	Job, CronJob
networking.k8s.io/v1	Ingress, IngressClass, NetworkPolicy
rbac.authorization.k8s.io/v1	Role, ClusterRole, RoleBinding, ClusterRoleBinding
autoscaling/v2	HorizontalPodAutoscaler
gateway.networking.k8s.io/v1	Gateway, HTTPRoute. Installed separately, as CRDs

ON THE EXAM

A bare v1 means the core group. Everything else is group/version.

The URLs behind kubectl

Illustrative: what the API server actually receives.

```
$ kubectl get pods -n quayside
  GET /api/v1/namespaces/quayside/pods

$ kubectl get deployment booking-api -n quayside
  GET /apis/apps/v1/namespaces/quayside/deployments/booking-api

$ kubectl create -f nightly-export-job.yaml
  POST /apis/batch/v1/namespaces/quayside/jobs
```

2 The core group lives under /api/v1.

5 Named groups live under /apis/group/version.

8 Create is a POST. Get is a GET. It is plain REST over HTTPS.

Alpha, beta, stable: what each version promises

EXAMPLE	LEVEL	THE PROMISE
v1alpha1	Alpha	Experimental. Off by default. May change or vanish in any release.
v2beta3	Beta	Well tested, but may still change incompatibly. New beta APIs are off by default.
v1	Stable (GA)	Will not be removed within a major version of Kubernetes.

ON THE EXAM

The version name tells you the promise. v2beta3 means: third beta of version 2, still allowed to change.

Currency check: the deprecation policy

- **Stable APIs:** may be deprecated, but never removed within Kubernetes 1.x.
- **Beta APIs:** still served at least 9 months or 3 minor releases after deprecation.
- **Alpha APIs:** can be removed in any release, with no notice.
- The **12 months** often quoted comes from an older rule. Today it applies to kubectl flags.
- Kubernetes ships about three minor releases a year, one every four months or so.

> ON THE EXAM
>
> Calling a deprecated API still works, but the response carries a warning. Treat it as a deadline.

The life of an API version

01

v1alpha1

Experiment. May change freely.

›

02

v1beta1

Tested. Alpha removed.

›

03

v1beta2

v1beta1 deprecated.

›

04

v1

Stable. Betas deprecated.

›

05

Betas removed

After their deprecation window.

ON THE EXAM

Old manifests break on upgrade when they still name a beta version that has since been removed.

Asking the cluster about its own API

COMMAND	ANSWERS
`kubectl api-resources`	Which kinds exist, their short names, group, and whether namespaced
`kubectl api-versions`	Which group/version pairs this cluster serves
`kubectl explain deployment.spec.replicas`	What a field means and its type, from the live schema
`kubectl get deploy booking-api -o yaml`	The full object as stored, spec and status

ON THE EXAM

Field documentation: explain. List of kinds: api-resources. The live object: get -o yaml.

Three ways to manage objects

TECHNIQUE	EXAMPLE	TRADE-OFF
Imperative commands	kubectl scale deploy web --replicas=5	Fast to type. No file, no history, hard to review.
Imperative object configuration	kubectl create -f web.yaml	A file exists, but you say create, replace or delete.
Declarative object configuration	kubectl apply -f manifests/	The files are the truth. Kubernetes works out the changes.

ON THE EXAM

Declarative is the foundation of GitOps: the files in Git are the desired state.

Why reconciliation is so robust

- **Level-triggered.** Controllers compare current state with desired state, not a list of events.
- **Missed events do not matter.** A restarted controller just looks again and acts.
- **Drift gets corrected.** Delete a Deployment's Pod by hand and a new one appears.
- **Hand edits to managed objects are overwritten.** Change the owner, not the child.

> ON THE EXAM
>
> To change what a controller manages, change the controller's spec. Editing its children is undone.

Kubernetes can learn **new kinds of object**. That is how most of the ecosystem plugs in.

Two ways to extend the API

CUSTOMRESOURCEDEFINITION

Declare a new kind

- Apply one YAML object
- API server stores it in etcd
- No extra server to run
- Used by most projects

AGGREGATION LAYER

Run your own API server

- You build and run an extension API server
- kube-apiserver proxies a group to it
- Your own storage and logic
- Example: metrics-server

THE TIEBREAKER

Almost always a CRD. Aggregation when the data is not stored objects, like live metrics from metrics.k8s.io.

A custom resource at Quayside

After a CRD for FerryRoute is installed, this is an ordinary object.

```
1  apiVersion: quayside.io/v1
2  kind: FerryRoute
3  metadata: {name: dover-calais}
4  spec:
5    departurePort: Dover
6    arrivalPort: Calais
7    sailingsPerDay: 12
8
9  $ kubectl get ferryroutes
10 NAME           AGE
11 dover-calais   4m
```

1-2 Quayside's own group and kind, defined by a CRD.

9 kubectl works with it like any built-in kind.

On its own, a CRD only stores the object. Nothing happens until a controller acts on it.

The operator pattern

- **Operator = custom resource + custom controller.** The controller reconciles the resource.
- **It encodes a human operator's know-how:** install, upgrade, back up, fail over.
- **Examples:** Prometheus Operator, cert-manager, CloudNativePG, Rook for Ceph storage.
- You declare a database or a certificate; the operator keeps it running correctly.

> REMEMBER
>
> A CRD adds a noun. A controller adds a verb. Together, they are an operator.

A vendor's scheduling add-on for Quayside offers its API at three versions: v1alpha2, v1beta1 and v1.

The platform lead asks what, exactly, the v1beta1 version promises before any team relies on it.

What does the v1beta1 version promise?

A Nothing; it may vanish in any release without notice

B It is final and will never be changed or removed

C It is well tested, but may still change incompatibly

D It can only be used on managed Kubernetes services

◆ PAUSE AND DECIDE

What does the v1beta1 version promise?

A Nothing; it may vanish in any release without notice

That is the alpha promise, not beta.

B It is final and will never be changed or removed

Closer to stable, and even stable APIs can be deprecated.

C It is well tested, but may still change incompatibly

Exactly the beta promise. Production should prefer v1.

D It can only be used on managed Kubernetes services

Version levels have nothing to do with where a cluster runs.

THE SIGNAL

Alpha: may vanish. Beta: may change. Stable: stays within the major version.

> Quayside upgrades an old test cluster to a current Kubernetes release. Re-applying a years-old manifest now fails.
>
> The error says: no matches for kind Ingress in version extensions/v1beta1. Other Ingress manifests apply cleanly.

What is the cause, and the fix?

A Ingress was removed; rewrite everything using the Gateway API

B That beta version was removed; use networking.k8s.io/v1

C The Ingress CRD is missing and needs to be installed

D An alpha feature gate must be enabled on the API server

◆ PAUSE AND DECIDE

What is the cause, and the fix?

A Ingress was removed; rewrite everything using the Gateway API

The Ingress API is still stable. Only this old version went.

B That beta version was removed; use networking.k8s.io/v1

The kind lives on; the manifest names a removed beta version.

C The Ingress CRD is missing and needs to be installed

Ingress is built in, not a CRD.

D An alpha feature gate must be enabled on the API server

The version was beta, and removed versions cannot be switched back on.

THE SIGNAL

'No matches for kind X in version Y' while other X objects work: the version was removed, not the kind.

> Quayside wants developers to declare a FerryRoute object with kubectl. For each route, the cluster should create and maintain the right Deployments and Services.
>
> The team wants the standard, least-effort approach, with no custom API server to run.

What should Quayside build?

A An aggregated extension API server for ferry routes

B ConfigMaps holding routes, plus a script run by a CronJob

C A CustomResourceDefinition for FerryRoute, and nothing else

D A FerryRoute CRD plus a custom controller: an operator

◆ PAUSE AND DECIDE

What should Quayside build?

A. An aggregated extension API server for ferry routes

Works, but it is the custom API server they said they do not want.

B. ConfigMaps holding routes, plus a script run by a CronJob

No real kind, no validation, and drift is only fixed on a schedule.

C. A CustomResourceDefinition for FerryRoute, and nothing else

Stores the routes, but nothing creates or maintains anything.

D. A FerryRoute CRD plus a custom controller: an operator

The CRD gives the kind; the controller reconciles it.

THE SIGNAL

'Declare it and have the cluster maintain it' needs both a CRD and a controller.

> A Quayside engineer is editing the booking API Deployment and wants to know which fields spec.strategy accepts, and what each means.
>
> They want the answer from the cluster itself, without leaving the terminal.

Which command gives exactly that?

A kubectl explain deployment.spec.strategy

B kubectl api-resources

C kubectl describe deployment booking-api

D kubectl get deployment booking-api -o yaml

◆ PAUSE AND DECIDE

Which command gives exactly that?

A kubectl explain deployment.spec.strategy

Documents each field and its type, from the cluster's own schema.

B kubectl api-resources

Lists kinds, not the fields inside one.

C kubectl describe deployment booking-api

Shows one live object and its events, not field documentation.

D kubectl get deployment booking-api -o yaml

Shows current values, not what the fields mean or allow.

THE SIGNAL

'What does this field mean' is explain. 'What is in this object' is get or describe.

Module 4 in five lines

- Every object is **apiVersion, kind, metadata, spec**; the system adds **status**.
- **Core group is plain v1**; everything else is group/version, served over REST.
- **Alpha may vanish, beta may change, stable stays** within Kubernetes 1.x.
- **Declarative apply** plus level-triggered controllers corrects drift automatically.
- **CRD adds a noun, controller adds a verb**: together, an operator.

All of this state lives in one place. Next: **etcd, high availability** and the control plane at work.

MODULE 05 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

etcd, high availability and the control plane at work

Quorum arithmetic, HA topologies, and the jobs the control plane does in the background.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 5 of 35

admin@sthithapragna.com

etcd does not need every member to agree. It needs a majority.

admin@sthithapragna.com

What etcd is

- **A distributed key-value store.** Kubernetes keeps every object in it.
- **Raft consensus.** One elected leader; followers replicate its log.
- **A write is committed** only when a majority of members have it.
- **A watch API** streams changes. That is what Kubernetes watches are built on.

> ON THE EXAM
>
> Strong consistency is the point: every API server sees the same committed state.

Quorum arithmetic

MEMBERS	QUORUM NEEDED	FAILURES TOLERATED
1	1	0
2	2	0
3	2	1
4	3	1
5	3	2
6	4	2
7	4	3

ON THE EXAM

Quorum is a majority: half the members, rounded down, plus one. Odd sizes give the best value.

What etcd needs to stay healthy

- **Fast disks.** Every committed write is flushed to disk. Slow disks slow the whole cluster.
- **Low network latency** between members, so they can agree quickly.
- **Regular snapshots.** etcdctl snapshot save, stored off the machine.
- **Tight access.** etcd holds every Secret. Only the API server should reach it.

> ON THE EXAM
>
> If asked which resources matter most for etcd performance: disk I/O and the network.

Stacked versus external etcd

STACKED

etcd on each control plane node

- Fewer machines: at least 3 in total
- Simpler to set up; kubeadm's default
- Lose a node, lose an etcd member too
- Coupled failure

EXTERNAL

etcd on its own hosts

- More machines: at least 3 plus 3, so 6
- More to set up and manage
- Control plane and etcd fail independently
- Decoupled failure

THE TIEBREAKER

Stacked is cheaper. External survives more, at the cost of at least six hosts.

A highly available control plane

01

Load balancer

One address for kubectl and every kubelet.

02

API servers

Three or more, all active at once.

03

Scheduler, controllers

One copy each is leader; the rest wait.

04

etcd

Three or five members, keeping quorum.

ON THE EXAM

Spread control plane instances across failure zones, so one zone going dark does not take the majority with it.

Active-active versus leader election

API SERVER

All active

- Stateless: state lives in etcd
- Any copy can serve any request
- Scale out behind a load balancer
- More copies, more capacity

SCHEDULER, CONTROLLER MANAGER

One leader

- Two leaders would fight over the same Pods
- Leader holds a Lease object
- Standbys take over if it stops renewing
- More copies, faster failover only

THE TIEBREAKER

API servers scale out. The scheduler and controller manager fail over.

How the control plane runs itself: static Pods

- **The puzzle:** the API server runs as a Pod, but Pods need an API server.
- **The answer:** the kubelet reads manifests from a folder on disk and runs them directly.
- With kubeadm, that folder is /etc/kubernetes/manifests on each control plane node.
- A read-only **mirror Pod** appears in kube-system so you can see it with kubectl.

> ON THE EXAM
>
> Static Pods are managed by the kubelet, not the API server. Delete the mirror and it simply comes back.

When a node goes silent

01

Heartbeats

The kubelet renews its Lease in kube-node-lease about every 10 seconds.

02

Silence

Heartbeats stop: power loss, crash, network cut.

03

Unknown

After a grace period of under a minute, Ready becomes Unknown.

04

Tainted

The node controller adds a NoExecute unreachable taint.

05

Evicted

Pods tolerate it for 300 seconds by default, then are evicted.

ON THE EXAM

By default, Pods on a dead node are evicted after about five minutes.

After eviction: who comes back?

OWNED BY A CONTROLLER

Replaced

- Deployment, ReplicaSet, StatefulSet, Job
- Controller sees too few Pods
- Creates a new Pod elsewhere
- New UID, and usually a new name

A BARE POD

Gone

- Created directly, e.g. kubectl run
- No controller watching it
- Deleted with the node
- Nothing recreates it

THE TIEBREAKER

Pods do not heal themselves. Controllers heal them. That is why you almost never create bare Pods.

What garbage collection cleans up

TARGET	HOW
Dependents whose owner is deleted	ownerReferences link them; the collector deletes them
Terminated Pods	Removed once they pass a cluster-wide threshold
Finished Jobs	Removed after ttlSecondsAfterFinished, if set
Unused images and containers	The kubelet cleans them on each node
Released volumes	Deleted when the reclaim policy is Delete
Expired certificate requests	Removed automatically

ON THE EXAM

Owner references are the key idea: delete a Deployment and its ReplicaSets and Pods go too.

Cascading deletion: three choices

KUBECTL DELETE --CASCADE=	WHAT HAPPENS
`background (default)`	Owner deleted at once; dependents cleaned up afterwards.
`foreground`	Dependents deleted first; owner waits, marked as being deleted.
`orphan`	Owner deleted; dependents kept, with their owner reference removed.

> ON THE EXAM
>
> Orphan is how you replace a controller without restarting the Pods it manages.

Finalizers: why deletion sometimes hangs

- **A finalizer** is a key on an object saying: do not remove me until I have cleaned up.
- Delete sets a deletion timestamp; the object stays until every finalizer is removed.
- **Good use:** a volume is not released while a Pod still uses it.
- **Stuck in Terminating** usually means a finalizer whose controller has gone.

> ON THE EXAM
>
> Terminating for hours: look for a finalizer, not a broken scheduler or etcd.

How big can a cluster be?

DIMENSION	DESIGNED LIMIT
Pods per node	110
Nodes	5,000
Total Pods	150,000
Total containers	300,000

ON THE EXAM

Unchanged in the Kubernetes 1.37 docs. Beyond this, run more clusters, not a bigger one.

> Quayside's self-managed cluster runs three etcd members. An engineer proposes adding a fourth member for extra resilience.
>
> The platform lead asks how many member failures the four-member cluster could survive.

How many failures could four members tolerate?

A One, because four members need three for a quorum

B Two, because each extra member adds one failure

C Three, because only one member must remain up

D None, because even counts cannot form a quorum

◆ PAUSE AND DECIDE

How many failures could four members tolerate?

A One, because four members need three for a quorum

Majority of four is three, so only one can fail. Same as three members.

B Two, because each extra member adds one failure

Only every second member adds tolerance. Five would give two.

C Three, because only one member must remain up

etcd needs a majority, not a single survivor.

D None, because even counts cannot form a quorum

Even counts work; they just waste a member.

THE SIGNAL

Going from three to four adds cost and a failure point, not tolerance. Go to five.

> For production, Quayside needs a highly available control plane. Losing an etcd member must never also take out a control plane instance.
>
> They accept running extra machines, but want the minimum number that meets the requirement.

Which layout meets the requirement with the fewest hosts?

A Stacked etcd spread across three control plane nodes

B Stacked etcd spread across four control plane nodes

C External etcd sharing the three control plane hosts

D External etcd: 3 control plane hosts, 3 etcd hosts

◆ PAUSE AND DECIDE

Which layout meets the requirement with the fewest hosts?

A Stacked etcd spread across three control plane nodes

Fewest hosts, but each node failure takes both layers down together.

B Stacked etcd spread across four control plane nodes

Still coupled, and four etcd members add no tolerance.

C External etcd sharing the three control plane hosts

Sharing hosts is just stacked etcd under another name.

D External etcd: 3 control plane hosts, 3 etcd hosts

Decoupled failure, at the minimum of six hosts.

THE SIGNAL

'Must fail independently' means external etcd, and that means at least six hosts.

> At 10:00, a worker node loses power. It was running three web front-end Pods from a Deployment, and a debugging Pod an engineer created with kubectl run.
>
> Default settings apply. The on-call engineer checks the cluster at 10:07.

What will the engineer most likely find?

A All four Pods have been recreated on other nodes

B The web Pods were replaced elsewhere; the debug Pod is gone

C Nothing has changed; Pods wait until the node returns

D Everything was replaced within ten seconds of power loss

◆ PAUSE AND DECIDE

What will the engineer most likely find?

A All four Pods have been recreated on other nodes

Nothing owns the debug Pod, so nothing recreates it.

B The web Pods were replaced elsewhere; the debug Pod is gone

Evicted after about five minutes; only the Deployment recreates its Pods.

C Nothing has changed; Pods wait until the node returns

True before about five minutes. By 10:07 eviction has happened.

D Everything was replaced within ten seconds of power loss

There is a grace period, then a 300-second toleration.

THE SIGNAL

Seven minutes is past the five-minute eviction. Ownership decides what comes back.

> Quayside deletes an old test namespace. Hours later, kubectl still shows the namespace as Terminating.
>
> Other namespaces can be created and deleted normally, and all workloads are healthy.

What is the most likely cause?

A. etcd has lost quorum, so the deletion cannot be stored

B. The scheduler is down, so nothing can be removed

C. A finalizer is waiting for a controller that is not acting

D. Foreground deletion always takes several hours

◆ PAUSE AND DECIDE

What is the most likely cause?

A etcd has lost quorum, so the deletion cannot be stored

Then nothing could change anywhere. Other namespaces work fine.

B The scheduler is down, so nothing can be removed

The scheduler places Pods. It plays no part in deletion.

C A finalizer is waiting for a controller that is not acting

Deletion was accepted but is blocked until the finalizer is removed.

D Foreground deletion always takes several hours

It waits for dependents, which normally takes seconds.

THE SIGNAL

One object stuck in Terminating while everything else works: a finalizer.

Module 5 in five lines

- etcd needs a **majority**: 3 members tolerate 1 failure, 5 tolerate 2. Never 4.
- **Stacked**: 3 hosts, coupled. **External**: 6 hosts, independent.
- **API servers are all active**; scheduler and controllers use leader election.
- Silent node: Pods **evicted after about 5 minutes**; only owned Pods come back.
- **Owner references** drive garbage collection; **finalizers** can block deletion.

We have the whole cluster. Next: the Pod, the smallest thing it runs.

admin@sthithapragna.com

MODULE 06 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Pods

The smallest thing Kubernetes runs: what it shares, what it contains, and how it lives and dies.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 6 of 35

admin@sthithapragna.com

A Pod is **a shared context** for one or more containers: one IP address, shared volumes, one lifecycle.

Why Pods, and not bare containers?

- **The smallest deployable unit.** Kubernetes creates, schedules and deletes Pods, never single containers.
- **Co-location.** Every container in a Pod lands on the same node, together.
- **Usually one app container.** Extra containers are helpers for that one app.
- **Scale by adding Pods,** not by adding containers to a Pod.

> ON THE EXAM
>
> Hierarchy to remember: cluster, then nodes, then Pods, then containers.

What containers in one Pod share

THING	SHARED?
Network namespace: IP address, ports, localhost	Yes, always
IPC namespace: shared memory	Yes
Hostname (UTS)	Yes
Volumes	Yes, if each container mounts them
Process list (PID namespace)	Only with shareProcessNamespace: true
Root filesystem	No. Each container has its own image
CPU and memory limits	No. Set per container

ON THE EXAM

Network and storage are the two things the exam says containers in a Pod share.

Consequences of sharing a network

- **One port space.** Two containers in a Pod cannot both listen on port 8080.
- **localhost works.** A helper reaches the app at localhost, no Service needed.
- **One Pod, one IP.** Other Pods reach it by that IP, or through a Service.
- The shared namespaces are held open by a tiny **pause** container, created by the runtime.

> ON THE EXAM
>
> Different Pods never clash on ports: each Pod has its own IP address.

Two containers in one Pod, or two Pods?

SAME POD

Tightly coupled

- Must run on the same node
- Share files or localhost
- Start and stop together
- Example: app plus its log shipper

SEPARATE PODS

Loosely coupled

- Scale independently
- Fail independently
- Talk over a Service
- Example: web front end and database

THE TIEBREAKER

One Pod only when they must share a node and a lifecycle. Otherwise, separate Pods.

Four kinds of container in a Pod

KIND	WHERE DECLARED	BEHAVIOUR
App	containers	The main program.
Init	initContainers	Runs to completion, in order, before the app.
Sidecar	initContainers, restartPolicy: Always	Starts first, keeps running, stops last.
Ephemeral	added to a running Pod	Temporary, for debugging. Never restarted.

ON THE EXAM

Native sidecars: stable since Kubernetes 1.33. Init containers that never finish.

Quayside's booking API Pod, all three at once

```
spec:
  initContainers:
  - name: migrate
    image: quayside/booking-migrate:2.4.1
  - name: log-shipper
    image: fluent/fluent-bit:4.0
    restartPolicy: Always
  containers:
  - name: api
    image: quayside/booking-api:2.4.1
```

3 A regular init container: migrates the database schema, then exits.

7 restartPolicy: Always turns this init container into a sidecar.

9 The app starts only after migrate succeeds and the sidecar has started.

Start-up and shut-down order

01
Init containers
Run one by one; each must succeed.

›

02
Sidecars start
In order, and keep running.

›

03
App containers
Start once all of the above are done.

›

04
Shut-down
App containers stop first.

›

05
Sidecars stop
Last, in reverse order.

ON THE EXAM

Sidecars wrap the app: first in, last out. So logs and proxies cover the app's whole life.

Old-style sidecar versus native sidecar

OLD STYLE

A second app container

- Listed under containers
- No start-up ordering
- May stop before the app finishes
- Keeps Job Pods from completing

NATIVE, STABLE SINCE 1.33

An init container that keeps running

- initContainers, restartPolicy: Always
- Guaranteed to start first
- Stops after the app
- Job completes when the app exits

THE TIEBREAKER

If a Job never finishes because a helper keeps running, the fix is a native sidecar.

Multi-container patterns

PATTERN	THE HELPER...	EXAMPLE
Sidecar	extends or enhances the app	Log shipper, service mesh proxy
Ambassador	proxies the app's calls to the outside world	Local proxy to a remote database
Adapter	reshapes the app's output for others	Converts custom metrics to Prometheus format
Init	prepares things before the app starts	Schema migration, fetching config

ON THE EXAM

All three helper patterns are just containers in the same Pod. Only the job differs.

A Pod is born, runs and dies. It is **never resurrected**. A replacement is always a new Pod.

The five Pod phases

PHASE	MEANS
Pending	Accepted, but containers are not running yet: waiting to schedule, or pulling images.
Running	Bound to a node; at least one container is running, starting or restarting.
Succeeded	All containers exited successfully and will not be restarted.
Failed	All containers have stopped, and at least one failed.
Unknown	The Pod's state cannot be obtained, usually lost contact with the node.

ON THE EXAM

CrashLoopBackOff, ContainerCreating and Terminating are not phases. They are what the STATUS column shows.

Container states and their reasons

STATE	COMMON REASONS
Waiting	ContainerCreating, ImagePullBackOff, ErrImagePull, CrashLoopBackOff
Running	The process is running. Probes may still fail.
Terminated	Completed, Error, OOMKilled, with an exit code

> ON THE EXAM
>
> kubectl get pods shows the most important container reason in STATUS, which is why it looks like a phase.

restartPolicy: set once for the whole Pod

VALUE	RESTARTS CONTAINERS...	TYPICAL USE
`Always (default)`	whenever they exit, even successfully	Deployments, StatefulSets, DaemonSets
`OnFailure`	only after a non-zero exit	Jobs that should retry in place
`Never`	never	Jobs that retry by making new Pods, or one-off tasks

ON THE EXAM

Restarts back off: 10, 20, 40 seconds and so on, capped at five minutes, reset after ten healthy minutes.

What CrashLoopBackOff really means

- **A waiting reason, not a phase.** The kubelet is pausing before the next restart.
- **The container keeps exiting.** It started, then crashed or was killed, again and again.
- **Usual causes:** the app crashes at start, bad config, a missing dependency, OOMKilled, a failing liveness probe.
- **Diagnose with** kubectl logs --previous and kubectl describe pod.

> ON THE EXAM
>
> CrashLoopBackOff says the container starts and then dies. ImagePullBackOff says it never started at all.

Graceful termination

01
Delete
Pod marked Terminating; removed from Service endpoints.

02
preStop hook
Runs first, if defined.

03
SIGTERM
Sent to each container's main process.

04
Grace period
30 seconds by default, to finish work.

05
SIGKILL
Anything still running is killed.

ON THE EXAM

Apps should handle SIGTERM and finish in-flight work. Otherwise every deploy drops requests.

> Quayside adds a caching proxy container to the booking API Pod. Both the API and the proxy are configured to listen on port 8080.
>
> The API starts normally. The proxy crashes repeatedly with the error: address already in use.

Why does the proxy fail?

A Another Pod on the same node is already using port 8080

B The Service can only route to one port per Pod

C kube-proxy reserves port 8080 on every node

D Containers in a Pod share one network namespace

◆ PAUSE AND DECIDE

Why does the proxy fail?

A Another Pod on the same node is already using port 8080

Each Pod has its own IP and port space. Other Pods cannot clash.

B The Service can only route to one port per Pod

Services can expose several ports, and a Service never blocks binding.

C kube-proxy reserves port 8080 on every node

kube-proxy reserves no such port inside Pods.

D Containers in a Pod share one network namespace

One IP, one port space: the second listener on 8080 collides.

THE SIGNAL

A port clash inside one Pod means shared network namespace. Change one container's port.

> Quayside's nightly manifest export runs as a Job. The team added a log-shipper as a second entry under containers.
>
> Since then, the export finishes its work each night, but the Job never completes. The Pod stays Running.

What is the best fix?

A Move the log shipper to initContainers with restartPolicy: Always

B Set the Pod's restartPolicy to Never so nothing restarts

C Add activeDeadlineSeconds so the Job is stopped after an hour

D Move the log shipper to initContainers, leaving restartPolicy unset

◆ PAUSE AND DECIDE

What is the best fix?

A Move the log shipper to initContainers with restartPolicy: Always

A native sidecar. The Job completes when the export container exits.

B Set the Pod's restartPolicy to Never so nothing restarts

The shipper still never exits, so the Pod still never finishes.

C Add activeDeadlineSeconds so the Job is stopped after an hour

Stops it, but the Job is then marked Failed every night.

D Move the log shipper to initContainers, leaving restartPolicy unset

A regular init container must finish first; the export would never start.

THE SIGNAL

A helper that never exits, blocking a Job: make it a native sidecar.

> kubectl get pods shows the payment worker with STATUS CrashLoopBackOff and RESTARTS 7. Its restartPolicy is the default.
>
> A colleague says the Pod is in the CrashLoopBackOff phase. You check the Pod's actual phase field.

What phase will the Pod most likely report?

A Pending, because the container is not currently up

B Failed, because the container keeps crashing

C Running, while the kubelet keeps restarting it

D Unknown, because its state cannot be obtained

◆ PAUSE AND DECIDE

What phase will the Pod most likely report?

A Pending, because the container is not currently up

Pending ends once the Pod is bound and its containers created.

B Failed, because the container keeps crashing

With restartPolicy Always, a Pod never reaches Failed this way.

C Running, while the kubelet keeps restarting it

Running includes containers being restarted; the reason is CrashLoopBackOff.

D Unknown, because its state cannot be obtained

The kubelet is reporting fine. Unknown means lost contact.

THE SIGNAL

Phase and STATUS are different fields. Restarting in place keeps the phase at Running.

> Quayside is writing a Job for a monthly fare recalculation. If the container exits with an error, it should be restarted in the same Pod.
>
> If it exits successfully, it must not run again.

Which restartPolicy should the Job's Pod template use?

A Always, so any exit triggers a restart

B OnFailure, so only error exits restart

C Never, so each failure creates a new Pod

D Leave it unset and accept the default

◆ PAUSE AND DECIDE

Which restartPolicy should the Job's Pod template use?

A Always, so any exit triggers a restart
Would rerun after success, and a Job does not accept it.

B OnFailure, so only error exits restart
Restarts in place on error; leaves a successful exit alone.

C Never, so each failure creates a new Pod
Retries happen, but in new Pods, not the same one.

D Leave it unset and accept the default
The default is Always, which a Job rejects.

THE SIGNAL

'Restart in the same Pod, only on error' is OnFailure, word for word.

Module 6 in five lines

- A Pod is **one IP and shared volumes** for containers that must live together.
- **Init** runs first and finishes; a **native sidecar** is an init container with restartPolicy: Always.
- Five phases: **Pending, Running, Succeeded, Failed, Unknown**. CrashLoopBackOff is not one.
- **restartPolicy**: Always by default, OnFailure or Never for Jobs.
- Delete sends **SIGTERM**, waits **30 seconds**, then SIGKILL.

You rarely create Pods yourself. Next: **ReplicaSets and Deployments**, which create them for you.

MODULE 07 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

ReplicaSets and Deployments

Keeping the right number of Pods, and changing them without downtime.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 7 of 35

admin@sthithapragna.com

A ReplicaSet keeps a number of Pods running. A Deployment manages change.

What a ReplicaSet is made of

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata: {name: web}
spec:
  replicas: 3
  selector:
    matchLabels: {app: web}
  template:
    metadata:
      labels: {app: web}
    spec:
      containers:
      - {name: web, image: quayside/web:5.2.0}
```

5 How many Pods to keep running.

7 Which Pods count: any Pod whose labels match.

10 The template's labels must match the selector, or the API rejects it.

The selector gotcha

- **It counts any matching Pod,** including ones it did not create.
- **Create a Pod by hand** with the same labels, and the ReplicaSet may delete one to stay at its number.
- **A Deployment's selector is immutable** once created. Changing it means a new Deployment.
- **ReplicationController** is the legacy predecessor. You will still see it named.

> ON THE EXAM
>
> Labels are the glue. Unique, deliberate labels keep controllers from adopting Pods they should not.

One Deployment, several ReplicaSets

Summarised from kubectl get replicasets and kubectl rollout history

REPLICASET	REVISION	IMAGE	PODS
booking-api-7d9f6c5b8d	3	quayside/booking-api:2.4.1	3
booking-api-5b8c4d7f9a	2	quayside/booking-api:2.4.0	0
booking-api-66f7a1c2e4	1	quayside/booking-api:2.3.9	0

ON THE EXAM

Each template version gets its own ReplicaSet, named with a pod-template-hash. Old ones are kept at zero for rollback.

ReplicaSet versus Deployment

REPLICASET

Keeps a count

- Maintains N matching Pods
- No rollout logic
- No history or rollback
- Usually created by a Deployment

DEPLOYMENT

Manages versions

- Owns and manages ReplicaSets
- Rolling updates or Recreate
- Revision history and rollback
- Pause, resume and restart

THE TIEBREAKER

For a stateless app, create a Deployment. The Deployment manages ReplicaSets, which manage Pods.

What triggers a rollout

- **Only changes to the Pod template,** spec.template: image, environment, resources, template labels.
- **Scaling is not a rollout.** Changing replicas creates no new ReplicaSet and no new revision.
- **kubectl rollout restart** forces a fresh rollout by stamping a timestamp into the template.
- Changing a ConfigMap the Pods read does **not** trigger a rollout on its own.

> ON THE EXAM
>
> New template, new ReplicaSet, new revision. Anything else changes the current ReplicaSet in place.

A rolling update, step by step

4 replicas, maxSurge 1, maxUnavailable 0

01
Surge
New ReplicaSet scales to 1. Five Pods exist.

02
Ready
The new Pod passes its readiness check.

03
Retire
Old ReplicaSet scales from 4 to 3.

04
Repeat
New 2, old 2; then new 3, old 1.

05
Done
New 4, old 0, kept for rollback.

ON THE EXAM

Capacity never drops below four, because no old Pod leaves until a new one is Ready.

maxSurge and maxUnavailable

SETTING	DEFAULT	MEANS	WITH 4 REPLICAS
`maxSurge`	25%, rounded up	Extra Pods allowed above replicas	Up to 5 Pods
`maxUnavailable`	25%, rounded down	Pods allowed to be unavailable	At least 3 available

ON THE EXAM

Both cannot be zero. maxUnavailable 0 means no capacity loss; a bigger maxSurge means a faster rollout.

RollingUpdate versus Recreate

ROLLINGUPDATE (DEFAULT)

Gradual, no downtime

- Old and new run side by side briefly
- Paced by maxSurge and maxUnavailable
- Needs versions that can coexist
- Stalls safely if new Pods never get Ready

RECREATE

All out, then all in

- Every old Pod is stopped first
- Then all new Pods start
- A short outage on every deploy
- For versions that must never overlap

THE TIEBREAKER

Default to RollingUpdate. Choose Recreate only when two versions running at once would break something.

Readiness is what paces a rollout

- **A new Pod counts as available** only once it is Ready, and stays Ready for minReadySeconds.
- **A broken release stalls,** it does not spread: old Pods keep serving.
- **progressDeadlineSeconds**, 600 by default, marks it as failed with ProgressDeadlineExceeded.
- **There is no automatic rollback.** Someone, or some tool, must undo it.

> ON THE EXAM
>
> A stuck rollout is safe but not self-fixing. Roll back, or fix forward.

Driving a rollout from the command line

```
$ kubectl set image deployment/booking-api api=quayside/booking-api:2.5.0
$ kubectl rollout status deployment/booking-api
Waiting for rollout to finish: 2 of 4 updated replicas are available...
$ kubectl rollout history deployment/booking-api
REVISION  CHANGE-CAUSE
3         image 2.4.1
4         image 2.5.0
$ kubectl rollout undo deployment/booking-api
$ kubectl rollout undo deployment/booking-api --to-revision=2
```

1. Changes the template's image, so a rollout starts.
2. Watches progress until it completes, or fails.
4. Lists revisions. CHANGE-CAUSE comes from an annotation.
8-9. Back one revision, or to a specific one.

The rollout family, and its neighbours

COMMAND	DOES
`kubectl rollout status`	Waits for a rollout to finish, or reports it stuck
`kubectl rollout history`	Lists revisions; --revision=N shows one in detail
`kubectl rollout undo`	Returns to the previous, or a named, revision
`kubectl rollout pause / resume`	Batches several template changes into one rollout
`kubectl rollout restart`	Replaces every Pod without changing the spec
`kubectl scale --replicas=N`	Changes the count. Not a rollout

ON THE EXAM

status, history, undo and restart also work on DaemonSets and StatefulSets. Pause and resume are for Deployments only.

What a rollback really does

- **Undo copies an old template** back into the Deployment. The old ReplicaSet scales back up.
- **It becomes a new revision.** Revision 3 rolled back becomes revision 5; 3 leaves the list.
- **With GitOps, roll back in Git.** Otherwise the GitOps tool re-applies the broken version.
- Record why with the kubernetes.io/change-cause annotation.

> ON THE EXAM
>
> A rollback is just another rollout, to an older template.

> Quayside's booking API Deployment has 4 replicas and the default RollingUpdate settings. A new image is rolled out.
>
> The capacity planner asks for the limits that hold at every moment during the rollout.

What are the limits during this rollout?

A. At most 4 Pods exist, and at least 4 are available

B. At most 6 Pods exist, and at least 2 are available

C. At most 5 Pods exist, and at least 3 are available

D. At most 8 Pods exist, and none need be available

◆ PAUSE AND DECIDE

What are the limits during this rollout?

A At most 4 Pods exist, and at least 4 are available
That needs both settings at zero, which is not allowed.

B At most 6 Pods exist, and at least 2 are available
That would be 50% surge and 50% unavailable.

C At most 5 Pods exist, and at least 3 are available
25% of 4 is 1: surge of 1, unavailable of 1.

D At most 8 Pods exist, and none need be available
Describes nothing real; Recreate would allow zero, but never eight.

THE SIGNAL

Defaults: 25% surge rounded up, 25% unavailable rounded down. Do the arithmetic on the replica count.

> Quayside's payment worker holds an exclusive session with the card processor. Two versions running at once, even briefly, cause duplicate charges.
>
> A short outage during each release is acceptable to the business.

Which Deployment strategy fits?

A RollingUpdate with maxSurge 1 and maxUnavailable 0

B RollingUpdate with the default 25% settings

C A canary, sending a small share to the new version first

D Recreate, so all old Pods stop before new ones start

◆ PAUSE AND DECIDE

Which Deployment strategy fits?

A RollingUpdate with maxSurge 1 and maxUnavailable 0

Guarantees overlap: old and new run together by design.

B RollingUpdate with the default 25% settings

Still runs both versions side by side during the rollout.

C A canary, sending a small share to the new version first

A canary is overlap on purpose, the opposite of what is needed.

D Recreate, so all old Pods stop before new ones start

No overlap at all, at the cost of the outage they accepted.

THE SIGNAL

'Must never overlap' plus 'outage acceptable' is Recreate.

> An engineer sets a new image on the web Deployment, which has 3 replicas and default settings. The image tag has a typo.
>
> kubectl rollout status hangs. One new Pod shows ImagePullBackOff. The three original Pods are still Running.

What is the situation, and the best next step?

A Old Pods still serve; run kubectl rollout undo

B It will roll back automatically at the deadline

C Delete the old ReplicaSet to force the change

D Scale to zero and back up to restart the rollout

◆ PAUSE AND DECIDE

What is the situation, and the best next step?

A Old Pods still serve; run kubectl rollout undo

The rollout stalled safely. Undo returns to the working template.

B It will roll back automatically at the deadline

The deadline only reports failure. Nothing rolls back by itself.

C Delete the old ReplicaSet to force the change

Deletes the only working Pods: a self-inflicted outage.

D Scale to zero and back up to restart the rollout

Takes the site down, and the new template is still broken.

THE SIGNAL

A stalled rollout protects you. Undo it, or fix the tag and roll forward.

> Yesterday Quayside scaled the booking API from 3 to 6 replicas. Today they changed one environment variable in the Pod template.
>
> kubectl rollout history shows only one new revision since last week.

Why is there only one new revision?

A revisionHistoryLimit is set to 1

B Scaling never changes the Pod template

C Revisions are recorded only with --record

D The scale revision is stored on the Pods

◆ PAUSE AND DECIDE

Why is there only one new revision?

A revisionHistoryLimit is set to 1

Possible in theory, but nothing suggests it; the default is 10.

B Scaling never changes the Pod template

Only template changes create revisions. The env change made one.

C Revisions are recorded only with --record

--record only filled in the change cause, and it is deprecated.

D The scale revision is stored on the Pods

There is no such thing as a scale revision.

THE SIGNAL

Revisions track the Pod template. Replicas are not part of it.

Module 7 in five lines

- **ReplicaSet keeps a count**; a **Deployment manages ReplicaSets** and change.
- **Only template changes** create a new ReplicaSet and revision. Scaling does not.
- Defaults: **maxSurge 25% up, maxUnavailable 25% down**. Readiness paces the rollout.
- **RollingUpdate** for zero downtime; **Recreate** when versions must not overlap.
- **No automatic rollback.** Use rollout status, history and undo.

Deployments suit stateless apps. Next: **choosing a workload** for everything else.

MODULE 08 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Choosing a workload

StatefulSets, DaemonSets, Jobs and CronJobs, and how to pick the right one in seconds.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 8 of 35

admin@sthithapragna.com

Ask one question: are the Pods identical, individual, everywhere, or finite?

admin@sthithapragna.com

The workload decision table

IF THE PODS...	USE	QUAYSIDE EXAMPLE
are interchangeable copies	Deployment	Web front end, booking API
each need a stable name and their own storage	StatefulSet	Bookings database
must run once on every node	DaemonSet	Log agent
run once to completion	Job	A one-off fare recalculation
run to completion on a schedule	CronJob	Nightly manifest export

ON THE EXAM

Most exam scenarios contain one phrase that points straight at a row of this table. Find it.

Deployment versus StatefulSet

DEPLOYMENT

Cattle

- Random names: web-7d9f6c-x2k4p
- Pods are interchangeable
- Shared or no storage
- Started and stopped in any order

STATEFULSET

Named individuals

- Ordinal names: bookings-db-0, -1, -2
- Each Pod keeps its identity
- Its own volume per Pod
- Ordered start-up and shut-down

THE TIEBREAKER

Both run Pods from one identical template. The difference is identity and storage, not the container.

What a StatefulSet guarantees

- **A stable name.** bookings-db-0 is always bookings-db-0, even after being rescheduled.
- **A stable DNS name** through a headless Service: bookings-db-0.bookings-db.quayside.svc.cluster.local.
- **Stable storage.** Each Pod's own claim, such as data-bookings-db-0, is reattached when it is rescheduled.
- **Order.** Created 0, 1, 2 by default; removed in reverse; updated from the highest ordinal down.

> ON THE EXAM
>
> Identity protects against split brain: two Pods can never both believe they are member 0.

Quayside's bookings database as a StatefulSet

Trimmed. Selector and Pod template are as for a Deployment.

```
 1 apiVersion: apps/v1
 2 kind: StatefulSet
 3 metadata: {name: bookings-db}
 4 spec:
 5   serviceName: bookings-db
 6   replicas: 3
 7   # selector and Pod template as usual;
 8   # the container mounts volume "data"
 9   volumeClaimTemplates:
10   - metadata: {name: data}
11     spec:
12       accessModes: [ReadWriteOnce]
13       resources: {requests: {storage: 50Gi}}
```

5 The headless Service that gives each Pod its DNS name.

9-10 A template for claims: one volume per Pod, named data-bookings-db-0 and so on.

Headless Services, and what StatefulSets do not do

- **Headless Service:** clusterIP: None. DNS returns each Pod's own IP instead of one virtual IP.
- **No clustering magic.** Replication and failover are the database's job, or an operator's.
- **Claims are kept by default** when you scale down or delete. The retention policy can change that.
- **For real databases,** an operator built on a StatefulSet is often the better answer.

> ON THE EXAM
>
> A StatefulSet gives identity and storage. Making Postgres replicate is still Postgres's job.

DaemonSets: one Pod on every node

- **One copy per node,** or per selected node using nodeSelector or affinity.
- **New nodes get a Pod automatically;** removed nodes' Pods are cleaned up.
- **Typical uses:** log collectors, node monitoring agents, network plugins, storage drivers.
- kube-proxy itself usually runs as a DaemonSet. Tolerations let it reach control plane nodes too.

> ON THE EXAM
>
> Anything that must exist on every node, including nodes that do not exist yet, is a DaemonSet.

DaemonSet versus a Deployment sized to nodes

DEPLOYMENT, REPLICAS = NODES

Looks equivalent

- Scheduler may stack two on one node
- Leaves some nodes with none
- Ignores nodes added later
- Needs manual resizing

DAEMONSET

Is equivalent

- Exactly one per eligible node
- Every node covered
- Follows nodes as they come and go
- No count to maintain

THE TIEBREAKER

'On every node, including new ones' is always a DaemonSet.

Jobs: work that finishes

- **Runs Pods to completion** and records success. restartPolicy must be OnFailure or Never.
- **completions** is how many successes are needed; **parallelism** is how many run at once.
- **backoffLimit**, 6 by default, is how many failures before the Job gives up.
- **activeDeadlineSeconds** caps total run time; **ttlSecondsAfterFinished** cleans it up afterwards.

> ON THE EXAM
>
> A Deployment restarts a finished program forever. A Job knows when to stop.

Job shapes

SHAPE	SETTINGS	GOOD FOR
One-off	completions 1, parallelism 1	A single task, such as a migration
Fixed count, in parallel	completions 20, parallelism 5	20 pieces of work, 5 at a time
Indexed	completionMode: Indexed	Each Pod gets its own index, and its own slice of work
Work queue	parallelism N, completions unset	Workers pull from a queue until it is empty

ON THE EXAM

Split a big batch into numbered chunks: an Indexed Job with completions and parallelism.

What happens when a CronJob fires

01
Schedule time
The CronJob's schedule matches the clock.

02
CronJob controller
Creates a Job from the jobTemplate.

03
Job controller
Creates a Pod from the Job's template.

04
Scheduler, kubelet
Place the Pod and run it to completion.

05
History
Keeps 3 successful and 1 failed Job by default.

ON THE EXAM

CronJob creates Jobs. Jobs create Pods. The kubelet never reads a CronJob.

Quayside's nightly manifest export

```
apiVersion: batch/v1
kind: CronJob
metadata: {name: manifest-export}
spec:
  schedule: "30 2 * * *"
  timeZone: Europe/London
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      backoffLimit: 3
      template:
        spec:
          restartPolicy: OnFailure
          containers: [{name: export, image: quayside/export:1.8.0}]
```

5-6 02:30 every day, London time rather than the controller's clock.

7 If last night's run is still going, skip tonight's.

13 Retry the container in place on failure.

Reading a cron schedule

Five fields: minute, hour, day of month, month, day of week.

SCHEDULE	RUNS
`"0 * * * *"`	At the start of every hour
`"30 2 * * *"`	At 02:30 every day
`"0 9 * * 1"`	At 09:00 every Monday
`"*/15 * * * *"`	Every 15 minutes
`"@daily"`	A shortcut for midnight every day

ON THE EXAM

Read left to right: minute first, then hour. "30 2" is 02:30, not 2:30 in the afternoon.

Controlling overlapping runs

SETTING	EFFECT
`concurrencyPolicy: Allow`	Default. A new run starts even if the last is still going.
`concurrencyPolicy: Forbid`	Skip the new run while the last is still going.
`concurrencyPolicy: Replace`	Stop the running Job and start the new one.
`startingDeadlineSeconds`	Skip a run that could not start within this many seconds.
`suspend: true`	Pause future runs. Jobs already running carry on.

ON THE EXAM

Allow is the default, and the cause of most 'why is this running twice' incidents.

Eight decisions. In each one, find the phrase that picks the workload.

> Quayside's manifest export must run at 02:30 London time every night, finish, and not run again until the next night.

Which workload should run the export?

A. A Deployment whose container sleeps until 02:30

B. A CronJob with a schedule and a time zone

C. A Job, started by an engineer each night

D. A DaemonSet so the export runs on every node

◆ PAUSE AND DECIDE

Which workload should run the export?

A. A Deployment whose container sleeps until 02:30
Runs forever, and restarts the program after each finish.

B. A CronJob with a schedule and a time zone
Creates a Job on schedule; the Job runs to completion.

C. A Job, started by an engineer each night
Works, but depends on a person every single night.

D. A DaemonSet so the export runs on every node
Would run the export once per node: many copies.

THE SIGNAL

'Finishes, then runs again on a schedule' is a CronJob.

> Quayside's log agent reads container logs from each node's disk and ships them to storage.
>
> It must run on every node, including nodes the autoscaler adds during the evening rush.

Which workload fits the log agent?

A. A Deployment with replicas equal to the node count

B. A StatefulSet so each agent has a stable identity

C. A CronJob that runs the agent every minute

D. A DaemonSet, one Pod on each eligible node

◆ PAUSE AND DECIDE

Which workload fits the log agent?

A A Deployment with replicas equal to the node count

Can stack copies on one node, and ignores new nodes.

B A StatefulSet so each agent has a stable identity

Identity is irrelevant; coverage of nodes is the need.

C A CronJob that runs the agent every minute

Would miss logs between runs, and still not cover every node.

D A DaemonSet, one Pod on each eligible node

One per node, and new nodes get a Pod automatically.

THE SIGNAL

'Every node, including new ones' is a DaemonSet.

> Quayside runs Postgres as three replicas. Each replica keeps its own data on disk, and the replicas find each other by name to replicate.
>
> If a replica is rescheduled, it must come back with the same name and the same data.

Which approach fits the database?

A. A StatefulSet, volumeClaimTemplates, headless Service

B. A Deployment of three replicas sharing one volume claim

C. A DaemonSet pinned to three labelled nodes

D. A Deployment of three replicas using emptyDir volumes

◆ PAUSE AND DECIDE

Which approach fits the database?

A A StatefulSet, volumeClaimTemplates, headless Service

Stable names, stable DNS and a volume per replica.

B A Deployment of three replicas sharing one volume claim

Random names, and three databases writing one volume.

C A DaemonSet pinned to three labelled nodes

Ties the database to nodes; no per-replica identity or claims.

D A Deployment of three replicas using emptyDir volumes

emptyDir is deleted with the Pod: data lost on reschedule.

THE SIGNAL

'Same name and same data after rescheduling' is a StatefulSet.

> After a fare change, Quayside must recalculate every stored fare once. The task takes about twenty minutes.
>
> If it fails, it should be retried. When it succeeds, it must not run again.

Which workload should run the recalculation?

A. A CronJob scheduled for a date far in the future

B. A Deployment of one replica that exits when done

C. A Job with a sensible backoffLimit

D. A bare Pod created with kubectl run

◆ PAUSE AND DECIDE

Which workload should run the recalculation?

A A CronJob scheduled for a date far in the future

A schedule for a one-off task is a workaround, not a design.

B A Deployment of one replica that exits when done

A Deployment restarts it after success, forever.

C A Job with a sensible backoffLimit

Runs once to completion and retries failures.

D A bare Pod created with kubectl run

No retry if its node fails, and no record of success.

THE SIGNAL

'Once, retry on failure, never again after success' is a Job.

> An engineer asks what actually happens inside the cluster when the manifest-export CronJob's scheduled time arrives.

Which description is correct?

A The kubelet watches CronJobs and starts the Pod itself

B The scheduler watches CronJobs and runs them on time

C The CronJob controller creates a Pod and waits for it

D The CronJob controller makes a Job; the Job makes a Pod

◆ PAUSE AND DECIDE

Which description is correct?

A The kubelet watches CronJobs and starts the Pod itself

The kubelet only runs Pods bound to its node.

B The scheduler watches CronJobs and runs them on time

The scheduler places Pods; it knows nothing about schedules.

C The CronJob controller creates a Pod and waits for it

It skips a step: CronJobs never create Pods directly.

D The CronJob controller makes a Job; the Job makes a Pod

CronJob, then Job, then Pod, each by its own controller.

THE SIGNAL

CronJob creates Job; Job creates Pod. Each object has its own controller.

> A reconciliation CronJob runs every 15 minutes. Usually it takes 5 minutes, but at month end it can take 25.
>
> Two runs at once corrupt the same report file. Finishing each run matters more than starting on time.

Which setting prevents the problem?

A concurrencyPolicy: Forbid

B concurrencyPolicy: Replace

C concurrencyPolicy: Allow

D startingDeadlineSeconds: 60

◆ PAUSE AND DECIDE

Which setting prevents the problem?

A concurrencyPolicy: Forbid
Skips a run while the previous one is still going.

B concurrencyPolicy: Replace
Kills the running job, which must be allowed to finish.

C concurrencyPolicy: Allow
The default, which is exactly what causes the overlap.

D startingDeadlineSeconds: 60
Limits late starts; does nothing about overlap.

THE SIGNAL

'Never overlap, and let the current run finish' is Forbid.

> Quayside's ticket validator is a stateless Go service. Every copy is identical and keeps no local data.
>
> It should scale from 4 to 20 copies with demand, as quickly as possible.

Which workload fits the validator?

A. A StatefulSet, so each copy has a stable name

B. A DaemonSet, so copies follow the nodes

C. A Deployment, scaled by demand

D. A Job with parallelism set to 20

◆ PAUSE AND DECIDE

Which workload fits the validator?

A. A StatefulSet, so each copy has a stable name
Adds ordering that slows scaling, for identity it does not need.

B. A DaemonSet, so copies follow the nodes
Ties the copy count to node count, not to demand.

C. A Deployment, scaled by demand
Interchangeable copies that scale freely.

D. A Job with parallelism set to 20
Jobs finish; the validator must keep serving.

THE SIGNAL

'Identical, stateless, scale freely' is a Deployment.

> Quayside must process one million archived tickets, already split into 20 numbered chunks. Each Pod should handle one chunk.
>
> The database can cope with at most 5 workers at a time.

How should this be run?

A. A CronJob that fires every minute, 20 times

B. An Indexed Job: completions 20, parallelism 5

C. A Deployment of 5 replicas that share the chunks

D. 20 bare Pods, created by a shell loop

◆ PAUSE AND DECIDE

How should this be run?

A. A CronJob that fires every minute, 20 times
Scheduling is the wrong tool; runs could overlap or be missed.

B. An Indexed Job: completions 20, parallelism 5
Each Pod gets a chunk index; five run at once.

C. A Deployment of 5 replicas that share the chunks
Deployments never finish, and nothing assigns chunks.

D. 20 bare Pods, created by a shell loop
No retries, no limit of five, no completion record.

THE SIGNAL

'Numbered chunks, a cap on concurrency' is an Indexed Job.

Module 8 in five lines

- **Identical**: Deployment. **Individual**: StatefulSet. **Everywhere**: DaemonSet. **Finite**: Job or CronJob.
- StatefulSets give **stable names, DNS via a headless Service, and a volume per Pod**.
- A DaemonSet **follows the nodes**; a Deployment sized to the node count does not.
- **CronJob creates a Job; the Job creates a Pod.** Default concurrency is Allow.
- Jobs: **completions, parallelism, backoffLimit** (6), Indexed for numbered chunks.

Every one of these finds its Pods by label.
Next: **labels, namespaces, ConfigMaps and Secrets**.

MODULE 09 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Labels, namespaces, ConfigMaps and Secrets

How objects are organised and found, and how configuration gets into Pods.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 9 of 35

admin@sthithapragna.com

Labels **identify and select**. Annotations **describe**. Neither changes what a Pod does.

Labels

- **Key-value pairs** on any object: app=booking-api, tier=backend, env=prod.
- **Not unique.** Many objects share a label; one object has many labels.
- **Selectors find objects by label.** Services, Deployments and kubectl all use them.
- Keys may have a prefix, such as app.kubernetes.io/name. Names and values: up to 63 characters.

> ON THE EXAM
>
> Labels are how loose coupling works: a Service never lists its Pods, it selects them.

Two kinds of selector

KIND	EXAMPLES	WHO SUPPORTS IT
Equality-based	app=web, tier!=cache	Everything, including Services
Set-based	env in (prod,staging), tier notin (cache), !legacy	Deployments, ReplicaSets, Jobs, DaemonSets, kubectl

ON THE EXAM

A Service selector is a plain map: equality only. matchExpressions is where set-based selectors live.

Labels from the command line

```
$ kubectl get pods -l app=booking-api
$ kubectl get pods -l 'env in (prod,staging),tier!=cache'
$ kubectl label pod booking-api-7d9f6c-x2k4p debug=true
$ kubectl get pods --show-labels
```

1. -l filters by an equality selector.
2. Set-based and equality selectors combined; comma means AND.
3. Add a label to a live object.

THE TAKEAWAY

Removing a Pod's app label takes it out of its Service and its ReplicaSet, which then creates a replacement.

Recommended labels

LABEL	QUAYSIDE VALUE
app.kubernetes.io/name	booking-api
app.kubernetes.io/instance	booking-api-prod
app.kubernetes.io/version	2.4.1
app.kubernetes.io/component	api
app.kubernetes.io/part-of	quayside
app.kubernetes.io/managed-by	helm

ON THE EXAM

A shared vocabulary, so dashboards and tools can understand any team's objects.

Labels versus annotations

LABELS

Identify and select

- Used by selectors
- Short values, 63 characters
- Meaningful to Kubernetes
- Example: app=booking-api

ANNOTATIONS

Describe

- Cannot be selected on
- Values can be large
- For people and tools
- Example: git commit, build URL

THE TIEBREAKER

If something selects on it, it is a label. If a person or tool just reads it, it is an annotation.

Namespaces

- **Virtual clusters inside one cluster.** A scope for names: two teams can each have a web Service.
- **A unit for policy:** RBAC permissions, resource quotas and network policies usually apply per namespace.
- **Not a security boundary on their own.** Same nodes, same kernel, open network by default.
- **Deleting a namespace deletes everything in it.**

> ON THE EXAM
>
> Namespaces organise and scope. Isolation needs RBAC, quotas and NetworkPolicy on top.

The four namespaces every cluster starts with

NAMESPACE	HOLDS
`default`	Objects created without a namespace. Fine for learning, poor for production.
`kube-system`	Objects created by Kubernetes itself: CoreDNS, kube-proxy, control plane Pods.
`kube-public`	Readable by everyone, including unauthenticated users. Mostly unused.
`kube-node-lease`	One Lease per node: the heartbeats from module 5.

ON THE EXAM

Four initial namespaces. Exam questions ask for the count and the names.

Namespaced versus cluster-scoped

NAMESPACED

Live inside a namespace

- Pods, Deployments, Services
- ConfigMaps, Secrets, PVCs
- Roles, RoleBindings
- ServiceAccounts, NetworkPolicies

CLUSTER-SCOPED

Belong to the whole cluster

- Nodes, Namespaces
- PersistentVolumes, StorageClasses
- ClusterRoles, ClusterRoleBindings
- CustomResourceDefinitions

REMEMBER

kubectl api-resources --namespaced=false lists every cluster-scoped kind.

Working across namespaces

- **kubectl -n quayside** targets one namespace; **-A** means all namespaces.
- **kubectl config set-context --current --namespace=quayside** changes your default.
- **DNS crosses namespaces:** booking-api.quayside.svc.cluster.local reaches that Service from anywhere.
- A short name like booking-api only resolves within the caller's own namespace.

> ON THE EXAM
>
> Service DNS reads service.namespace.svc.cluster.local: smallest to largest.

Keep configuration **out of the image**, so one image runs in every environment.

A ConfigMap, and two ways to use it

```
 1 apiVersion: v1
 2 kind: ConfigMap
 3 metadata: {name: booking-config}
 4 data:
 5   LOG_LEVEL: info
 6   fares.yaml: |
 7     currency: GBP
 8 ---
 9 # inside the booking API's Pod spec:
10 envFrom: [{configMapRef: {name: booking-config}}]
11 volumes: [{name: cfg, configMap: {name: booking-config}}]
```

4-7 Simple values, or whole files. Up to 1 MiB, not confidential.

10 As environment variables.

11 As files, once the volume is mounted.

Environment variables versus mounted files

ENVIRONMENT VARIABLES

Read once

- Set when the container starts
- Edit the ConfigMap: nothing changes
- Needs a Pod restart to pick up
- Simple for small settings

MOUNTED VOLUME

Kept up to date

- Files refreshed by the kubelet
- Changes appear after a short delay
- App must reread the file
- Not updated if mounted with subPath

THE TIEBREAKER

Edited a ConfigMap and nothing changed? If it is consumed as env vars, restart the Pods.

Immutable ConfigMaps and Secrets

- **immutable: true** locks the data. Edits are rejected.
- **Protects against accidents:** nobody changes production config in place by mistake.
- **Lighter on the cluster:** kubelets stop watching it for changes.
- **To change it,** create a new one with a new name, and point the Pods at it.

> ON THE EXAM
>
> Versioned, immutable config plus a rollout is a safer pattern than editing config in place.

Secrets: what they are, and are not

- **Same shape as a ConfigMap,** meant for passwords, tokens and keys. Up to 1 MiB.
- **Base64 is encoding, not encryption.** Anyone who can read it can decode it.
- **Stored unencrypted in etcd by default,** unless encryption at rest is configured.
- **Anyone who can create a Pod** in a namespace can mount, and so read, its Secrets.
- **Mounted Secrets live in memory** (tmpfs) on the node, never written to its disk.

> ON THE EXAM
>
> Some material claims Secrets are encrypted by default. They are not. Base64 only.

Base64 is not protection

```
1 $ kubectl get secret db-credentials -o jsonpath='{.data.password}'
2 cGE1NXdvcmQtMjAyNg==
3
4 $ echo 'cGE1NXdvcmQtMjAyNg==' | base64 -d
5 pa55word-2026
```

2 What the API returns: encoded, not encrypted.

5 One command later: the password, in plain text.

THE TAKEAWAY

Protect Secrets with RBAC and encryption at rest, not with base64.

How to actually protect Secrets

LAYER	WHAT IT DOES
Encryption at rest	The API server encrypts Secrets before writing to etcd, ideally with a KMS provider.
RBAC, least privilege	Very few people and ServiceAccounts may get, list or watch Secrets.
External secret stores	Vault or a cloud secret manager, synced in by External Secrets or a CSI driver.
Files, not env vars	Environment variables leak into logs, crash dumps and child processes.
Never plain in Git	Use Sealed Secrets or SOPS if encrypted Secrets must live in a repository.

ON THE EXAM

If asked how to secure Secrets beyond the defaults: encryption at rest plus RBAC.

Secret types

TYPE	HOLDS
`Opaque`	The default: any key-value data.
`kubernetes.io/tls`	A TLS certificate and its private key.
`kubernetes.io/dockerconfigjson`	Registry credentials, used as imagePullSecrets.
`kubernetes.io/basic-auth`	A username and password.
`kubernetes.io/service-account-token`	A legacy long-lived token. Prefer short-lived tokens.

ON THE EXAM

Pulling from a private registry needs a dockerconfigjson Secret named in imagePullSecrets.

> For audits, Quayside wants every Deployment to record the Git commit and the full CI build URL that produced it.
>
> No Service, controller or kubectl selector will ever filter on these values.

Where should this information go?

A Annotations on each Deployment

B Labels on each Deployment

C A ConfigMap for each Deployment

D A namespace named after each commit

◆ PAUSE AND DECIDE

Where should this information go?

A Annotations on each Deployment

Descriptive metadata for people and tools; long values allowed.

B Labels on each Deployment

Nothing selects on it, and a URL breaks the 63-character limit.

C A ConfigMap for each Deployment

ConfigMaps feed apps. This describes the object itself.

D A namespace named after each commit

Namespaces organise workloads, not version history.

THE SIGNAL

'Nothing selects on it' plus long values: an annotation.

> The booking API reads LOG_LEVEL from a ConfigMap, using envFrom. To debug an issue, an engineer changes it from info to debug.
>
> An hour later the running Pods are still logging at info. The ConfigMap shows debug.

What explains this, and what is the fix?

A The kubelet caches ConfigMaps for a day; wait

B ConfigMaps are immutable by default; recreate it

C Env vars are read at start; restart the Pods

D The ConfigMap must be moved into kube-system

◆ PAUSE AND DECIDE

What explains this, and what is the fix?

A The kubelet caches ConfigMaps for a day; wait

Mounted files refresh within minutes; env vars never do.

B ConfigMaps are immutable by default; recreate it

Immutable is opt-in, and the edit clearly succeeded.

C Env vars are read at start; restart the Pods

envFrom is read once. kubectl rollout restart picks up the change.

D The ConfigMap must be moved into kube-system

A Pod can only use ConfigMaps in its own namespace.

THE SIGNAL

Config edited but nothing changed, consumed via env: restart the Pods.

> A security auditor opens a copy of Quayside's etcd backup file and finds the database password, readable after a simple base64 decode.
>
> The password was stored as a Kubernetes Secret, exactly as documented.

Why was the password readable?

A. The backup tool decrypted Secrets while copying them

B. The password was accidentally saved in a ConfigMap

C. The Secret used type Opaque instead of an encrypted type

D. Secrets are just base64 unless encryption at rest is on

◆ PAUSE AND DECIDE

Why was the password readable?

A The backup tool decrypted Secrets while copying them

There was nothing to decrypt: they were never encrypted.

B The password was accidentally saved in a ConfigMap

The scenario says it was a Secret, and base64 is the Secret format.

C The Secret used type Opaque instead of an encrypted type

No Secret type encrypts. Type only describes the data's shape.

D Secrets are just base64 unless encryption at rest is on

The default. Enable encryption at rest, ideally with KMS.

THE SIGNAL

Secret readable in etcd or a backup: encryption at rest was never enabled.

> The payment worker runs in the payments namespace. It must call the booking-api Service, which lives in the quayside namespace.
>
> Both use the default cluster domain, cluster.local.

Which address should the payment worker use?

A booking-api

B booking-api.quayside.svc.cluster.local

C quayside.booking-api.svc.cluster.local

D booking-api.payments.svc.cluster.local

◆ PAUSE AND DECIDE

Which address should the payment worker use?

A booking-api

A short name resolves in the caller's own namespace: payments.

B booking-api.quayside.svc.cluster.local

Service, then namespace, then svc and the cluster domain.

C quayside.booking-api.svc.cluster.local

Namespace and Service are the wrong way round.

D booking-api.payments.svc.cluster.local

Looks in the caller's namespace, where no such Service exists.

THE SIGNAL

Crossing namespaces: service.namespace.svc.cluster.local.

Module 9 in five lines

- **Labels select, annotations describe.** Services use equality selectors only.
- Four initial namespaces: **default, kube-system, kube-public, kube-node-lease**.
- Cross-namespace DNS: **service.namespace.svc.cluster.local**.
- ConfigMap as **env vars: read once**; as **files: refreshed**.
- Secrets are **base64, not encrypted**. Add encryption at rest and RBAC.

You have met a dozen kubectl commands.
Next: **kubectl fluency**, all in one place.

MODULE 10 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

kubectl fluency

The commands, flags and habits that exam questions assume you know.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 10 of 35

admin@sthithapragna.com

Every kubectl command is a verb, a resource type, a name, and some flags.

The grammar of a command

```
1 $ kubectl get deployment booking-api -n quayside -o yaml
2
3 #         verb           name                    output
4 #             type                    namespace
5
6 $ kubectl describe pod web-7d9f6c-x2k4p
7 $ kubectl delete svc web -n quayside
8 $ kubectl get pods,svc -A
```

1 Verb, type, name, then flags. The order of flags does not matter.

8 Several types at once, across all namespaces.

ON THE EXAM

Types accept singular, plural or a short name: pod, pods and po all work.

Short names worth knowing

SHORT	KIND	SHORT	KIND
po	pods	deploy	deployments
svc	services	rs	replicasets
ns	namespaces	sts	statefulsets
cm	configmaps	ds	daemonsets
pvc	persistentvolumeclaims	pv	persistentvolumes
sa	serviceaccounts	no	nodes

ON THE EXAM

kubectl api-resources lists every short name. Exam commands often use them.

kubeconfig: how kubectl knows where to go

- **A file,** by default ~/.kube/config, or any path in the KUBECONFIG variable.
- **clusters:** each API server's URL and CA certificate.
- **users:** credentials: a client certificate, a token, or an exec plugin.
- **contexts:** a cluster plus a user plus a default namespace. current-context picks one.

> ON THE EXAM
>
> A context is not a cluster. It is a named pairing of cluster, user and namespace.

Switching contexts

```
$ kubectl config get-contexts
CURRENT   NAME             CLUSTER        AUTHINFO     NAMESPACE
*         quayside-stage   stage-cluster  ci-deployer  quayside
          quayside-prod    prod-cluster   ci-deployer  quayside

$ kubectl config use-context quayside-prod
$ kubectl config set-context --current --namespace=payments
$ kubectl get pods --context quayside-stage
```

3 The asterisk marks the current context.

6 Switch every later command to production.

7 Change the current context's default namespace.

8 Target a context for one command only.

create versus apply

KUBECTL CREATE -F

Imperative

- Makes a new object
- Fails if it already exists
- You choose the operation
- Good for one-off objects

KUBECTL APPLY -F

Declarative

- Creates it, or updates it to match
- Safe to run again and again
- Works on files and whole folders
- The basis of GitOps

THE TIEBREAKER

apply is idempotent: run it twice and nothing changes the second time. create is not.

Generating manifests instead of writing them

```
1 $ kubectl create deployment web --image=quayside/web:5.2.0 \
2     --replicas=3 --dry-run=client -o yaml > web.yaml
3
4 $ kubectl expose deployment web --port=80 --target-port=8080
5
6 $ kubectl run debug --image=busybox:1.37 --restart=Never -it --rm -- sh
```

1-2 --dry-run=client sends nothing; -o yaml prints the manifest to a file.

4 expose creates a Service for existing Pods.

6 run creates a single Pod. Here: a throwaway shell, deleted on exit.

kubectl run: just a Pod

- **kubectl run creates one bare Pod.** Not a Deployment, not a Job.
- **Old material says otherwise:** years ago, run created Deployments. It no longer does.
- **A bare Pod is not healed.** Fine for a quick test, wrong for anything that must stay up.
- **For a Deployment,** use kubectl create deployment, or apply a manifest.

> ON THE EXAM
>
> If an option claims kubectl run creates a Deployment or a ReplicaSet, it is out of date.

Getting output in the shape you need

FLAG	GIVES YOU
`-o wide`	Extra columns, including the node and Pod IP
`-o yaml, -o json`	The full object, spec and status
`-o jsonpath='{.status.podIP}'`	One field, for scripts
`-o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName`	Your own table
`-w, --watch`	Keep streaming changes
`--sort-by=.metadata.creationTimestamp`	Rows in a chosen order

ON THE EXAM

'Which node is each Pod on?' is get pods -o wide.

get versus describe

KUBECTL GET

Lists and data

- One line per object
- Machine-friendly -o formats
- Good for scripts and quick checks
- Shows STATUS, not reasons

KUBECTL DESCRIBE

Detail and events

- Everything about one object
- Human-readable
- Recent Events at the bottom
- Shows why, not just what

THE TIEBREAKER

'Why is it Pending?' is describe. The Events section holds the answer.

Changing live objects

COMMAND	DOES
`kubectl edit deploy/web`	Opens the live object in an editor; saving applies it
`kubectl patch deploy/web -p '...'`	Changes just the fields you give
`kubectl set image | env | resources`	Targeted changes to a Pod template
`kubectl scale deploy/web --replicas=5`	Changes the replica count
`kubectl label | annotate`	Adds or changes metadata
`kubectl diff -f web.yaml`	Previews what apply would change, without changing it

> ON THE EXAM
>
> diff before apply shows exactly what will change. It is the safest habit in this table.

Three small changes

```
$ kubectl patch deployment web -p '{"spec":{"replicas":5}}'
$ kubectl set env deployment/booking-api LOG_LEVEL=debug
$ kubectl diff -f web.yaml
-  replicas: 5
+  replicas: 3
```

1 A merge patch: only replicas changes.

2 Changes the template, so it triggers a rollout.

3-5 The file says 3; the cluster says 5. apply would scale back down.

Imperative edits and the source of truth

- **edit, patch, scale and set** change the cluster, not your files.
- **The next apply from Git** puts the file's values back.
- **A GitOps tool** will revert them even sooner, as drift.
- **Emergencies are fine,** then copy the change into Git straight away.

> ON THE EXAM
>
> If the file and the cluster disagree, the next apply makes the cluster match the file.

Deleting, safely

COMMAND	RESULT
`kubectl delete -f web.yaml`	Deletes whatever the file defines
`kubectl delete pod web-7d9f6c-x2k4p`	Its controller creates a replacement
`kubectl delete deploy web`	The Deployment, its ReplicaSets and Pods
`kubectl delete ns staging`	Everything in the namespace
`--grace-period=0 --force`	Skips graceful shutdown. Last resort only

ON THE EXAM

Deleting a managed Pod is a restart, not a removal. To remove it, delete its owner.

Inspection commands, covered later

COMMAND	FOR	MODULE
`kubectl logs`	A container's output	25
`kubectl exec -it pod -- sh`	A shell inside a container	25
`kubectl port-forward`	Reach a Pod from your laptop	25
`kubectl debug`	An ephemeral debug container	25
`kubectl top pods`	CPU and memory use, via metrics-server	26
`kubectl auth can-i`	Check a permission	17

ON THE EXAM

kubectl top needs metrics-server. Without it, top reports that metrics are unavailable.

Versions, plugins and completion

- **Version skew:** kubectl is supported within one minor version of the API server.
- A 1.37 kubectl works with 1.36, 1.37 and 1.38 control planes.
- **Plugins:** any executable named kubectl-something becomes kubectl something. Krew installs them.
- **Completion:** kubectl completion bash or zsh enables tab completion.

> ON THE EXAM
>
> Old kubectl against a much newer cluster is unsupported. Keep it within one minor version.

> An engineer needs a starting manifest for a new Deployment called web, using image quayside/web:5.2.0 with 3 replicas.
>
> Nothing may be created in the cluster yet; the file will be reviewed in Git first.

Which command produces the manifest?

A kubectl run web --image=quayside/web:5.2.0

B kubectl get deployment web -o yaml > web.yaml

C kubectl explain deployment --recursive > web.yaml

D kubectl create deploy web … --dry-run=client -o yaml

◆ PAUSE AND DECIDE

Which command produces the manifest?

A kubectl run web --image=quayside/web:5.2.0

Creates a real bare Pod, not a Deployment manifest.

B kubectl get deployment web -o yaml > web.yaml

The Deployment does not exist yet, so there is nothing to get.

C kubectl explain deployment --recursive > web.yaml

Saves field documentation, not a manifest.

D kubectl create deploy web ... --dry-run=client -o yaml

Builds the object locally and prints it. Nothing is sent.

THE SIGNAL

'Generate YAML, create nothing' is --dry-run=client -o yaml.

> During an incident, the on-call engineer needs to see which node each booking API Pod is running on, and each Pod's IP address.
>
> They want one quick command, with one line per Pod.

Which command gives exactly that?

A kubectl describe deployment booking-api

B kubectl get pods -l app=booking-api -o wide

C kubectl top pods -l app=booking-api

D kubectl get deployment booking-api -o yaml

◆ PAUSE AND DECIDE

Which command gives exactly that?

A kubectl describe deployment booking-api
Describes the Deployment, not where each Pod landed.

B kubectl get pods -l app=booking-api -o wide
One line per Pod, with NODE and IP columns.

C kubectl top pods -l app=booking-api
Shows CPU and memory use, not nodes or IPs.

D kubectl get deployment booking-api -o yaml
The Deployment's spec and status. Pods are not listed.

THE SIGNAL

Node and IP per Pod: get pods -o wide.

> An engineer deleted the web Deployment in production, believing they were connected to staging. Both contexts use the same namespace name.
>
> The team wants a habit that would most directly have prevented this.

Which habit would most directly have prevented it?

A Always add --force when deleting, to finish quickly

B Use kubectl run for changes instead of delete

C Check the current context, or pass --context explicitly

D Keep production and staging in one shared context

◆ PAUSE AND DECIDE

Which habit would most directly have prevented it?

A Always add --force when deleting, to finish quickly

Makes a mistake faster and harsher, not less likely.

B Use kubectl run for changes instead of delete

run creates Pods; it has nothing to do with the target cluster.

C Check the current context, or pass --context explicitly

The command went to the wrong cluster. Context decides that.

D Keep production and staging in one shared context

A context points at one cluster; this only adds confusion.

THE SIGNAL

Wrong cluster means wrong context. Check it before anything destructive.

> The web Deployment is managed from Git, with a pipeline running kubectl apply on every merge. The file sets replicas: 3.
>
> During an incident, on-call ran kubectl scale to 10. The next morning, after an unrelated merge, there are 3 replicas again.

Why did the count return to 3?

A The pipeline's apply made the cluster match the file again

B kubectl scale changes expire after about twelve hours

C The ReplicaSet restored its original replica count

D etcd rolled back the change during the overnight backup

◆ PAUSE AND DECIDE

Why did the count return to 3?

A The pipeline's apply made the cluster match the file again

The file still says 3, and apply makes the cluster match it.

B kubectl scale changes expire after about twelve hours

No kubectl change expires. It lasts until something changes it.

C The ReplicaSet restored its original replica count

A ReplicaSet follows its Deployment; it keeps no memory of old counts.

D etcd rolled back the change during the overnight backup

Backups copy state; they never roll it back.

THE SIGNAL

Imperative change, then an apply from Git: the file wins. Put emergency changes into Git.

Module 10 in five lines

- **Verb, type, name, flags.** Short names like po, svc, deploy, ns, cm.
- A **context** is cluster + user + namespace, stored in **kubeconfig**.
- **apply is idempotent;** create fails if it exists. **run** makes a bare Pod.
- **--dry-run=client -o yaml** generates manifests; **-o wide** shows node and IP.
- **describe for why**; imperative edits are undone by the next **apply**.

You can drive a cluster. Next: **administering one**, from kubeadm to upgrades and draining nodes.

MODULE 11 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Administering a cluster

Building clusters, upgrading them, maintaining nodes, and sharing capacity fairly.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 11 of 35

admin@sthithapragna.com

Good administration is **changing things without anyone noticing**: upgrades, maintenance, and fair shares.

Ways to get a cluster

OPTION	WHO RUNS THE CONTROL PLANE	TYPICAL USE
Managed: EKS, GKE, AKS	The cloud provider	Most production clusters
kubeadm	You	Self-managed clusters, learning the internals
k3s, k0s, MicroK8s	You, in a lightweight package	Edge, IoT, small machines
kind, minikube	Your laptop	Local development and CI tests
OpenShift, Rancher	A platform on top of Kubernetes	Enterprise platforms

ON THE EXAM

The CNCF's Certified Kubernetes programme checks that a distribution passes the conformance tests.

kubeadm: building a cluster yourself

- **kubeadm init** sets up a control plane: certificates, kubeconfig files, static Pod manifests.
- **kubeadm join** adds nodes, using a bootstrap token printed by init.
- **kubeadm installs no network plugin.** Until you add a CNI plugin, nodes stay NotReady.
- **kubeadm upgrade plan / apply** upgrades the control plane, one minor version at a time.
- Its certificates last one year; **kubeadm certs** checks and renews them, and upgrades renew them too.

> ON THE EXAM
>
> Fresh kubeadm cluster, nodes NotReady, CoreDNS Pending: the CNI plugin is missing.

A kubeadm cluster in three commands

```
1 # on the first control plane node
2 $ kubeadm init --pod-network-cidr=10.244.0.0/16
3
4 $ kubectl apply -f <your CNI plugin's manifest>
5
6 # on each worker node
7 $ kubeadm join 10.0.0.10:6443 --token abcdef.0123456789abcdef \
8     --discovery-token-ca-cert-hash sha256:<hash from init>
```

2 Builds the control plane and prints the join command.

4 The step people forget: install a network plugin.

7-8 Token proves the node may join; the hash proves the cluster is genuine.

How Kubernetes releases

- **About three minor releases a year,** one every four months or so.
- **The three most recent minor versions are maintained:** today 1.37, 1.36 and 1.35.
- **Patch releases** such as 1.37.2 carry fixes, including security fixes.
- **Fall behind and you lose patches.** Plan an upgrade at least once a year.

> ON THE EXAM
>
> Versions read major.minor.patch. Upgrades are planned by minor version.

Version skew: what may run together

COMPONENT	ALLOWED VERSIONS, RELATIVE TO THE API SERVER
`kube-apiserver (HA)`	All instances within one minor version of each other
`controller-manager, scheduler`	Not newer; at most one minor older
`kubelet`	Not newer; up to three minors older
`kube-proxy`	Not newer; up to three minors older
`kubectl`	One minor older or newer

ON THE EXAM

With the API server at 1.37, kubelets may run 1.34 to 1.37. Never newer.

The upgrade order

01

Back up etcd

A snapshot, stored off the cluster.

02

API servers

First, one minor version at a time.

03

Other control plane

Controller manager and scheduler.

04

Nodes, one by one

Drain, upgrade kubelet, uncordon.

05

Add-ons

CNI, CoreDNS, and so on, per their docs.

ON THE EXAM

Control plane before nodes, never skip a minor version, and back up etcd first.

cordon versus drain

KUBECTL CORDON

Close to new bookings

- Node marked SchedulingDisabled
- No new Pods are placed there
- Existing Pods keep running
- Reverse with kubectl uncordon

KUBECTL DRAIN

Move the diners out

- Cordons the node first
- Then evicts its Pods gracefully
- Respects PodDisruptionBudgets
- Node is then safe to work on

THE TIEBREAKER

Cordon stops new arrivals. Drain also moves the current residents. Both are undone with uncordon.

Maintaining a node

```
$ kubectl drain node-3 --ignore-daemonsets --delete-emptydir-data
node/node-3 cordoned
evicting pod quayside/web-7d9f6c-x2k4p
evicting pod quayside/booking-api-5b8c4d-q8r2m
node/node-3 drained

# patch, reboot, upgrade the kubelet...

$ kubectl uncordon node-3
node/node-3 uncordoned
```

1. DaemonSet Pods cannot move, so drain must be told to ignore them.
2. Drain cordons first, automatically.
9. Nothing moves back on its own. New Pods may now land here.

PodDisruptionBudgets

- **A promise about availability:** keep at least N Pods, or lose at most N, for a label selector.
- **Drain uses the Eviction API,** which refuses an eviction that would break the budget.
- **Only voluntary disruptions** are covered: drains, upgrades, autoscaler scale-down.
- **A budget that allows zero disruptions** blocks every drain until someone intervenes.

> ON THE EXAM
>
> minAvailable equal to the replica count means no Pod may ever be evicted. Leave headroom.

A disruption budget for the booking API

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: booking-api-pdb}
spec:
  minAvailable: 2
  selector: {matchLabels: {app: booking-api}}
```

5 Or maxUnavailable. A number or a percentage.

6 Which Pods the budget protects.

THE TAKEAWAY

With 3 replicas this allows one eviction at a time. With 2 replicas, it allows none.

Voluntary versus involuntary disruptions

VOLUNTARY

Someone chose it

- kubectl drain
- Cluster or node upgrades
- Autoscaler removing a node
- PDBs apply

INVOLUNTARY

It just happened

- Hardware or kernel failure
- A cloud VM disappearing
- Node running out of resources
- PDBs cannot prevent it

THE TIEBREAKER

Budgets manage planned change. Protection from failure comes from replicas spread across nodes and zones.

Teams share one cluster. Quotas and limit ranges keep one team from starving the rest.

ResourceQuota versus LimitRange

RESOURCEQUOTA

Caps the namespace

- Totals for the whole namespace
- CPU, memory, storage
- Counts: Pods, Services, PVCs
- Over the cap: creation rejected

LIMITRANGE

Shapes each container

- Rules per container or Pod
- Default requests and limits
- Minimum and maximum sizes
- Fills gaps in Pod specs

THE TIEBREAKER

Quota caps the namespace. LimitRange shapes each container. They are often used together.

Quota and defaults for the payments team

```yaml
apiVersion: v1
kind: ResourceQuota
metadata: {name: team-quota, namespace: payments}
spec:
  hard: {requests.cpu: "8", requests.memory: 16Gi, pods: "40"}
---
apiVersion: v1
kind: LimitRange
metadata: {name: defaults, namespace: payments}
spec:
  limits:
  - type: Container
    defaultRequest: {cpu: 250m, memory: 256Mi}
    default: {cpu: 500m, memory: 512Mi}
```

5 The whole namespace: 8 CPUs of requests, 16 GiB, 40 Pods.

13-14 Requests and limits filled in for containers that set none.

The quota trap

- **With a CPU or memory quota,** every new Pod must declare requests or limits for it.
- **Pods without them are rejected,** and the Deployment shows no Pods at all.
- **A LimitRange with defaults** fixes this by filling the values in automatically.
- **Where to look:** the ReplicaSet's events show FailedCreate, with the quota reason.

> ON THE EXAM
>
> Deployment with zero Pods in a quota'd namespace: describe the ReplicaSet and read its events.

Keeping clusters healthy over time

- **Back up etcd** on a schedule, and practise restoring it.
- **Renew certificates** before they expire, or let regular upgrades do it.
- **Patch nodes by rotation:** drain, patch or replace, uncordon, one at a time.
- **Cluster API** manages whole clusters declaratively, as Kubernetes objects.

> ON THE EXAM
>
> Treat clusters like workloads: declared, versioned and replaceable.

> Quayside's control plane runs Kubernetes 1.37 on every API server. The team is reviewing the kubelet versions across its nodes.
>
> One of the node pools below is running an unsupported combination.

Which kubelet version is not supported with this control plane?

A kubelet 1.34

B kubelet 1.38

C kubelet 1.36

D kubelet 1.35

◆ PAUSE AND DECIDE

Which kubelet version is not supported with this control plane?

A kubelet 1.34

Three minors older: the oldest allowed.

B kubelet 1.38

Newer than the API server, which is never allowed.

C kubelet 1.36

One minor older: allowed.

D kubelet 1.35

Two minors older: allowed.

THE SIGNAL

Kubelets may lag by three minors, but may never lead.

> An engineer must patch the kernel on node-3 and reboot it. No new Pods should be placed there, and the Pods already on it must move elsewhere first.
>
> The booking API's disruption budget must be respected.

Which command does this?

A kubectl drain node-3 --ignore-daemonsets

B kubectl cordon node-3

C kubectl delete node node-3

D kubectl taint nodes node-3 maint=true:NoSchedule

◆ PAUSE AND DECIDE

Which command does this?

A kubectl drain node-3 --ignore-daemonsets

Cordons, then evicts gracefully through the budget-aware Eviction API.

B kubectl cordon node-3

Stops new Pods, but the existing ones stay put.

C kubectl delete node node-3

Removes the node object; Pods are not evicted through their budget.

D kubectl taint nodes node-3 maint=true:NoSchedule

Like cordon: no new Pods, but existing ones stay.

THE SIGNAL

'No new Pods and move the existing ones' is drain. Cordon does only half.

> A drain of node-5 hangs, repeating: cannot evict pod as it would violate the pod's disruption budget.
>
> The booking API runs 2 replicas, and its PodDisruptionBudget sets minAvailable: 2.

What is the cause, and the best fix?

A Drain is stuck; rerun it with --force

B DaemonSet Pods block it; add --ignore-daemonsets

C The node is cordoned twice; uncordon it once

D The budget allows no evictions; add a replica

◆ PAUSE AND DECIDE

What is the cause, and the best fix?

A. Drain is stuck; rerun it with --force
Force affects bare Pods; the budget still blocks eviction.

B. DaemonSet Pods block it; add --ignore-daemonsets
A real flag, but for a different error message.

C. The node is cordoned twice; uncordon it once
Cordoning is not counted. Uncordoning undoes the drain.

D. The budget allows no evictions; add a replica
With 3 replicas and minAvailable 2, one Pod can be evicted.

THE SIGNAL

minAvailable equal to replicas means zero allowed disruptions. Add headroom or relax the budget.

> The payments namespace has a ResourceQuota on requests.cpu and requests.memory. It has no LimitRange.
>
> A new Deployment is created, but no Pods ever appear. Its ReplicaSet shows FailedCreate: must specify requests.cpu.

What is the best fix?

A Raise the namespace's CPU quota

B Deploy it into the default namespace

C Add CPU and memory requests to the containers

D Delete the ResourceQuota from the namespace

◆ PAUSE AND DECIDE

What is the best fix?

A Raise the namespace's CPU quota

The quota is not full; the Pods simply lack requests.

B Deploy it into the default namespace

Dodges the team's controls rather than meeting them.

C Add CPU and memory requests to the containers

Meets the quota's requirement. A LimitRange with defaults also works.

D Delete the ResourceQuota from the namespace

Removes the protection that exists for good reason.

THE SIGNAL

'Must specify requests' under a quota: set requests, or add LimitRange defaults.

Module 11 in five lines

- kubeadm **init and join**; you must add a **CNI plugin**. Certificates last a year.
- Three minors maintained. **Kubelets never newer**, up to three minors older.
- Upgrade **control plane first**, one minor at a time, after an etcd backup.
- **Cordon** stops new Pods; **drain** also evicts, respecting PDBs.
- **ResourceQuota caps the namespace; LimitRange shapes each container.**

Quotas decide how much a team may use. Next: **scheduling**, which decides where each Pod runs.

MODULE 12 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Scheduling

How Kubernetes picks a node, and every tool you have to steer it.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 12 of 35

admin@sthithapragna.com

The scheduler answers one question per Pod: **which node?** It decides from **requests and rules**, not actual usage.

How the scheduler places a Pod

01
Watch
A new Pod appears with no node.

02
Filter
Drop nodes that cannot run it: resources, taints, selectors, affinity.

03
Score
Rank the rest: preferences, spreading, balance.

04
Bind
Write the winning node's name into the Pod.

05
Run
That node's kubelet starts it.

ON THE EXAM

No node survives the filter: the Pod stays Pending, with a FailedScheduling event giving the reasons.

Requests versus limits

REQUESTS

Decide placement

- Read by the scheduler
- A reservation on the node
- Sum of requests must fit allocatable
- Also the Pod's guaranteed share

LIMITS

Decide punishment

- Enforced by the kernel via cgroups
- A ceiling while running
- Over CPU limit: throttled
- Over memory limit: OOMKilled

THE TIEBREAKER

Requests decide placement. Limits decide punishment. Set only limits, and requests are copied from them.

Declaring resources

```
containers:
- name: api
  image: quayside/booking-api:2.4.1
  resources:
    requests: {cpu: 250m, memory: 256Mi}
    limits:   {cpu: "1", memory: 512Mi}
```

5 250m is a quarter of a CPU. Mi is mebibytes, 1024-based.

6 "1" is one whole CPU core or vCPU.

THE UNITS

CPU in cores or millicores; memory in Mi and Gi. 1000m equals one CPU.

The scheduler reads requests, not usage

- **A quiet node can be full.** If its capacity is already requested, nothing more fits.
- **A busy node can look empty.** Pods with no requests reserve nothing.
- **The message to recognise:** 0/6 nodes are available: 6 Insufficient memory.
- **Right-size requests** from real usage, so reservations match reality.

> ON THE EXAM
>
> 'Insufficient cpu' while dashboards show idle nodes: requests are reserved, not used.

Quality of service classes

CLASS	WHEN	UNDER NODE MEMORY PRESSURE
Guaranteed	Every container sets CPU and memory, with requests equal to limits	Least likely to be evicted
Burstable	Some requests or limits set, but not Guaranteed	In between
BestEffort	No requests or limits at all	Most likely to be evicted

ON THE EXAM

Kubernetes assigns the class automatically from your resource settings. You never set it directly.

Node-pressure eviction, precisely

- **The kubelet evicts Pods** when a node runs short of memory, disk or process IDs.
- **It ranks by:** usage above requests first, then Pod priority, then how far above.
- **QoS predicts the result:** BestEffort always exceeds its zero requests; Guaranteed rarely does.
- Different from a drain, which uses the Eviction API, and from an OOM kill of one container.

> ON THE EXAM
>
> Exam shorthand: BestEffort first, Burstable next, Guaranteed last.

Selectors and affinity attract. Taints repel. Spread constraints balance.

admin@sthithapragna.com

The placement toolkit

TOOL	SET ON	EFFECT
nodeSelector	Pod	Only nodes with exactly these labels
Node affinity	Pod	Required or preferred node label rules, with operators
Pod affinity / anti-affinity	Pod	Near, or away from, other Pods, per topology
Taints	Node	Repel Pods that do not tolerate them
Tolerations	Pod	Permission to land on a tainted node
Topology spread	Pod	Even out Pods across zones or nodes

ON THE EXAM

Everything is set on the Pod except taints, which are set on the node.

Node affinity: required and preferred

```
1  affinity:
2    nodeAffinity:
3      requiredDuringSchedulingIgnoredDuringExecution:
4        nodeSelectorTerms:
5        - matchExpressions:
6          - {key: topology.kubernetes.io/zone, operator: In, values: [zone-a, zone-b]}
7      preferredDuringSchedulingIgnoredDuringExecution:
8      - weight: 50
9        preference:
10         matchExpressions:
11         - {key: node.quayside.io/disk, operator: In, values: [nvme]}
```

3 A hard rule. No matching node means Pending.

7-8 A soft rule: adds to a node's score, weighted 1 to 100.

IgnoredDuringExecution: if node labels change later, running Pods are not moved.

Required versus preferred

REQUIRED...

A filter

- Nodes that fail are removed
- No match: Pod stays Pending
- Use for true constraints
- Licensing, compliance, hardware

PREFERRED...

A score

- Nodes that match score higher
- No match: placed elsewhere
- Use for nice-to-haves
- Faster disks, same zone as cache

THE TIEBREAKER

If the rule must never be broken, required. If breaking it is merely worse, preferred.

Pod affinity and anti-affinity

- **Anti-affinity** keeps replicas apart: never two booking API Pods on one node.
- **Affinity** keeps Pods together: put the cache in the same zone as the API.
- **topologyKey** sets the unit: kubernetes.io/hostname for nodes, topology.kubernetes.io/zone for zones.
- **Required anti-affinity with more replicas than nodes** leaves the extras Pending.

> ON THE EXAM
>
> Affinity rules about Pods need a topologyKey: near or apart, measured by node or by zone.

Taint effects

EFFECT	NEW PODS WITHOUT A TOLERATION	PODS ALREADY RUNNING
`NoSchedule`	Not scheduled here	Stay
`PreferNoSchedule`	Avoided if possible	Stay
`NoExecute`	Not scheduled here	Evicted, after any tolerationSeconds

ON THE EXAM

Control plane nodes carry a NoSchedule taint, which is why ordinary Pods never land there.

Tainting a node, and tolerating it

```
$ kubectl taint nodes gpu-1 dedicated=ml:NoSchedule

# in the ML Pod's spec:
tolerations:
- {key: dedicated, operator: Equal, value: ml, effect: NoSchedule}

$ kubectl taint nodes gpu-1 dedicated=ml:NoSchedule-
```

1. key=value:effect, on the node.
5. The matching toleration, on the Pod.
7. A trailing minus removes the taint.

Taints versus node affinity

TAINT + TOLERATION

Keeps others out

- Set on the node
- Repels Pods without a toleration
- A toleration permits, never attracts
- Tolerating Pods may still go elsewhere

NODE AFFINITY

Pulls yours in

- Set on the Pod
- Attracts it to matching nodes
- Does not stop other Pods arriving
- Other Pods may fill the node

THE TIEBREAKER

Dedicated nodes need both: a taint to keep others out, affinity to bring yours in.

Topology spread constraints

- **Even out replicas** across zones or nodes, rather than just keeping them apart.
- **maxSkew:** the largest allowed difference in Pod count between any two zones.
- **whenUnsatisfiable:** DoNotSchedule is a hard rule; ScheduleAnyway is a preference.
- **Quayside:** booking API, maxSkew 1 across zones, so losing a zone loses a third.

> ON THE EXAM
>
> Anti-affinity says never together. Spread says evenly. Spread scales past one Pod per node.

Priority and preemption

- **A PriorityClass** gives Pods a number. Higher numbers are scheduled first.
- **Preemption:** if a high-priority Pod cannot fit, lower-priority Pods may be evicted to make room.
- **Built in:** system-cluster-critical and system-node-critical, for core components.
- Priority also feeds node-pressure eviction ranking, after usage above requests.

> ON THE EXAM
>
> 'Critical Pods must always find room, even if others make way' is a PriorityClass.

Special cases

- **nodeName** skips the scheduler entirely and pins a Pod to one node. Rarely wise.
- **schedulerName** hands a Pod to a custom or second scheduler.
- **GPUs and devices:** Dynamic Resource Allocation, stable since 1.34, requests devices through ResourceClaims.
- The older approach, extended resources such as nvidia.com/gpu, still works.

> ON THE EXAM
>
> With nodeName set, filters, taints and resource checks by the scheduler are all bypassed.

Currency check: in-place Pod resize

- **Stable since Kubernetes 1.35:** change a running Pod's CPU and memory without recreating it.
- **Through the resize subresource,** for example kubectl patch pod … --subresource resize.
- **resizePolicy per resource:** NotRequired, the default, applies it live; RestartContainer restarts.
- **Old answer, now wrong:** 'resources are immutable, you must delete and recreate the Pod'.

> ON THE EXAM
>
> If the node lacks room for the new requests, the resize waits rather than moving the Pod.

Eight placement decisions. In each, ask: **attract, repel, balance**, or is it about **requests**?

> New booking API Pods stay Pending with the event: 0/6 nodes are available: 6 Insufficient memory.
>
> Monitoring shows every node using only about 40% of its memory.

What is the most likely explanation?

A The monitoring system is under-reporting memory use

B The kubelet on every node has stopped reporting

C Existing Pods' memory requests already reserve the nodes

D The new Pods' memory limits are set too low

◆ PAUSE AND DECIDE

What is the most likely explanation?

A The monitoring system is under-reporting memory use

Nothing suggests monitoring is wrong; the gap is expected.

B The kubelet on every node has stopped reporting

Then nodes would be NotReady, not 'Insufficient memory'.

C Existing Pods' memory requests already reserve the nodes

The scheduler counts requests, not usage. Reservations fill nodes.

D The new Pods' memory limits are set too low

Low limits cause OOM kills later, not Pending now.

THE SIGNAL

'Insufficient' on idle nodes: requests are reserved, not used.

> When a node runs short of memory, Quayside wants the payment worker to be among the last Pods evicted.
>
> It must be done through the payment worker's own resource settings.

Which settings achieve this?

A Requests equal to limits for CPU and memory, everywhere

B No requests or limits at all, so it reserves no capacity

C Low requests with much higher limits, to allow bursting

D Deploy it into the kube-system namespace instead

◆ PAUSE AND DECIDE

Which settings achieve this?

A Requests equal to limits for CPU and memory, everywhere

Guaranteed QoS: usage rarely exceeds requests, so it is ranked last.

B No requests or limits at all, so it reserves no capacity

BestEffort: the most likely to be evicted.

C Low requests with much higher limits, to allow bursting

Burstable: usage above requests puts it early in the queue.

D Deploy it into the kube-system namespace instead

Namespaces play no part in eviction ranking.

THE SIGNAL

'Last to be evicted', from resource settings alone: Guaranteed.

> The booking API runs 3 replicas on a 5-node cluster. A node failure must never take out more than one replica.
>
> So no two booking API Pods may ever share a node.

Which rule enforces this?

A. A nodeSelector that lists three chosen nodes

B. A NoSchedule taint on every node in the cluster

C. Setting replicas equal to the number of nodes

D. Required pod anti-affinity with topologyKey hostname

◆ PAUSE AND DECIDE

Which rule enforces this?

A. A nodeSelector that lists three chosen nodes
Pins to three nodes; still allows two Pods on one of them.

B. A NoSchedule taint on every node in the cluster
Repels everything, including the booking API itself.

C. Setting replicas equal to the number of nodes
Changes the count; says nothing about placement.

D. Required pod anti-affinity with topologyKey hostname
A hard rule: no two matching Pods per node.

THE SIGNAL

'Never two on one node' is required anti-affinity by hostname.

> Quayside adds two GPU nodes. Only machine-learning Pods may run on them, and every machine-learning Pod must run on them.
>
> Ordinary Pods must never land on the GPU nodes.

What configuration achieves both requirements?

A. A toleration on ML Pods, with no taint on the GPU nodes

B. Taint the GPU nodes; give ML Pods toleration and affinity

C. A nodeSelector on the ML Pods only, and no taint anywhere

D. A high PriorityClass for every machine-learning Pod

◆ PAUSE AND DECIDE

What configuration achieves both requirements?

A. A toleration on ML Pods, with no taint on the GPU nodes
With no taint, a toleration does nothing at all.

B. Taint the GPU nodes; give ML Pods toleration and affinity
Taint keeps others out; affinity pulls ML Pods in.

C. A nodeSelector on the ML Pods only, and no taint anywhere
ML Pods go there, but ordinary Pods can still land there too.

D. A high PriorityClass for every machine-learning Pod
Priority affects order and preemption, not which nodes.

THE SIGNAL

Two requirements, others out and ours in: taint plus affinity.

> GPU nodes carry the taint dedicated=ml:NoSchedule. The team adds a matching toleration to the ML Pods, and nothing else.
>
> Some ML Pods land on the GPU nodes, but many land on ordinary nodes.

Why?

A A toleration permits the node; it does not attract to it

B The taint effect should have been NoExecute, not NoSchedule

C The scheduler ignores tolerations on Deployment Pods

D The taint should have been placed on the Pods instead

◆ PAUSE AND DECIDE

Why?

A A toleration permits the node; it does not attract to it

Tolerating Pods may go anywhere. Add node affinity to pull them in.

B The taint effect should have been NoExecute, not NoSchedule

NoExecute evicts running Pods; it attracts nothing either.

C The scheduler ignores tolerations on Deployment Pods

Tolerations work for every Pod, however it was created.

D The taint should have been placed on the Pods instead

Taints belong on nodes; tolerations belong on Pods.

THE SIGNAL

A toleration is permission, not direction.

> An engineer adds the taint maintenance=true:NoExecute to node-4. Several running Pods on node-4 have no toleration for it.
>
> None of them sets tolerationSeconds, and no other change is made.

What happens to those running Pods?

A Nothing, until each Pod is next restarted

B Only newly created Pods are kept away

C They are evicted from node-4 straight away

D node-4 is removed from the cluster

◆ PAUSE AND DECIDE

What happens to those running Pods?

A Nothing, until each Pod is next restarted

That describes no taint effect at all.

B Only newly created Pods are kept away

That is NoSchedule. NoExecute also acts on running Pods.

C They are evicted from node-4 straight away

NoExecute evicts running Pods that do not tolerate it.

D node-4 is removed from the cluster

Taints never delete nodes.

THE SIGNAL

NoExecute is the effect that evicts running Pods.

> The web front end has required pod anti-affinity by hostname. The cluster has 3 nodes, and the team scales web to 4 replicas.
>
> Three Pods run; one stays Pending. Spreading is desirable, but not essential.

What is the best fix?

A Increase maxSurge on the Deployment

B Change the rule to preferred anti-affinity

C Remove the Pods' resource requests

D Give the web Pods a higher PriorityClass

◆ PAUSE AND DECIDE

What is the best fix?

A Increase maxSurge on the Deployment
maxSurge only matters during rollouts.

B Change the rule to preferred anti-affinity
Spread when possible, share a node when needed.

C Remove the Pods' resource requests
Requests are not the blocker; the rule is.

D Give the web Pods a higher PriorityClass
Preemption cannot satisfy anti-affinity either.

THE SIGNAL

Hard rule, not enough nodes, and the rule is only 'desirable': make it preferred.

> During a ticket sale, each booking API Pod needs more CPU. The number of Pods must stay the same, and Pods must not restart.
>
> The cluster runs Kubernetes 1.37, and the nodes have spare capacity.

What should the team do?

A Delete each Pod so it is recreated with more CPU

B Edit the Deployment template's CPU and let it roll out

C Add a HorizontalPodAutoscaler to add more Pods

D Resize the running Pods through the resize subresource

◆ PAUSE AND DECIDE

What should the team do?

A Delete each Pod so it is recreated with more CPU

Recreates every Pod: exactly what they must avoid.

B Edit the Deployment template's CPU and let it roll out

A template change is a rollout: new Pods replace old ones.

C Add a HorizontalPodAutoscaler to add more Pods

Changes the Pod count, which must stay the same.

D Resize the running Pods through the resize subresource

In-place resize, stable since 1.35: new CPU, same Pods.

THE SIGNAL

'More CPU, same Pods, no restart' is in-place resize.

Module 12 in five lines

- Filter, score, bind. **Requests decide placement; limits decide punishment.**
- QoS: **Guaranteed** (requests = limits), **Burstable**, **BestEffort** (none). BestEffort goes first.
- **Affinity attracts, taints repel, spread balances.** Required filters; preferred scores.
- **A toleration permits, never attracts.** NoExecute evicts running Pods.
- **PriorityClass** for preemption; **in-place resize** is stable since 1.35.

That completes Kubernetes Fundamentals. Next domain: **Container Orchestration**, starting with the network.

MODULE 13 · DOMAIN 2 - CONTAINER ORCHESTRATION

The network model

Why every Pod gets its own IP, who wires it up, and how Service traffic finds a Pod.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 13 of 35

admin@sthithapragna.com

Every Pod gets **its own IP address**, and every Pod can reach every other Pod **without NAT**.

admin@sthithapragna.com

Four networking problems, four answers

PROBLEM	SOLVED BY	MODULE
Container to container, in one Pod	Shared network namespace: localhost	6
Pod to Pod, across nodes	The flat Pod network, built by a CNI plugin	13
Pod to Service	A stable virtual IP, made real by kube-proxy	13, 14
Outside world to Service	NodePort, LoadBalancer, Ingress, Gateway API	14, 15

ON THE EXAM

Keep these four apart, and every networking question has an obvious home.

The rules of the Kubernetes network model

- **Each Pod has a unique IP** across the whole cluster.
- **Pods reach Pods on any node without NAT.** The address a Pod sees for itself is the one others see.
- **Node agents,** such as the kubelet, can reach every Pod on their node.
- **Kubernetes defines the rules; it ships no implementation.** A plugin must provide one.

> ON THE EXAM
>
> No NAT between Pods: an application can hand out its own IP, and peers can use it.

Three address ranges that must never overlap

RANGE	ASSIGNED TO	EXAMPLE
Node network	The nodes themselves	10.0.0.0/16
Pod CIDR	Pods; each node usually gets its own slice	10.244.0.0/16
Service CIDR	Virtual ClusterIPs for Services	10.96.0.0/12

ON THE EXAM

Overlapping ranges cause traffic to vanish in confusing ways. They are chosen when the cluster is built.

CNI: the Container Network Interface

- **A specification and libraries** for plugins that connect containers to a network. A CNCF project.
- **The runtime calls the plugin** when it creates a Pod's network namespace, and again when it deletes it.
- **The plugin** creates the interface, assigns an IP (IPAM), and sets up routes.
- **No plugin, no Pod network:** nodes stay NotReady and Pods cannot start.

> ON THE EXAM
>
> CNI for networks, CSI for storage, CRI for runtimes: three plug-in standards.

What happens to the network when a Pod starts

01
Scheduled
The Pod is bound to a node.

›

02
Sandbox
The kubelet asks the runtime for a Pod sandbox.

›

03
Namespace
The runtime creates the Pod's network namespace.

›

04
CNI ADD
The plugin adds an interface, an IP and routes.

›

05
Containers
App containers start inside that namespace.

ON THE EXAM

The network exists before any app container starts, which is why every container in the Pod shares it.

Network plugins you should recognise

PLUGIN	KNOWN FOR
Calico	Routed networking with BGP, or an overlay; strong NetworkPolicy; optional eBPF data plane
Cilium	eBPF data plane; NetworkPolicy including layer 7; can replace kube-proxy. CNCF graduated
Flannel	A simple overlay network. Does not enforce NetworkPolicy on its own
Cloud VPC plugins	Pods get real addresses from the cloud network, such as the AWS VPC CNI
Multus	A meta-plugin: several network interfaces per Pod

ON THE EXAM

Which plugin you choose decides whether NetworkPolicy is enforced at all.

Overlay versus routed networking

OVERLAY

Tunnel between nodes

- Pod packets wrapped, e.g. in VXLAN
- Works on almost any underlying network
- Small overhead per packet
- Flannel, or Calico in overlay mode

ROUTED

Real routes to Pods

- Pod IPs routed directly, no wrapping
- Needs the network to know Pod routes
- No encapsulation overhead
- Calico with BGP, cloud VPC plugins

THE TIEBREAKER

Both satisfy the model: flat, no NAT between Pods. They differ in how packets cross between nodes.

NetworkPolicy depends on the plugin

- **Kubernetes owns the NetworkPolicy API.** You can always create the objects.
- **The plugin does the enforcing.** A plugin without policy support ignores them.
- **Silently.** No error, no warning: the policies simply do nothing.
- **Check the plugin first** when policies seem to have no effect. Module 19 covers the rules.

> ON THE EXAM
>
> Policies accepted but not enforced: the network plugin does not support NetworkPolicy.

Dual-stack: IPv4 and IPv6 together

- **Stable since Kubernetes 1.23.** Pods and Services can have both an IPv4 and an IPv6 address.
- **Services choose** with ipFamilyPolicy: SingleStack, PreferDualStack or RequireDualStack.
- **Needs support throughout:** the plugin, the nodes, and dual Pod and Service ranges.

> ON THE EXAM
>
> IPv6-only and dual-stack clusters are both possible, if the network plugin supports them.

Pod IPs change whenever Pods are replaced. A **Service** gives a stable address, and **kube-proxy** makes it work.

How a packet to a Service reaches a Pod

01

Send

A client Pod sends to the Service's ClusterIP and port.

02

Match

Rules on the node, written by kube-proxy, match that virtual IP.

03

Choose

One ready backend is picked from the EndpointSlices.

04

Rewrite

The destination becomes that Pod's real IP.

05

Deliver

The Pod network carries it there.

ON THE EXAM

A ClusterIP is virtual. Nothing listens on it; rules on each node rewrite traffic sent to it.

kube-proxy modes

MODE	HOW	STATUS
`iptables`	One set of iptables rules per Service	Default on Linux
`nftables`	The newer nftables framework; scales far better	Stable since 1.33; recommended
`ipvs`	The kernel's IP Virtual Server load balancer	Deprecated in 1.35; to be removed
`No kube-proxy`	The CNI plugin handles Services, e.g. Cilium with eBPF	Plugin-dependent

ON THE EXAM

Old advice said 'use IPVS for big clusters'. Today's answer is nftables.

Currency check: kube-proxy and endpoints

- **IPVS mode is deprecated** since 1.35. It now logs a warning and is on the way out.
- **nftables** is the recommended mode for large clusters. iptables remains the default.
- **The Endpoints API is deprecated** since 1.33, in favour of EndpointSlices.
- Older material says 'switch to IPVS for scale' and shows Endpoints objects. Both are dated.

> ON THE EXAM
>
> Scale problem with kube-proxy today: nftables. Backend lists today: EndpointSlices.

EndpointSlices

- **The list of backends for a Service:** Pod IPs, ports, and whether each is ready.
- **Maintained by the EndpointSlice controller** from the Service's selector and Pod readiness.
- **Split into slices,** up to 100 endpoints each by default, so big Services update cheaply.
- **Readiness decides membership.** A Pod failing its readiness check stops receiving traffic.

> ON THE EXAM
>
> Service with no traffic reaching Pods? Check the EndpointSlice: empty usually means a selector or readiness problem.

Looking at a Service's backends

```
$ kubectl get endpointslices -l kubernetes.io/service-name=booking-api
NAME                ADDRESSTYPE   PORTS   ENDPOINTS                          AGE
booking-api-7x2kq   IPv4          8080    10.244.1.14,10.244.2.9,10.244.3.21   3d
```

1 Slices are linked to their Service by a standard label.

3 Three ready Pod IPs, on three different nodes' Pod ranges.

THE TAKEAWAY

If ENDPOINTS shows none, compare the Service selector with the Pods' labels, then check readiness.

> In a Quayside test cluster, Pods can reach other Pods on the same node by IP. Pods on different nodes cannot reach each other at all.
>
> The test uses Pod IPs directly, not Services or DNS names. No NetworkPolicies exist.

Where is the fault most likely to be?

A The Pod network is not routing between nodes

B kube-proxy has stopped programming Service rules

C CoreDNS is failing to resolve the Pods' names

D A default-deny NetworkPolicy is blocking traffic

◆ PAUSE AND DECIDE

Where is the fault most likely to be?

A The Pod network is not routing between nodes

Same node works, cross-node fails: the plugin's overlay or routes.

B kube-proxy has stopped programming Service rules

kube-proxy handles Service IPs. This test uses Pod IPs.

C CoreDNS is failing to resolve the Pods' names

No names are used; the test goes straight to IPs.

D A default-deny NetworkPolicy is blocking traffic

None exist, and one would block same-node traffic too.

THE SIGNAL

Pod IP to Pod IP, failing only across nodes: the CNI plugin's cross-node path.

> Quayside creates NetworkPolicies so only the booking API can reach the bookings database. kubectl shows the policies created without error.
>
> Yet every Pod can still connect to the database. The cluster uses Flannel as its only network plugin.

What is the most likely reason?

A NetworkPolicies only apply to namespaces labelled for them

B kube-proxy enforces policies and must use nftables mode

C New policies take effect after the Pods are next restarted

D The network plugin in use does not enforce NetworkPolicy

◆ PAUSE AND DECIDE

What is the most likely reason?

A NetworkPolicies only apply to namespaces labelled for them

No such label is needed. Policies select Pods directly.

B kube-proxy enforces policies and must use nftables mode

kube-proxy handles Services, not policy.

C New policies take effect after the Pods are next restarted

Enforcing plugins apply policies to running Pods.

D The network plugin in use does not enforce NetworkPolicy

Flannel alone accepts the objects but enforces nothing.

THE SIGNAL

Policies created fine but ignored: check whether the plugin enforces them.

> Quayside's largest cluster runs Kubernetes 1.37 with about 20,000 Services. kube-proxy runs in IPVS mode, and its logs warn that the mode is deprecated.
>
> The team wants a supported mode that stays fast with this many Services.

Which kube-proxy mode should they move to?

A Stay on IPVS and ignore the warning

B nftables mode

C The userspace proxy mode

D iptables mode, the default

◆ PAUSE AND DECIDE

Which kube-proxy mode should they move to?

A Stay on IPVS and ignore the warning

Deprecated since 1.35 and due for removal.

B nftables mode

Stable since 1.33, and built to scale where iptables struggles.

C The userspace proxy mode

Removed from kube-proxy years ago.

D iptables mode, the default

Supported, but slows down with tens of thousands of Services.

THE SIGNAL

'Supported and fast at scale' today means nftables.

> The booking API Service selects 3 Pods. One Pod starts failing its readiness check, but keeps running and does not restart.
>
> Clients keep calling the Service's ClusterIP throughout.

What happens to the traffic?

A. It is still spread evenly across all three Pods

B. The Service stops routing until all Pods are ready

C. The unready Pod gets none; the other two share it

D. kube-proxy restarts the failing Pod to recover it

◆ PAUSE AND DECIDE

What happens to the traffic?

A It is still spread evenly across all three Pods

Readiness exists precisely to stop this.

B The Service stops routing until all Pods are ready

Ready Pods keep serving; one bad Pod does not stop a Service.

C The unready Pod gets none; the other two share it

It is marked not ready in the EndpointSlice, so no traffic.

D kube-proxy restarts the failing Pod to recover it

kube-proxy never restarts Pods; liveness probes cause restarts.

THE SIGNAL

Readiness removes a Pod from traffic without restarting it.

Module 13 in five lines

- **Every Pod its own IP; Pod to Pod without NAT.** Kubernetes sets the rules, plugins implement them.
- **CNI**: the runtime calls the plugin to add an interface, an IP and routes.
- **NetworkPolicy** is only enforced if the plugin supports it. Flannel alone does not.
- **kube-proxy** makes ClusterIPs work: iptables default, **nftables** recommended, IPVS deprecated.
- **EndpointSlices** list ready backends; readiness decides who gets traffic.

We have seen how a ClusterIP works. Next: **every type of Service**, and how DNS names them.

MODULE 14 · DOMAIN 2 - CONTAINER ORCHESTRATION

Services and DNS

Every Service type, when to use each, and how CoreDNS names them.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 14 of 35

admin@sthithapragna.com

A Service is a stable name and address in front of a changing set of Pods.

A Service for the booking API

```
1 apiVersion: v1
2 kind: Service
3 metadata: {name: booking-api, namespace: quayside}
4 spec:
5   selector: {app: booking-api}
6   ports:
7   - name: http
8     port: 80
9     targetPort: 8080
```

5 Which Pods are behind it: an equality selector.

8 port: what clients connect to on the Service.

9 targetPort: where the container actually listens.

ON THE EXAM

No type given, so this is a ClusterIP Service: reachable only inside the cluster.

Service types at a glance

TYPE	REACHABLE FROM	HOW
`ClusterIP (default)`	Inside the cluster	A stable virtual IP
`NodePort`	Outside, via any node	A port from 30000-32767 opened on every node
`LoadBalancer`	Outside, via one address	A cloud load balancer in front of the NodePorts
`ExternalName`	Inside, pointing out	A DNS alias to an outside hostname; no proxying
`Headless (clusterIP: None)`	Inside the cluster	DNS returns the Pods' own IPs; no virtual IP

ON THE EXAM

Four types, plus headless as a variation of ClusterIP. Most questions ask which one fits.

Each type builds on the one before

01
Client
On the internet.

›

02
LoadBalancer
The cloud's load balancer, one external address.

›

03
NodePort
Traffic arrives at a port on some node.

›

04
ClusterIP
kube-proxy's rules pick a backend.

›

05
Pod
A ready Pod, possibly on another node.

ON THE EXAM

A LoadBalancer Service also has a NodePort and a ClusterIP. Each layer adds reach on top of the last.

ClusterIP versus NodePort

CLUSTERIP

Internal only

- The default
- A virtual IP inside the cluster
- For service-to-service calls
- Not reachable from outside

NODEPORT

A door on every node

- Same port open on every node
- Reach it at any node IP and port
- High port numbers only
- Clients must know node IPs

THE TIEBREAKER

Internal traffic: ClusterIP. Quick outside access without a cloud load balancer: NodePort.

NodePort versus LoadBalancer

NODEPORT

You bring the front door

- No external load balancer
- Clients track node addresses
- Works anywhere, including bare metal
- Fine for testing, awkward for users

LOADBALANCER

The cloud builds it

- One stable external address
- Created by the cloud-controller-manager
- Needs cloud integration, or MetalLB
- One load balancer per Service: cost

THE TIEBREAKER

Production internet traffic on a cloud: LoadBalancer, or an Ingress or Gateway in front of many Services.

externalTrafficPolicy: Cluster versus Local

CLUSTER (DEFAULT)

Any node forwards

- Every node accepts the traffic
- May hop to a Pod on another node
- Client IP replaced by a node IP
- Even spread across Pods

LOCAL

Only nodes with local Pods

- Only nodes running a Pod accept it
- No extra hop
- The real client IP is preserved
- Spread can be uneven

THE TIEBREAKER

The app needs to see the real client IP: externalTrafficPolicy: Local.

ExternalName, and Services without selectors

- **ExternalName:** the Service's DNS name becomes an alias (CNAME) for an outside hostname.
- **No selector, no proxying:** pure DNS. Apps use an in-cluster name for something outside.
- **A Service without a selector** plus your own EndpointSlice points at fixed IP addresses.
- **Why bother?** Move the thing into the cluster later, and only the Service changes.

> ON THE EXAM
>
> 'Give an outside dependency an in-cluster name' is ExternalName, or a selectorless Service.

Headless Services

- **clusterIP: None.** No virtual IP, and kube-proxy does not balance it.
- **DNS returns every ready Pod's IP.** The client chooses which to use.
- **With a StatefulSet,** each Pod also gets its own name: bookings-db-0.bookings-db.
- **Used by** databases and clustered software whose members must address each other individually.

> ON THE EXAM
>
> 'Reach a specific Pod by name' or 'discover all Pod IPs' means headless.

Every Service gets **a DNS name**, so no client ever needs to know an IP address.

CoreDNS

- **The cluster DNS server.** A CNCF graduated project, built from plugins.
- **Runs as a Deployment in kube-system,** behind a Service still named kube-dns.
- **Watches Services and EndpointSlices** through the API server, and answers from them.
- **The kubelet points each Pod** at it, through the Pod's /etc/resolv.conf.

> ON THE EXAM
>
> The Service named kube-dns is served by CoreDNS. The old name stayed for compatibility.

The DNS names Kubernetes creates

RECORD	NAME	RESOLVES TO
Service	booking-api.quayside.svc.cluster.local	The ClusterIP
Headless Service	bookings-db.quayside.svc.cluster.local	Every ready Pod IP
StatefulSet Pod	bookings-db-0.bookings-db.quayside.svc.cluster.local	That one Pod's IP
SRV, named port	_http._tcp.booking-api.quayside.svc.cluster.local	Port number and host
Pod	10-244-1-14.quayside.pod.cluster.local	That Pod's IP

ON THE EXAM

Services read smallest to largest: service, namespace, svc, cluster domain.

Inside a Pod: how short names work

```
$ kubectl exec -n quayside web-7d9f6c-x2k4p -- cat /etc/resolv.conf
nameserver 10.96.0.10
search quayside.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

2 The kube-dns Service's ClusterIP.

3 Search domains: why a bare name like booking-api works.

4 Names with fewer than five dots try the search list first.

THE TAKEAWAY

booking-api resolves in the Pod's own namespace; booking-api.payments reaches another.

dnsPolicy

VALUE	THE POD USES
`ClusterFirst`	Cluster DNS, forwarding outside names upstream. The default.
`Default`	The node's own resolver. Despite its name, not the default.
`ClusterFirstWithHostNet`	Cluster DNS for a Pod on the host network
`None`	Only what you put in dnsConfig

> ON THE EXAM
>
> The value called Default is not the default. ClusterFirst is.

The older way: environment variables

- **The kubelet injects variables** such as BOOKING_API_SERVICE_HOST and _SERVICE_PORT.
- **Only for Services that existed** before the Pod started. Later Services are invisible.
- **DNS has no such ordering problem,** which is why it is the recommended method.

> ON THE EXAM
>
> Two discovery methods: environment variables and DNS. DNS is preferred.

Eight Service decisions. For each, ask: **who needs to reach it, and how?**

> The payment worker calls the booking API. Nothing outside the cluster should ever reach the booking API directly.

Which Service type should the booking API use?

A NodePort

B LoadBalancer

C ExternalName

D ClusterIP

◆ PAUSE AND DECIDE

Which Service type should the booking API use?

A NodePort

Opens a port on every node to the outside.

B LoadBalancer

Creates a public load balancer.

C ExternalName

An alias to something outside; nothing to proxy.

D ClusterIP

Internal only, stable virtual IP: exactly this.

THE SIGNAL

Internal callers only: ClusterIP, the default.

> Quayside's web front end, running on a public cloud, must be reachable from the internet at one stable address.
>
> The team wants the cloud to manage that address and spread traffic across nodes.

Which Service type fits?

A NodePort

B LoadBalancer

C ClusterIP

D A headless Service

◆ PAUSE AND DECIDE

Which Service type fits?

A NodePort

Clients would need node IPs, which change.

B LoadBalancer

One external address, built by the cloud.

C ClusterIP

Not reachable from the internet.

D A headless Service

No virtual IP and no external reach.

THE SIGNAL

Internet, one stable address, cloud-managed: LoadBalancer.

> Quayside's apps use a legacy payments database hosted outside the cluster at payments-db.legacy.example.com.
>
> The team wants apps to use an in-cluster name now, so moving the database later only changes one object.

What should they create?

A An ExternalName Service pointing at that hostname

B A headless Service selecting the database Pods

C A NodePort Service in front of the database

D A ConfigMap holding the database hostname

◆ PAUSE AND DECIDE

What should they create?

A An ExternalName Service pointing at that hostname

A DNS alias: apps use payments-db, the Service holds the real name.

B A headless Service selecting the database Pods

There are no database Pods in the cluster to select.

C A NodePort Service in front of the database

NodePort brings outside traffic in, not the reverse.

D A ConfigMap holding the database hostname

Works, but every app must read it; not a DNS name.

THE SIGNAL

In-cluster name for an outside hostname: ExternalName.

> The three bookings database replicas must find and connect to each other individually, by stable names.
>
> Load balancing across them would break replication.

Which Service suits the replicas?

A A ClusterIP Service with a selector

B A LoadBalancer Service

C A headless Service, clusterIP: None

D An ExternalName Service

◆ PAUSE AND DECIDE

Which Service suits the replicas?

A. A ClusterIP Service with a selector
One virtual IP that load-balances: exactly what breaks replication.

B. A LoadBalancer Service
External, and still load-balanced.

C. A headless Service, clusterIP: None
DNS gives each replica its own name and IP.

D. An ExternalName Service
Points outside the cluster.

THE SIGNAL

'Individually, by name, no load balancing': headless.

> Quayside runs a test cluster on bare-metal servers in its office. There is no cloud and no load-balancer software installed.
>
> Testers on the office network need quick access to the web front end.

What is the simplest option that works?

A A LoadBalancer Service

B A NodePort Service

C A ClusterIP Service

D An ExternalName Service

◆ PAUSE AND DECIDE

What is the simplest option that works?

A — A LoadBalancer Service
With no cloud or MetalLB, its external IP stays pending.

B — A NodePort Service
A port on every node, reachable from the office network.

C — A ClusterIP Service
Only reachable inside the cluster.

D — An ExternalName Service
Points outwards, not inwards.

THE SIGNAL

Bare metal, no load balancer, quick access: NodePort.

> The web front end sits behind a LoadBalancer Service. The fraud team needs each request's real client IP address.
>
> Application logs show only the nodes' own IP addresses as the source.

What change preserves the client IP?

A Set sessionAffinity: ClientIP on the Service

B Change the Service type to NodePort

C Make it a headless Service instead

D Set externalTrafficPolicy to Local

◆ PAUSE AND DECIDE

What change preserves the client IP?

A Set sessionAffinity: ClientIP on the Service
Makes a client stick to one Pod; the IP is still replaced.

B Change the Service type to NodePort
NodePort forwards the same way; the IP is still replaced.

C Make it a headless Service instead
Headless gives no external access at all.

D Set externalTrafficPolicy to Local
No extra hop, so the client IP is kept.

THE SIGNAL

Real client IP needed: externalTrafficPolicy: Local.

> The bookings database is a StatefulSet in the quayside namespace, with a headless Service called bookings-db.
>
> A tool in the payments namespace must connect to replica 0 specifically.

Which DNS name should it use?

A bookings-db.quayside.svc.cluster.local

B bookings-db-0.bookings-db.payments.svc.cluster.local

C bookings-db-0.bookings-db.quayside.svc.cluster.local

D bookings-db-0.pod.cluster.local

◆ PAUSE AND DECIDE

Which DNS name should it use?

A bookings-db.quayside.svc.cluster.local

Resolves to every replica, not replica 0 alone.

B bookings-db-0.bookings-db.payments.svc.cluster.local

Right shape, wrong namespace: that is the caller's.

C bookings-db-0.bookings-db.quayside.svc.cluster.local

Pod name, then the headless Service, then namespace.

D bookings-db-0.pod.cluster.local

Not a name Kubernetes creates.

THE SIGNAL

One StatefulSet Pod by name: pod.service.namespace.svc.cluster.local.

> The booking API Service resolves correctly in DNS, but every connection to it times out.
>
> Its EndpointSlice lists no endpoints. The Service selects app=booking-api; the Pods are labelled app=booking.

What is the fix?

A Match the Service selector to the Pod labels

B Restart CoreDNS so the Service name is refreshed

C Change the Service type to NodePort

D Restart kube-proxy on every node

◆ PAUSE AND DECIDE

What is the fix?

A Match the Service selector to the Pod labels

No matching Pods means no endpoints. Fix the selector or labels.

B Restart CoreDNS so the Service name is refreshed

DNS already resolves correctly.

C Change the Service type to NodePort

A new type still has no Pods behind it.

D Restart kube-proxy on every node

kube-proxy has no endpoints to route to.

THE SIGNAL

DNS fine, EndpointSlice empty: a selector and label mismatch.

Module 14 in five lines

- **ClusterIP** internal (default); **NodePort** 30000-32767 on every node; **LoadBalancer** one cloud address.
- **ExternalName** is a DNS alias outwards; **headless** returns Pod IPs, no virtual IP.
- **externalTrafficPolicy: Local** keeps the client IP; Cluster spreads evenly.
- **CoreDNS** behind the kube-dns Service; names read **service.namespace.svc.cluster.local**.
- dnsPolicy **ClusterFirst** is the default; the value named Default is not.

One LoadBalancer per Service gets expensive. Next: **Ingress, and its successor the Gateway API**.

MODULE 15 · DOMAIN 2 - CONTAINER ORCHESTRATION

Getting traffic in: Ingress and the Gateway API

One entrance for many Services, the Ingress NGINX retirement, and what replaces it.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 15 of 35

admin@sthithapragna.com

One entrance, many Services: route **by hostname and path**, at the HTTP layer.

LoadBalancer Service versus an HTTP entrance

LOADBALANCER SERVICE

Layer 4

- One external address per Service
- Any TCP or UDP traffic
- Knows ports, not URLs
- Cost grows with every Service

INGRESS OR GATEWAY

Layer 7

- One address for many Services
- Routes by hostname and path
- Terminates TLS in one place
- One load balancer, shared

THE TIEBREAKER

Many HTTP services, one address, routing by host and path: Ingress or Gateway, not a LoadBalancer each.

Ingress: an API object and a controller

- **The Ingress object** lists rules: which host and path go to which Service.
- **An Ingress controller** makes it real. Kubernetes ships none. Without one, nothing happens.
- **Controllers include** Traefik, HAProxy, Contour, cloud load balancer controllers, and F5 NGINX.
- **The Ingress API is stable** since 1.19 and still supported, but feature-frozen.

> ON THE EXAM
>
> Ingress created, no address, no traffic: there is no Ingress controller running.

An Ingress for Quayside

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: {name: quayside}
spec:
  ingressClassName: traefik
  rules:
  - host: book.quayside.example
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend: {service: {name: booking-api, port: {number: 80}}}
      - {path: /, pathType: Prefix, backend: {service: {name: web, port: {number: 80}}}}
```

`5` Which controller should handle this Ingress.

`10-13` /api goes to the booking API; everything else goes to web.

The parts of an Ingress

FIELD	PURPOSE
`ingressClassName`	Picks the controller, through an IngressClass object
`rules: host`	Match on the requested hostname
`paths: pathType`	Exact, Prefix, or ImplementationSpecific
`backend`	A Service name and port
`tls`	Hostnames plus a kubernetes.io/tls Secret holding the certificate
`defaultBackend`	Where unmatched requests go

ON THE EXAM

Ingress TLS comes from a kubernetes.io/tls Secret. The cert-manager tool often creates it.

Where Ingress falls short

- **HTTP and HTTPS only.** No TCP, UDP or gRPC-specific routing in the spec.
- **Anything beyond host and path** needs controller-specific annotations: rewrites, headers, canaries.
- **Annotations are not portable.** Switching controllers means rewriting them.
- **One object mixes** platform concerns, like TLS, with each app's routing.

> ON THE EXAM
>
> 'Portable canary weights without vendor annotations' is exactly what Ingress cannot offer.

Currency check: Ingress NGINX is retired

- **November 2025:** SIG Network and the Security Response Committee announced its retirement.
- **Since March 2026:** no further releases, no bug fixes, no security patches.
- **Existing installs still run,** but become less safe with every unpatched vulnerability.
- **The Ingress API is not deprecated.** Only this one community controller was retired.
- **F5's NGINX Ingress Controller** is a different, still-maintained product.

ON THE EXAM

Recommended path: move to a Gateway API implementation, or at least another Ingress controller.

The Gateway API is **role-oriented, expressive and portable**: the future of traffic into Kubernetes.

Gateway API resources

RESOURCE	OWNED BY	DEFINES
GatewayClass	Infrastructure provider	Which implementation: Envoy Gateway, Istio, a cloud…
Gateway	Cluster operator	Listeners: ports, protocols, hostnames, TLS
HTTPRoute	Application developer	Hosts, paths, headers and weights, to Services
GRPCRoute	Application developer	gRPC services and methods, to Services
ReferenceGrant	The target namespace	Permission for a cross-namespace reference

ON THE EXAM

Installed as CRDs, usually by the implementation. That is why its apiVersion is gateway.networking.k8s.io.

Who owns what

01

Infrastructure provider

Offers a GatewayClass.

02

Cluster operator

Creates a Gateway in the infra namespace, with TLS.

03

App teams

Write HTTPRoutes in their own namespaces.

04

Attach

Routes attach to the Gateway through parentRefs.

05

Services

Routes send traffic to ordinary Services.

ON THE EXAM

The platform team controls the entrance; each app team controls only its own routes.

An HTTPRoute with a canary, no annotations

```
 1 apiVersion: gateway.networking.k8s.io/v1
 2 kind: HTTPRoute
 3 metadata: {name: booking, namespace: quayside}
 4 spec:
 5   parentRefs: [{name: public, namespace: infra}]
 6   hostnames: [book.quayside.example]
 7   rules:
 8   - matches: [{path: {type: PathPrefix, value: /api}}]
 9     backendRefs:
10     - {name: booking-api, port: 80, weight: 90}
11     - {name: booking-api-canary, port: 80, weight: 10}
```

5 Attaches to the Gateway in infra, whose listener must allow this namespace.

10-11 90% to the stable version, 10% to the canary. Part of the spec.

Ingress versus Gateway API

INGRESS

Simple and frozen

- One object for everything
- HTTP and HTTPS
- Extras through vendor annotations
- Built in; stable; no new features

GATEWAY API

Expressive and growing

- Split by role across resources
- HTTP and gRPC; more protocols in progress
- Headers, weights, mirrors in the spec
- Installed as CRDs; actively developed

THE TIEBREAKER

Role separation, portable traffic splitting, or cross-namespace sharing: Gateway API.

Moving from Ingress to Gateway API

- **ingress2gateway,** a Kubernetes SIG tool, converts Ingress objects into Gateway API resources.
- **Many implementations support both,** so the two can run side by side during migration.
- **Popular implementations:** Envoy Gateway, Istio, Cilium, Traefik, Kong, NGINX Gateway Fabric.
- **Gateway API also reaches inside the cluster:** service meshes use it through GAMMA, in module 16.

> ON THE EXAM
>
> New work in Kubernetes networking happens in the Gateway API.

> On a fresh kubeadm cluster, an engineer creates an Ingress routing book.quayside.example to the web Service.
>
> kubectl shows the Ingress with no address, and requests never arrive. The web Service and its Pods are healthy.

What is the most likely cause?

A The backend Service must be type LoadBalancer

B The DNS record for the host has not been created

C No Ingress controller is running in the cluster

D The pathType must be Exact for a root path

◆ PAUSE AND DECIDE

What is the most likely cause?

A The backend Service must be type LoadBalancer

Backends are normally plain ClusterIP Services.

B The DNS record for the host has not been created

DNS would matter later, but the Ingress still gets no address.

C No Ingress controller is running in the cluster

Kubernetes ships none. No controller, no address, no traffic.

D The pathType must be Exact for a root path

Prefix with / is the usual catch-all.

THE SIGNAL

Fresh cluster, Ingress with no address: install an Ingress controller.

> Quayside currently runs two LoadBalancer Services, one for web and one for the booking API, and pays for two cloud load balancers.
>
> They want one public address: book.quayside.example/api to the API, everything else to web.

What should they do?

A One Ingress or Gateway, routing by path to both Services

B Change both Services to NodePort and publish node addresses

C Make both Services ExternalName pointing at the same host

D Make both Services headless and publish the Pod addresses

◆ PAUSE AND DECIDE

What should they do?

A One Ingress or Gateway, routing by path to both Services

One entrance, path routing, one load balancer to pay for.

B Change both Services to NodePort and publish node addresses

No path routing, and users would chase node IPs.

C Make both Services ExternalName pointing at the same host

ExternalName points outwards; it serves nothing.

D Make both Services headless and publish the Pod addresses

Headless is internal, and Pod IPs change constantly.

THE SIGNAL

One address, routing by path to several Services: an HTTP entrance.

> In September 2026, Quayside's security team finds the community ingress-nginx controller still running in two clusters.
>
> They ask for an accurate statement of its status and the right response.

Which statement is accurate?

A The Ingress API is deprecated and will soon be removed from Kubernetes

B It is retired and unpatched since March 2026; plan a migration

C It still gets security patches until Kubernetes 2.0 is released

D Every Ingress it serves has already stopped working

◆ PAUSE AND DECIDE

Which statement is accurate?

A The Ingress API is deprecated and will soon be removed from Kubernetes

The API is stable and frozen, not deprecated. Only the controller retired.

B It is retired and unpatched since March 2026; plan a migration

Still runs, gets no fixes. Move to Gateway API or another controller.

C It still gets security patches until Kubernetes 2.0 is released

No patches of any kind since March 2026.

D Every Ingress it serves has already stopped working

Existing installations keep running; they are just unmaintained.

THE SIGNAL

Controller retired, API alive. Running but unpatched means migrate, not panic.

> Quayside's platform team wants to own the public entrance and its TLS certificates. Each app team should manage only its own routing, in its own namespace.
>
> The booking team also wants to send 10% of traffic to a canary, without vendor-specific annotations.

Which approach fits?

A One Ingress per team, with controller annotations for the canary

B A LoadBalancer Service for each team's applications

C NodePort Services, with the platform team's own proxy in front

D A shared Gateway, with each team's HTTPRoutes using weights

◆ PAUSE AND DECIDE

Which approach fits?

A One Ingress per team, with controller annotations for the canary

Uses the very vendor annotations they want to avoid.

B A LoadBalancer Service for each team's applications

Many entrances, no shared TLS, no weighted routing.

C NodePort Services, with the platform team's own proxy in front

Hand-built plumbing that the Gateway API already provides.

D A shared Gateway, with each team's HTTPRoutes using weights

Role split by design; weights in the standard spec.

THE SIGNAL

Separate roles plus portable weights: Gateway and HTTPRoute.

Module 15 in five lines

- **One HTTP entrance for many Services,** routing by host and path, instead of a LoadBalancer each.
- **Ingress needs a controller;** Kubernetes ships none. The API is stable and frozen.
- **Ingress NGINX is retired:** no patches since March 2026. The Ingress API is not deprecated.
- **Gateway API:** GatewayClass, Gateway, HTTPRoute, split by role, installed as CRDs.
- **Weights and headers in the spec,** portable across implementations. ingress2gateway converts.

That is traffic from outside. Next: **service meshes**, for traffic between services inside the cluster.

MODULE 16 · DOMAIN 2 - CONTAINER ORCHESTRATION

Service mesh

Proxies that give every call encryption, identity, retries and metrics, without touching the code.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 16 of 35

admin@sthithapragna.com

A service mesh moves **networking concerns out of the application** and into a layer of proxies.

What a mesh gives you

AREA	EXAMPLES
Traffic management	Retries, timeouts, circuit breaking, traffic splitting, mirroring
Security	Mutual TLS on every call, workload identity, authorisation policies
Observability	Request rate, errors and latency for every service-to-service call

ON THE EXAM

Same features in every language, with no code changes. That uniformity is the selling point.

Data plane versus control plane

DATA PLANE

Handles every request

- Proxies next to or near each workload
- Encrypt, route, retry, measure
- Examples: Envoy, ztunnel, linkerd2-proxy
- Keeps working with its last config

CONTROL PLANE

Configures the proxies

- Pushes routing and policy to proxies
- Issues and rotates certificates
- Examples: istiod, Linkerd's control plane
- Handles no application traffic

THE TIEBREAKER

Same split as Kubernetes itself: the control plane decides, the data plane does the work.

Sidecar versus sidecarless

SIDECAR

A proxy in every Pod

- Full layer 7 features per Pod
- CPU and memory cost in every Pod
- Upgrading means restarting Pods
- The classic design

SIDECARLESS

Proxies per node

- A shared proxy per node for L4 and mTLS
- Layer 7 proxies only where needed
- Far less overhead, no Pod restarts
- Istio ambient mode, Cilium

THE TIEBREAKER

Mesh overhead too high, or Pod restarts on every upgrade: consider a sidecarless design.

Istio ambient mode

- **ztunnel:** a per-node proxy giving every Pod mTLS, identity and layer 4 policy.
- **Waypoint proxies:** Envoy, deployed only where layer 7 routing or policy is needed.
- **Generally available since Istio 1.24,** November 2024. Sidecar mode is still supported.
- Opt in by labelling a namespace. No change to the Pods themselves.

> ON THE EXAM
>
> Older material presents sidecars as the only way to run a mesh. That is no longer true.

Meshes and proxies you should recognise

NAME	KNOWN FOR
Envoy	The layer 7 proxy inside many meshes and gateways. CNCF graduated
Istio	Feature-rich mesh on Envoy; sidecar or ambient. CNCF graduated
Linkerd	Simplicity; its own lightweight proxy written in Rust. CNCF graduated
Cilium	eBPF-based, sidecarless mesh from the network plugin
Kuma, Consul	Envoy-based meshes from Kong and HashiCorp

ON THE EXAM

Envoy is a proxy, not a mesh. Istio and Linkerd are meshes. Questions like to blur that line.

Mutual TLS: **both sides prove who they are**, and everything between them is encrypted.

admin@sthithapragna.com

TLS versus mutual TLS

TLS

The server proves itself

- Client checks the server's certificate
- Server does not know who the client is
- Encrypted in transit
- Typical HTTPS on the web

MUTUAL TLS

Both sides prove themselves

- Both present certificates
- Each side knows the other's identity
- Encrypted in transit
- Service-to-service in a mesh

THE TIEBREAKER

'Encrypt and authenticate both sides of every internal call': mutual TLS, usually via a mesh.

Workload identity

- **Each workload gets a certificate** tied to its ServiceAccount.
- **SPIFFE format:** spiffe://cluster.local/ns/quayside/sa/booking-api.
- **Short-lived and rotated automatically** by the control plane. Nobody handles keys.
- **Policies use identity, not IP addresses,** which change constantly in Kubernetes.

> ON THE EXAM
>
> SPIFFE and its runtime, SPIRE, are graduated CNCF projects for workload identity.

One call through the mesh

01

App sends

The payment worker sends plain HTTP to booking-api.

02

Proxy intercepts

Its local proxy picks up the call.

03

Handshake

Proxies exchange mesh-issued certificates.

04

Policy

Identity checked: may payments call bookings?

05

Deliver

Encrypted across the network; plain HTTP to the app.

ON THE EXAM

Neither application knows TLS happened. That is the whole point.

What a mesh costs

- **Latency:** every call passes through one or two extra proxies.
- **Resources:** CPU and memory for proxies, most of all with sidecars.
- **Complexity:** another control plane to run, upgrade and debug.
- **Worth it** with many services, strict encryption or zero-trust needs, or poor visibility.

> ON THE EXAM
>
> A mesh is not a default. A handful of services may be better served by libraries and NetworkPolicy.

Currency check: from SMI to GAMMA

- **SMI, the Service Mesh Interface,** was archived by the CNCF in October 2023.
- **GAMMA** uses the Gateway API for mesh traffic, inside the cluster.
- **In the Gateway API's standard channel since v1.1**, released in 2024.
- Older material calls SMI 'the' mesh standard. Its maintainers moved to GAMMA.

> ON THE EXAM
>
> Portable mesh traffic rules today: Gateway API HTTPRoutes, attached to a Service.

GAMMA: an HTTPRoute attached to a Service

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: {name: booking-api-split, namespace: quayside}
spec:
  parentRefs:
  - {group: "", kind: Service, name: booking-api, port: 80}
  rules:
  - backendRefs:
    - {name: booking-api-v2, port: 80, weight: 90}
    - {name: booking-api-v3, port: 80, weight: 10}
```

6 The parent is a Service, not a Gateway: this is mesh traffic.

9-10 Calls to booking-api are split 90 to 10 between versions.

THE TAKEAWAY

Same HTTPRoute as module 15. Parent a Gateway for traffic in; parent a Service for traffic between services.

> Quayside's security team requires every call between its 40 internal services to be encrypted, with both caller and receiver authenticated.
>
> The services are written in five languages, and teams cannot change their code this year.

Which approach meets the requirement?

A NetworkPolicies allowing only the expected callers

B TLS termination at the Ingress or Gateway

C Certificates in Secrets, loaded by each service

D A service mesh enforcing mutual TLS

◆ PAUSE AND DECIDE

Which approach meets the requirement?

A NetworkPolicies allowing only the expected callers

Restricts who connects, but encrypts nothing and proves no identity.

B TLS termination at the Ingress or Gateway

Covers traffic coming in, not calls between services.

C Certificates in Secrets, loaded by each service

Needs code changes in five languages, which is ruled out.

D A service mesh enforcing mutual TLS

Encrypts and authenticates both sides, with no code changes.

THE SIGNAL

Encrypt and authenticate every internal call, without code changes: mesh mTLS.

> Quayside runs a sidecar mesh across 400 small Pods. The proxies use more memory than many of the apps, and every mesh upgrade restarts every Pod.
>
> The team still needs mutual TLS everywhere, and layer 7 routing for only a few services.

What change best addresses this?

A Add memory to every node and keep the sidecars

B Turn off mutual TLS to reduce proxy load

C Move to a sidecarless mode such as Istio ambient

D Run Envoy as a DaemonSet, with no mesh control plane

◆ PAUSE AND DECIDE

What change best addresses this?

A Add memory to every node and keep the sidecars

Pays for the problem instead of removing it; restarts remain.

B Turn off mutual TLS to reduce proxy load

Drops the one requirement that must stay.

C Move to a sidecarless mode such as Istio ambient

Per-node mTLS, waypoints only where needed, no Pod restarts.

D Run Envoy as a DaemonSet, with no mesh control plane

No certificates, identity or configuration: not a mesh.

THE SIGNAL

Sidecar overhead and restarts, mTLS everywhere, L7 in a few places: sidecarless.

> An internal runbook from 2021 says to use an SMI TrafficSplit object for canary releases inside Quayside's mesh.
>
> In 2026 the team wants a supported, portable way to split internal traffic 90 to 10 between versions.

What should they use now?

A. A Gateway API HTTPRoute whose parent is the Service

B. SMI TrafficSplit, which is still the mesh standard

C. Ingress annotations from the ingress controller

D. The Deployment's maxSurge and maxUnavailable

◆ PAUSE AND DECIDE

What should they use now?

A A Gateway API HTTPRoute whose parent is the Service

GAMMA: the standard, portable way, since Gateway API v1.1.

B SMI TrafficSplit, which is still the mesh standard

SMI was archived in 2023.

C Ingress annotations from the ingress controller

Ingress handles traffic coming in, not calls between services.

D The Deployment's maxSurge and maxUnavailable

Paces a rollout; gives no steady 90 to 10 split.

THE SIGNAL

Mesh traffic splitting today: GAMMA, meaning an HTTPRoute with a Service parent.

> istiod, Istio's control plane, crashes and stays down for fifteen minutes. The data plane proxies are all healthy.
>
> Services were exchanging mutually authenticated traffic when it failed.

What happens to that service-to-service traffic?

A All traffic between services stops at once

B It keeps flowing on the proxies' last config

C It falls back to plain, unencrypted HTTP traffic

D Pods restart without their mesh proxies

◆ PAUSE AND DECIDE

What happens to that service-to-service traffic?

A All traffic between services stops at once
The control plane carries no traffic, so nothing stops.

B It keeps flowing on the proxies' last config
The data plane runs on; only changes and new certificates wait.

C It falls back to plain, unencrypted HTTP traffic
Proxies keep their existing certificates; no silent downgrade.

D Pods restart without their mesh proxies
Nothing restarts Pods because the control plane is down.

THE SIGNAL

Control plane down: change stops, traffic continues. The same rule as Kubernetes.

Module 16 in five lines

- A mesh moves **encryption, identity, retries and metrics** into proxies, with no code changes.
- **Data plane** proxies carry traffic; the **control plane** configures them and issues certificates.
- **Sidecar**: a proxy per Pod. **Sidecarless**: per-node proxies, e.g. Istio ambient, GA in 1.24.
- **Mutual TLS**: both sides authenticated and encrypted, identity tied to ServiceAccounts.
- **SMI is archived;** mesh routing now uses the Gateway API through **GAMMA**.

Mesh identity is built on ServiceAccounts. Next: **access control**, who may do what in a cluster.

MODULE 17 · DOMAIN 2 - CONTAINER ORCHESTRATION

Access control

The 4Cs, how the API server checks every request, RBAC, and ServiceAccounts.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 17 of 35

admin@sthithapragna.com

Every API request answers three questions: **who are you**, **may you do this**, and **is it acceptable**?

The 4Cs of cloud native security

LAYER	PROTECT IT WITH...
Cloud	Provider security, network access to the control plane, node hardening
Cluster	RBAC, admission policy, NetworkPolicy, Secrets encryption, audit logs
Container	Minimal, scanned, signed images; non-root; sandboxed runtimes
Code	Secure coding, dependency scanning, TLS, no secrets in source

ON THE EXAM

Each layer relies on the one outside it. Perfect code cannot save a compromised cloud account.

Currency check: how the docs frame it now

- **The 4Cs** remain the widely taught model, and exam questions use it.
- **Since 2024, kubernetes.io** frames security by lifecycle phase, from the CNCF security white paper.
- **The four phases:** Develop, Distribute, Deploy and Runtime.
- Same ideas, different slicing: layers of the stack, or stages of an application's life.

> ON THE EXAM
>
> Asked for the 4Cs: Cloud, Cluster, Container, Code. Asked for lifecycle phases: Develop, Distribute, Deploy, Runtime.

The path of every API request

01
Request
From kubectl, a pipeline, a controller or a Pod.

›

02
Authentication
Who is this? Certificate, token or OIDC.

›

03
Authorisation
May they do this? Usually RBAC.

›

04
Admission
Mutate, then validate the object.

›

05
Persist
Validated, then stored in etcd.

ON THE EXAM

Fails authentication: 401 Unauthorized. Authenticated but not permitted: 403 Forbidden.

Authentication: who are you?

- **Kubernetes has no user objects.** People are managed outside the cluster.
- **People** authenticate with client certificates, or OIDC tokens from an identity provider.
- **Workloads** authenticate with ServiceAccount tokens.
- **Users belong to groups,** taken from the certificate or the identity provider.
- Unauthenticated requests become the user system:anonymous.

> ON THE EXAM
>
> You cannot kubectl create user. Users come from certificates or an identity provider.

Authorisation modes

MODE	DECIDES BY
`RBAC`	Roles and bindings stored as Kubernetes objects. The standard.
`Node`	Kubelets may only touch objects for Pods on their own node
`Webhook`	Asking an external service
`ABAC`	Attribute rules in a file. Legacy
`AlwaysAllow, AlwaysDeny`	Everything, or nothing. For testing only

ON THE EXAM

kubeadm enables Node and RBAC. Modes are checked in order; if none allows, the request is denied.

RBAC in one slide

- **A subject:** a user, a group, or a ServiceAccount.
- **A rule:** verbs on resources in API groups, such as get, list, watch on pods.
- **A role holds rules; a binding gives a role to subjects.**
- **Permissions only add up.** There are no deny rules; anything not granted is refused.

> ON THE EXAM
>
> RBAC is default-deny and additive. To remove access, remove a binding. You cannot write a deny.

Role versus ClusterRole

ROLE

One namespace

- Lives in a namespace
- Grants access only inside it
- For namespaced resources
- Most application permissions

CLUSTERROLE

Cluster-wide or reusable

- Not in any namespace
- Needed for nodes, PVs, namespaces
- Can be reused in many namespaces
- The built-in view, edit, admin

THE TIEBREAKER

Cluster-scoped resources such as nodes can only be granted through a ClusterRole.

RoleBinding versus ClusterRoleBinding

ROLEBINDING

Grants in one namespace

- Lives in a namespace
- Can reference a Role
- Or a ClusterRole, scoped down to here
- The safe default

CLUSTERROLEBINDING

Grants everywhere

- Cluster-wide
- References a ClusterRole
- Applies in every namespace
- Use sparingly

THE TIEBREAKER

The binding decides the scope. A ClusterRole bound by a RoleBinding only applies in that namespace.

Read-only Pod access for the support team

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: pod-reader, namespace: quayside}
rules:
- apiGroups: [""]
  resources: [pods, pods/log]
  verbs: [get, list, watch]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: {name: support-reads-pods, namespace: quayside}
subjects: [{kind: Group, name: support, apiGroup: rbac.authorization.k8s.io}]
roleRef: {kind: Role, name: pod-reader, apiGroup: rbac.authorization.k8s.io}
```

5-7 Core group, Pods and their logs, read-only verbs.

12-13 The support group gets that Role, in quayside only.

Built-in ClusterRoles

CLUSTERROLE	ALLOWS
`cluster-admin`	Everything, everywhere. Reserve for break-glass use
`admin`	Full control within a namespace, including its Roles and bindings
`edit`	Read and write most objects in a namespace; not Roles or bindings
`view`	Read-only on most objects; not Secrets

ON THE EXAM

Bind view, edit or admin with a RoleBinding to give one namespace's worth of access.

People have accounts outside the cluster.
Workloads have ServiceAccounts inside it.

ServiceAccounts

- **A namespaced object.** Every namespace starts with one called default.
- **Each Pod runs as one,** set by serviceAccountName, or default if unset.
- **Bound to roles like any subject.** Its username: system:serviceaccount:quayside:booking-api.
- **Default has no special permissions,** and should stay that way.

> ON THE EXAM
>
> A Pod's permissions are exactly those bound to its ServiceAccount.

Legacy versus projected ServiceAccount tokens

LEGACY SECRET TOKENS

Long-lived

- Stored in a Secret
- Never expired
- No longer created automatically since 1.24
- A stolen one worked forever

PROJECTED TOKENS

Short-lived and bound

- Issued by the TokenRequest API
- Typically valid for about an hour
- Rotated automatically by the kubelet
- Invalid once the Pod is gone

THE TIEBREAKER

Modern clusters mount short-lived, audience-bound tokens. Old material describing Secret tokens is dated.

Hardening ServiceAccounts

- **One ServiceAccount per workload,** with only the permissions it needs.
- **automountServiceAccountToken: false** for Pods that never call the API.
- **Never bind broad roles** such as cluster-admin to a default ServiceAccount.
- **Check with kubectl auth can-i,** impersonating the ServiceAccount.

> ON THE EXAM
>
> A compromised Pod can do whatever its ServiceAccount can do. Keep that small.

Checking permissions with auth can-i

```
1 $ kubectl auth can-i create deployments -n quayside
2 yes
3
4 $ kubectl auth can-i list secrets -n quayside \
5     --as=system:serviceaccount:quayside:booking-api
6 no
7
8 $ kubectl auth can-i --list -n quayside
```

1 Can I do this?

4-5 Can this ServiceAccount do it? Impersonation, with --as.

8 Everything I may do in this namespace.

> Quayside's support team needs to read Pods and their logs in the quayside namespace. They must not change anything.
>
> They must not see anything in any other namespace.

Which RBAC setup fits?

A The view ClusterRole with a ClusterRoleBinding

B A read-only Role and a RoleBinding in quayside

C cluster-admin, removed again after a month

D The edit ClusterRole bound by a RoleBinding

◆ PAUSE AND DECIDE

Which RBAC setup fits?

A The view ClusterRole with a ClusterRoleBinding
Read-only, but in every namespace.

B A read-only Role and a RoleBinding in quayside
Only the verbs needed, only in quayside.

C cluster-admin, removed again after a month
Everything, everywhere, for a month.

D The edit ClusterRole bound by a RoleBinding
Right namespace, but allows changes.

THE SIGNAL

One namespace, read-only: Role plus RoleBinding in that namespace.

> The same read-only Pod access is now needed in twelve of Quayside's twenty namespaces.
>
> The platform team wants one definition of the permissions, not twelve copies.

How should they set it up?

A One ClusterRole with one ClusterRoleBinding

B A copy of the Role and a RoleBinding in each namespace

C A ClusterRoleBinding plus deny rules for eight namespaces

D One ClusterRole, a RoleBinding in each of the twelve

◆ PAUSE AND DECIDE

How should they set it up?

A One ClusterRole with one ClusterRoleBinding

One definition, but it applies in all twenty namespaces.

B A copy of the Role and a RoleBinding in each namespace

Works, but twelve copies is what they want to avoid.

C A ClusterRoleBinding plus deny rules for eight namespaces

RBAC has no deny rules.

D One ClusterRole, a RoleBinding in each of the twelve

Defined once; each RoleBinding scopes it to one namespace.

THE SIGNAL

Reuse without copying: a ClusterRole, bound per namespace with RoleBindings.

> An attacker compromised a booking API Pod and used its token to list every Secret in the cluster.
>
> The investigation finds the Pod ran as the default ServiceAccount, which someone had bound to cluster-admin.

What change would most directly have prevented this?

A Rotating ServiceAccount tokens far more frequently

B A NetworkPolicy limiting the Pod's traffic

C A dedicated ServiceAccount with minimal permissions

D Encrypting Secrets at rest in etcd

◆ PAUSE AND DECIDE

What change would most directly have prevented this?

A Rotating ServiceAccount tokens far more frequently

A fresh token with cluster-admin is just as dangerous.

B A NetworkPolicy limiting the Pod's traffic

Useful, but the Pod still needs to reach the API server.

C A dedicated ServiceAccount with minimal permissions

The token could only do what the booking API needs.

D Encrypting Secrets at rest in etcd

The API server still returns them decrypted to anyone allowed.

THE SIGNAL

The blast radius of a compromised Pod is its ServiceAccount's permissions.

> A developer logs in through Quayside's identity provider and runs kubectl get pods -n staging.
>
> The API server returns 403 Forbidden. kubectl auth can-i list pods -n staging says no.

Which stage rejected the request?

A Authorisation: known user, no permission

B Authentication: the user was not recognised

C Admission control: the object failed validation

D etcd: the Pods could not be read from storage

◆ PAUSE AND DECIDE

Which stage rejected the request?

A Authorisation: known user, no permission

403 means authenticated but not allowed: RBAC said no.

B Authentication: the user was not recognised

That returns 401 Unauthorized, not 403.

C Admission control: the object failed validation

Admission checks writes. A get creates no object.

D etcd: the Pods could not be read from storage

Storage errors are not 403s, and can-i agrees it is a permission issue.

THE SIGNAL

401: who are you? 403: I know you, and the answer is no.

Module 17 in five lines

- **4Cs:** Cloud, Cluster, Container, Code. Docs now also use Develop, Distribute, Deploy, Runtime.
- Requests pass **authentication, authorisation, admission**. 401 versus 403.
- **RBAC is additive, default-deny:** Roles and ClusterRoles, bound by RoleBindings and ClusterRoleBindings.
- **The binding decides the scope;** a ClusterRole via RoleBinding is one namespace only.
- **ServiceAccounts** for workloads, with short-lived projected tokens. One per workload.

RBAC decides who may create a Pod. Next: **what that Pod may do** once it runs.

MODULE 18 · DOMAIN 2 - CONTAINER ORCHESTRATION

Pod security

Pod Security Standards and Admission, securityContext, and admission policy engines.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 18 of 35

admin@sthithapragna.com

RBAC decides who may create a Pod. Pod security decides **what that Pod may do to the node**.

The settings that make a Pod dangerous

- **privileged: true** gives the container almost all of the host's power.
- **hostNetwork, hostPID, hostIPC** share the node's own namespaces.
- **hostPath volumes** mount the node's filesystem into the container.
- **Running as root, extra capabilities,** and allowing privilege escalation.

> ON THE EXAM
>
> Each of these turns a container escape into a node compromise. Pod security blocks them.

The three Pod Security Standards

LEVEL	WHAT IT ALLOWS	FOR
Privileged	Everything. No restrictions at all	Trusted system components: CNI, CSI, monitoring agents
Baseline	Blocks known escalations: privileged, host namespaces, hostPath, dangerous capabilities	Most ordinary applications, with no changes
Restricted	Baseline, plus non-root, no privilege escalation, all capabilities dropped, seccomp	Hardened, security-critical workloads

ON THE EXAM

Privileged, Baseline, Restricted: from anything goes, to safe by default, to hardened.

Currency check: PodSecurityPolicy is gone

- **PodSecurityPolicy was removed in Kubernetes 1.25.**
- **Pod Security Admission** replaced it: built in, and stable since 1.25.
- **Simpler:** no policy objects to write; a namespace label picks a standard.
- Older material describing PSP objects and their bindings is out of date.

> ON THE EXAM
>
> Asked what replaced PodSecurityPolicy: Pod Security Admission, applying the Pod Security Standards.

Pod Security Admission modes

MODE	WHAT HAPPENS TO A VIOLATING POD
`enforce`	Rejected. It is never created
`audit`	Allowed, but recorded in the audit log
`warn`	Allowed, but the user sees a warning

ON THE EXAM

Each mode can use a different level. Enforce what you are sure of; warn and audit what you are testing.

Setting the standard for a namespace

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: quayside
  labels:
    pod-security.kubernetes.io/enforce: baseline
    pod-security.kubernetes.io/warn: restricted
    pod-security.kubernetes.io/audit: restricted
```

6 Reject anything below Baseline, from now on.

7-8 Report what Restricted would reject, without blocking it.

THE TAKEAWAY

Labels can also pin a version, such as enforce-version: v1.37, so upgrades do not tighten rules by surprise.

The gotcha: enforce checks Pods, not Deployments

- **A Deployment that breaks the standard is accepted.** It is not a Pod.
- **Its Pods are then rejected,** so the Deployment shows zero ready replicas.
- **The reason is in the ReplicaSet's events:** FailedCreate, violates PodSecurity.
- **warn and audit do check workload templates,** so you see warnings at apply time.

> ON THE EXAM
>
> Deployment accepted, no Pods, in a namespace with enforce: check the ReplicaSet events.

A Pod that meets Restricted

```
1  spec:
2    securityContext:
3      runAsNonRoot: true
4      seccompProfile: {type: RuntimeDefault}
5    containers:
6    - name: api
7      image: quayside/booking-api:2.4.1
8      securityContext:
9        allowPrivilegeEscalation: false
10       readOnlyRootFilesystem: true
11       capabilities: {drop: [ALL]}
```

3-4 Pod level: never root, and the runtime's default seccomp filter.

9-11 Container level: no escalation, no capabilities. Read-only root is extra hardening.

securityContext settings

SETTING	EFFECT
`runAsNonRoot, runAsUser`	Refuse root; or run as a specific user ID
`allowPrivilegeEscalation: false`	Block setuid tricks that gain privileges
`capabilities: drop / add`	Remove or grant specific Linux capabilities
`privileged`	Near-full host power. Avoid
`readOnlyRootFilesystem`	The container cannot modify its own files
`seccompProfile`	Filter which system calls the process may make

ON THE EXAM

Pod-level settings apply to every container; container-level settings override them.

Currency check: user namespaces

- **hostUsers: false** gives a Pod its own user namespace.
- **Root inside maps to an unprivileged user on the host,** so a breakout is not root on the node.
- **Stable since Kubernetes 1.36,** April 2026. Linux only.
- **Needs a recent kernel and runtime,** and supporting volume types.

> ON THE EXAM
>
> Isolation stronger than runAsNonRoot alone, without the cost of a sandboxed runtime.

Admission controllers are the last check before storage: they can **change** a request, or **reject** it.

Where admission fits

01

Authenticated, authorised

The request passed module 17's checks.

02

Mutating

Defaults and changes: built-ins, webhooks, policies.

03

Schema check

Is the object well formed?

04

Validating

Accept or reject: PodSecurity, quotas, webhooks, policies.

05

Stored

Written to etcd.

ON THE EXAM

Mutating runs first, so validating sees the final object.

Built-in admission controllers you have already met

CONTROLLER	DOES
NamespaceLifecycle	Refuses new objects in a namespace being deleted
LimitRanger	Applies LimitRange defaults and limits (module 11)
ResourceQuota	Enforces namespace quotas (module 11)
ServiceAccount	Sets the default ServiceAccount and token (module 17)
PodSecurity	Applies the Pod Security Standards
DefaultStorageClass	Adds the default StorageClass to claims (module 20)

ON THE EXAM

Two more built-ins call out to webhooks: MutatingAdmissionWebhook and ValidatingAdmissionWebhook.

Built-in policies versus policy engines

BUILT-IN ADMISSION POLICIES

CEL, in the API server

- ValidatingAdmissionPolicy, stable since 1.30
- MutatingAdmissionPolicy, stable in 1.36
- Rules as CEL expressions
- No webhook to run or keep available

POLICY ENGINES VIA WEBHOOKS

OPA Gatekeeper, Kyverno

- Run as webhooks beside the cluster
- Richer: generate resources, verify images
- Reports and policy libraries
- One more service to operate

THE TIEBREAKER

Simple validation or mutation: the built-in policies. Generating resources or verifying signatures: an engine.

Policy tools you should recognise

TOOL	KNOWN FOR
`OPA Gatekeeper`	Open Policy Agent for Kubernetes; policies in the Rego language. OPA is CNCF graduated
`Kyverno`	Policies written as Kubernetes YAML; validate, mutate, generate, verify images. CNCF graduated 2026
`ValidatingAdmissionPolicy`	Built in; CEL expressions; nothing extra to install

> ON THE EXAM
>
> Rego is OPA's language. Kyverno needs no new language: its policies are YAML.

> Quayside wants to block privileged containers, host namespaces and hostPath volumes in its application namespaces.
>
> Typical application Pods must keep working unchanged, and the team wants a built-in feature.

What should they do?

A Label the namespaces pod-security enforce: baseline

B Label the namespaces pod-security enforce: restricted

C Create PodSecurityPolicy objects and bind them

D Apply a default-deny NetworkPolicy to each namespace

◆ PAUSE AND DECIDE

What should they do?

A Label the namespaces pod-security enforce: baseline

Blocks exactly those settings; ordinary apps pass unchanged.

B Label the namespaces pod-security enforce: restricted

Also blocks them, but many unchanged apps would fail.

C Create PodSecurityPolicy objects and bind them

Removed in 1.25.

D Apply a default-deny NetworkPolicy to each namespace

Controls traffic, not what a Pod may do to its node.

THE SIGNAL

Block known escalations, keep typical apps working: Baseline.

> The payments namespace enforces Baseline. The team wants to move it to Restricted next quarter.
>
> First they want to know which workloads would break, without blocking anything today.

What should they do now?

A Switch enforce to restricted over a quiet weekend

B Set every namespace to the Privileged level for now

C Keep enforce baseline; add warn and audit at restricted

D Delete any Pod that does not look compliant

◆ PAUSE AND DECIDE

What should they do now?

A Switch enforce to restricted over a quiet weekend

Blocks things today, which is exactly what they must not do.

B Set every namespace to the Privileged level for now

Removes protection and reveals nothing.

C Keep enforce baseline; add warn and audit at restricted

Reports every future violation, blocks nothing new.

D Delete any Pod that does not look compliant

Guesswork, and causes outages.

THE SIGNAL

'Find out what would break, block nothing': warn and audit at the stricter level.

> The quayside namespace now enforces Restricted. A new Deployment is created without error, but it never gets any Pods.
>
> The ReplicaSet's events say: violates PodSecurity restricted: allowPrivilegeEscalation != false.

What explains this, and what is the fix?

A The Deployment was rejected; re-create it with --force

B RBAC forbids the ReplicaSet from creating Pods

C The namespace ResourceQuota has run out of Pods

D Pods are rejected; fix the template's securityContext

◆ PAUSE AND DECIDE

What explains this, and what is the fix?

A The Deployment was rejected; re-create it with --force

It was accepted. Only its Pods are rejected.

B RBAC forbids the ReplicaSet from creating Pods

The event names PodSecurity, not a permission.

C The namespace ResourceQuota has run out of Pods

A quota event would say exceeded quota.

D Pods are rejected; fix the template's securityContext

Set allowPrivilegeEscalation: false, and the rest of Restricted.

THE SIGNAL

Deployment accepted, zero Pods, PodSecurity in the events: fix the Pod template.

> Quayside wants two organisation-wide rules. Only images from registry.quayside.example may run.
>
> And every new namespace must automatically get a default NetworkPolicy created inside it.

Which tool fits both rules?

A Pod Security Admission at the Restricted level

B A policy engine such as Kyverno

C RBAC roles limiting who may create Pods

D NetworkPolicies in every namespace

◆ PAUSE AND DECIDE

Which tool fits both rules?

A Pod Security Admission at the Restricted level

Checks the fixed standards only; knows nothing of registries.

B A policy engine such as Kyverno

Validates image sources and generates resources in new namespaces.

C RBAC roles limiting who may create Pods

Controls who, not which images or what gets created.

D NetworkPolicies in every namespace

The thing to be generated, not the tool that generates it.

THE SIGNAL

Custom rules plus generating resources: a policy engine.

Module 18 in five lines

- Three standards: **Privileged, Baseline, Restricted**.
- **Pod Security Admission** replaced PodSecurityPolicy in 1.25. Namespace labels pick the level.
- Modes: **enforce** rejects, **audit** logs, **warn** warns. Enforce checks Pods, not Deployments.
- Restricted needs **non-root, no escalation, drop ALL, seccomp**. User namespaces stable in 1.36.
- Custom rules: **ValidatingAdmissionPolicy** (CEL) built in; **Gatekeeper** and **Kyverno** go further.

Pods are locked down. Next: **locking down the network between them**, and the supply chain behind them.

MODULE 19 · DOMAIN 2 - CONTAINER ORCHESTRATION

Network policy, secrets and the supply chain

Limiting traffic, protecting stored data, watching what runs, and trusting what you deploy.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 19 of 35

admin@sthithapragna.com

Defence in depth: limit who can talk, protect what is stored, watch what runs, trust what you deploy.

NetworkPolicy: the defaults

- **By default, every Pod can reach every other Pod,** in every namespace.
- **Once a policy selects a Pod,** that direction becomes isolated: only allowed traffic gets through.
- **Policies add up.** The allowed traffic is the union of all policies. There are no deny rules.
- Namespaced, layer 3 and 4 only, and enforced by the network plugin, as module 13 showed.

> ON THE EXAM
>
> No policy selects a Pod: everything allowed. Any policy selects it: only what policies allow.

Choosing who is allowed

SELECTOR	ALLOWS TRAFFIC FROM, OR TO...
podSelector	Pods with these labels, in the policy's own namespace
namespaceSelector	All Pods in namespaces with these labels
Both, in one entry	Pods with these labels, in namespaces with those labels
ipBlock	A CIDR range, typically outside the cluster
ports	Which ports and protocols, such as TCP 5432

ON THE EXAM

Policies select Pods by label, never by IP. Pod IPs change; labels do not.

Step 1: deny all incoming traffic

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: default-deny-ingress, namespace: quayside}
spec:
  podSelector: {}
  policyTypes: [Ingress]
```

5 An empty selector selects every Pod in the namespace.

6 Ingress, with no rules: nothing is allowed in.

THE TAKEAWAY

Start from default-deny, then allow each flow you need. Anything not written down is blocked.

Step 2: let only the booking API in

```
1  apiVersion: networking.k8s.io/v1
2  kind: NetworkPolicy
3  metadata: {name: db-allow-booking-api, namespace: quayside}
4  spec:
5    podSelector: {matchLabels: {app: bookings-db}}
6    policyTypes: [Ingress]
7    ingress:
8    - from:
9      - podSelector: {matchLabels: {app: booking-api}}
10     ports: [{protocol: TCP, port: 5432}]
```

5 This policy protects the database Pods.

9-10 Only booking API Pods, and only on the Postgres port.

The dash that changes everything

TWO ENTRIES: OR

Wider than it looks

```
- from:
  - namespaceSelector:
      matchLabels: {team: payments}
  - podSelector:
      matchLabels: {app: worker}
```

Any Pod in payments namespaces, or worker Pods in this one.

ONE ENTRY: AND

What was meant

```
- from:
  - namespaceSelector:
      matchLabels: {team: payments}
    podSelector:
      matchLabels: {app: worker}
```

Only worker Pods, in payments namespaces.

THE TIEBREAKER

One dash: AND. Two dashes: OR. A single extra dash can open a policy right up.

Egress policies, and the DNS trap

- **Egress rules** limit where Pods may send traffic: other Pods, or ipBlock ranges.
- **Default-deny egress also blocks DNS.** Every name lookup fails.
- **Allow port 53, UDP and TCP,** to CoreDNS in kube-system, alongside real destinations.
- Symptom to recognise: 'could not resolve host', even for in-cluster Services.

> ON THE EXAM
>
> Egress locked down and name lookups fail: allow DNS to kube-dns on port 53.

What NetworkPolicy cannot do

- **No layer 7:** it sees IPs and ports, not HTTP paths or methods. Meshes and some plugins go further.
- **No encryption:** it restricts traffic but never encrypts it. That is mTLS, module 16.
- **No effect without a supporting plugin,** as module 13 showed.
- Cluster-wide administrator policies are being developed by SIG Network.

> ON THE EXAM
>
> NetworkPolicy restricts. mTLS encrypts and authenticates. Strong setups use both.

Base64 is not protection. Encrypt Secrets at rest, and record who touched what.

Encryption at rest: providers

PROVIDER	KEY HELD	NOTE
`identity`	None	The default: no encryption at all
`aescbc, aesgcm, secretbox`	A key file on the control plane	Encrypts, but the key sits beside the data
`kms (v2)`	An external key management service	Envelope encryption. Recommended

ON THE EXAM

Set in an EncryptionConfiguration passed to the API server. Existing Secrets are encrypted when next written.

Audit logging

- **The API server can log every request:** who, what, when, and the result.
- **An audit policy sets the detail per resource:** None, Metadata, Request, RequestResponse.
- **Log Secrets at Metadata only.** Logging their bodies would copy the secrets into logs.
- **Ship logs off the cluster,** so an attacker cannot erase their tracks.

> ON THE EXAM
>
> 'Who deleted that Deployment at 3am?' is answered by the API server's audit log.

Prevention is never perfect. **Runtime security watches what containers actually do.**

Falco

- **A CNCF graduated project** for runtime threat detection.
- **Watches system calls** through the kernel, usually with eBPF.
- **Rules flag suspicious behaviour:** a shell opened in a container, a write under /etc, a sensitive file read.
- **It detects and alerts;** it does not block by default. Tools like Tetragon can also enforce.

> ON THE EXAM
>
> 'Alert when someone opens a shell in a production container': runtime security, such as Falco.

The software supply chain

01
Source
Code review; dependency scanning.

02
Build
Trusted CI; provenance recorded.

03
Image
SBOM generated; vulnerabilities scanned.

04
Registry
Image signed; trusted registry only.

05
Admission
Signature and source verified before running.

ON THE EXAM

Each stage adds evidence. Admission is where the cluster checks that evidence.

Supply chain terms and tools

TERM	MEANS	EXAMPLES
SBOM	Every component inside an image	SPDX, CycloneDX
Scanning	Components checked against known CVEs	Trivy, Grype
Signing	Who built it, and that it is unchanged	Sigstore cosign, Notary
Provenance	How and where it was built	SLSA levels
Trusted registry	Stores, scans and serves signed images	Harbor (CNCF graduated)

ON THE EXAM

New CVE announced? An SBOM shows in minutes which images contain it.

cert-manager: certificates as Kubernetes objects

- **A CNCF graduated project** that issues and renews TLS certificates.
- **Issuers** connect to a source: ACME services such as Let's Encrypt, Vault, or a private CA.
- **A Certificate object** asks for a certificate; the result lands in a kubernetes.io/tls Secret.
- **Renewal is automatic,** so certificates on Ingresses and Gateways never quietly expire.

> ON THE EXAM
>
> 'TLS certificates that renew themselves inside the cluster': cert-manager.

> Quayside applied a default-deny ingress policy to the quayside namespace. Now the web front end cannot reach the booking API.
>
> Only web Pods should be able to reach the booking API, and only on port 8080.

What should the team add?

A Delete the default-deny policy so traffic flows again

B An egress policy on web Pods allowing traffic to the API

C An ingress policy on booking API Pods allowing web on 8080

D Change the booking API Service to type NodePort

◆ PAUSE AND DECIDE

What should the team add?

A Delete the default-deny policy so traffic flows again

Opens everything, including the database, to everyone.

B An egress policy on web Pods allowing traffic to the API

Egress was never blocked. The API's ingress still denies it.

C An ingress policy on booking API Pods allowing web on 8080

Selects the receiver and allows exactly one flow in.

D Change the booking API Service to type NodePort

Service types do not bypass NetworkPolicy.

THE SIGNAL

Incoming traffic blocked: add an ingress rule on the receiving Pods.

> The team applies a default-deny egress policy to the payments namespace, then adds a rule allowing traffic to booking API Pods.
>
> Every call from the payment worker now fails with: could not resolve host booking-api.quayside.

What is wrong?

A CoreDNS crashed when the policy was applied

B DNS traffic is blocked; allow port 53 to kube-dns

C kube-proxy cannot route with egress policies

D Egress policies are not supported by Kubernetes

◆ PAUSE AND DECIDE

What is wrong?

A CoreDNS crashed when the policy was applied
CoreDNS is fine; the worker just cannot reach it.

B DNS traffic is blocked; allow port 53 to kube-dns
Default-deny egress blocked lookups. Allow UDP and TCP 53.

C kube-proxy cannot route with egress policies
The failure is resolving the name, before any routing.

D Egress policies are not supported by Kubernetes
They are supported, and are working all too well.

THE SIGNAL

'Could not resolve host' right after default-deny egress: allow DNS.

> Quayside's security team wants an alert whenever someone opens an interactive shell inside a production container.
>
> They also want alerts when a container reads sensitive files such as /etc/shadow.

Which kind of tool fits?

A Pod Security Admission at the Restricted level

B A default-deny NetworkPolicy in production

C Vulnerability scanning of images in CI

D Runtime threat detection, such as Falco

◆ PAUSE AND DECIDE

Which kind of tool fits?

A Pod Security Admission at the Restricted level

Checks Pods when created; sees nothing that happens later.

B A default-deny NetworkPolicy in production

Controls network traffic, not processes or file reads.

C Vulnerability scanning of images in CI

Finds known flaws before deployment, not live behaviour.

D Runtime threat detection, such as Falco

Watches system calls and alerts on exactly these behaviours.

THE SIGNAL

Alert on behaviour inside running containers: runtime security.

> Quayside wants only images built by its own CI system to run in production.
>
> When a new vulnerability is announced, the team also wants to know within minutes which running images contain the affected library.

Which combination meets both needs?

A Sign images and make SBOMs in CI; verify at admission

B Scan all running images for vulnerabilities once a month

C Enforce Pod Security Restricted on production namespaces

D Always deploy the latest tag so images stay up to date

◆ PAUSE AND DECIDE

Which combination meets both needs?

A Sign images and make SBOMs in CI; verify at admission

Signatures prove the source; SBOMs answer 'which images?' fast.

B Scan all running images for vulnerabilities once a month

Slow, and says nothing about who built the images.

C Enforce Pod Security Restricted on production namespaces

Controls Pod privileges, not image origin.

D Always deploy the latest tag so images stay up to date

Makes origin and content less certain, not more.

THE SIGNAL

Trusted origin plus fast inventory: signing, SBOMs, and admission verification.

Module 19 in five lines

- **All traffic is allowed** until a policy selects a Pod; then only what policies allow. No deny rules.
- Start with **default-deny**, allow each flow. One dash AND, two dashes OR. **Allow DNS** for egress.
- **Encrypt Secrets at rest,** ideally with KMS v2; **audit log** who did what.
- **Falco** detects suspicious behaviour at runtime, from system calls.
- **Supply chain:** SBOMs, scanning, signing, verified at admission. **cert-manager** for TLS.

Security covered. Next: storage, and how data outlives the Pods that write it.

admin@sthithapragna.com

MODULE 20 · DOMAIN 2 - CONTAINER ORCHESTRATION

Storage

Volumes, persistent volumes and claims, StorageClasses, access modes, and CSI.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 20 of 35

admin@sthithapragna.com

Containers are disposable. **Data must not be.** Volumes separate data from the container's lifetime.

How long does the data live?

WHERE DATA IS WRITTEN	LIVES UNTIL	TYPICAL USE
The container's writable layer	The container is replaced	Nothing you want to keep
emptyDir volume	The Pod is deleted	Scratch space, caches, sharing between containers
hostPath volume	The node is rebuilt	Node agents only; a security risk for apps
configMap, secret volumes	The object changes	Configuration, from module 9
PersistentVolumeClaim	You delete the storage	Databases, uploads, anything to keep

ON THE EXAM

Data that must outlive the Pod needs a PersistentVolumeClaim.

emptyDir versus a persistent volume

EMPTYDIR

Tied to the Pod

- Created empty when the Pod starts
- Shared by the Pod's containers
- Deleted with the Pod
- Can live in memory: medium: Memory

PERSISTENTVOLUMECLAIM

Independent of the Pod

- Storage outside any one Pod
- Survives Pod deletion and rescheduling
- Reattached to replacement Pods
- Deleted only by its reclaim policy

THE TIEBREAKER

Container restarts keep emptyDir data; Pod deletion does not. Only persistent storage survives both.

Three objects behind persistent storage

OBJECT	SCOPE	WHAT IT IS
PersistentVolume (PV)	Cluster	A real piece of storage, such as one cloud disk
PersistentVolumeClaim (PVC)	Namespace	An app's request: size, access mode, class
StorageClass	Cluster	A kind of storage, and how to create it on demand

ON THE EXAM

Apps ask with claims; claims bind one-to-one with volumes; classes create volumes when asked.

An analogy: booking a hotel room

- **The StorageClass is the room type:** standard, or suite with a sea view.
- **The PVC is your booking:** one suite, for two nights. You never pick the room.
- **The PV is the actual room** you are given, and nobody else gets it while you have it.
- **The provisioner is a builder** who constructs a new room the moment you book. That is dynamic provisioning.

> ON THE EXAM
>
> You book a type of room; the hotel assigns an actual room. Claims are bookings, volumes are rooms.

Dynamic provisioning, step by step

01
Claim
A PVC asks for 20Gi of class fast-ssd.

›

02
Provisioner
That class's CSI driver sees the claim.

›

03
Disk
It creates a real disk in the cloud or storage system.

›

04
Bind
A PV is created and bound to the PVC.

›

05
Mount
The kubelet mounts it into the Pod.

ON THE EXAM

With dynamic provisioning, nobody creates PVs by hand. Omit storageClassName and the default class is used.

Quayside's manifest archive, stored properly

```
1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata: {name: manifest-archive, namespace: quayside}
4  spec:
5    storageClassName: fast-ssd
6    accessModes: [ReadWriteOnce]
7    resources: {requests: {storage: 20Gi}}
8  ---
9  # the Pod refers to the claim by name:
10 volumes: [{name: archive, persistentVolumeClaim: {claimName: manifest-archive}}]
```

5-7 Which class, how it may be mounted, and how big.

10 The Pod names the claim, never the volume behind it.

THE TAKEAWAY

Module 1's lost uploads, fixed: data now lives in a claim, not the writable layer.

Access modes

MODE	SHORT	MEANS
ReadWriteOnce	RWO	Read-write by one node at a time
ReadOnlyMany	ROX	Read-only by many nodes
ReadWriteMany	RWX	Read-write by many nodes. Needs a shared filesystem
ReadWriteOncePod	RWOP	Read-write by exactly one Pod. Stable since 1.29

ON THE EXAM

ReadWriteOnce means one node, not one Pod. For a single Pod, use ReadWriteOncePod.

What a StorageClass sets

FIELD	CONTROLS
`provisioner`	Which CSI driver creates the volumes
`parameters`	Driver options, such as disk type or IOPS
`reclaimPolicy`	Delete (default) or Retain, when the claim is deleted
`volumeBindingMode`	Immediate, or WaitForFirstConsumer
`allowVolumeExpansion`	Whether claims can be grown later

ON THE EXAM

One class can be the default, marked with the annotation storageclass.kubernetes.io/is-default-class.

Reclaim policy: Delete versus Retain

DELETE

Claim gone, data gone

- The default for dynamic provisioning
- Deleting the PVC deletes the PV
- And the real disk behind it
- Right for disposable data

RETAIN

Claim gone, data kept

- The PV becomes Released
- The disk and its data remain
- An admin must clean up or reuse it
- Right for precious data

THE TIEBREAKER

With the default class, deleting a claim deletes the disk. Precious data wants Retain, and backups.

Immediate versus WaitForFirstConsumer

IMMEDIATE

Create the disk now

- Disk made as soon as the PVC exists
- Zone chosen before the Pod is placed
- Pod may need a node in another zone
- Result: Pending, volume node affinity conflict

WAITFORFIRSTCONSUMER

Wait for the Pod

- Nothing made until a Pod uses the PVC
- Scheduler places the Pod first
- Disk created in that node's zone
- The safe choice for zonal disks

THE TIEBREAKER

Zonal disks cannot follow a Pod to another zone. Let the Pod choose first: WaitForFirstConsumer.

CSI: the Container Storage Interface

- **A standard for storage plugins,** like CRI for runtimes and CNI for networks.
- **Drivers are maintained by vendors,** outside Kubernetes itself, and released on their own schedule.
- **A controller part** creates and attaches volumes; **a node part**, usually a DaemonSet, mounts them.
- **The old in-tree cloud volume plugins** have moved to CSI drivers.

> ON THE EXAM
>
> CRI, CNI, CSI: runtimes, networks, storage. Three plug-in standards.

Storage features and projects

- **Expansion:** raise a claim's size if the class allows it. Shrinking is not supported.
- **Snapshots:** VolumeSnapshot objects capture a point-in-time copy, through CSI.
- **Protection:** a claim still used by a Pod is not removed until that Pod is gone.
- **Rook**, CNCF graduated, runs Ceph storage inside the cluster as an operator. **Longhorn** is another CNCF option.

> ON THE EXAM
>
> Rook is an operator: it runs a storage system inside Kubernetes. CSI drivers connect to one.

> Quayside's web front end runs 4 replicas across different nodes. Every replica must read and write the same folder of uploaded ferry photos.
>
> The photos must survive Pod replacement.

What storage fits?

A One ReadWriteOnce claim, mounted by all four Pods

B An emptyDir volume in each of the four Pods

C A hostPath volume pointing at /data on each node

D A ReadWriteMany claim on a shared-filesystem class

◆ PAUSE AND DECIDE

What storage fits?

A One ReadWriteOnce claim, mounted by all four Pods

One node at a time: Pods on other nodes cannot mount it.

B An emptyDir volume in each of the four Pods

Four separate folders, each deleted with its Pod.

C A hostPath volume pointing at /data on each node

Different data on every node, and a security risk.

D A ReadWriteMany claim on a shared-filesystem class

Many nodes, read-write, shared, and persistent.

THE SIGNAL

Many Pods on many nodes, all writing, sharing: ReadWriteMany.

> A new Pod with a PVC stays Pending: volume node affinity conflict. The disk was created in zone a as soon as the PVC appeared.
>
> Only nodes in zone b have spare capacity. The StorageClass uses volumeBindingMode: Immediate.

What change prevents this in future?

A Set the StorageClass to WaitForFirstConsumer

B Switch the claim's access mode to ReadWriteMany

C Increase the size requested by the claim

D Remove any node affinity rules from the Pod

◆ PAUSE AND DECIDE

What change prevents this in future?

A Set the StorageClass to WaitForFirstConsumer
The Pod is placed first; the disk is then made in its zone.

B Switch the claim's access mode to ReadWriteMany
Changes sharing, not the zone the disk lives in.

C Increase the size requested by the claim
A bigger disk in the wrong zone is still in the wrong zone.

D Remove any node affinity rules from the Pod
The affinity comes from the zonal volume, not the Pod spec.

THE SIGNAL

'Volume node affinity conflict' with Immediate binding: switch to WaitForFirstConsumer.

> An engineer deletes a PVC in a test namespace by mistake. Its volume was created dynamically by the cluster's default StorageClass.
>
> Nobody changed that class's reclaim policy, and there are no backups.

What has most likely happened to the data?

A. It is kept in a Released PV, ready to rebind

B. The PV and its disk were deleted with the claim

C. It was moved into an emptyDir for safekeeping

D. The kubelet saved a snapshot before deletion

◆ PAUSE AND DECIDE

What has most likely happened to the data?

A It is kept in a Released PV, ready to rebind
Only with Retain. The default for dynamic volumes is Delete.

B The PV and its disk were deleted with the claim
Default reclaim policy Delete: claim gone, disk gone.

C It was moved into an emptyDir for safekeeping
Kubernetes never moves data between volume types.

D The kubelet saved a snapshot before deletion
Snapshots are only taken when you ask for them.

THE SIGNAL

Default class, dynamic volume, claim deleted: the data is gone. Use Retain and backups for anything precious.

> A Quayside job converts ferry videos. Each Pod needs fast scratch space for temporary files while it works.
>
> The files are useless afterwards and must not be shared with other Pods or nodes.

Which volume fits?

A. A PersistentVolumeClaim per Pod, with Retain

B. A hostPath volume pointing at /tmp on the node

C. An emptyDir volume, in memory if space allows

D. A ConfigMap mounted as a writable folder

◆ PAUSE AND DECIDE

Which volume fits?

A A PersistentVolumeClaim per Pod, with Retain

Keeps useless files forever, and costs disks.

B A hostPath volume pointing at /tmp on the node

Shared with everything on the node, and a security risk.

C An emptyDir volume, in memory if space allows

Private to the Pod, fast, and cleaned up automatically.

D A ConfigMap mounted as a writable folder

ConfigMap volumes are read-only configuration.

THE SIGNAL

Per-Pod scratch space, thrown away afterwards: emptyDir.

Module 20 in five lines

- **emptyDir** dies with the Pod; a **PersistentVolumeClaim** outlives it.
- **PVC** is the request, **PV** the storage, **StorageClass** the kind, created on demand.
- **RWO** is one node, **RWX** many nodes, **RWOP** one Pod.
- Dynamic volumes default to **Delete**; use **Retain** for precious data.
- Zonal disks: **WaitForFirstConsumer**. Plugins speak **CSI**.

That completes the building blocks. Next: **troubleshooting**, when any of them goes wrong.

MODULE 21 · DOMAIN 2 - CONTAINER ORCHESTRATION

Troubleshooting

A method, and the usual causes of Pending, crash loops, pull failures, OOM kills, NotReady nodes and DNS trouble.

DOMAIN 2 of 4 EXAM WEIGHT 28% MODULE 21 of 35

admin@sthithapragna.com

Work **from the symptom inwards**: read the status, read the events, then follow the layer they point to.

admin@sthithapragna.com

A method that works every time

01
get
What state is it in? STATUS, READY, RESTARTS.

02
describe
Why? The Events at the bottom.

03
logs
With --previous: its last words before crashing.

04
Check the layer
Node, network, storage, config, quota.

05
Fix and verify
Change one thing, then watch it recover.

ON THE EXAM

get, describe, logs: in that order. Most exam troubleshooting answers come from one of the three.

From symptom to likely cause

SYMPTOM	USUALLY MEANS	LOOK AT
Pending	Cannot be scheduled, or a volume cannot bind	describe Pod: FailedScheduling
ContainerCreating, for long	Image still pulling, volume mount, or network plugin	describe Pod events
ErrImagePull / ImagePullBackOff	Wrong name or tag, or no registry credentials	describe Pod events
CrashLoopBackOff	The app starts, then exits or is killed, repeatedly	logs --previous
Running, but 0/1 READY	The readiness probe is failing	describe Pod: probe events
Deployment with no Pods	Pods rejected: quota or Pod Security	describe ReplicaSet

ON THE EXAM

The status tells you which layer. That alone rules out most answer options.

Reading a Pending Pod's events

EVENT SAYS...	CAUSE	MODULE
Insufficient cpu / memory	Requests do not fit any node	12
had untolerated taint	Every node has a taint the Pod does not tolerate	12
didn't match node affinity/selector	No node has the required labels	12
unbound immediate PersistentVolumeClaims	The claim cannot be satisfied	20
volume node affinity conflict	The disk is in another zone	20
No events at all	The scheduler itself may be down	3

ON THE EXAM

The FailedScheduling message lists every reason, node by node. Read it before changing anything.

Image pull failures

- **ErrImagePull** is the first failure; **ImagePullBackOff** is the kubelet waiting before retrying.
- **A typo** in the repository, tag or registry. The commonest cause by far.
- **A private registry** without an imagePullSecret: 'pull access denied' or 'unauthorized'.
- **No image for this CPU architecture,** an unreachable registry, or a rate limit.

> ON THE EXAM
>
> The container never started, so there are no logs. The answer is in describe's events.

Exit codes worth knowing

EXIT CODE	MEANS	TYPICAL CAUSE
0	Exited successfully	A short task run as a Deployment
1	Application error	Bad config, missing variable, a crash
126 / 127	Cannot execute / command not found	Wrong command or entrypoint
137	Killed with SIGKILL (128 + 9)	OOMKilled, or killed after the grace period
143	Stopped with SIGTERM (128 + 15)	A normal graceful shutdown

ON THE EXAM

137 plus reason OOMKilled means the memory limit. 137 alone means something killed it.

CrashLoopBackOff: the usual causes

- **Missing configuration:** a ConfigMap, Secret or variable the app expects at start-up.
- **A dependency is down:** the database is unreachable and the app exits.
- **Wrong command or arguments** in the Pod spec.
- **A liveness probe kills a slow starter** before it is ready. A startup probe fixes that.
- **OOMKilled,** over and over, under load.

> ON THE EXAM
>
> Always read logs with --previous: the current container may have only just started.

A troubleshooting session

```
$ kubectl get pods -n quayside
NAME                     READY   STATUS             RESTARTS   AGE
booking-api-5b8c4d-q8r2m   0/1     CrashLoopBackOff   6          9m
$ kubectl logs booking-api-5b8c4d-q8r2m -n quayside --previous
Error: environment variable DB_PASSWORD is not set
$ kubectl describe pod booking-api-5b8c4d-q8r2m -n quayside
    Last State:  Terminated
      Reason:    Error
      Exit Code: 1
```

3 What state: crashing, six restarts.

5 Why: the previous container's last words.

9 Exit code 1: an application error, not OOM.

THE TAKEAWAY

Fix: add the Secret reference to the Pod template, then roll out.

OOMKilled versus Evicted

OOMKILLED

One container, its own limit

- The kernel kills the container
- It exceeded its own memory limit
- Restarts inside the same Pod
- Exit code 137; RESTARTS climbs

EVICTED

Whole Pod, node pressure

- The kubelet removes the whole Pod
- The node itself ran short of memory or disk
- Pod ends Failed; a controller replaces it
- Often another Pod's fault

THE TIEBREAKER

OOMKilled: raise the limit or fix the app. Evicted: look at the node and at other Pods' requests.

Node conditions

CONDITION	TRUE MEANS
`Ready`	The kubelet is healthy and can run Pods. False or Unknown is the problem
`MemoryPressure`	The node is low on memory; evictions may follow
`DiskPressure`	The node is low on disk space
`PIDPressure`	Too many processes on the node
`NetworkUnavailable`	The node's network is not configured

ON THE EXAM

kubectl describe node shows every condition with a reason. Start there for NotReady.

Why a node goes NotReady

- **The kubelet stopped:** crashed, or not started after a reboot.
- **The container runtime is down,** so the kubelet cannot run anything.
- **No working network plugin:** the classic fresh-cluster cause.
- **Resource exhaustion, an expired certificate,** or the node losing its network.
- **On the node itself,** check the kubelet's service status and its logs.

> ON THE EXAM
>
> Remember module 5: after about five minutes NotReady, Pods on the node are evicted.

A Service is unreachable: check in this order

01
DNS
Does the name resolve from a Pod?

›

02
Service
Does it exist, with the right port and targetPort?

›

03
Endpoints
Does its EndpointSlice list ready Pods?

›

04
Pod
Is the app listening on targetPort?

›

05
Policy
Is a NetworkPolicy blocking the path?

ON THE EXAM

An empty EndpointSlice is the single most common cause: a selector mismatch or no ready Pods.

DNS problems

- **Are the CoreDNS Pods running** in kube-system, and is the kube-dns Service there?
- **Test from inside a Pod:** nslookup kubernetes.default from a small utility Pod.
- **An egress policy blocking port 53** breaks every lookup, as module 19 showed.
- **A short name from another namespace** fails; use service.namespace.

> ON THE EXAM
>
> If kubernetes.default resolves but your Service does not, the problem is your name or namespace, not DNS.

Eight failures. For each, ask: what does the status say, and which layer does it point to?

admin@sthithapragna.com

> The booking API's new Pods show ImagePullBackOff. The image is registry.quayside.example/booking-api:2.4.1, and the tag exists.
>
> describe shows: pull access denied, repository may require authorization.

What is the fix?

A Raise the container's memory limit

B Restart the kubelet on every node

C Add an imagePullSecret for the registry

D Change the Pod's restartPolicy to Never

◆ PAUSE AND DECIDE

What is the fix?

A Raise the container's memory limit
Memory has nothing to do with pulling images.

B Restart the kubelet on every node
The kubelet is working; the registry refused it.

C Add an imagePullSecret for the registry
The registry is private; nodes need credentials.

D Change the Pod's restartPolicy to Never
Restart policy applies after a container starts.

THE SIGNAL

'Pull access denied' from a private registry: missing credentials.

> A new Pod stays Pending. Its event reads: 0/5 nodes are available: 5 node(s) had untolerated taint {dedicated: ml}.

What is the cause?

A Every node has a taint the Pod does not tolerate

B The nodes lack enough memory for its requests

C The image cannot be pulled from the registry

D The scheduler has stopped working

◆ PAUSE AND DECIDE

What is the cause?

A Every node has a taint the Pod does not tolerate

Exactly what the event says, on all five nodes.

B The nodes lack enough memory for its requests

That would read Insufficient memory.

C The image cannot be pulled from the registry

Image pulls happen after scheduling, not before.

D The scheduler has stopped working

A dead scheduler writes no events at all.

THE SIGNAL

Read the FailedScheduling reason literally. It names the cause.

> The payment worker restarts every few minutes during the evening rush. describe shows Last State: Terminated, Reason: OOMKilled, Exit Code: 137.
>
> Its memory limit is 256Mi.

What is the most appropriate fix?

A Raise the container's CPU limit

B Add a readiness probe to the payment worker

C Set imagePullPolicy to Always

D Raise the memory limit, or cut memory use

◆ PAUSE AND DECIDE

What is the most appropriate fix?

A Raise the container's CPU limit

CPU over its limit is throttled, never killed.

B Add a readiness probe to the payment worker

Readiness controls traffic, not memory.

C Set imagePullPolicy to Always

Pulling has nothing to do with memory kills.

D Raise the memory limit, or cut memory use

It exceeds its own memory limit under load.

THE SIGNAL

OOMKilled plus 137: the container's own memory limit. Raise it or fix the usage.

> A team runs a data-import script as a Deployment. The script does its work and exits successfully after five seconds.
>
> The Pod shows CrashLoopBackOff, and describe shows Last State: Terminated, Reason: Completed, Exit Code: 0.

Why is it crash-looping?

A The image is corrupt, so the container fails to start

B Deployments restart containers after success too

C The container is being OOMKilled

D A liveness probe keeps killing it

◆ PAUSE AND DECIDE

Why is it crash-looping?

A The image is corrupt, so the container fails to start

It starts fine and even completes its work.

B Deployments restart containers after success too

restartPolicy Always restarts every exit. Use a Job instead.

C The container is being OOMKilled

Exit code 0 and Completed rule out a kill.

D A liveness probe keeps killing it

A probe kill would not report Completed with code 0.

THE SIGNAL

Exit code 0 in a crash loop: finite work running as a Deployment. It should be a Job.

> After a kernel patch and reboot, node-4 shows NotReady. describe node reports the kubelet has stopped posting status.
>
> On the node, the kubelet service is inactive: it was never enabled to start at boot.

What is the right fix?

A Delete node-4 and join it again from scratch

B Start the kubelet and enable it at boot

C Drain node-4 so its Pods move elsewhere

D Restart the CoreDNS Pods in kube-system

◆ PAUSE AND DECIDE

What is the right fix?

A Delete node-4 and join it again from scratch

Drastic, and the same fault returns on the next reboot.

B Start the kubelet and enable it at boot

The kubelet is simply not running. Start it, and enable it.

C Drain node-4 so its Pods move elsewhere

Moves work away but leaves the node broken.

D Restart the CoreDNS Pods in kube-system

DNS has nothing to do with node readiness.

THE SIGNAL

NotReady with the kubelet stopped: start the kubelet, and make it start at boot.

> A booking API Pod is Running but shows 0/1 READY, and receives no traffic. Its logs look normal.
>
> The readiness probe checks port 8080 at /healthz. The app was recently changed to listen on port 9090.

What is wrong?

A The liveness probe will soon restart it

B A NetworkPolicy is blocking its traffic

C DNS cannot resolve the booking API name

D The readiness probe checks the old port

◆ PAUSE AND DECIDE

What is wrong?

A The liveness probe will soon restart it

Liveness is not mentioned, and the app is healthy.

B A NetworkPolicy is blocking its traffic

Probes come from the kubelet on the node; the probe itself fails.

C DNS cannot resolve the booking API name

DNS is irrelevant: the Pod is marked not ready.

D The readiness probe checks the old port

Probe on 8080, app on 9090: never ready, so no traffic.

THE SIGNAL

Running but not Ready, healthy logs: suspect the readiness probe itself.

> From the payment worker, nslookup booking-api.quayside fails. So do lookups of every other in-cluster name.
>
> In kube-system, the CoreDNS Pods show CrashLoopBackOff. There are no NetworkPolicies in either namespace.

Where is the problem?

A Cluster DNS itself: CoreDNS is not running

B The booking API Service's selector is wrong

C A NetworkPolicy protects the booking API

D kube-proxy is running in the wrong mode

◆ PAUSE AND DECIDE

Where is the problem?

A Cluster DNS itself: CoreDNS is not running

Every lookup fails and CoreDNS is crash-looping.

B The booking API Service's selector is wrong

A wrong selector still resolves; it just reaches nothing.

C A NetworkPolicy protects the booking API

There are no policies, and policy does not affect DNS answers.

D kube-proxy is running in the wrong mode

Lookups fail before any Service traffic is sent.

THE SIGNAL

Every name fails, not just one: cluster DNS itself.

> The export-reports Deployment shows 0/3 ready, and no Pods exist for it at all.
>
> The ReplicaSet's events say: exceeded quota: team-quota, requested: pods=1, used: pods=40, limited: pods=40.

What is the cause?

A The scheduler is down, so Pods are not placed

B The image cannot be pulled

C The namespace has reached its Pod quota

D Pod Security Admission rejects the Pods

◆ PAUSE AND DECIDE

What is the cause?

A The scheduler is down, so Pods are not placed
Pods would still exist, stuck in Pending.

B The image cannot be pulled
A pull failure needs a Pod to exist first.

C The namespace has reached its Pod quota
40 of 40 Pods used: creation is refused.

D Pod Security Admission rejects the Pods
That event would name PodSecurity, not a quota.

THE SIGNAL

Deployment with no Pods: read the ReplicaSet's events. Here, the quota.

Module 21 in five lines

- Method: **get** for state, **describe** for events, **logs --previous** for crashes.
- **Pending** is scheduling or storage; **ImagePullBackOff** is name or credentials; no logs exist.
- Exit **137 + OOMKilled** is the memory limit; **exit 0 crash loop** is finite work in a Deployment.
- **NotReady:** describe node, then kubelet, runtime, network plugin, resources.
- **Unreachable Service:** DNS, Service, EndpointSlice, Pod, policy. Empty endpoints is most common.

That completes Container Orchestration. Next domain: **Cloud Native Application Delivery**.

admin@sthithapragna.com

MODULE 22 · DOMAIN 3 - CLOUD NATIVE APPLICATION DELIVERY

Delivery fundamentals

CI and CD, build once and promote, and the strategies for releasing a new version.

DOMAIN 3 of 4 EXAM WEIGHT 16% MODULE 22 of 35

admin@sthithapragna.com

Delivery is **the path from a commit to production**. The goal is to make it frequent, automatic and boring.

Continuous integration versus continuous delivery

CI

Integrate and verify

- Merge small changes often
- Every change is built automatically
- Tests and scans run on every change
- Output: a tested artefact, such as an image

CD

Get it into environments

- Takes the artefact from CI
- Deploys to staging, then production
- Promotes the same artefact
- Output: a running release

THE TIEBREAKER

CI produces a trustworthy artefact. CD moves that artefact safely into environments.

Continuous delivery versus continuous deployment

CONTINUOUS DELIVERY

Always releasable

- Every change could go to production
- A person approves the final step
- Common where releases need sign-off
- Release is a business decision

CONTINUOUS DEPLOYMENT

Always released

- Every passing change goes to production
- No human approval step
- Needs strong automated tests
- Release is automatic

THE TIEBREAKER

The only difference is the human gate before production. Same initials, CD, for both.

A typical pipeline

01

Commit

A developer merges a small change.

02

Build

CI builds the image, once.

03

Test and scan

Unit tests, vulnerability scan, SBOM, signature.

04

Publish

Push to the registry, tagged with a version.

05

Deploy

Staging first, then promote the same image to production.

ON THE EXAM

Supply chain security from module 19 lives in the test-and-scan stage.

Build once, promote everywhere

- **One image, built once,** runs in every environment. Identified by version tag or digest.
- **Only configuration differs** between environments: ConfigMaps, Secrets, overlays.
- **Never rebuild for production.** A rebuild can pull different dependencies.
- **Never deploy latest.** Module 1 showed why: tags move.

> ON THE EXAM
>
> What you tested must be exactly what you ship. Promote the artefact; do not recreate it.

CI and CD tools you should recognise

TOOL	KNOWN FOR
Jenkins	Long-standing automation server with a huge plugin ecosystem
GitHub Actions, GitLab CI	Pipelines built into the code-hosting platform
Tekton	Kubernetes-native pipelines, defined as custom resources
Argo Workflows	Kubernetes-native workflow engine; part of the graduated Argo project
Argo CD, Flux	GitOps continuous delivery, in module 24

ON THE EXAM

Kubernetes-native tools run each pipeline step as a Pod, and are defined as Kubernetes objects.

Infrastructure as code

- **Infrastructure declared in files,** kept in version control, reviewed like code.
- **Repeatable:** the same files build the same environment every time.
- **Drift is visible:** compare what exists with what the files say.
- **Tools:** Terraform and OpenTofu, Pulumi, and Crossplane, a CNCF project managing cloud resources as Kubernetes objects.

> ON THE EXAM
>
> Kubernetes manifests are infrastructure as code for workloads. The same discipline applies.

Push versus pull deployment

PUSH

The pipeline deploys

- CI runs kubectl or helm against the cluster
- CI holds cluster credentials
- Cluster changes only when a pipeline runs
- Drift goes unnoticed

PULL

The cluster fetches

- An agent in the cluster watches Git
- No cluster credentials outside it
- Changes applied continuously
- Drift is detected and corrected

THE TIEBREAKER

Pull-based delivery is GitOps. Module 24 covers it in depth.

A new version can reach users all at once, gradually, side by side, or to a few first.

Release strategies at a glance

STRATEGY	HOW	TRADE-OFF
Recreate	Stop the old version, then start the new	Simple; causes downtime
Rolling update	Replace Pods gradually	The Deployment default; brief version mix
Blue-green	Run the new version alongside; switch all traffic at once	Instant rollback; double capacity
Canary	Send a small share of users to the new version first	Small blast radius; needs good metrics
A/B testing	Route by user attribute to compare variants	A business experiment, not just a release
Shadow	Copy live traffic to the new version; discard its replies	Real load, no user impact

ON THE EXAM

Deployments give you Recreate and rolling update. The others need extra Services, routing or tools.

Blue-green, step by step

01

Blue is live

All users on version 1, the blue environment.

02

Deploy green

Version 2 runs alongside, taking no traffic.

03

Test green

Smoke tests against green directly.

04

Switch

Point the Service at green. All users move at once.

05

Keep blue

Switch back instantly if needed; retire it later.

ON THE EXAM

The switch is one change, so the rollback is one change too.

Blue-green with plain Kubernetes

Two Deployments, web-blue and web-green, label their Pods track: blue and track: green.

```
apiVersion: v1
kind: Service
metadata: {name: web, namespace: quayside}
spec:
  selector: {app: web, track: green}
  ports: [{port: 80, targetPort: 8080}]

# rollback: set track back to blue
```

5 The switch: one label in the selector moves all traffic.

THE TAKEAWAY

Instant switch and instant rollback, at the cost of running both versions in full.

Canary, step by step

01

5%

A few users get version 2.

02

Watch

Compare errors and latency with version 1.

03

25%

Healthy? Send more.

04

50%

Still healthy? More again.

05

100%

Done. Any bad signal: back to 0%.

ON THE EXAM

A canary limits the damage of a bad release to the few users who saw it.

Canary by replica count versus by traffic weight

REPLICA RATIO

Coarse

- 1 canary Pod among 10 is about 10%
- Share tied to the number of Pods
- Hard to do 1%: you need 100 Pods
- Works with a plain Service

TRAFFIC WEIGHTS

Precise

- Gateway API or mesh weights: exactly 5%
- Share independent of Pod count
- Can route by header, e.g. staff only
- Module 15's HTTPRoute, 90 to 10

THE TIEBREAKER

For precise canaries, split traffic by weight at the Gateway or mesh, not by counting Pods.

Blue-green versus canary

BLUE-GREEN

All at once, instantly reversible

- Every user moves at the same moment
- Rollback is one switch
- Needs a full second environment
- No gradual real-user signal

CANARY

A few first, then more

- Real users exposed gradually
- Bad releases hit few users
- Needs metrics and traffic splitting
- Takes longer to complete

THE TIEBREAKER

'Switch everyone at once, instant rollback': blue-green. 'Expose a few users first': canary.

Progressive delivery: automating the canary

- **Tools step the canary for you,** checking metrics at each stage.
- **Argo Rollouts** replaces a Deployment with a Rollout resource that supports canary and blue-green.
- **Flagger**, from the Flux family, automates canaries with a mesh or gateway.
- **Metrics decide:** error rate or latency above a threshold triggers an automatic rollback.

> ON THE EXAM
>
> 'Canary with automatic rollback on bad metrics': progressive delivery, e.g. Argo Rollouts or Flagger.

Feature flags: deploy is not release

- **Ship code switched off,** then turn features on without a new deployment.
- **Target users:** staff first, then a region, then everyone. Turn off instantly if needed.
- **Separates deployment** (code running) from **release** (users seeing it).
- **OpenFeature**, a CNCF project, is a vendor-neutral API for feature flags.

> ON THE EXAM
>
> Deployment is a technical event; release is a business decision. Flags let them happen at different times.

> At Quayside, every merge to the main branch is built, tested, scanned and then deployed to production automatically.
>
> No person approves any step between the merge and production.

Which practice does this describe?

A Continuous integration, and nothing more

B Continuous deployment

C Continuous delivery with a release gate

D Infrastructure as code

◆ PAUSE AND DECIDE

Which practice does this describe?

A Continuous integration, and nothing more

CI stops at a tested artefact; this goes on to production.

B Continuous deployment

Every passing change reaches production with no human gate.

C Continuous delivery with a release gate

That has a person approving production. This has none.

D Infrastructure as code

About declaring infrastructure, not the release path.

THE SIGNAL

No human approval before production: continuous deployment.

> For the new booking API, Quayside wants every user moved to the new version at the same moment, and an instant way back if anything goes wrong.
>
> It can afford double capacity for an hour during the switch.

Which strategy fits?

A A rolling update with default settings

B A canary, starting at 5% of users

C Recreate: stop old, then start new

D Blue-green, switching the Service at once

◆ PAUSE AND DECIDE

Which strategy fits?

A A rolling update with default settings

Users are on mixed versions during the rollout.

B A canary, starting at 5% of users

Deliberately moves users gradually, not at once.

C Recreate: stop old, then start new

All at once, but with downtime and a slow way back.

D Blue-green, switching the Service at once

One switch forward, one switch back, double capacity.

THE SIGNAL

'Everyone at once, instant rollback, double capacity acceptable': blue-green.

> Quayside's new fare engine is risky. The team wants 5% of real users on it first, then gradually more.
>
> If its error rate rises above the current version's, it must roll back automatically.

What should they use?

A. A canary, automated by Argo Rollouts or similar

B. Blue-green with a manual switch after testing

C. The Recreate strategy on a Deployment

D. Shadow traffic sent to the new fare engine

◆ PAUSE AND DECIDE

What should they use?

A A canary, automated by Argo Rollouts or similar
Gradual real-user exposure with metric-driven rollback.

B Blue-green with a manual switch after testing
Moves everyone at once; no gradual real-user signal.

C The Recreate strategy on a Deployment
Everyone at once, with downtime.

D Shadow traffic sent to the new fare engine
Real users never see its answers, so user errors are not measured.

THE SIGNAL

'A few real users, gradually more, automatic rollback': an automated canary.

> Quayside's pipeline builds the booking API image twice from the same commit: once for staging, once for production.
>
> A production bug never appeared in staging. The production build had pulled a newer version of a library.

Which practice prevents this?

A Rebuild more carefully, pinning the OS packages

B Tag both builds latest so they always match

C Build once, then promote that same image

D Skip staging and test only in production

◆ PAUSE AND DECIDE

Which practice prevents this?

A Rebuild more carefully, pinning the OS packages

Helps, but two builds can still differ. One build cannot.

B Tag both builds latest so they always match

A shared tag hides the difference; it does not remove it.

C Build once, then promote that same image

Production runs exactly the bytes staging tested.

D Skip staging and test only in production

Removes the safety net instead of fixing the pipeline.

THE SIGNAL

Two builds from one commit can differ. Build once, promote by digest.

Module 22 in five lines

- **CI** builds and tests an artefact; **CD** moves it into environments.
- Continuous **delivery** has a human gate; continuous **deployment** does not.
- **Build once, promote the same image;** only configuration differs.
- **Blue-green:** everyone at once, instant rollback. **Canary:** a few first, then more.
- **Progressive delivery** automates canaries; **feature flags** separate deploy from release.

Every environment needs slightly different manifests. Next: **Helm and Kustomize**.

MODULE 23 · DOMAIN 3 - CLOUD NATIVE APPLICATION DELIVERY

Packaging: Helm and Kustomize

Charts, values and releases; bases and overlays; and when to use each.

DOMAIN 3 of 4 EXAM WEIGHT 16% MODULE 23 of 35

admin@sthithapragna.com

Helm **templates** YAML and packages it. Kustomize **patches** plain YAML. Both end as ordinary manifests.

The problem both tools solve

- **One app, many environments:** development, staging, production.
- **Small differences:** replica counts, image tags, hostnames, resources, namespaces.
- **Copying the YAML per environment** drifts: a fix lands in one copy and not the others.
- **The goal:** one source of truth, plus a small, explicit list of differences.

> ON THE EXAM
>
> Copy-and-edit YAML per environment is the anti-pattern both tools replace.

Helm: the package manager for Kubernetes

- **A chart** is a package: templates plus default values. A graduated CNCF project.
- **Values** are the inputs: your settings override the chart's defaults.
- **A release** is one installed copy of a chart, with a numbered revision history.
- **Charts are shared** through repositories and OCI registries; Artifact Hub helps you find them.

> ON THE EXAM
>
> Chart: the package. Values: the settings. Release: an installed instance.

Inside a chart

FILE OR FOLDER	HOLDS
`Chart.yaml`	Name, chart version, and appVersion of the software inside
`values.yaml`	Default values for every setting
`templates/`	Manifests with placeholders, rendered using the values
`charts/`	Other charts this one depends on
`values.schema.json`	Optional: rules that validate supplied values

> ON THE EXAM
>
> version is the chart's own version; appVersion is the version of the application it installs.

A template, and the values that fill it

```
# templates/deployment.yaml (excerpt)
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: api
        image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
---
# values.yaml
replicaCount: 2
image: {repository: quayside/booking-api, tag: 2.4.1}
```

3 A placeholder, filled from the values when the chart is rendered.

11-12 The chart's defaults. Anything you supply overrides them.

The Helm commands you will see

```
$ helm repo add quayside https://charts.quayside.example
$ helm install booking quayside/booking-api -n quayside -f prod-values.yaml
$ helm upgrade booking quayside/booking-api -n quayside --set replicaCount=6
$ helm history booking -n quayside
$ helm rollback booking 3 -n quayside
$ helm uninstall booking -n quayside
```

2 Release name, then chart, then your values file.

3 New values or a new chart version: a new revision.

5 Back to revision 3, recorded as a new revision.

More commands worth knowing

COMMAND OR FLAG	DOES
`helm template`	Renders the manifests locally. Installs nothing
`helm show values`	Prints a chart's defaults, to see what you can set
`helm lint`	Checks a chart for problems
`helm list`	Lists releases in a namespace
`helm upgrade --install`	Installs if missing, upgrades if present
`--rollback-on-failure`	Rolls back automatically if the upgrade fails. Formerly --atomic

ON THE EXAM

'See the exact manifests without installing': helm template.

Which value wins?

- **Lowest:** the chart's own values.yaml.
- **Then:** each -f values file, in order; a later file beats an earlier one.
- **Highest:** --set on the command line.
- **Best practice:** keep environment values in files in Git; keep --set for rare one-offs.

> ON THE EXAM
>
> Chart defaults, then files in order, then --set. The most specific, latest source wins.

Releases and their history

- **Every install, upgrade and rollback** creates a new numbered revision.
- **Revisions are stored as Secrets** in the release's namespace.
- **No server component:** the old in-cluster Tiller was removed back in Helm 3.
- helm uninstall deletes the release's resources, but not claims kept by a StatefulSet.

> ON THE EXAM
>
> A Helm rollback, like a Deployment rollback, is recorded as a new revision.

Currency check: Helm 4

- **Helm 4 went GA on 12 November 2025,** the first major version in six years.
- **Existing charts work:** apiVersion v2 charts and Helm 3 releases carry over unchanged.
- **Renamed flags:** --atomic is now --rollback-on-failure; --force is now --force-replace.
- **New releases use server-side apply** by default.
- **Helm 3:** final feature release 9 September 2026; security fixes end 10 February 2027.

> ON THE EXAM
>
> Old flags still work, with a deprecation warning. Expect either name in questions.

Kustomize keeps **plain YAML**, and describes each environment as **patches on a shared base**.

Kustomize

- **No templates, no new language.** Just YAML and a kustomization.yaml file.
- **A base** holds the common manifests; **overlays** hold each environment's changes.
- **Built into kubectl:** kubectl apply -k, and kubectl kustomize to preview.
- **Common changes:** patches, image tags, namespace, name prefixes, labels, replicas, generated ConfigMaps.

> ON THE EXAM
>
> Kustomize needs no install: kubectl apply -k folder/ builds and applies it.

A base and two overlays

```
booking-api/
  base/         deployment.yaml  service.yaml  kustomization.yaml
  overlays/
    staging/    kustomization.yaml
    prod/       kustomization.yaml  replicas-patch.yaml

# overlays/prod/kustomization.yaml
resources: [../../base]
namespace: quayside
images: [{name: quayside/booking-api, newTag: 2.4.1}]
patches: [{path: replicas-patch.yaml}]
```

8 Start from the shared base.

10-11 Production's differences: its image tag and a replica patch.

THE TAKEAWAY

kubectl apply -k overlays/prod builds base plus production changes, and applies them.

configMapGenerator: config changes that roll out

- **Kustomize generates the ConfigMap** from files or literals.
- **It appends a hash of the contents** to the name: booking-config-7g2k9m.
- **Change the contents, and the name changes,** so the Pod template changes, so a rollout happens.
- Solves module 9's problem: editing a ConfigMap alone never restarts Pods.

> ON THE EXAM
>
> Hashed names turn every config change into a normal, reversible rollout.

Helm versus Kustomize

HELM

Templates and packages

- Templates plus values
- Packages to share and version
- Release history and rollback
- Best for installing others' software

KUSTOMIZE

Patches on plain YAML

- Plain YAML plus patches
- No packaging of its own
- Built into kubectl
- Best for your own apps' environments

THE TIEBREAKER

Installing third-party software: Helm. Your own app across environments, plain YAML: Kustomize. Many teams use both.

Six packaging decisions. For each, ask: **whose software**, and **which differences**?

> Quayside wants to install a widely used open-source monitoring stack, with sensible defaults and a few settings changed.
>
> Later it must upgrade to new versions, and roll back if an upgrade goes wrong.

What is the most practical approach?

A Install the project's Helm chart with a values file

B Write all of the manifests by hand from scratch

C A Kustomize overlay with no base to start from

D kubectl run for each component of the stack

◆ PAUSE AND DECIDE

What is the most practical approach?

A. Install the project's Helm chart with a values file
Vendor defaults, your overrides, versioned upgrades and rollback.

B. Write all of the manifests by hand from scratch
Huge effort, and you own every future upgrade.

C. A Kustomize overlay with no base to start from
An overlay needs a base, which you would have to write.

D. kubectl run for each component of the stack
Creates bare Pods: no Services, no upgrades.

THE SIGNAL

Someone else's software, upgrades and rollback: a Helm chart.

> A chart's values.yaml sets replicaCount: 2. The team installs it with -f prod-values.yaml, which sets replicaCount: 4.
>
> They also pass --set replicaCount=6 on the command line.

How many replicas are created?

A 2, from the chart's own defaults

B 4, from the production values file

C 6, from the --set flag

D None: Helm rejects the conflict

◆ PAUSE AND DECIDE

How many replicas are created?

A 2, from the chart's own defaults

Chart defaults have the lowest priority.

B 4, from the production values file

Files beat defaults, but --set beats files.

C 6, from the --set flag

--set has the highest priority.

D None: Helm rejects the conflict

Overrides are normal; nothing conflicts.

THE SIGNAL

Defaults, then -f files, then --set. --set wins.

> Quayside's own booking API runs in development, staging and production. The only differences are replica counts, image tags and namespaces.
>
> The team wants plain YAML, no templating language, and nothing extra to install beyond kubectl.

Which approach fits?

A Write a Helm chart with a values file per environment

B A Kustomize base with one overlay per environment

C Copy the YAML folder once for each environment

D One set of YAML, edited by hand before each deploy

◆ PAUSE AND DECIDE

Which approach fits?

A Write a Helm chart with a values file per environment

Works, but adds a templating language they wanted to avoid.

B A Kustomize base with one overlay per environment

Plain YAML, small patches, built into kubectl.

C Copy the YAML folder once for each environment

Copies drift: fixes land in one and not the others.

D One set of YAML, edited by hand before each deploy

Error-prone, and nothing records the differences.

THE SIGNAL

Own app, plain YAML, small differences, kubectl only: Kustomize.

> A Helm 4 upgrade of the booking API failed halfway: the new Pods crash and the release is left in a broken state.
>
> For future upgrades, the team wants Helm to roll back automatically whenever an upgrade fails.

Which flag should the pipeline add?

A --force-replace

B helm template instead of upgrade

C --dry-run

D --rollback-on-failure

◆ PAUSE AND DECIDE

Which flag should the pipeline add?

A --force-replace

Replaces resources forcibly; it does not roll back.

B helm template instead of upgrade

Renders manifests; it deploys nothing at all.

C --dry-run

Previews the upgrade without doing it.

D --rollback-on-failure

Formerly --atomic: roll back if the upgrade fails.

THE SIGNAL

Automatic rollback on a failed upgrade: --rollback-on-failure, once called --atomic.

> Before merging a change, a reviewer wants to see the exact manifests a chart will produce with the team's production values.
>
> Nothing may be installed or changed in any cluster.

Which command fits?

A helm install with the production values

B helm history for the release

C helm template with the production values

D helm lint on the chart folder

◆ PAUSE AND DECIDE

Which command fits?

A helm install with the production values
Installs the release, which is not allowed.

B helm history for the release
Lists past revisions, not future manifests.

C helm template with the production values
Renders the manifests locally; installs nothing.

D helm lint on the chart folder
Reports problems, but is not a full manifest review.

THE SIGNAL

'Exact manifests, install nothing': helm template.

> The team uses a Kustomize configMapGenerator for the booking API's settings file. They change one setting and run kubectl apply -k.
>
> Every booking API Pod is replaced by a rolling update, although nobody touched the Deployment.

Why did the Pods roll?

A Its name has a content hash, so the Pod template changed

B The kubelet restarts Pods whenever a ConfigMap changes

C Kustomize deletes Pods directly after every apply

D ConfigMaps are immutable, so all Pods were recreated

◆ PAUSE AND DECIDE

Why did the Pods roll?

A Its name has a content hash, so the Pod template changed

New contents, new name, new template: a normal rollout.

B The kubelet restarts Pods whenever a ConfigMap changes

Module 9: ConfigMap edits alone never restart Pods.

C Kustomize deletes Pods directly after every apply

Kustomize only builds manifests; it never deletes Pods.

D ConfigMaps are immutable, so all Pods were recreated

Immutability is opt-in, and would not cause a rollout.

THE SIGNAL

configMapGenerator hashes the contents into the name, so config changes roll out.

Module 23 in five lines

- **Helm:** charts (templates + values), releases with revisions and rollback. CNCF graduated.
- **Value priority:** chart defaults, then -f files in order, then --set.
- **helm template** renders without installing; **--rollback-on-failure** was --atomic.
- **Helm 4** GA November 2025; Helm 3 security fixes end February 2027.
- **Kustomize:** plain YAML, base plus overlays, kubectl apply -k; generators hash names.

Charts and overlays live in Git. Next: GitOps, where Git itself drives the cluster.

admin@sthithapragna.com

MODULE 24 · DOMAIN 3 - CLOUD NATIVE APPLICATION DELIVERY

GitOps

The four principles, Argo CD and ApplicationSets, Flux, drift, and the mistakes to avoid.

DOMAIN 3 of 4 EXAM WEIGHT 16% MODULE 24 of 35

admin@sthithapragna.com

In GitOps, **Git is the source of truth**, and **an agent in the cluster** keeps reality matching it.

The four OpenGitOps principles

PRINCIPLE	MEANS
Declarative	The desired state is expressed declaratively, not as steps
Versioned and immutable	Stored so that every version is kept, with complete history
Pulled automatically	Agents pull the desired state from the source
Continuously reconciled	Agents keep observing, and correct any difference

ON THE EXAM

Defined by the OpenGitOps project, part of the CNCF. Pushed by a pipeline is not one of them.

The GitOps loop

01
Propose
A change is made in Git, as a pull request.

02
Review
Colleagues review and approve; it is merged.

03
Notice
The agent sees the new commit.

04
Compare
Git's desired state versus the cluster's actual state.

05
Apply
Differences are applied; the agent keeps watching.

ON THE EXAM

Every change to production is a reviewed, recorded commit.

Why teams adopt GitOps

- **Audit:** git log shows who changed what, when, and who approved it.
- **Rollback:** revert the commit, and the cluster follows.
- **Security:** nothing outside the cluster holds cluster credentials.
- **Drift correction:** manual changes are detected, and can be reverted.
- **Recovery:** a lost cluster can be rebuilt from the repository.

> ON THE EXAM
>
> Most GitOps benefits come from one fact: the cluster's desired state is a Git repository.

Argo CD

- **A graduated CNCF project,** part of the Argo family.
- **An Application** links a source in Git to a destination cluster and namespace.
- **Understands plain YAML, Kustomize and Helm** as sources.
- **A web UI and CLI** show every application's sync and health status.

> ON THE EXAM
>
> Argo CD's core object is the Application: what to deploy, from where, to where.

An Argo CD Application for the booking API

```
 1 apiVersion: argoproj.io/v1alpha1
 2 kind: Application
 3 metadata: {name: booking-api-prod, namespace: argocd}
 4 spec:
 5   project: default
 6   source:
 7     repoURL: https://git.quayside.example/platform/deploy.git
 8     targetRevision: main
 9     path: booking-api/overlays/prod
10   destination: {server: https://kubernetes.default.svc, namespace: quayside}
11   syncPolicy:
12     automated: {prune: true, selfHeal: true}
```

9 Module 23's Kustomize overlay, straight from Git.

10 Where to deploy: this cluster, the quayside namespace.

12 Sync automatically, delete what Git removes, revert drift.

Sync status versus health status

STATUS	VALUES	ANSWERS
Sync	Synced, OutOfSync	Does the cluster match Git?
Health	Healthy, Progressing, Degraded, Missing	Is the application actually working?

ON THE EXAM

Synced but Degraded: the cluster matches Git, and what Git describes is broken. Fix it in Git.

prune versus selfHeal

PRUNE

Deletes what Git removed

- A manifest is deleted from Git
- The matching object is deleted in the cluster
- Without it: the object lingers
- Stops leftovers building up

SELFHEAL

Reverts manual changes

- Someone edits an object in the cluster
- Argo CD changes it back to match Git
- Without it: drift stays until the next commit
- Makes Git the only way to change things

THE TIEBREAKER

Both are off by default, even with automated sync. Turn them on deliberately.

ApplicationSets: Applications from a template

- **An ApplicationSet** is a template plus generators that produce Applications.
- **List generator:** a fixed list, such as three environments.
- **Cluster generator:** one Application per registered cluster.
- **Git generator:** one Application per folder or file in a repository.
- **Matrix and merge** combine generators; a pull request generator builds preview environments.

> ON THE EXAM
>
> 'An Application for every folder, including ones added later': an ApplicationSet with a Git generator.

One Application per service, automatically

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata: {name: quayside-prod, namespace: argocd}
spec:
  generators:
  - git:
      repoURL: https://git.quayside.example/platform/deploy.git
      revision: main
      directories: [{path: "*/overlays/prod"}]
  template:
    metadata: {name: "{{path[0]}}-prod"}
    # spec: source path "{{path}}", destination as before
```

9 Every folder matching this pattern becomes an Application.

11 Named from the folder: booking-api-prod, web-prod, and so on.

THE TAKEAWAY

Add a new service's folder in Git, and its Application appears. Delete it, and the Application goes.

Flux

- **A graduated CNCF project,** built as a set of controllers: the GitOps Toolkit.
- **source-controller** fetches Git repositories, OCI artefacts and Helm repositories.
- **kustomize-controller** applies manifests, through a Flux Kustomization object.
- **helm-controller** manages HelmRelease objects; others handle notifications and image updates.

> ON THE EXAM
>
> A Flux Kustomization is a custom resource, not the kustomization.yaml file. The names catch people out.

Argo CD versus Flux

ARGO CD

Application-centred, with a UI

- Application and ApplicationSet objects
- Rich web UI for many clusters
- Central instance can manage many clusters
- Plain YAML, Kustomize, Helm

FLUX

A toolkit of controllers

- GitRepository, Kustomization, HelmRelease
- CLI-first; UIs come from elsewhere
- Often runs inside each cluster
- Built-in image update automation

THE TIEBREAKER

Both are graduated, pull-based GitOps. Pick by the detail: UI or ApplicationSet means Argo CD; HelmRelease or toolkit means Flux.

Promotion and rollback, the GitOps way

- **Promote** with a pull request that changes the image tag in the production overlay.
- **Roll back** with git revert. The agent syncs the previous state.
- **Not with kubectl rollout undo:** self-heal puts the bad version straight back.
- **Automate tag bumps** with Argo CD Image Updater or Flux image automation.

> ON THE EXAM
>
> In GitOps, every change goes through Git, including emergencies and rollbacks.

Secrets in a GitOps repository

- **Never commit plain Secrets.** Base64 is not encryption, as module 9 showed.
- **Sealed Secrets:** encrypt for the cluster; only its controller can decrypt.
- **SOPS:** encrypted files in Git, decrypted by Flux or a plugin at apply time.
- **External Secrets:** Git holds a reference; values come from Vault or a cloud secret manager.

> ON THE EXAM
>
> Git holds everything except secret values, which are encrypted or fetched from a secret store.

GitOps anti-patterns

- **kubectl edit in production:** reverted by self-heal, or left as silent drift.
- **Two writers:** a CI pipeline also running kubectl apply against the same cluster.
- **A long-lived branch per environment:** merges get painful and branches drift apart.
- **Prefer one branch, a folder or overlay per environment,** promoted by pull request.

> ON THE EXAM
>
> One source of truth, one writer. Anything else is not really GitOps.

Six GitOps decisions. For each, ask: **what does Git say**, and **what will the agent do about it**?

> Quayside's platform team is writing an internal guide to GitOps, listing the four OpenGitOps principles.
>
> A reviewer spots that one item on the draft list is not a principle.

Which item is not an OpenGitOps principle?

A Desired state is expressed declaratively

B Desired state is pulled automatically

C The system is continuously reconciled

D Changes are pushed by the CI pipeline

◆ PAUSE AND DECIDE

Which item is not an OpenGitOps principle?

A Desired state is expressed declaratively

Principle 1: declarative.

B Desired state is pulled automatically

Principle 3: pulled automatically.

C The system is continuously reconciled

Principle 4: continuously reconciled.

D Changes are pushed by the CI pipeline

The opposite of GitOps: agents pull; pipelines do not push.

THE SIGNAL

Declarative, versioned and immutable, pulled, reconciled. Pushed is the odd one out.

> During an incident, an engineer runs kubectl scale to raise the booking API from 3 to 10 replicas in production.
>
> Within a minute it is back to 3. The Argo CD Application uses automated sync with selfHeal: true. No autoscaler exists.

Why did the replica count return to 3?

A An autoscaler scaled it back down

B Argo CD self-healed the drift from Git

C The Deployment controller reverted it

D etcd restored an earlier snapshot

◆ PAUSE AND DECIDE

Why did the replica count return to 3?

A An autoscaler scaled it back down

The scenario says no autoscaler exists.

B Argo CD self-healed the drift from Git

Git says 3; self-heal reverts the manual change.

C The Deployment controller reverted it

It makes Pods match the spec; it never edits the spec.

D etcd restored an earlier snapshot

etcd never restores state on its own.

THE SIGNAL

Manual change reverted within a minute, selfHeal on: drift correction. Change Git instead.

> The team deletes the old nightly-report CronJob's manifest from Git. Argo CD applies other changes automatically.
>
> But the CronJob still runs every night, and Argo CD marks it as needing to be pruned.

Which setting makes Argo CD delete such objects automatically?

A prune: true in the automated sync policy

B selfHeal: true in the automated sync policy

C targetRevision: HEAD instead of main

D A Replace=true sync option

◆ PAUSE AND DECIDE

Which setting makes Argo CD delete such objects automatically?

A prune: true in the automated sync policy

Deletes objects whose manifests were removed from Git.

B selfHeal: true in the automated sync policy

Reverts edits to existing objects; it does not delete removed ones.

C targetRevision: HEAD instead of main

Changes which commit is tracked, not deletion.

D A Replace=true sync option

Changes how objects are applied, not whether they are deleted.

THE SIGNAL

Removed from Git but still in the cluster: turn on prune.

> Quayside has thirty microservices, each with a production overlay folder in the deployment repository.
>
> It wants one Argo CD Application per service, created automatically, including for services added next year.

What should the team use?

A Thirty Applications, written and maintained by hand

B One Application pointing at the repository root

C A CronJob that runs kubectl apply on every folder

D An ApplicationSet with a Git directory generator

◆ PAUSE AND DECIDE

What should the team use?

A Thirty Applications, written and maintained by hand

Works today, but every new service needs a new file.

B One Application pointing at the repository root

One giant app: no per-service health, sync or rollback.

C A CronJob that runs kubectl apply on every folder

Push-style, no reconciliation: not GitOps.

D An ApplicationSet with a Git directory generator

A folder appears in Git; its Application appears too.

THE SIGNAL

'One Application per folder, including future ones': ApplicationSet, Git generator.

> A bad release of the web front end reached production through Argo CD, with selfHeal enabled.
>
> The team wants to return to the previous version quickly, in a way that sticks.

What should they do?

A. Run kubectl rollout undo on the web Deployment

B. Delete the Argo CD Application for web

C. Revert the release commit in Git

D. Edit the image tag directly in the cluster

◆ PAUSE AND DECIDE

What should they do?

A Run kubectl rollout undo on the web Deployment

Self-heal sees drift and puts the bad version back.

B Delete the Argo CD Application for web

Stops management, and may delete the running app.

C Revert the release commit in Git

Git goes back; the agent syncs the old version.

D Edit the image tag directly in the cluster

Drift again: reverted within a minute.

THE SIGNAL

GitOps rollback: git revert. Anything done only in the cluster is undone.

> The Argo CD dashboard shows the booking API Application as Synced, but Degraded.
>
> The on-call engineer asks what that combination means.

What does it mean?

A The cluster and Git differ, and Argo CD will fix it

B The cluster matches Git, but the app is unhealthy

C Argo CD cannot reach the Git repository

D Some resources defined in Git are missing

◆ PAUSE AND DECIDE

What does it mean?

A The cluster and Git differ, and Argo CD will fix it

That would be OutOfSync.

B The cluster matches Git, but the app is unhealthy

Git was applied faithfully; what Git describes is broken.

C Argo CD cannot reach the Git repository

That shows as a sync error, not Degraded.

D Some resources defined in Git are missing

That shows as health Missing, or OutOfSync.

THE SIGNAL

Synced plus Degraded: the problem is in what Git says. Fix it in Git.

Module 24 in five lines

- OpenGitOps: **declarative, versioned and immutable, pulled automatically, continuously reconciled**.
- **Argo CD:** Applications, sync versus health, prune and selfHeal (both off by default).
- **ApplicationSets** generate Applications: list, cluster, Git, matrix, pull request.
- **Flux:** GitOps Toolkit controllers, Kustomization and HelmRelease objects.
- **Roll back with git revert;** encrypt or externalise secrets; one writer only.

GitOps deploys it. Next: **debugging applications** when what you deployed misbehaves.

MODULE 25 · DOMAIN 3 - CLOUD NATIVE APPLICATION DELIVERY

Debugging applications

Probes, logs, events, exec, ephemeral debug containers, port-forward and top.

DOMAIN 3 of 4 EXAM WEIGHT 16% MODULE 25 of 35

admin@sthithapragna.com

The kubelet asks your app three questions: **have you started**, **are you alive**, and **are you ready**?

The three probes

PROBE	ASKS	IF IT FAILS
`startupProbe`	Has the app finished starting?	Keeps retrying; after its threshold, the container is restarted
`livenessProbe`	Is the app still alive, not stuck?	The container is restarted
`readinessProbe`	Can it take traffic right now?	Removed from Service endpoints. Not restarted

ON THE EXAM

Liveness restarts. Readiness removes from traffic. Startup holds the other two back until the app is up.

Liveness versus readiness

LIVENESS

Restart me

- For states only a restart fixes
- Deadlocks, a wedged event loop
- Checks only the process itself
- Failure: container restarted

READINESS

Do not send me traffic

- For temporary unavailability
- Warming up, overloaded, a dependency down
- May check what it needs to serve
- Failure: removed from endpoints

THE TIEBREAKER

If a restart would not fix it, it belongs in readiness, not liveness.

How probes check, and their defaults

MECHANISM OR SETTING	DETAIL
`httpGet`	Success on any 2xx or 3xx response
`tcpSocket`	Success if a TCP connection opens
`exec`	Success if the command exits with 0
`grpc`	Uses the standard gRPC health check
`periodSeconds / timeoutSeconds`	Defaults: every 10 seconds, 1-second timeout
`failureThreshold`	Default 3 failures in a row before acting

ON THE EXAM

Liveness with defaults acts after about 30 seconds of failure: three checks, ten seconds apart.

All three probes on the booking API

```
containers:
- name: api
  image: quayside/booking-api:2.4.1
  startupProbe:
    httpGet: {path: /healthz, port: 8080}
    failureThreshold: 30
    periodSeconds: 5
  livenessProbe:
    httpGet: {path: /healthz, port: 8080}
  readinessProbe:
    httpGet: {path: /ready, port: 8080}
```

6-7 Up to 30 x 5 = 150 seconds to start, before liveness begins.

9 Liveness: is the process healthy? Default timings once started.

11 Readiness: a separate endpoint that may check dependencies.

Probe mistakes that cause outages

- **Liveness checks a dependency.** The database blips, and every Pod restarts at once.
- **Slow start, no startup probe.** Liveness kills the app before it is ready: a crash loop.
- **No readiness probe.** Traffic reaches Pods before they can serve, on every rollout.
- **Heavy probe endpoints, checked too often,** add load and fail under pressure.

> ON THE EXAM
>
> A probe that checks the wrong thing is worse than no probe at all.

Look before you touch: logs and events first, then step inside only if you must.

kubectl logs, and the flags that matter

COMMAND	SHOWS
`kubectl logs pod`	The container's standard output and error
`-f`	Follow: keep streaming new lines
`-c api`	A specific container, in a multi-container Pod
`--previous (-p)`	The last instance, before it crashed
`--since=10m, --tail=100`	Only recent lines
`-l app=web`	Logs from every Pod with that label

ON THE EXAM

Crashed container: --previous. Multi-container Pod: -c. Both at once is common.

What logs can and cannot tell you

- **Only standard output and standard error.** Apps should log there, not to files.
- **Stored on the node and rotated.** Old lines disappear.
- **Gone with the Pod,** or the node. The previous instance is the furthest back you get.
- **For history,** ship logs to a central store: module 26.

> ON THE EXAM
>
> kubectl logs is for now. Central logging is for yesterday.

Events

- **kubectl describe** shows the events for one object, at the bottom.
- **kubectl get events -n quayside --sort-by=.lastTimestamp** shows a namespace's recent events.
- **Warning events** flag trouble: failed scheduling, failed pulls, failed probes, back-offs.
- **Kept for about an hour by default,** then deleted. Yesterday's events are gone.

> ON THE EXAM
>
> Events are the cluster's short-term memory. For anything older, you need monitoring and logs.

kubectl exec versus kubectl debug

KUBECTL EXEC

Use what is already there

- Runs a command in an existing container
- Needs the tool, even a shell, in the image
- Fails on distroless images
- Quick, when the tools exist

KUBECTL DEBUG

Bring your own tools

- Adds an ephemeral container to a running Pod
- Uses any image, such as busybox
- Works on distroless images
- Can see the app's processes with --target

THE TIEBREAKER

No shell in the image: kubectl debug with an ephemeral container.

Debugging a distroless container

```
$ kubectl exec -it ticket-validator-6c9d-x7k2p -- sh
error: exec: "sh": executable file not found in $PATH

$ kubectl debug -it ticket-validator-6c9d-x7k2p --image=busybox:1.37 --target=validator
/ # ps
PID   USER     COMMAND
    1 65532    /validator
```

2 Distroless: there is no shell to exec into.

4 An ephemeral busybox container, sharing the validator's process namespace.

7 The app itself, visible from the debug container.

Three ways to use kubectl debug

MODE	CREATES	USE WHEN
`kubectl debug pod --image=... --target=app`	An ephemeral container in the running Pod	Inspect a live Pod without changing it
`kubectl debug pod --copy-to=pod-debug ...`	A copy of the Pod, with changes such as a new image or command	The Pod crashes too fast to attach to
`kubectl debug node/node-3 -it --image=...`	A Pod on that node, using its namespaces; its filesystem at /host	Investigate the node itself

ON THE EXAM

Crash-looping too fast to attach? Debug a copy, with its command changed to a shell.

port-forward, top and cp

- **kubectl port-forward svc/booking-api 8080:80** reaches it at localhost:8080 on your laptop.
- **Nothing is exposed:** no Service type change, no Ingress. It tunnels through the API server.
- **kubectl top pods / nodes:** current CPU and memory, from metrics-server. Not history.
- **kubectl cp:** copies files to or from a container. Needs tar in the image.

> ON THE EXAM
>
> 'Reach it from my laptop without exposing it': kubectl port-forward.

Six debugging decisions. For each, ask: **which question is failing**, and **which tool answers it**?

> A new Java reporting service takes about 90 seconds to start. It has a liveness probe with default settings and no startup probe.
>
> It never becomes ready: every Pod is restarted after about 30 seconds, and ends in CrashLoopBackOff.

What is the best fix?

A. Remove the readiness probe from the container

B. Raise the CPU limit so it might start faster

C. Add a startup probe that allows enough time

D. Set the Pod's restartPolicy to Never

◆ PAUSE AND DECIDE

What is the best fix?

A Remove the readiness probe from the container
Readiness never restarts anything.

B Raise the CPU limit so it might start faster
May help a little; guarantees nothing.

C Add a startup probe that allows enough time
Holds liveness back until the app has started.

D Set the Pod's restartPolicy to Never
Stops restarts, but the app is still killed and never serves.

THE SIGNAL

Slow start killed by liveness: add a startup probe.

> During a two-minute database maintenance window, every booking API Pod was restarted, and the outage lasted fifteen minutes.
>
> The liveness probe's endpoint checks the database connection.

What change prevents this?

A Keep liveness to the process; check the DB in readiness

B Raise the liveness failureThreshold to 100 checks

C Remove every probe from the booking API

D Add more replicas of the booking API

◆ PAUSE AND DECIDE

What change prevents this?

A Keep liveness to the process; check the DB in readiness

A database blip then only removes Pods from traffic, briefly.

B Raise the liveness failureThreshold to 100 checks

Masks real deadlocks for many minutes.

C Remove every probe from the booking API

Loses restarts and traffic control entirely.

D Add more replicas of the booking API

More replicas would all restart at once too.

THE SIGNAL

Liveness checking a dependency causes cascading restarts. Move it to readiness.

> The distroless ticket validator is misbehaving in production. kubectl exec fails: there is no shell in the image.
>
> An engineer needs to inspect its processes and network from inside the running Pod, without restarting it.

What should they use?

A Rebuild the image with a shell, and redeploy it

B kubectl logs -f on the validator container

C kubectl port-forward to the validator Pod

D kubectl debug, an ephemeral container with --target

◆ PAUSE AND DECIDE

What should they use?

A Rebuild the image with a shell, and redeploy it
Replaces the misbehaving Pod, and the evidence with it.

B kubectl logs -f on the validator container
Shows output, but gives no view inside the container.

C kubectl port-forward to the validator Pod
Reaches its port; still no view of processes.

D kubectl debug, an ephemeral container with --target
Adds a tool container to the live Pod; no restart.

THE SIGNAL

No shell, must not restart, need to look inside: kubectl debug.

> The booking API Pod runs two containers: api and a log-shipper sidecar. The api container crashed and has just restarted.
>
> The engineer needs the api container's output from just before the crash.

Which command gives exactly that?

A kubectl logs booking-api-pod -f

B kubectl logs booking-api-pod -c api --previous

C kubectl describe pod booking-api-pod

D kubectl logs booking-api-pod --all-containers

◆ PAUSE AND DECIDE

Which command gives exactly that?

A kubectl logs booking-api-pod -f

Needs a container name, and shows only the current instance.

B kubectl logs booking-api-pod -c api --previous

The api container's previous, crashed instance.

C kubectl describe pod booking-api-pod

Shows events and exit codes, not the app's output.

D kubectl logs booking-api-pod --all-containers

Current instances only, mixed with the sidecar's lines.

THE SIGNAL

Crashed container in a multi-container Pod: -c plus --previous.

> A developer wants to test the staging booking API from their laptop for an hour.
>
> It must not be exposed outside the cluster, and nothing about the Service may change.

What should they use?

A. kubectl port-forward svc/booking-api 8080:80

B. Change the Service to type LoadBalancer

C. Add an Ingress rule for the booking API

D. Change the Service to type NodePort

◆ PAUSE AND DECIDE

What should they use?

A kubectl port-forward svc/booking-api 8080:80
A private tunnel for as long as the command runs.

B Change the Service to type LoadBalancer
Exposes it publicly and changes the Service.

C Add an Ingress rule for the booking API
Exposes it through the shared entrance.

D Change the Service to type NodePort
Opens it on every node, and changes the Service.

THE SIGNAL

'From my laptop, not exposed, nothing changed': port-forward.

> An engineer investigates a Pod that failed repeatedly yesterday afternoon. Today, kubectl get events shows nothing about it.
>
> The Pod still exists, and the engineer's permissions have not changed.

Why are the events missing?

A RBAC hides events older than one day

B Events are only recorded for Deployments

C Events expire after about an hour by default

D Events are cleared whenever kubectl restarts

◆ PAUSE AND DECIDE

Why are the events missing?

A RBAC hides events older than one day
Permissions did not change; RBAC has no age filter.

B Events are only recorded for Deployments
Pods have events too, as describe shows.

C Events expire after about an hour by default
They were deleted long ago. Use central logs and metrics.

D Events are cleared whenever kubectl restarts
kubectl holds no state; events live in the cluster.

THE SIGNAL

Yesterday's events are gone: they are short-term memory. History needs monitoring.

Six mixed decisions from **delivery, packaging, GitOps and debugging**, the whole domain in one pass.

> Quayside installs the same booking API chart twice, each with its own values file.
>
> One release is booking-staging, in the staging namespace. The other is booking-prod, in production.

In Helm terms, what are booking-staging and booking-prod?

A Two separate charts

B Two releases of one chart

C Two chart repositories

D Two revisions of one release

◆ PAUSE AND DECIDE

In Helm terms, what are booking-staging and booking-prod?

A Two separate charts

There is one chart, installed twice.

B Two releases of one chart

Each installed copy of a chart is a release.

C Two chart repositories

Repositories hold charts; nothing was added.

D Two revisions of one release

Revisions belong to one release over time.

THE SIGNAL

One chart, installed twice: two releases.

> The web front end has 9 stable Pods behind its Service. The team adds 1 canary Pod with the same labels, so the Service selects all 10.
>
> No gateway or mesh weights are configured.

Roughly what share of requests reaches the canary?

A About half, since there are two versions

B About 1%, the usual canary share

C None, until the canary is promoted

D About 10%, one Pod in ten

◆ PAUSE AND DECIDE

Roughly what share of requests reaches the canary?

A About half, since there are two versions
Services balance across Pods, not versions.

B About 1%, the usual canary share
Nothing sets 1%; the share follows Pod count.

C None, until the canary is promoted
The Service selects it, so it gets traffic now.

D About 10%, one Pod in ten
One of ten matching Pods: roughly a tenth.

THE SIGNAL

Canary by replica ratio: the share follows the Pod count.

> Quayside's second cluster uses Flux. The team wants to install a Helm chart declaratively, with its values kept in Git.
>
> Flux should then install and upgrade it automatically.

Which Flux object describes the chart and its values?

A. A Flux Kustomization

B. A HelmRelease

C. A GitRepository

D. An Argo CD Application

◆ PAUSE AND DECIDE

Which Flux object describes the chart and its values?

A. A Flux Kustomization
Applies manifests from a path; not a Helm chart.

B. A HelmRelease
The helm-controller's object: chart plus values.

C. A GitRepository
Where source files come from, not what to install.

D. An Argo CD Application
Argo CD's object, not Flux's.

THE SIGNAL

Flux plus a Helm chart: HelmRelease.

> Every rolling update of the web front end causes about twenty seconds of errors for some users.
>
> New Pods receive traffic the moment they start, but need about fifteen seconds to load their caches. There is no readiness probe.

What change fixes this?

A Increase maxSurge on the Deployment

B Switch the strategy to Recreate

C Add a liveness probe to the container

D Add a readiness probe to the container

◆ PAUSE AND DECIDE

What change fixes this?

A Increase maxSurge on the Deployment

More new Pods at once: more errors, not fewer.

B Switch the strategy to Recreate

Causes a full outage on every deploy instead.

C Add a liveness probe to the container

Restarts stuck containers; does not gate traffic.

D Add a readiness probe to the container

Traffic waits until each new Pod is ready.

THE SIGNAL

Errors during rollouts from Pods not yet warm: add readiness.

> Quayside runs Argo CD with six registered clusters, and adds more every quarter.
>
> The log agent must be deployed to every registered cluster, including ones added in future.

Which ApplicationSet generator fits?

A The cluster generator

B The list generator

C The Git directory generator

D The pull request generator

◆ PAUSE AND DECIDE

Which ApplicationSet generator fits?

A **The cluster generator**
One Application per registered cluster, new ones included.

B **The list generator**
A fixed list: new clusters need manual edits.

C **The Git directory generator**
One per folder in Git, not per cluster.

D **The pull request generator**
One per open pull request, for previews.

THE SIGNAL

'Every registered cluster, including future ones': cluster generator.

> Quayside's GitOps repository uses Kustomize: a shared base, and an overlay per environment that pins the image tag.
>
> Staging already runs booking API 2.5.0 successfully. Now production should move to 2.5.0.

What is the GitOps way to promote it?

A Run kubectl set image on the production Deployment

B Rebuild the image and retag it as latest

C A pull request changing newTag in the prod overlay

D Change the image tag in the shared base

◆ PAUSE AND DECIDE

What is the GitOps way to promote it?

A Run kubectl set image on the production Deployment

Drift: self-heal reverts it, and Git never records it.

B Rebuild the image and retag it as latest

Breaks build once, promote; latest moves.

C A pull request changing newTag in the prod overlay

A reviewed, recorded, revertable change.

D Change the image tag in the shared base

Changes every environment at once, not just prod.

THE SIGNAL

Promote by changing the production overlay in Git, through review.

Module 25 in five lines

- **Startup** holds the others back; **liveness** restarts; **readiness** removes from traffic.
- Liveness checks **only the process**; dependencies go in **readiness**. Defaults: 10s, 1s, 3.
- **logs -c name --previous** for a crashed container; logs vanish with the Pod.
- **kubectl debug** adds an ephemeral container: works on distroless, no restart.
- **port-forward** reaches it privately; events last **about an hour**.

That completes Application Delivery. Final domain: **Cloud Native Architecture**, starting with observability.

MODULE 26 · DOMAIN 4 - CLOUD NATIVE ARCHITECTURE

Observability

Metrics, logs and traces: Prometheus, Grafana, OpenTelemetry, Jaeger, Fluent Bit, and SLOs.

DOMAIN 4 of 4 EXAM WEIGHT 12% MODULE 26 of 35

admin@sthithapragna.com

Monitoring tells you **whether** something is wrong. Observability lets you ask **why**.

The three signals

SIGNAL	IS	BEST FOR
Metrics	Numbers sampled over time	Dashboards, trends, alerts. Cheap to keep
Logs	Timestamped records of events	Detail about what happened, and debugging
Traces	One request's path through every service	Finding which service made a request slow

ON THE EXAM

Metrics say something is wrong; traces say where; logs say what exactly happened.

Prometheus

- **The CNCF's second graduated project,** after Kubernetes itself.
- **Pulls metrics:** scrapes each target's /metrics endpoint over HTTP, on an interval.
- **Stores time series** in its own database, and queries them with PromQL.
- **Finds targets automatically** through Kubernetes service discovery.

> ON THE EXAM
>
> Prometheus pulls. That single design choice explains most Prometheus exam questions.

How the Prometheus stack fits together

01
Targets
Apps and exporters expose /metrics.

02
Scrape
Prometheus pulls every 15 to 60 seconds.

03
Store
Time series database, queried with PromQL.

04
Alert
Rules fire; Alertmanager groups, routes, silences.

05
Visualise
Grafana dashboards query Prometheus.

ON THE EXAM

Prometheus decides that an alert fires. Alertmanager decides who hears about it.

Pull versus push

PULL (PROMETHEUS'S DEFAULT)

Prometheus scrapes

- Prometheus controls timing
- A failed scrape shows the target is down
- Targets need no idea where Prometheus is
- Suits long-running services

PUSH

The job sends

- The workload sends its own metrics
- Needed when the job ends before a scrape
- Pushgateway holds them for scraping
- Also how OTLP senders work

THE TIEBREAKER

A batch job that finishes before the next scrape pushes to the Pushgateway.

The four metric types

TYPE	BEHAVES LIKE	EXAMPLE
Counter	Only ever goes up; resets to zero on restart	http_requests_total
Gauge	Goes up and down	memory in use, queue length
Histogram	Counts observations into buckets	request duration buckets
Summary	Pre-calculated quantiles in the client	request duration, 0.95 quantile

ON THE EXAM

Never graph a raw counter. Use rate() to turn it into 'per second'.

PromQL you should be able to read

```
1 # requests per second over the last 5 minutes, per service
2 sum by (service) (rate(http_requests_total[5m]))
3
4 # share of booking API requests that fail with a 5xx code
5 sum(rate(http_requests_total{service="booking-api",code=~"5.."}[5m]))
6   / sum(rate(http_requests_total{service="booking-api"}[5m]))
7
8 # 95th percentile latency, calculated from a histogram
9 histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

2 rate() over a counter, then summed per service.

5-6 Errors divided by all requests: the error ratio.

9 A percentile from histogram buckets, grouped by le.

Where Kubernetes metrics come from

SOURCE	PROVIDES
`node_exporter`	Node CPU, memory, disk and network; runs as a DaemonSet
`kube-state-metrics`	Object state: desired versus available replicas, Pod phases
`cAdvisor, via the kubelet`	Per-container CPU and memory use
`Component /metrics`	The API server, scheduler, etcd and others, in Prometheus format
`metrics-server`	Current CPU and memory, for kubectl top and autoscalers

ON THE EXAM

Usage from cAdvisor; object state from kube-state-metrics. Different questions, different sources.

metrics-server versus Prometheus

METRICS-SERVER

Right now, CPU and memory

- Current values only, held in memory
- CPU and memory only
- Serves the metrics API: metrics.k8s.io
- Feeds kubectl top and autoscalers

PROMETHEUS

Anything, over time

- Stores history
- Any metric at all
- Queries with PromQL, alerts with rules
- Feeds dashboards and alerting

THE TIEBREAKER

kubectl top or HPA not working: metrics-server. History, dashboards or alerts: Prometheus.

Around Prometheus

- **Alertmanager** groups, deduplicates, routes and silences alerts.
- **Grafana** draws dashboards from Prometheus, logs and traces. A Grafana Labs project, not CNCF.
- **Thanos and Cortex,** CNCF projects, add long-term storage and high availability.
- **The Prometheus Operator** manages Prometheus with custom resources, from module 4.

> ON THE EXAM
>
> Grafana visualises; it does not collect. Prometheus collects and stores.

Logs die with the Pod. **Ship them off the node** before they disappear.

Three ways to collect logs

PATTERN	HOW	WHEN
Node agent	A DaemonSet reads every container's log files on its node	The default choice. One agent per node
Sidecar	A shipper container beside the app	The app writes to files, not stdout
Direct	The app sends logs straight to the backend	Rarely; ties the app to the backend

ON THE EXAM

Apps log to stdout; a node-level agent ships everything. That is the standard pattern.

Logging tools

- **Fluentd:** a CNCF graduated log collector and router, with many plugins.
- **Fluent Bit:** its lightweight sibling, ideal as the node agent. Part of the same CNCF project.
- **Storage and search:** Loki, Elasticsearch or OpenSearch, or a cloud service.
- **Structured JSON logs** make searching and filtering far easier.

> ON THE EXAM
>
> Fluentd and Fluent Bit collect and forward. They are not the store you search.

A trace follows **one request** through every service it touches.

How tracing works

- **A span** is one operation, with a start time, a duration and attributes.
- **A trace** is the tree of spans for one request, sharing one trace ID.
- **Context propagation:** each service passes the trace ID on, in a header such as traceparent.
- **Every service must propagate it,** or the trace breaks into pieces.
- **Sampling** keeps a share of traces, to control cost.

> ON THE EXAM
>
> Tracing needs instrumentation in every hop. One service that drops the header breaks the chain.

OpenTelemetry

- **Vendor-neutral APIs, SDKs and a protocol, OTLP,** for traces, metrics and logs.
- **Instrument once,** send to any backend: Prometheus, Jaeger, Loki, or a commercial vendor.
- **The Collector** receives, processes and exports telemetry.
- **Formed from the merger** of OpenTracing and OpenCensus. Graduated in the CNCF, May 2026.

> ON THE EXAM
>
> 'Vendor-neutral instrumentation for all three signals': OpenTelemetry.

An OpenTelemetry pipeline

01
Instrument
SDK or auto-instrumentation in each service.

›

02
Send
Telemetry sent over OTLP.

›

03
Receive
The Collector's receivers accept it.

›

04
Process
Batch, sample, add attributes, redact.

›

05
Export
To Prometheus, Jaeger, Loki or a vendor.

ON THE EXAM

The Collector decouples apps from backends: change the backend without touching the code.

Currency check: the observability landscape

- **OpenTelemetry graduated** on 21 May 2026. Banks and guides that say incubating are dated.
- **OpenTracing is archived.** The official curriculum text still names it; teach OpenTelemetry.
- **Jaeger v2** is built on the OpenTelemetry Collector. Jaeger is CNCF graduated.
- **Prometheus 3** can ingest OpenTelemetry metrics sent over OTLP.

> ON THE EXAM
>
> A question offering OpenTracing as the tracing standard today is testing whether you know it was superseded.

SLIs, SLOs, SLAs and error budgets

TERM	IS	QUAYSIDE EXAMPLE
SLI	A measurement of service	Share of bookings answered in under 300 ms
SLO	A target for an SLI	99.9% of bookings under 300 ms, over 30 days
SLA	A contract, with consequences if missed	Refunds if availability falls below 99.5%
Error budget	100% minus the SLO	0.1% of bookings may be slow: spend it on releases

ON THE EXAM

Indicator measures, objective targets, agreement promises. SLOs are internal; SLAs are contractual.

What to measure

METHOD	MEASURES	FOR
Four golden signals	Latency, traffic, errors, saturation	Any user-facing service
RED	Rate, errors, duration	Request-driven services
USE	Utilisation, saturation, errors	Resources: CPU, memory, disks

ON THE EXAM

RED for services, USE for resources, the four golden signals for both.

> Quayside's nightly manifest export runs as a CronJob and finishes in about 40 seconds. Prometheus scrapes every 60 seconds.
>
> The team wants the job's duration and record count stored in Prometheus after every run.

What should the job do?

A Nothing; lower the scrape interval to one second

B Push its metrics to a Prometheus Pushgateway

C Report its numbers through metrics-server

D Print them so kubectl top can read them

◆ PAUSE AND DECIDE

What should the job do?

A Nothing; lower the scrape interval to one second

Expensive everywhere, and still races the job's end.

B Push its metrics to a Prometheus Pushgateway

Built for short jobs: it holds the metrics for scraping.

C Report its numbers through metrics-server

Serves only current CPU and memory, not custom metrics.

D Print them so kubectl top can read them

kubectl top shows resource usage, not job output.

THE SIGNAL

A job that ends before the scrape: Pushgateway.

> Some bookings take four seconds instead of half a second. Each request passes through the web front end, the booking API, the payment worker and two other services.
>
> Dashboards show higher overall latency, but not which service causes it.

What will most directly show where the time goes?

A. A dashboard of every node's CPU usage

B. Searching each service's logs for errors

C. kubectl top pods across the namespace

D. Tracing with OpenTelemetry and Jaeger

◆ PAUSE AND DECIDE

What will most directly show where the time goes?

A A dashboard of every node's CPU usage
Shows load, not which call in the request was slow.

B Searching each service's logs for errors
Slow is not an error, and logs are not linked per request.

C kubectl top pods across the namespace
Current resource use only; no view of one request.

D Tracing with OpenTelemetry and Jaeger
Spans show exactly which service used the time.

THE SIGNAL

'Which service in the chain is slow?': traces.

> A new cluster already runs Prometheus and Grafana. But kubectl top pods fails with: Metrics API not available.
>
> An autoscaler based on CPU cannot read any CPU figures either.

What is missing?

A. A Grafana dashboard for Pod CPU

B. A Jaeger backend for traces

C. metrics-server, for the metrics API

D. An Alertmanager route for CPU alerts

◆ PAUSE AND DECIDE

What is missing?

A — A Grafana dashboard for Pod CPU

Dashboards display data; they do not serve the metrics API.

B — A Jaeger backend for traces

Traces have nothing to do with CPU figures.

C — metrics-server, for the metrics API

kubectl top and CPU autoscaling read metrics.k8s.io.

D — An Alertmanager route for CPU alerts

Routes alerts; provides no metrics.

THE SIGNAL

'Metrics API not available': install metrics-server. Prometheus alone does not provide it.

> An old study guide calls OpenTracing the CNCF standard for tracing.
>
> Quayside must choose one vendor-neutral way to instrument its services for traces, metrics and logs, and to switch backends later.

Which should Quayside adopt?

A OpenTelemetry

B OpenTracing

C Jaeger's own client libraries

D Prometheus client libraries alone

◆ PAUSE AND DECIDE

Which should Quayside adopt?

A OpenTelemetry

All three signals, vendor-neutral, graduated; the Collector swaps backends.

B OpenTracing

Archived, and merged into OpenTelemetry. Traces only.

C Jaeger's own client libraries

Retired in favour of OpenTelemetry; ties you to one backend.

D Prometheus client libraries alone

Metrics only: no traces or logs.

THE SIGNAL

Vendor-neutral, all three signals: OpenTelemetry. OpenTracing is its archived ancestor.

Module 26 in five lines

- **Metrics** say something is wrong, **traces** say where, **logs** say what.
- **Prometheus pulls**; short jobs push to the **Pushgateway**; **Alertmanager** routes alerts.
- Counter, gauge, histogram, summary. **rate()** on counters. **metrics-server** is for top and autoscaling.
- **Fluent Bit** DaemonSets ship logs; **OpenTelemetry** (graduated 2026) instruments; **Jaeger** stores traces.
- **SLI** measures, **SLO** targets, **SLA** promises; the error budget is 100% minus the SLO.

Metrics also drive scaling. Next: **cloud native principles and autoscaling**.

MODULE 27 · DOMAIN 4 - CLOUD NATIVE ARCHITECTURE

Cloud native principles and autoscaling

What makes a system cloud native, resilience patterns, the four autoscalers, serverless, and cost.

DOMAIN 4 of 4 EXAM WEIGHT 12% MODULE 27 of 35

admin@sthithapragna.com

Cloud native systems are **loosely coupled**, **resilient** and **automated**, so they can change often and fail safely.

The CNCF's definition, in plain words

- **Build and run scalable applications** in dynamic environments: public, private and hybrid clouds.
- **Typical techniques:** containers, service meshes, microservices, immutable infrastructure, declarative APIs.
- **The result:** loosely coupled systems that are resilient, manageable and observable.
- **With strong automation,** engineers make big changes often and predictably, with little toil.

> ON THE EXAM
>
> Cloud native is about how you build and run, not where. It applies on-premises too.

Monolith versus microservices

MONOLITH

One deployable

- Simple to build, test and run at first
- One release for everyone
- Scales only as a whole
- One bug can take everything down

MICROSERVICES

Many small services

- Each deployed and scaled independently
- Teams own and release their service
- Network calls, versioning, tracing needed
- Failures can be contained

THE TIEBREAKER

Microservices trade code complexity for operational complexity. Kubernetes exists to handle the latter.

Traits of a cloud native application

TRAIT	IN PRACTICE
Immutable	Replace, never patch in place: new image, new Pods
Declarative	Describe the desired state; controllers make it so
Disposable	Start fast, stop gracefully on SIGTERM
Stateless where possible	State lives in databases and stores, not in Pods
Observable	Metrics, logs and traces by design
Configured from outside	The same image in every environment

ON THE EXAM

These echo the twelve-factor app: config in the environment, disposable processes, logs as streams.

Resilience patterns

- **Redundancy:** several replicas, spread across nodes and zones (module 12).
- **Timeouts and retries with back-off,** so one slow call does not stall everything.
- **Circuit breakers:** stop calling a failing service for a while; fail fast instead.
- **Graceful degradation:** show cached timetables when live fares are unavailable.
- **Chaos engineering:** break things on purpose to prove resilience. LitmusChaos and Chaos Mesh are CNCF projects.

> ON THE EXAM
>
> Resilience is designed in. Kubernetes restarts containers; it cannot make your calls time out sensibly.

Kubernetes scales in three directions: more Pods, bigger Pods, and more nodes.

The four autoscalers

AUTOSCALER	CHANGES	TRIGGERED BY
HorizontalPodAutoscaler (HPA)	The number of Pods	CPU, memory or custom metrics per Pod
VerticalPodAutoscaler (VPA)	Each Pod's requests and limits	Observed usage over time
Cluster Autoscaler, Karpenter	The number of nodes	Pods Pending for lack of room
KEDA	The number of Pods, including zero	Events: queue length, lag, schedules

ON THE EXAM

HPA is built in. VPA, the node autoscalers and KEDA are add-ons.

How the HPA decides

01
Measure
Reads metrics, e.g. CPU from metrics-server.

02
Compare
Average usage against the target, say 70%.

03
Calculate
desired = current x (actual / target), rounded up.

04
Scale
Sets the Deployment's replicas, within min and max.

05
Stabilise
Waits 5 minutes before scaling down, to avoid flapping.

ON THE EXAM

Four Pods at 140% of a 70% target: 4 x 140/70 = 8 Pods.

An HPA for the booking API

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: booking-api, namespace: quayside}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: booking-api}
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource: {name: cpu, target: {type: Utilization, averageUtilization: 70}}
```

5 Scales the booking API Deployment.

6-7 Never fewer than 3, never more than 20.

10 70% of each Pod's requested CPU. Pods must declare requests.

HPA details the exam likes

- **Utilisation is a percentage of requests.** No requests: the target shows unknown.
- **Custom and external metrics** come through adapters, such as the Prometheus Adapter, or KEDA.
- **Scale-down waits five minutes** by default: the stabilisation window.
- **The HPA owns replicas.** Hand edits, or a replica count in Git, fight it.

> ON THE EXAM
>
> With GitOps, leave replicas out of Git for HPA-managed workloads, or the two will argue.

HPA versus VPA

HPA

More Pods

- Adds or removes identical Pods
- For stateless apps that scale out
- Reacts to load quickly
- Built into Kubernetes

VPA

Right-sized Pods

- Sets requests and limits from real usage
- For apps that cannot scale out
- Recommends, or applies automatically
- Can use in-place resize (module 12)

THE TIEBREAKER

Do not let HPA and VPA both act on the same CPU or memory metric; they fight.

Scaling Pods versus scaling nodes

HPA, KEDA

More Pods

- Triggered by load on the app
- Needs room on existing nodes
- Fails to help if nodes are full
- New Pods stay Pending

CLUSTER AUTOSCALER, KARPENTER

More nodes

- Triggered by Pending Pods
- Adds nodes that fit them
- Removes nodes that sit underused
- Karpenter picks node sizes itself

THE TIEBREAKER

Pods overloaded: HPA. Pods Pending with Insufficient cpu: a node autoscaler. They work together.

KEDA: event-driven autoscaling

- **A graduated CNCF project** since 2023: Kubernetes Event-Driven Autoscaling.
- **Scales on events:** queue length, Kafka lag, a Prometheus query, a schedule.
- **Scales to zero** when idle, and back up when work arrives.
- **A ScaledObject** describes the trigger; KEDA manages an HPA underneath.

> ON THE EXAM
>
> 'Scale to zero, and scale on queue length': KEDA.

Serverless: scale to zero, pay only while code runs, and let the platform handle the rest.

Serverless on Kubernetes

- **Knative**, CNCF graduated in October 2025: serverless and event-driven apps on Kubernetes.
- **Knative Serving:** scale to zero, request-based autoscaling, revisions, traffic splitting.
- **Knative Eventing:** connects event sources to services.
- **CloudEvents,** a CNCF specification, describes event data the same way everywhere.
- **The trade-off: cold starts.** The first request after idle waits for a Pod to start.

> ON THE EXAM
>
> 'Scale HTTP services to zero on Kubernetes': Knative. 'A common format for events': CloudEvents.

Cost: FinOps for Kubernetes

- **See the cost:** by namespace, team and label. OpenCost is a CNCF project for this.
- **Right-size requests:** over-requesting reserves capacity nobody uses (module 12).
- **Scale down, even to zero,** outside busy hours.
- **Spot or pre-emptible nodes** for work that tolerates interruption.

> ON THE EXAM
>
> In Kubernetes, you pay for what you request, not what you use. Requests drive cost.

Six architecture decisions. For scaling, ask: **what is short**, Pods, size, nodes, or none at all?

> The stateless booking API sees five times more traffic every evening. CPU per Pod climbs sharply at the peak.
>
> Quayside wants more Pods automatically when load rises, and fewer again afterwards.

Which autoscaler fits?

A. A VerticalPodAutoscaler

B. A Cluster Autoscaler

C. A HorizontalPodAutoscaler on CPU

D. A CronJob that scales at fixed times

◆ PAUSE AND DECIDE

Which autoscaler fits?

A A VerticalPodAutoscaler

Makes Pods bigger, not more numerous.

B A Cluster Autoscaler

Adds nodes, not Pods.

C A HorizontalPodAutoscaler on CPU

More identical Pods as CPU rises; fewer after.

D A CronJob that scales at fixed times

Fixed times miss unexpected peaks.

THE SIGNAL

Stateless app, load-driven, more Pods: HPA.

> An HPA targets 70% CPU for the web front end. kubectl get hpa shows the target as unknown/70%, and it never scales.
>
> metrics-server is running and kubectl top pods works. The web containers set no resource requests.

What is the fix?

A Add CPU requests to the web containers

B Install Grafana to supply the metrics

C Raise the HPA's maxReplicas to 50

D Switch the Deployment strategy to Recreate

◆ PAUSE AND DECIDE

What is the fix?

A Add CPU requests to the web containers

Utilisation is a share of requests; without them there is no ratio.

B Install Grafana to supply the metrics

Grafana displays metrics; it supplies nothing to the HPA.

C Raise the HPA's maxReplicas to 50

The HPA cannot scale at all yet; the cap is irrelevant.

D Switch the Deployment strategy to Recreate

Rollout strategy has nothing to do with scaling.

THE SIGNAL

'unknown' target with metrics-server working: missing requests.

> During a sale, the booking API's HPA asks for 12 replicas. Eight run; four stay Pending with: Insufficient cpu.
>
> Every node is already full of other workloads.

What should add the missing capacity?

A Raise the HPA's maxReplicas

B Add a VerticalPodAutoscaler

C Set the Pods' CPU requests to zero

D A Cluster Autoscaler or Karpenter

◆ PAUSE AND DECIDE

What should add the missing capacity?

A Raise the HPA's maxReplicas

The HPA already asked for 12; the nodes are the limit.

B Add a VerticalPodAutoscaler

Bigger Pods fit even worse on full nodes.

C Set the Pods' CPU requests to zero

Hides the shortage and overloads the nodes.

D A Cluster Autoscaler or Karpenter

Pending Pods trigger new nodes that fit them.

THE SIGNAL

Pods Pending with Insufficient cpu: a node autoscaler.

> The payment worker reads jobs from a message queue. At night the queue is empty for hours; at peak it holds thousands of jobs.
>
> Quayside wants zero workers when the queue is empty, and more workers as it grows.

Which tool fits?

A An HPA on the workers' CPU

B KEDA, scaling on queue length

C A VerticalPodAutoscaler

D A DaemonSet, one worker per node

◆ PAUSE AND DECIDE

Which tool fits?

A An HPA on the workers' CPU

Cannot reach zero on its own, and CPU lags the queue.

B KEDA, scaling on queue length

Scales on the queue itself, down to zero.

C A VerticalPodAutoscaler

Resizes Pods; never removes them.

D A DaemonSet, one worker per node

Ties workers to nodes, not to the queue.

THE SIGNAL

'Scale to zero on queue length': KEDA.

> Quayside's legacy reporting service can only ever run as a single instance.
>
> Nobody knows its right size. It is OOMKilled at month end, and wastes memory the rest of the time.

Which autoscaler helps most?

A A HorizontalPodAutoscaler

B KEDA with a schedule trigger

C A Cluster Autoscaler

D A VerticalPodAutoscaler

◆ PAUSE AND DECIDE

Which autoscaler helps most?

A A HorizontalPodAutoscaler

Adds instances, which this service cannot run.

B KEDA with a schedule trigger

Scales instance count, which is fixed at one.

C A Cluster Autoscaler

Adds nodes; the Pod's own size is the problem.

D A VerticalPodAutoscaler

Learns real usage and right-sizes requests and limits.

THE SIGNAL

Cannot scale out, size unknown: VPA.

> Quayside's booking and web features live in one monolith, released weekly after every team coordinates.
>
> Teams want to release independently, and to scale booking separately from the website during sales.

Which direction fits these goals?

A Split into microservices, deployed independently

B Run the monolith on much larger servers

C Put every component in one multi-container Pod

D Run the monolith as a StatefulSet

◆ PAUSE AND DECIDE

Which direction fits these goals?

A Split into microservices, deployed independently

Independent release and scaling, at the cost of more operational work.

B Run the monolith on much larger servers

Scales everything together; releases stay coupled.

C Put every component in one multi-container Pod

One Pod scales as one unit, like the monolith.

D Run the monolith as a StatefulSet

Adds identity, not independence.

THE SIGNAL

Independent releases and independent scaling: microservices.

Module 27 in five lines

- Cloud native: **loosely coupled, resilient, observable, automated**. How, not where.
- **HPA** more Pods (needs requests); **VPA** bigger Pods; don't combine them on the same metric.
- **Cluster Autoscaler / Karpenter** add nodes for Pending Pods.
- **KEDA** scales on events, to zero. **Knative** (graduated 2025) for serverless.
- **You pay for requests.** Right-size, scale down, use spot for tolerant work.

Last teaching module: **the community behind all of it**, and how cloud native projects are governed.

MODULE 28 · DOMAIN 4 - CLOUD NATIVE ARCHITECTURE

Community and collaboration

The CNCF, project maturity, how Kubernetes is governed and released, open standards, and the roles people play.

DOMAIN 4 of 4 EXAM WEIGHT 12% MODULE 28 of 35

admin@sthithapragna.com

Kubernetes is not one company's product. It is **open source, governed in the open**, by a community.

The Cloud Native Computing Foundation

- **Founded in 2015** under the Linux Foundation, with Kubernetes as its first project.
- **A vendor-neutral home** for cloud native open-source projects.
- **Runs KubeCon + CloudNativeCon,** the certifications, and the CNCF Landscape.
- Google donated Kubernetes; no single company controls it now.

> ON THE EXAM
>
> The CNCF is part of the Linux Foundation. The OCI is a separate Linux Foundation project (module 2).

Who does what inside the CNCF

BODY	RESPONSIBLE FOR
Governing Board	Budget, marketing and business decisions
Technical Oversight Committee (TOC)	Technical vision; admits projects; approves moves between maturity levels
Technical Advisory Groups (TAGs)	Expertise in areas such as security, to support the TOC and projects
End User community	Organisations that use cloud native, feeding back their needs

ON THE EXAM

Technical decisions about projects, including graduation, belong to the TOC, not the Governing Board.

Project maturity levels

LEVEL	MEANS	TYPICAL ADOPTERS
Sandbox	Early, experimental, low barrier to entry	Innovators
Incubating	Growing production use, healthy contributor base	Early adopters
Graduated	Widely adopted, diverse maintainers, security audit, mature governance	Early majority
Archived	No longer active; kept for reference	Should migrate away

ON THE EXAM

Sandbox, incubating, graduated. The TOC decides each move.

A project's journey

01
Apply
The project applies to join the CNCF.

›

02
Sandbox
Accepted by the TOC as an early-stage project.

›

03
Incubating
Due diligence: adopters, contributors, governance.

›

04
Graduated
TOC vote, security audit, broad adoption.

›

05
Archived
Possible at any stage, if a project winds down.

ON THE EXAM

Graduation requires, among other things, a third-party security audit and committers from several organisations.

Projects from this course, and their level

PROJECT	LEVEL	MODULE
Kubernetes, etcd, containerd, CoreDNS	Graduated	3, 5, 2, 14
Prometheus, Fluentd, Jaeger, OpenTelemetry	Graduated	26
Envoy, Istio, Linkerd, Cilium	Graduated	13, 16
Helm, Argo, Flux, Knative, KEDA	Graduated	23, 24, 27
Falco, Kyverno, cert-manager, Harbor	Graduated	18, 19
SMI, OpenTracing	Archived	16, 26

ON THE EXAM

Check cncf.io/projects before the exam: levels change several times a year.

Currency check: recent moves

- **Graduated:** Knative, October 2025; Dragonfly, January 2026; Kyverno, March 2026.
- **Graduated:** OpenTelemetry, May 2026; Cloud Native Buildpacks, August 2026.
- **Archived:** SMI in 2023, replaced by GAMMA; OpenTracing, replaced by OpenTelemetry.
- Question banks that call these incubating, or current, are simply dated.

> ON THE EXAM
>
> If an answer depends on a maturity level, trust the most recent source you have seen.

Kubernetes work is organised by special interest groups, and changed through enhancement proposals.

admin@sthithapragna.com

How the Kubernetes project is governed

GROUP	ROLE
Steering Committee	Overall governance of the Kubernetes project
Special interest groups (SIGs)	Long-lived; each owns an area of the code and its decisions
Working groups	Temporary, cross-SIG, focused on one goal
Committees	Specific duties: Code of Conduct, Security Response

ON THE EXAM

SIGs own code. Working groups own a temporary goal. The Steering Committee oversees the whole.

SIGs, and where you met their work

SIG	OWNS	MODULE
SIG Node	The kubelet, runtimes, Pods	2, 6
SIG Network	Services, DNS, kube-proxy, Gateway API	13, 14, 15
SIG Storage	Volumes, PVs, CSI	20
SIG Auth	Authentication, RBAC, ServiceAccounts	17
SIG Scheduling	The scheduler	12
SIG Apps	Workload controllers	7, 8
SIG Cluster Lifecycle	kubeadm, Cluster API	11
SIG Autoscaling	HPA, VPA, Cluster Autoscaler, Karpenter	27

ON THE EXAM

A feature's owner is the SIG whose area it touches: networking questions belong to SIG Network.

How a feature becomes stable: the KEP process

01
Discuss
The idea is raised with the owning SIG.

02
Write a KEP
Motivation, design, risks, graduation criteria.

03
Approve
The SIG reviews and approves it.

04
Alpha
Behind a feature gate, off by default.

05
Beta, then GA
Proven in use, then stable. Deprecation rules apply.

ON THE EXAM

Every alpha, beta and stable feature you met in this course arrived through a KEP.

The Kubernetes release cycle

- **About three minor releases a year,** each cycle roughly fifteen weeks.
- **A volunteer Release Team** runs each cycle, with shadows learning each role.
- **Key dates:** enhancements freeze, then code freeze, then release.
- **Patch releases** for the three maintained minor versions, roughly monthly.

> ON THE EXAM
>
> Features that miss the enhancements freeze wait for the next release. Nothing is rushed in.

Contributing

- **A contributor ladder:** member, reviewer, approver, subproject owner.
- **OWNERS files** in each folder list who can review and approve changes there.
- **Not only code:** documentation, issue triage, testing, release work, events.
- **Everything is open:** meetings, recordings, Slack, and a Code of Conduct for all.

> ON THE EXAM
>
> Anyone can contribute. Responsibility grows through the ladder, earned by contribution.

Open standards that tie the ecosystem together

STANDARD	COVERS	MODULE
OCI specifications	Images, runtimes, registries	2
CRI, CNI, CSI	Plugging in runtimes, networks, storage	2, 13, 20
Gateway API	Traffic into and between services	15, 16
OpenTelemetry, OTLP	Telemetry: traces, metrics, logs	26
CloudEvents	Describing events	27
SPIFFE	Workload identity	16

ON THE EXAM

Standardise the interface, compete on the implementation.

Why open standards matter

- **No lock-in:** swap containerd for CRI-O, or Calico for Cilium, without touching workloads.
- **Interoperability:** any OCI image runs on any compliant runtime.
- **Competition on quality,** not on proprietary interfaces.
- **Portability:** conformant Kubernetes behaves the same on every certified distribution.

> ON THE EXAM
>
> 'Change the runtime or network plugin without changing workloads' is made possible by CRI and CNI.

Roles in a cloud native organisation

ROLE	FOCUSES ON
Application developer	Writing the software; using the platform
Platform engineer	Building the internal platform: self-service, golden paths
Site reliability engineer (SRE)	Reliability, SLOs, error budgets, incident response
DevOps engineer	Bridging development and operations: pipelines, automation
Security engineer	Shifting security left, into every stage (DevSecOps)
Cluster operator	Running, upgrading and securing the clusters

ON THE EXAM

Platform engineers build the paved road; developers drive on it; SREs keep it safe.

Platform engineering

- **An internal developer platform** hides Kubernetes' complexity behind self-service.
- **Golden paths:** supported templates for a new service, with CI, GitOps and monitoring included.
- **Platform as a product:** developers are its customers; their feedback shapes it.
- **Backstage,** a CNCF project, is a common developer portal for platforms.

> ON THE EXAM
>
> Developers should not need to master every module of this course to ship a service.

The community, and where to learn

- **KubeCon + CloudNativeCon:** the CNCF's flagship conferences, on several continents.
- **Kubernetes Community Days (KCDs):** local, community-run events.
- **CNCF Ambassadors,** meetups, and the Kubernetes Slack.
- **Certifications:** KCNA, KCSA, CKA, CKAD, CKS. Holding all five makes you a Kubestronaut.

> ON THE EXAM
>
> KCNA is the entry point. KCSA is its security sibling; CKA, CKAD and CKS are hands-on.

Six community questions. For each, ask: **who decides**, **who owns**, or **which standard**?

> A CNCF incubating project has growing adoption and has completed a security audit.
> Its maintainers apply to become a graduated project.

Who approves the move to graduated?

A The CNCF Governing Board

B The CNCF Technical Oversight Committee

C The Kubernetes Steering Committee

D The project's own maintainers

◆ PAUSE AND DECIDE

Who approves the move to graduated?

A The CNCF Governing Board

Handles budget and business, not technical maturity.

B The CNCF Technical Oversight Committee

The TOC admits projects and approves every level change.

C The Kubernetes Steering Committee

Governs Kubernetes only, not other CNCF projects.

D The project's own maintainers

They apply; they cannot approve their own graduation.

THE SIGNAL

Project maturity is a technical decision: the TOC.

> A new open-source tool, built by one company, has a handful of early users and an evolving design.
>
> The CNCF accepts it this month.

At which maturity level does it most likely enter?

A Graduated

B Incubating

C Sandbox

D Archived

◆ PAUSE AND DECIDE

At which maturity level does it most likely enter?

A **Graduated**
Needs broad adoption, diverse maintainers and an audit.

B **Incubating**
Needs real production adopters and due diligence.

C **Sandbox**
The entry point for early, experimental projects.

D **Archived**
For projects winding down, not joining.

THE SIGNAL

Early, experimental, one company, few users: sandbox.

> A contributor wants to add a significant new field to the Pod API, changing how the kubelet behaves.
>
> They want to follow the Kubernetes project's normal process.

What should they do first?

A Write a KEP and work through the owning SIG

B Open a pull request straight to the main branch

C Ask the CNCF TOC to approve the feature

D Email the Steering Committee for permission

◆ PAUSE AND DECIDE

What should they do first?

A Write a KEP and work through the owning SIG
Significant changes start with a KEP in the right SIG.

B Open a pull request straight to the main branch
Big API changes are not merged without a KEP.

C Ask the CNCF TOC to approve the feature
The TOC governs CNCF projects, not Kubernetes features.

D Email the Steering Committee for permission
The Steering Committee does not approve individual features.

THE SIGNAL

Significant Kubernetes change: a KEP, through the owning SIG.

> A proposed change touches Services, EndpointSlices and the Gateway API.
>
> The contributor needs to know which group owns this area of Kubernetes.

Which SIG owns it?

A SIG Node

B SIG Apps

C SIG Network

D SIG Storage

◆ PAUSE AND DECIDE

Which SIG owns it?

A **SIG Node**
Owns the kubelet, runtimes and Pods.

B **SIG Apps**
Owns workload controllers such as Deployments.

C **SIG Network**
Owns Services, DNS, kube-proxy and the Gateway API.

D **SIG Storage**
Owns volumes and CSI.

THE SIGNAL

Services and traffic belong to SIG Network.

> Quayside plans to replace containerd with CRI-O on its nodes, and Flannel with Cilium as its network plugin.
>
> No application manifest needs to change.

What makes this possible?

A Helm charts for every application

B GitOps with Argo CD

C A service mesh in every namespace

D The CRI and CNI open standards

◆ PAUSE AND DECIDE

What makes this possible?

A Helm charts for every application

Packages apps; says nothing about runtimes or networks.

B GitOps with Argo CD

Deploys apps; does not decouple the runtime.

C A service mesh in every namespace

Adds a traffic layer; unrelated to swapping plugins.

D The CRI and CNI open standards

Kubernetes talks to runtimes and plugins through standard interfaces.

THE SIGNAL

Swap a runtime or network plugin freely: CRI and CNI.

> At Quayside, one team builds self-service templates, so that other teams get a new service with CI, GitOps and monitoring already wired in.
>
> Application teams can ship without learning most Kubernetes internals.

Which role best describes that team?

A Site reliability engineers

B Platform engineers

C Application developers

D FinOps practitioners

◆ PAUSE AND DECIDE

Which role best describes that team?

A Site reliability engineers

Focus on reliability, SLOs and incidents.

B Platform engineers

Build the internal platform and its golden paths.

C Application developers

The customers of the platform, not its builders.

D FinOps practitioners

Focus on cost visibility and optimisation.

THE SIGNAL

Self-service templates and golden paths: platform engineering.

Module 28 in five lines

- The **CNCF** (Linux Foundation, 2015) hosts projects; the **TOC** decides maturity.
- **Sandbox, incubating, graduated**, or archived. Graduation needs a security audit.
- Kubernetes: **Steering Committee, SIGs, working groups**; changes via **KEPs**.
- About **three releases a year**; alpha behind feature gates, then beta, then GA.
- **Open standards** (OCI, CRI, CNI, CSI, Gateway API, OpenTelemetry) prevent lock-in.

That completes the teaching. Next: what changed, then confusion pairs, then practice exams.

MODULE 29 · EXAM STRATEGY - ALL DOMAINS

What changed: spotting a dated question

Every currency flag from this course in one place, and how to answer when a question looks out of date.

SECTION Exam skills COVERS All domains MODULE 29 of 35

admin@sthithapragna.com

Cloud native moves fast. Much study material, and some questions, **describe a world that no longer exists**.

admin@sthithapragna.com

The curriculum itself changed

BEFORE 24 NOVEMBER 2025	FROM 24 NOVEMBER 2025
Five domains	Four domains
Fundamentals 46%, Orchestration 22%	Fundamentals 44%, Orchestration 28%
Architecture 16%, Observability 8%	Architecture 12%, observability folded into it
Application Delivery 8%	Application Delivery 16%: doubled
Troubleshooting and debugging implicit	Administration, Troubleshooting and Debugging named

ON THE EXAM

Newer questions lean on commands, troubleshooting and delivery; fewer ask pure CNCF trivia.

Dated answers, current answers (1)

IF AN OPTION SAYS...	TODAY THE ANSWER IS...
Docker Engine is the Kubernetes runtime	containerd or CRI-O, via the CRI
Use PodSecurityPolicy	Pod Security Admission (PSP removed in 1.25)
SMI is the service mesh standard	The Gateway API, through GAMMA
Use OpenTracing for tracing	OpenTelemetry
kubectl run creates a Deployment	kubectl run creates one bare Pod
Stable APIs go 12 months after deprecation	Never removed within Kubernetes 1.x

ON THE EXAM

Each row is a trap in a question bank somewhere. Recognise the left column on sight.

Dated answers, current answers (2)

IF AN OPTION SAYS...	TODAY THE ANSWER IS...
Use kube-proxy IPVS mode for scale	nftables mode; IPVS deprecated in 1.35
Services are backed by Endpoints	EndpointSlices; Endpoints deprecated 1.33
Install the Ingress NGINX controller	Retired March 2026: move to Gateway API
Resources are immutable	In-place resize, stable in 1.35
Helm's --atomic flag	--rollback-on-failure in Helm 4
SA tokens are long-lived Secrets	Short-lived projected tokens

ON THE EXAM

Recognise the left column on sight, and know the current fact.

Recently stable features

FEATURE	STABLE IN	MODULE
ValidatingAdmissionPolicy (CEL)	Kubernetes 1.30	18
Native sidecar containers	Kubernetes 1.33	6
kube-proxy nftables mode	Kubernetes 1.33	13
Dynamic Resource Allocation (DRA)	Kubernetes 1.34	12
In-place Pod resize	Kubernetes 1.35	12
User namespaces (hostUsers: false)	Kubernetes 1.36	18

ON THE EXAM

1.37 (August 2026) is current; 1.37, 1.36 and 1.35 are maintained.

Nodes and runtimes

CHANGE	WHEN	MODULE
dockershim removed	Kubernetes 1.24, May 2022	2
Last release supporting containerd 1.x	Kubernetes 1.35	2
Kubelet refuses cgroup v1 nodes by default	Kubernetes 1.35	1
Kubelets up to three minors older than the API server	Current skew policy	11

ON THE EXAM

Upgrading past 1.35 needs containerd 2 and cgroup v2 on every node.

The wider ecosystem

PROJECT	CHANGE	MODULE
Knative	Graduated, October 2025	27
Helm 4	GA, November 2025; Helm 3 security fixes end February 2027	23
Kyverno, Dragonfly	Graduated, March and January 2026	18
OpenTelemetry	Graduated, May 2026	26
Cloud Native Buildpacks	Graduated, August 2026	2
Istio ambient mode	GA in Istio 1.24, sidecarless	16
kaniko	Archived by Google, June 2025	2

ON THE EXAM

Maturity levels change several times a year. Check cncf.io/projects the week before your exam.

How the documentation itself changed

- **Security framing:** kubernetes.io moved from the 4Cs to lifecycle phases: Develop, Distribute, Deploy, Runtime.
- **The official curriculum text** still names OpenTracing among CNCF projects. Teach and answer with OpenTelemetry.
- Both framings, and both names, may appear in questions. Know which is current.

> ON THE EXAM
>
> A source that looks official can still be out of date. Dates matter more than logos.

If the options include the current answer, **choose it**. If they do not, **choose the best of what is offered**.

Answering a question that looks dated

- **Current answer offered?** Choose it. Exams are updated towards current facts, not away.
- **Only dated options?** Choose the one that is least wrong for the concept being tested.
- **Is it really about history?** 'What replaced PodSecurityPolicy' wants the history.
- **Test the concept, not the date.** KCNA rarely hinges on an exact version number.
- **Do not argue with the question.** Spend the time on the next one.

> ON THE EXAM
>
> Know what changed, then answer what the question is really testing.

> A practice question asks how containers are started on the nodes of a current Kubernetes 1.37 cluster.

Which statement is accurate today?

A Docker Engine must be installed on every node

B The dockershim translates CRI calls for containerd

C Images built with Docker must be rebuilt first

D containerd or CRI-O runs them, through the CRI

◆ PAUSE AND DECIDE

Which statement is accurate today?

A Docker Engine must be installed on every node
Dated: Docker Engine has not been needed since 1.24.

B The dockershim translates CRI calls for containerd
Dated: the dockershim was removed in 1.24.

C Images built with Docker must be rebuilt first
Never true: Docker builds OCI images.

D containerd or CRI-O runs them, through the CRI
Current: CRI runtimes, with no shim.

THE SIGNAL

Anything requiring Docker Engine or the dockershim on nodes is pre-2022.

> On a Kubernetes 1.37 cluster, the team must give a running Pod more CPU without recreating it.

Which statement is accurate today?

A Resize it in place, via the resize subresource

B It is impossible: resources are immutable once set

C Edit the Deployment; that is the only way

D Only a VPA in Recreate mode can change it

◆ PAUSE AND DECIDE

Which statement is accurate today?

A Resize it in place, via the resize subresource

Current: in-place resize is stable since 1.35.

B It is impossible: resources are immutable once set

Dated: true only before in-place resize.

C Edit the Deployment; that is the only way

Works, but recreates the Pods through a rollout.

D Only a VPA in Recreate mode can change it

Recreate mode recreates the Pod, by definition.

THE SIGNAL

'Resources are immutable' is a pre-1.35 answer.

> A question bank from 2024 says OpenTelemetry is an incubating CNCF project. You are sitting the exam in late 2026.

What is OpenTelemetry's CNCF level today?

A Sandbox

B Incubating

C Graduated

D Archived

◆ PAUSE AND DECIDE

What is OpenTelemetry's CNCF level today?

A Sandbox

Never; it entered as incubating.

B Incubating

Dated: true until May 2026.

C Graduated

Current: graduated on 21 May 2026.

D Archived

That is OpenTracing, its predecessor.

THE SIGNAL

OpenTelemetry graduated; OpenTracing is the archived one.

> A study guide recommends PodSecurityPolicy to stop containers running as root. The team runs Kubernetes 1.37.

What should they use instead?

A PodSecurityPolicy objects, with RBAC bindings

B Pod Security Admission labels at Restricted

C A default-deny NetworkPolicy

D A RuntimeClass for every Pod

◆ PAUSE AND DECIDE

What should they use instead?

A PodSecurityPolicy objects, with RBAC bindings
Removed in 1.25; the objects no longer exist.

B Pod Security Admission labels at Restricted
The built-in replacement; Restricted requires non-root.

C A default-deny NetworkPolicy
Controls traffic, not how containers run.

D A RuntimeClass for every Pod
Chooses a runtime; does not enforce non-root.

THE SIGNAL

What replaced PodSecurityPolicy: Pod Security Admission.

Module 29 in five lines

- **New curriculum** since November 2025: four domains, 44/28/16/12; Delivery doubled.
- **Replaced:** dockershim, PSP, SMI, OpenTracing, IPVS, Endpoints, Ingress NGINX, --atomic.
- **Now stable:** sidecars, nftables, DRA, in-place resize, user namespaces.
- **Graduated 2025-26:** Knative, Dragonfly, Kyverno, OpenTelemetry, Buildpacks.
- **Current answer offered? Choose it.** Otherwise, the least wrong option.

Next: confusion pairs, the look-alike concepts the exam loves to put side by side.

MODULE 30 · EXAM STRATEGY - ALL DOMAINS

Confusion pairs

Sixty-six look-alike pairs, each with the tiebreaker that separates it from its neighbour.

SECTION Exam skills COVERS All domains MODULE 30 of 35

admin@sthithapragna.com

Most wrong answers are the right answer's neighbour. Learn the one fact that tells them apart.

admin@sthithapragna.com

How to use this module

- **Each row is a pair** and its tiebreaker, the fact that decides between them.
- **Questions plant a detail** that points to one side: 'must not restart', 'every node', 'one namespace'.
- **Find the planted detail first,** then apply the tiebreaker.
- **Replay this module** the day before the exam. It is the course in about twelve minutes.

> ON THE EXAM
>
> The planted detail plus the tiebreaker answers most pair questions in seconds.

Fundamentals (1): containers and runtimes

PAIR	TIEBREAKER
Container / virtual machine	Container shares the host kernel; a VM brings its own
Namespaces / cgroups	Namespaces isolate what it sees; cgroups limit what it uses
CPU limit / memory limit	Over CPU: throttled. Over memory: OOMKilled
CRI / OCI	CRI: kubelet to runtime. OCI: image and runtime formats
containerd / runc	containerd pulls and manages; runc just starts the process

ON THE EXAM

Modules 1 and 2. The shared kernel explains most container questions.

Fundamentals (2): objects and workloads

PAIR	TIEBREAKER
spec / status	You write spec; the system writes status
ReplicaSet / Deployment	ReplicaSet keeps a count; Deployment manages versions
Deployment / StatefulSet	Identical copies; or named individuals with own storage
DaemonSet / Deployment	One per node, following nodes; or a count anywhere
Job / CronJob	Runs once to completion; or on a schedule
create / apply	create fails if it exists; apply is idempotent

ON THE EXAM

Identical, individual, everywhere, finite: the workload shortcut from module 8.

Fundamentals (3): admin and scheduling

PAIR	TIEBREAKER
cordon / drain	Close to new bookings; or also move the diners out
requests / limits	Requests decide placement; limits decide punishment
Taint / node affinity	Taint keeps others out; affinity pulls yours in
Required / preferred affinity	Required filters (Pending if unmet); preferred scores
Labels / annotations	Labels are selected on; annotations only describe
ResourceQuota / LimitRange	Quota caps the namespace; LimitRange shapes each container

ON THE EXAM

A toleration permits a tainted node; it never attracts to it.

Orchestration (1): Services and traffic

PAIR	TIEBREAKER
ClusterIP / NodePort	Inside the cluster only; or a port on every node
NodePort / LoadBalancer	You bring the front door; or the cloud builds one
Headless / ClusterIP	DNS returns Pod IPs; or one balanced virtual IP
Ingress / Gateway API	Frozen, one object; or role-split and expressive

ON THE EXAM

'Every Pod individually' is headless; 'one entrance for many Services' is Ingress or Gateway.

Orchestration (2): security and failures

PAIR	TIEBREAKER
Role / ClusterRole	One namespace; or cluster-wide and reusable
RoleBinding / ClusterRoleBinding	The binding decides the scope
NetworkPolicy / mTLS	Restricts who connects; or encrypts and authenticates
OOMKilled / Evicted	One container over its limit; or whole Pod, node pressure

ON THE EXAM

A ClusterRole bound by a RoleBinding only applies in that namespace.

Delivery (1): pipelines and releases

PAIR	TIEBREAKER
CI / CD	Build and test an artefact; or move it into environments
Continuous delivery / deployment	A human gate before production; or none
Blue-green / canary	Everyone at once, instant rollback; or a few users first
Rolling update / Recreate	Gradual, no downtime; or all out, then all in
Build once / rebuild per environment	Promote the same image; rebuilds can differ

ON THE EXAM

'Must never overlap' is Recreate; 'a few real users first' is canary.

Delivery (2): Helm and Kustomize

PAIR	TIEBREAKER
Helm / Kustomize	Templates and packages; or patches on plain YAML
Chart / release	The package; or one installed copy of it
Release / revision	One installed copy; or a numbered step in its history
helm template / helm install	Renders locally; or installs into the cluster
-f values file / --set	Files, in order; --set wins over both
Flux Kustomization / kustomization.yaml	A Flux custom resource; or Kustomize's own file

ON THE EXAM

Someone else's software: Helm. Your own app, plain YAML: Kustomize.

Delivery (3): GitOps

PAIR	TIEBREAKER
Push / pull deployment	Pipeline holds cluster credentials; or an agent pulls from Git
Argo CD / Flux	Application objects and a UI; or a toolkit of controllers
prune / selfHeal	Deletes what Git removed; or reverts manual changes
Synced / Healthy	Cluster matches Git; or the app actually works
git revert / kubectl rollout undo	Sticks under GitOps; or is reverted as drift
Application / ApplicationSet	One app; or a template generating many

ON THE EXAM

Synced but Degraded: Git was applied faithfully, and Git is wrong.

Delivery (4): probes and debugging

PAIR	TIEBREAKER
Liveness / readiness	Restart me; or do not send me traffic
Startup / liveness	Hold checks back while starting; or check once running
kubectl exec / kubectl debug	Use tools in the image; or bring your own
logs -f / logs --previous	Follow the live container; or read the one that crashed
get / describe	What state; or why, with events

ON THE EXAM

If a restart would not fix it, it belongs in readiness, not liveness.

Architecture (1): observability

PAIR	TIEBREAKER
Metrics / logs / traces	Something is wrong; what exactly; which service
Prometheus / Grafana	Collects and stores; or draws dashboards
metrics-server / Prometheus	Current CPU and memory for top and HPA; or history
Counter / gauge	Only goes up; or goes up and down
Histogram / summary	Buckets, percentiles at query time; or in the client
Scrape / Pushgateway	Prometheus pulls; short jobs push instead

> ON THE EXAM
>
> Never graph a raw counter. rate() turns it into per second.

Architecture (2): standards, SLOs and scaling

PAIR	TIEBREAKER
OpenTelemetry / OpenTracing	Current and graduated; or archived
SLI / SLO / SLA	A measurement; a target; a contract
HPA / VPA	More Pods; or bigger Pods
HPA / Cluster Autoscaler	Pods overloaded; or Pods Pending for lack of room
HPA / KEDA	CPU or memory; or events, down to zero
Monolith / microservices	One deployable; or many, independently released

ON THE EXAM

Pending with Insufficient cpu is a node problem, not a Pod-count problem.

Architecture (3): community

PAIR	TIEBREAKER
TOC / Governing Board	Technical decisions and maturity; or budget and business
Sandbox / incubating	Early and experimental; or growing production use
Incubating / graduated	Growing; or broad adoption plus a security audit
SIG / working group	Owns an area long-term; or a temporary goal
KEP / pull request	Designs a significant change; or implements code
Platform engineer / SRE	Builds the paved road; or keeps it reliable
CNCF / OCI	Both Linux Foundation; the OCI is not a CNCF project

ON THE EXAM

Project maturity is always decided by the CNCF's TOC.

Eight pair questions, six from Domains 3 and 4. In each, **find the planted detail** first.

> During a security investigation, node-6 must accept no new Pods for the next hour.
>
> The Pods already on it must keep serving customers throughout.

Which command fits?

A kubectl drain node-6

B kubectl cordon node-6

C A NoExecute taint on node-6

D kubectl delete node node-6

◆ PAUSE AND DECIDE

Which command fits?

A kubectl drain node-6

Also evicts the Pods that must keep serving.

B kubectl cordon node-6

No new Pods; existing ones stay.

C A NoExecute taint on node-6

Evicts running Pods without tolerations.

D kubectl delete node node-6

Removes the node from the cluster.

THE SIGNAL

No new Pods, existing ones stay: cordon.

> The security team requires traffic between the payment worker and the booking API to be encrypted in transit.
>
> Only those two services talk to each other already.

What meets the requirement?

A. A NetworkPolicy allowing only that pair

B. A ClusterIP Service between them

C. A headless Service for the booking API

D. Mutual TLS, typically through a mesh

◆ PAUSE AND DECIDE

What meets the requirement?

A. A NetworkPolicy allowing only that pair
Restricts who connects; encrypts nothing.

B. A ClusterIP Service between them
Addresses traffic; does not encrypt it.

C. A headless Service for the booking API
Changes DNS answers; no encryption.

D. Mutual TLS, typically through a mesh
Encrypts and authenticates each call.

THE SIGNAL

Encrypted in transit: mTLS. NetworkPolicy never encrypts.

> The fare engine container crashed and has just been restarted. kubectl logs shows only: Starting up...
>
> The engineer needs the error the container printed just before it crashed.

Which flag gives that?

A --previous

B -f

C --tail=100

D --since=1h

◆ PAUSE AND DECIDE

Which flag gives that?

A --previous

The last instance: the one that crashed.

B -f

Follows the new container, which has not failed.

C --tail=100

Recent lines of the current container only.

D --since=1h

A time filter on the current container only.

THE SIGNAL

Output from before a crash: --previous.

> Quayside's new search feature must be tried on real traffic from 2% of users for a week.
>
> Only if error rates stay low should everyone get it.

Which release strategy fits?

A Blue-green, switching all users at once

B Recreate, stopping the old version first

C A canary, starting at 2% of users

D A rolling update with default settings

◆ PAUSE AND DECIDE

Which release strategy fits?

A Blue-green, switching all users at once

Moves everyone together; no 2% phase.

B Recreate, stopping the old version first

Everyone at once, with downtime.

C A canary, starting at 2% of users

A small share first, expanded only if healthy.

D A rolling update with default settings

Replaces every Pod within minutes.

THE SIGNAL

'A few real users first, then everyone': canary.

> Quayside wants a metric for the number of bookings currently waiting in payment processing.
>
> The number rises when orders arrive and falls as they complete.

Which metric type fits?

A Counter

B Histogram

C Gauge

D Summary

◆ PAUSE AND DECIDE

Which metric type fits?

A **Counter**
Only goes up; cannot fall as bookings complete.

B **Histogram**
Distributions of observations, such as latency.

C **Gauge**
Goes up and down: a current level.

D **Summary**
Client-side percentiles, not a current level.

THE SIGNAL

A value that rises and falls: gauge.

> Quayside's team writes: 99.95% of fare lookups must complete in under 200 milliseconds, measured over 28 days.
>
> It is an internal goal. No customer contract refers to it.

What is this statement?

A An SLO

B An SLI

C An SLA

D An error budget

◆ PAUSE AND DECIDE

What is this statement?

A An SLO

A target for a measurement, used internally.

B An SLI

The measurement itself, without a target.

C An SLA

A contract with consequences for customers.

D An error budget

What the target leaves over: here 0.05%.

THE SIGNAL

Measurement plus target, internal: SLO. Add a contract: SLA.

> The booking API's error rate is already stored in Prometheus.
>
> The team now wants a dashboard that shows it on a wall screen in the operations room.

Which tool draws that dashboard?

A Alertmanager

B The Pushgateway

C Fluent Bit

D Grafana

◆ PAUSE AND DECIDE

Which tool draws that dashboard?

A Alertmanager

Routes alerts; draws no dashboards.

B The Pushgateway

Holds pushed metrics for short jobs.

C Fluent Bit

Ships logs from nodes.

D Grafana

Queries Prometheus and draws dashboards.

THE SIGNAL

Stored in Prometheus, shown on a dashboard: Grafana.

> A CNCF project has production adopters and maintainers from several companies.
>
> It has not yet completed a third-party security audit.

At which maturity level is it most likely?

A Sandbox

B Incubating

C Graduated

D Archived

◆ PAUSE AND DECIDE

At which maturity level is it most likely?

A Sandbox

It has already outgrown the early, experimental stage.

B Incubating

Growing production use, not yet graduated.

C Graduated

Graduation requires the security audit.

D Archived

It is active and growing.

THE SIGNAL

Production users, no audit yet: incubating, not graduated.

Module 30 in five lines

- **Wrong answers are neighbours.** Learn the tiebreaker for each pair.
- **Find the planted detail first:** 'must not restart', 'every node', 'a few users'.
- **Delivery:** blue-green vs canary, prune vs selfHeal, liveness vs readiness.
- **Architecture:** counter vs gauge, SLI vs SLO vs SLA, HPA vs VPA vs node autoscaler.
- **Replay this module** the day before the exam.

Next: how KCNA questions are written, and how to take the exam.

admin@sthithapragna.com

MODULE 31 · EXAM STRATEGY - ALL DOMAINS

How KCNA questions are written

The exam's format, the question styles, the distractor patterns, and how to use ninety minutes.

SECTION Exam skills COVERS All domains MODULE 31 of 35

admin@sthithapragna.com

KCNA tests **concepts and choices**, not typing. Each question has one best answer among **plausible neighbours**.

The exam at a glance

ITEM	DETAIL
Format	Online, remotely proctored, multiple choice, in a secure browser
Time	90 minutes
Questions	About 60, as widely reported. Read each question's instructions
Pass mark	75%
Results	Within 24 hours, scored automatically
Attempts and validity	Two attempts; the certification is valid for two years

ON THE EXAM

Check the Linux Foundation's exam page and candidate handbook the week you book: details can change.

Pacing ninety minutes

- **About 1.5 minutes per question** on average. Most need far less.
- **First pass:** answer what you know, flag anything that needs thought, move on.
- **Aim to finish the first pass** with about thirty minutes to spare.
- **Second pass:** the flagged questions, with the time you saved.
- **Never leave a question blank.** An unanswered question scores nothing.

> ON THE EXAM
>
> One stubborn question should never cost you three easy ones.

Question styles you will meet

STYLE	LOOKS LIKE	TESTS
Definition	What is a DaemonSet?	Vocabulary
Component	Which component stores cluster state?	Architecture
Command	Which command shows each Pod's node?	kubectl fluency
Scenario	A team needs X, without Y. What should they do?	Choosing between neighbours
Negative	Which is NOT a graduated project?	Careful reading
Ecosystem	Which project provides event-driven autoscaling?	CNCF landscape

ON THE EXAM

Newer questions lean towards scenarios and commands; fewer are pure definitions.

Reading the question

- **Read the last sentence first:** it tells you what is being asked.
- **Find the planted detail:** 'every node', 'without restarting', 'one namespace only'.
- **Mark the qualifiers:** BEST, MOST, FIRST, LEAST effort, NOT, EXCEPT.
- **List the constraints:** must, must not, cannot, only.

> ON THE EXAM
>
> A missed NOT turns a known answer into a wrong one. Read the stem twice.

True versus best

TRUE

Works, technically

- Would solve part of the problem
- May break a stated constraint
- May be more effort than needed
- Often the tempting distractor

BEST

Meets every constraint

- Solves the whole problem
- Respects every must and must not
- Least effort, least privilege
- The answer the exam wants

THE TIEBREAKER

Several options may be true. Only one satisfies every constraint in the question.

Six distractor patterns

PATTERN	EXAMPLE FROM THIS COURSE
The neighbour	Readiness offered when the answer is liveness
Right tool, wrong layer	kube-proxy offered for a DNS failure
The over-reach	cluster-admin, or Restricted when Baseline is asked for
The outdated fact	dockershim, PodSecurityPolicy, SMI, OpenTracing
The half answer	Cordon when the Pods must also move
The invented feature	RBAC deny rules; a day-long ConfigMap cache

> ON THE EXAM
>
> Name the pattern, and the distractor usually eliminates itself.

Eliminating to the answer

01
Remove invented
Features that do not exist.

02
Remove wrong layer
Right idea, wrong component.

03
Remove over-reach
More power or effort than asked.

04
Compare the last two
Reread the planted detail.

05
Commit
Change later only for a specific reason.

ON THE EXAM

Most questions come down to two options. The planted detail decides between them.

Command questions

- **Read commands as grammar:** verb, type, name, flags. Module 10.
- **Know the flags that matter:** -o wide, -n and -A, --previous, -c, --dry-run=client -o yaml, apply -k.
- **Watch for invented flags** and verbs that do the wrong thing, such as create instead of apply.
- **Short names are fair game:** po, svc, deploy, ns, cm, pvc, sts, ds.

> ON THE EXAM
>
> A command with one wrong flag is a wrong answer. Read every flag.

Absolute words: handle with care

- **'Always', 'never', 'only'** are often wrong in a flexible ecosystem.
- **But not always:** 'only the API server talks to etcd' is true.
- **And 'never newer than the API server'** is exactly the kubelet skew rule.
- **Test the absolute against a fact you know,** rather than rejecting it on sight.

> ON THE EXAM
>
> Absolutes are a clue, not a verdict. Check them against the facts.

Exam day

- **Run the system check** for the secure browser well before the day.
- **A clear desk, a single monitor, valid ID,** and a quiet, private room.
- **Start early** enough to allow the identity and room checks.
- **If something breaks,** contact the proctor through the exam's chat; do not just leave.

> ON THE EXAM
>
> Most exam-day failures are technical, not knowledge. Remove them in advance.

The last week

- **Take the four practice exams** in this course under timed conditions.
- **Review every wrong answer by domain,** and rewatch that module's scenarios.
- **Replay modules 29 and 30:** what changed, and the confusion pairs.
- **Check cncf.io/projects** for any maturity changes since this course.

> ON THE EXAM
>
> Aim for a consistent 85% or more in practice before booking. That leaves margin above 75%.

> This question tests careful reading of a negative stem.

Which of these is NOT a CNCF graduated project?

A Prometheus

B Envoy

C Grafana

D Helm

◆ PAUSE AND DECIDE

Which of these is NOT a CNCF graduated project?

A Prometheus

Graduated: the CNCF's second.

B Envoy

Graduated.

C Grafana

A Grafana Labs project, not CNCF at all.

D Helm

Graduated.

THE SIGNAL

NOT questions: find the one that fails the test, not the ones that pass.

> A new booking API Pod has been Pending for ten minutes. The on-call engineer does not yet know why.

What should the engineer do FIRST?

A Run kubectl describe pod on it

B Add more nodes to the cluster

C Delete the Pod so it is recreated

D Restart the kubelet on every node

◆ PAUSE AND DECIDE

What should the engineer do FIRST?

A Run kubectl describe pod on it

Read the cause before changing anything.

B Add more nodes to the cluster

Might help, if the cause is capacity. Unknown yet.

C Delete the Pod so it is recreated

A replacement hits the same cause.

D Restart the kubelet on every node

Drastic, and unrelated unless nodes are the issue.

THE SIGNAL

FIRST means diagnose before you act.

> Developers need to view Pods, Services and Deployments in the dev namespace only. Nothing else, and no changes.

Which grant follows least privilege?

A cluster-admin through a ClusterRoleBinding

B The edit ClusterRole through a RoleBinding in dev

C The view ClusterRole through a ClusterRoleBinding

D The view ClusterRole through a RoleBinding in dev

◆ PAUSE AND DECIDE

Which grant follows least privilege?

A cluster-admin through a ClusterRoleBinding

The ultimate over-reach.

B The edit ClusterRole through a RoleBinding in dev

Right scope, but allows changes.

C The view ClusterRole through a ClusterRoleBinding

Read-only, but in every namespace.

D The view ClusterRole through a RoleBinding in dev

Read-only, in dev only.

THE SIGNAL

The over-reach pattern: pick the least power that meets the need.

> A ServiceAccount has a RoleBinding to the edit ClusterRole in the quayside namespace. It must stop being able to delete Pods.
>
> It should keep its other permissions where possible.

What should the administrator do?

A Add an RBAC rule that denies delete on Pods

B Replace the binding with a narrower custom Role

C Apply a NetworkPolicy to the ServiceAccount

D Create a PodDisruptionBudget for the Pods

◆ PAUSE AND DECIDE

What should the administrator do?

A Add an RBAC rule that denies delete on Pods

Invented: RBAC has no deny rules.

B Replace the binding with a narrower custom Role

RBAC only adds; remove the power by granting less.

C Apply a NetworkPolicy to the ServiceAccount

Network rules do not govern API permissions.

D Create a PodDisruptionBudget for the Pods

Limits voluntary evictions, not API deletes.

THE SIGNAL

The invented-feature pattern: RBAC deny rules do not exist.

Module 31 in five lines

- **90 minutes, 75% to pass,** about 60 questions; read each question's instructions.
- **Two passes:** answer and flag, then return with the time saved.
- **Last sentence first,** then the planted detail and the qualifiers.
- **True is not best.** The best option meets every constraint.
- **Name the distractor:** neighbour, wrong layer, over-reach, outdated, half, invented.

Now test yourself: four practice exams, twenty questions each, weighted like the real exam.

PRACTICE EXAM 1 OF 4 · EXAM STRATEGY - ALL DOMAINS

Practice exam 1

Twenty questions across all four domains, weighted like the real exam.

QUESTIONS 20 TIME About 30 minutes MODULE 32 of 35

admin@sthithapragna.com

How to take this exam

- **Answer before the reveal.** Say or write your letter while the question holds.
- **Keep a tally by question number,** so you can score by domain at the end.
- **Use the technique from module 31:** last sentence first, planted detail, eliminate.
- **Do not pause to look things up.** This is a measure, not a lesson.

> ON THE EXAM
>
> Your pattern of misses by domain matters more than your total.

> A new Pod has been created and stored, but its spec has no node assigned yet.

Which component will assign it to a node?

A The kubelet

B The kube-scheduler

C The kube-controller-manager

D kube-proxy

◆ PAUSE AND DECIDE

Which component will assign it to a node?

A The kubelet

Runs Pods already bound to its node.

B The kube-scheduler

Watches for unassigned Pods and binds each to a node.

C The kube-controller-manager

Creates Pods; does not place them.

D kube-proxy

Programs Service rules on nodes.

THE SIGNAL

No node yet: the scheduler's one job.

> A developer writes a Service manifest but leaves out spec.type entirely.

What type of Service is created?

A NodePort

B LoadBalancer

C ExternalName

D ClusterIP

◆ PAUSE AND DECIDE

What type of Service is created?

A NodePort
Must be requested explicitly.

B LoadBalancer
Must be requested explicitly.

C ExternalName
Must be requested explicitly.

D ClusterIP
The default: a stable internal virtual IP.

THE SIGNAL

No type given: ClusterIP.

> A Pod runs an app container and a metrics-exporter sidecar.
>
> The exporter must read the app's statistics, which the app serves on port 9100.

How should the exporter address the app?

A localhost, on port 9100

B The name of a Service in front of the Pod

C The IP address of the node

D A separate IP belonging to the app container

◆ PAUSE AND DECIDE

How should the exporter address the app?

A localhost, on port 9100

Containers in a Pod share one network namespace.

B The name of a Service in front of the Pod

Works, but adds a needless hop through a Service.

C The IP address of the node

The node's IP does not reach the Pod's port.

D A separate IP belonging to the app container

Containers in a Pod have no separate IPs.

THE SIGNAL

Same Pod, shared network: localhost.

> Quayside runs Prometheus with its default design.

How does Prometheus normally collect metrics from a workload?

A The workload pushes its metrics to it over OTLP

B Fluent Bit forwards them from each node

C It scrapes each target's /metrics over HTTP

D The kubelet writes the metrics into etcd

◆ PAUSE AND DECIDE

How does Prometheus normally collect metrics from a workload?

A The workload pushes its metrics to it over OTLP
Push is possible, but not Prometheus's default model.

B Fluent Bit forwards them from each node
Fluent Bit ships logs, not metrics.

C It scrapes each target's /metrics over HTTP
Prometheus pulls on an interval.

D The kubelet writes the metrics into etcd
Metrics never go into etcd.

THE SIGNAL

Prometheus pulls.

> Quayside's stateless timetable service needs four identical replicas.
>
> New versions must roll out gradually, with the option to roll back.

Which workload fits?

A A StatefulSet

B A DaemonSet

C A Deployment

D A Job

◆ PAUSE AND DECIDE

Which workload fits?

A A StatefulSet
Adds identity and storage it does not need.

B A DaemonSet
One per node, not four identical copies.

C A Deployment
Identical replicas, rolling updates and rollback.

D A Job
Runs to completion; it would stop.

THE SIGNAL

Identical and stateless: Deployment.

> A new team member asks what a Helm chart actually is.

Which description is correct?

A. A package of templates and default values

B. One installed copy of an application

C. A repository that hosts packages

D. A numbered entry in an install's history

◆ PAUSE AND DECIDE

Which description is correct?

A. A package of templates and default values
The chart is the package.

B. One installed copy of an application
That is a release.

C. A repository that hosts packages
That is a chart repository.

D. A numbered entry in an install's history
That is a revision.

THE SIGNAL

Chart, release, repository, revision: four different nouns.

> An engineer wants to list only the Pods labelled tier=frontend, in the current namespace.

Which command does that?

A kubectl get pods --name=tier=frontend

B kubectl get pods -l tier=frontend

C kubectl describe pods tier=frontend

D kubectl get pods --field=tier=frontend

◆ PAUSE AND DECIDE

Which command does that?

A kubectl get pods --name=tier=frontend
No such flag for label filtering.

B kubectl get pods -l tier=frontend
-l takes a label selector.

C kubectl describe pods tier=frontend
Treats tier=frontend as a Pod name.

D kubectl get pods --field=tier=frontend
No such flag; field selectors work differently.

THE SIGNAL

Labels are selected with -l.

> A CI pipeline's ServiceAccount must create and update Deployments in the ci namespace, and nowhere else.

Which grant follows least privilege?

A A ClusterRole with a ClusterRoleBinding, for Deployments

B Adding the account to cluster-admin

C A Role and RoleBinding in default

D A Role and RoleBinding in the ci namespace

◆ PAUSE AND DECIDE

Which grant follows least privilege?

A A ClusterRole with a ClusterRoleBinding, for Deployments
Grants the power in every namespace.

B Adding the account to cluster-admin
Grants everything, everywhere.

C A Role and RoleBinding in default
Wrong namespace entirely.

D A Role and RoleBinding in the ci namespace
Exactly the verbs needed, in ci only.

THE SIGNAL

One namespace, specific verbs: Role plus RoleBinding there.

> On a kubeadm cluster, an engineer looks for the CoreDNS and kube-proxy Pods.

In which namespace will they be found?

A default

B kube-public

C kube-node-lease

D kube-system

◆ PAUSE AND DECIDE

In which namespace will they be found?

A default
For objects created without a namespace.

B kube-public
Readable by all; mostly unused.

C kube-node-lease
Holds node heartbeat Leases.

D kube-system
Objects created by Kubernetes itself.

THE SIGNAL

Kubernetes' own components live in kube-system.

> A single-replica internal database needs a persistent volume.
>
> It will be mounted read-write, by one node at a time.

Which access mode fits?

A ReadOnlyMany

B ReadWriteOnce

C ReadWriteMany

D ReadOnlyOnce

◆ PAUSE AND DECIDE

Which access mode fits?

A ReadOnlyMany

Read-only: the database must write.

B ReadWriteOnce

Read-write by one node at a time.

C ReadWriteMany

Many nodes: needs a shared filesystem it does not need.

D ReadOnlyOnce

Not a real access mode.

THE SIGNAL

One node, read-write: ReadWriteOnce.

> The booking API should get more Pods automatically when average CPU rises, and fewer when it falls.

Which object does this?

A A VerticalPodAutoscaler

B A PodDisruptionBudget

C A HorizontalPodAutoscaler

D A ResourceQuota

◆ PAUSE AND DECIDE

Which object does this?

A A VerticalPodAutoscaler
Changes Pod size, not count.

B A PodDisruptionBudget
Limits voluntary disruptions.

C A HorizontalPodAutoscaler
Adjusts the replica count from metrics.

D A ResourceQuota
Caps a namespace's totals.

THE SIGNAL

More or fewer Pods from metrics: HPA.

> New GPU nodes must repel every Pod that does not explicitly tolerate them.
>
> Pods already running on those nodes may stay.

What should be applied to the GPU nodes?

A A taint with the NoSchedule effect

B A taint with the NoExecute effect

C A nodeSelector on the GPU nodes

D A high PriorityClass

◆ PAUSE AND DECIDE

What should be applied to the GPU nodes?

A A taint with the NoSchedule effect
Repels new Pods; leaves running ones.

B A taint with the NoExecute effect
Would also evict running Pods.

C A nodeSelector on the GPU nodes
Selectors go on Pods, and attract rather than repel.

D A high PriorityClass
Controls order and preemption, not placement.

THE SIGNAL

Repel new Pods, keep running ones: NoSchedule.

> A platform lead explains GitOps to a new team.

How do changes reach the cluster in GitOps?

A An agent in the cluster pulls from Git and reconciles

B The CI pipeline runs kubectl apply after every build

C Engineers edit live objects, then commit to Git

D The scheduler reads manifests straight from Git

◆ PAUSE AND DECIDE

How do changes reach the cluster in GitOps?

A An agent in the cluster pulls from Git and reconciles

Pulled automatically, continuously reconciled.

B The CI pipeline runs kubectl apply after every build

That is push-based delivery.

C Engineers edit live objects, then commit to Git

Backwards: Git must lead, not follow.

D The scheduler reads manifests straight from Git

The scheduler only places Pods.

THE SIGNAL

GitOps pulls.

> A brand new namespace has no NetworkPolicies at all.

Which traffic can reach its Pods?

A None: namespaces deny by default

B Only traffic from the same namespace

C Traffic from any Pod in the cluster

D Only traffic from kube-system

◆ PAUSE AND DECIDE

Which traffic can reach its Pods?

A None: namespaces deny by default

Default deny must be created as a policy.

B Only traffic from the same namespace

Namespaces are not network boundaries by default.

C Traffic from any Pod in the cluster

Pods are non-isolated until a policy selects them.

D Only traffic from kube-system

Nothing special applies to kube-system.

THE SIGNAL

No policy selects a Pod: everything allowed.

> A student compares containers with virtual machines.

Which statement is correct?

A Each container runs its own kernel

B Containers need a hypervisor to run

C Containers boot a full guest operating system

D Containers share the host's kernel

◆ PAUSE AND DECIDE

Which statement is correct?

A Each container runs its own kernel
That describes virtual machines.

B Containers need a hypervisor to run
That describes virtual machines.

C Containers boot a full guest operating system
That describes virtual machines.

D Containers share the host's kernel
The key difference from VMs.

THE SIGNAL

Shared kernel: the whole difference.

> A booking API Pod's readiness probe starts failing. Its liveness probe keeps passing.

What happens to the Pod?

A The container is restarted

B It keeps running, but gets no Service traffic

C Its node is drained

D The Deployment rolls back to its previous revision

◆ PAUSE AND DECIDE

What happens to the Pod?

A The container is restarted

Only a failing liveness probe restarts.

B It keeps running, but gets no Service traffic

Readiness removes it from endpoints.

C Its node is drained

Probes never drain nodes.

D The Deployment rolls back to its previous revision

No automatic rollback happens.

THE SIGNAL

Readiness removes from traffic; liveness restarts.

> The web front end needs a non-confidential setting, a timetable refresh interval of 300 seconds, kept outside the image.

Where should the setting live?

A A Secret

B An annotation on the Deployment

C A ConfigMap

D A PersistentVolumeClaim

◆ PAUSE AND DECIDE

Where should the setting live?

A A Secret

For confidential data.

B An annotation on the Deployment

Describes the object; not consumed as config.

C A ConfigMap

Non-confidential configuration.

D A PersistentVolumeClaim

Storage for data, not settings.

THE SIGNAL

Non-confidential config: ConfigMap.

> Pods resolve names such as web.quayside.svc.cluster.local to Service addresses.

Which component answers those lookups?

A kube-proxy

B The kubelet

C etcd

D CoreDNS

◆ PAUSE AND DECIDE

Which component answers those lookups?

A kube-proxy

Programs traffic rules, not DNS.

B The kubelet

Points Pods at DNS, but does not answer lookups.

C etcd

Stores state; answers no DNS.

D CoreDNS

The cluster DNS server.

THE SIGNAL

Cluster DNS: CoreDNS.

> A team wants one vendor-neutral way to instrument its services for traces, metrics and logs.

Which CNCF project provides that?

A OpenTelemetry

B Jaeger

C Prometheus

D Fluentd

◆ PAUSE AND DECIDE

Which CNCF project provides that?

A **OpenTelemetry**
APIs, SDKs and OTLP for all three signals.

B **Jaeger**
A tracing backend.

C **Prometheus**
Metrics collection and storage.

D **Fluentd**
Log collection and routing.

THE SIGNAL

All three signals, vendor-neutral: OpenTelemetry.

> A monthly data-cleanup script must run once to completion, and be retried if it fails.

Which workload fits?

A A Deployment

B A Job

C A DaemonSet

D A StatefulSet

◆ PAUSE AND DECIDE

Which workload fits?

A A Deployment

Restarts the script after it succeeds.

B A Job

Runs to completion, with retries.

C A DaemonSet

One copy per node, running forever.

D A StatefulSet

For stateful long-running services.

THE SIGNAL

Finite work: Job.

Score yourself by domain

DOMAIN	QUESTIONS	YOUR SCORE
Kubernetes Fundamentals (44%)	1, 3, 5, 7, 9, 12, 15, 17, 20	of 9
Container Orchestration (28%)	2, 8, 10, 14, 18	of 5
Application Delivery (16%)	6, 13, 16	of 3
Cloud Native Architecture (12%)	4, 11, 19	of 3

ON THE EXAM

Which domain your misses cluster in matters more than your total.

What your score means

SCORE	MEANS	NEXT STEP
17 to 20 (85%+)	Ready, with margin	Keep going: practice exam 2
15 or 16 (75 to 80%)	On the pass line	Rewatch the modules behind your misses
Below 15	Not yet	Revisit the weakest domain's modules, then retake

ON THE EXAM

The real pass mark is 75%. Aim for a steady 85% in practice, to leave margin for nerves.

Practice exam 2: twenty new questions, no topic repeated from this paper.

PRACTICE EXAM 2 OF 4 · EXAM STRATEGY - ALL DOMAINS

Practice exam 2

Twenty new questions. Four pair with exam 1, where one changed detail changes the answer.

QUESTIONS 20 TIME About 30 minutes MODULE 33 of 35

admin@sthithapragna.com

> Quayside's production etcd cluster has five members.

How many members can fail while etcd keeps accepting writes?

A One

B Four

C Two

D Three

◆ PAUSE AND DECIDE

How many members can fail while etcd keeps accepting writes?

A One

Five members tolerate more than one.

B Four

That would leave one member. no majority.

C Two

Quorum is three of five, so two may fail.

D Three

That would leave two of five: no majority.

THE SIGNAL

Quorum is a majority: floor(n/2) + 1.

> The web front end must be reachable from the internet at one stable address.
>
> The cluster runs on a public cloud, which should create and manage that address.

Which Service type fits?

A LoadBalancer

B ClusterIP

C ExternalName

D A headless Service

◆ PAUSE AND DECIDE

Which Service type fits?

A **LoadBalancer**
The cloud provisions one external address.

B **ClusterIP**
Internal only, the default.

C **ExternalName**
An alias pointing outwards.

D **A headless Service**
No virtual IP; internal only.

THE SIGNAL

Internet, one stable cloud address: LoadBalancer.

> The new GPU nodes must repel every Pod that does not tolerate them.
>
> Pods already running there without a toleration must also be moved off.

What should be applied to the GPU nodes?

A A taint with the NoSchedule effect

B A taint with PreferNoSchedule

C A nodeSelector on the GPU Pods

D A taint with the NoExecute effect

◆ PAUSE AND DECIDE

What should be applied to the GPU nodes?

A A taint with the NoSchedule effect
Leaves running Pods in place.

B A taint with PreferNoSchedule
Only a preference; nothing is evicted.

C A nodeSelector on the GPU Pods
Attracts GPU Pods; repels nothing.

D A taint with the NoExecute effect
Repels new Pods and evicts running ones.

THE SIGNAL

Must evict running Pods too: NoExecute.

> For a new booking API release, Quayside wants every user moved at the same moment.
>
> If anything goes wrong, it must switch back instantly. Double capacity for an hour is affordable.

Which release strategy fits?

A A canary, starting small

B Blue-green, switching all at once

C Recreate, stopping old then new

D Shadow traffic, discarding replies

◆ PAUSE AND DECIDE

Which release strategy fits?

A. A canary, starting small
Moves users gradually, not at once.

B. Blue-green, switching all at once
One switch forward, one switch back.

C. Recreate, stopping old then new
Everyone at once, but with downtime.

D. Shadow traffic, discarding replies
Users never see the new version.

THE SIGNAL

Everyone at once, instant rollback: blue-green.

> During an incident, an engineer needs each booking API Pod's node and IP address, one line per Pod.

Which command gives that?

A kubectl get pods -l app=booking-api -o wide

B kubectl describe nodes

C kubectl top pods -l app=booking-api --containers

D kubectl get deploy booking-api -o yaml

◆ PAUSE AND DECIDE

Which command gives that?

A kubectl get pods -l app=booking-api -o wide

Adds node and IP columns, one line per Pod.

B kubectl describe nodes

Describes nodes, not where each Pod runs.

C kubectl top pods -l app=booking-api --containers

Shows CPU and memory use.

D kubectl get deploy booking-api -o yaml

Shows the Deployment, not its Pods.

THE SIGNAL

Node and IP per Pod: get pods -o wide.

> New Pods show ImagePullBackOff. The events say: manifest for quayside/web:5.3.0 not found. The registry is public, and older tags pull fine.

What is the most likely cause?

A The node has run out of CPU

B A liveness probe keeps failing

C The registry needs a pull secret

D Tag 5.3.0 is not in the registry

◆ PAUSE AND DECIDE

What is the most likely cause?

A The node has run out of CPU
CPU does not affect pulling.

B A liveness probe keeps failing
The container never started.

C The registry needs a pull secret
It is public, and older tags work.

D Tag 5.3.0 is not in the registry
'Not found' for one tag: a missing or mistyped tag.

THE SIGNAL

'Manifest not found' on a public registry: the tag is wrong.

> The booking API runs 3 replicas. During node drains, at least 2 must always stay available.

What enforces that during drains?

A A ResourceQuota on the namespace

B A LimitRange with default limits

C A PodDisruptionBudget, minAvailable 2

D A high PriorityClass for the Pods

◆ PAUSE AND DECIDE

What enforces that during drains?

A A ResourceQuota on the namespace
Caps namespace totals.

B A LimitRange with default limits
Sets per-container defaults.

C A PodDisruptionBudget, minAvailable 2
Drain's evictions respect it.

D A high PriorityClass for the Pods
Affects scheduling order, not evictions.

THE SIGNAL

Availability during voluntary disruptions: PodDisruptionBudget.

> Quayside's dashboard shows: 99.7% of booking requests completed in under 300 ms over the last 28 days.

What is that figure?

A An SLO

B An SLI

C An SLA

D An error budget

◆ PAUSE AND DECIDE

What is that figure?

A An SLO
A target, not a measurement.

B An SLI
A measured indicator of service.

C An SLA
A customer contract.

D An error budget
What an SLO leaves over.

THE SIGNAL

A measured value: SLI. A target: SLO.

> The data-cleanup script from exam 1 must now run every night at 02:00, to completion each time.

Which workload fits?

A A Job

B A CronJob

C A DaemonSet

D A Deployment

◆ PAUSE AND DECIDE

Which workload fits?

A A Job

Runs once; nothing repeats it.

B A CronJob

Creates a Job on a schedule.

C A DaemonSet

One copy per node, running forever.

D A Deployment

Restarts it after it finishes.

THE SIGNAL

Finite work on a schedule: CronJob.

> Four web Pods on different nodes must all read and write the same folder of uploaded photos.

Which access mode does the claim need?

A ReadWriteOnce

B ReadOnlyMany

C ReadWriteMany

D ReadWriteOncePod

◆ PAUSE AND DECIDE

Which access mode does the claim need?

A ReadWriteOnce
One node at a time.

B ReadOnlyMany
Read-only.

C ReadWriteMany
Many nodes, read-write.

D ReadWriteOncePod
A single Pod only.

THE SIGNAL

Many nodes, read-write: ReadWriteMany.

> An engineer is writing a Deployment manifest for a current cluster.

Which apiVersion should it use?

A apps/v1

B v1

C extensions/v1beta1

D batch/v1

◆ PAUSE AND DECIDE

Which apiVersion should it use?

A apps/v1
Deployments live in the apps group.

B v1
The core group: Pods, Services, ConfigMaps.

C extensions/v1beta1
Removed long ago.

D batch/v1
Jobs and CronJobs.

THE SIGNAL

Deployments: apps/v1.

> The booking API's production settings live in a Kustomize overlay at overlays/prod.
>
> The engineer wants to build and apply it with kubectl alone.

Which command does that?

A kubectl apply -f overlays/prod/kustomization.yaml

B helm install booking overlays/prod

C kubectl kustomize --apply overlays/prod

D kubectl apply -k overlays/prod

◆ PAUSE AND DECIDE

Which command does that?

A kubectl apply -f overlays/prod/kustomization.yaml
Applies the file itself, not the built overlay.

B helm install booking overlays/prod
Helm installs charts, not overlays.

C kubectl kustomize --apply overlays/prod
No such flag.

D kubectl apply -k overlays/prod
-k builds the overlay and applies it.

THE SIGNAL

Kustomize with kubectl: apply -k.

> An auditor asks how Kubernetes stores Secret values in etcd, on a cluster with default settings.

Which statement is accurate?

A Encrypted with the cluster CA by default

B Hashed, so they cannot be read back

C Held only in memory, never in etcd

D Base64-encoded, but not encrypted

◆ PAUSE AND DECIDE

Which statement is accurate?

A Encrypted with the cluster CA by default
There is no default encryption.

B Hashed, so they cannot be read back
They can always be read back.

C Held only in memory, never in etcd
They are stored in etcd.

D Base64-encoded, but not encrypted
Encryption at rest must be configured.

THE SIGNAL

Default Secrets: encoded, not encrypted.

> A container keeps restarting. describe shows Last State: Terminated, Reason: OOMKilled, Exit Code: 137.

What happened?

A It exceeded its CPU limit

B It exceeded its memory limit

C Its liveness probe failed

D Its image could not be pulled

◆ PAUSE AND DECIDE

What happened?

A It exceeded its CPU limit

CPU over the limit is throttled, not killed.

B It exceeded its memory limit

The kernel killed it at its memory limit.

C Its liveness probe failed

That would not say OOMKilled.

D Its image could not be pulled

It started; pulls happen before.

THE SIGNAL

OOMKilled with 137: the memory limit.

> A node joins a cluster, but its Pods get no IP addresses and cannot reach each other.

Which component is responsible for Pod IPs and Pod-to-Pod connectivity?

A kube-proxy

B CoreDNS

C The CNI network plugin

D The kubelet, on its own

◆ PAUSE AND DECIDE

Which component is responsible for Pod IPs and Pod-to-Pod connectivity?

A kube-proxy
Handles Service virtual IPs.

B CoreDNS
Answers DNS lookups.

C The CNI network plugin
Assigns IPs and wires the Pod network.

D The kubelet, on its own
Calls the runtime, which calls CNI.

THE SIGNAL

Pod IPs and the Pod network: CNI.

> An engineer asks how the kubelet tells containerd to start a container.

Which interface does it use?

A The CRI

B The CNI

C The CSI

D The OCI image spec

◆ PAUSE AND DECIDE

Which interface does it use?

A **The CRI**
The kubelet's interface to runtimes.

B **The CNI**
For network plugins.

C **The CSI**
For storage plugins.

D **The OCI image spec**
Describes image format, not the call.

THE SIGNAL

Kubelet to runtime: CRI.

> An engineer uses kubectl to raise a Deployment's replicas in a cluster managed by Argo CD. Within a minute, Argo CD changes it back to match Git.

Which Argo CD setting caused that?

A prune

B targetRevision

C selfHeal

D A sync wave

◆ PAUSE AND DECIDE

Which Argo CD setting caused that?

A prune
Deletes objects removed from Git.

B targetRevision
Chooses which commit is tracked.

C selfHeal
Reverts manual changes to match Git.

D A sync wave
Orders resources during a sync.

THE SIGNAL

Manual change reverted: selfHeal.

> After a default-deny egress policy is applied in the payments namespace, every outbound call fails with: could not resolve host.

What is the fix?

A Restart the CoreDNS Pods

B Switch kube-proxy to nftables mode

C Remove the namespace's labels

D Allow egress to kube-dns on port 53

◆ PAUSE AND DECIDE

What is the fix?

A Restart the CoreDNS Pods
CoreDNS is fine; it is unreachable.

B Switch kube-proxy to nftables mode
Unrelated to name lookups.

C Remove the namespace's labels
Labels do not block DNS.

D Allow egress to kube-dns on port 53
DNS traffic was blocked by the policy.

THE SIGNAL

Egress locked down and lookups fail: allow DNS.

> Quayside wants every container's logs collected centrally, with no changes to its applications.

Which approach is standard?

A Apps write to a shared NFS volume

B A node agent DaemonSet, such as Fluent Bit

C A CronJob that dumps kubectl logs every hour

D Store the log lines in etcd

◆ PAUSE AND DECIDE

Which approach is standard?

A Apps write to a shared NFS volume
Needs app changes; fragile.

B A node agent DaemonSet, such as Fluent Bit
Reads every container's logs on each node.

C A CronJob that dumps kubectl logs every hour
Misses anything between runs.

D Store the log lines in etcd
etcd is not a log store.

THE SIGNAL

Central logs, no app changes: a node-agent DaemonSet.

> Before the booking API starts, a database schema migration must run once and complete successfully.

What should run the migration?

A An init container

B A native sidecar container

C An ephemeral container

D A second app container

◆ PAUSE AND DECIDE

What should run the migration?

A An init container
Runs to completion before the app starts.

B A native sidecar container
Keeps running beside the app.

C An ephemeral container
For debugging only.

D A second app container
Starts alongside the app, not before.

THE SIGNAL

Must finish before the app starts: init container.

Score yourself by domain

DOMAIN	QUESTIONS	YOUR SCORE
Kubernetes Fundamentals (44%)	1, 3, 5, 7, 9, 11, 14, 16, 20	of 9
Container Orchestration (28%)	2, 6, 10, 13, 15, 18	of 6
Application Delivery (16%)	4, 12, 17	of 3
Cloud Native Architecture (12%)	8, 19	of 2

ON THE EXAM

Compare with exam 1. A domain that is weak twice is where to spend your time.

The pairs, and the constraint that changed

THIS EXAM	EXAM 1	WHAT CHANGED THE ANSWER
Q2 LoadBalancer	Q2 ClusterIP	Internet access with a cloud address
Q3 NoExecute	Q12 NoSchedule	Running Pods must also leave
Q9 CronJob	Q20 Job	It must repeat on a schedule
Q10 ReadWriteMany	Q10 ReadWriteOnce	Many nodes write, not one

> ON THE EXAM
>
> If you got one of a pair wrong, you missed a constraint, not a fact. Reread the stem.

Practice exam 3: twenty more, with more Delivery questions to match the domain's weight.

PRACTICE EXAM 3 OF 4 · EXAM STRATEGY - ALL DOMAINS

Practice exam 3

Twenty new questions, with four from Application Delivery to match its weight across the set.

QUESTIONS 20 TIME About 30 minutes MODULE 34 of 35

admin@sthithapragna.com

> The booking API Deployment is set to 3 replicas. An engineer deletes one of its Pods by hand.

What creates the replacement Pod?

A The kubelet on the same node

B The kube-scheduler

C etcd, from its stored copy

D The ReplicaSet controller

◆ PAUSE AND DECIDE

What creates the replacement Pod?

A The kubelet on the same node

Runs Pods; does not create them.

B The kube-scheduler

Places new Pods; does not create them.

C etcd, from its stored copy

Stores state; creates nothing.

D The ReplicaSet controller

Sees 2 of 3 and creates one more.

THE SIGNAL

Creating Pods to match a count: the ReplicaSet controller.

> Every change at Quayside is built, tested and ready to release.
>
> A manager approves each move into production.

Which practice is this?

A Continuous deployment

B Continuous delivery

C Continuous integration only

D Infrastructure as code

◆ PAUSE AND DECIDE

Which practice is this?

A Continuous deployment
No human gate at all.

B Continuous delivery
Always releasable, with a human gate.

C Continuous integration only
Stops before any environment.

D Infrastructure as code
About infrastructure, not release.

THE SIGNAL

Human approval before production: continuous delivery.

> Quayside wants each Deployment to record the full URL of the CI build that produced it. Nothing will ever select objects by this value.

Where should it be stored?

A In a label

B In a ConfigMap

C In an annotation

D In the namespace's name

◆ PAUSE AND DECIDE

Where should it be stored?

A In a label

For selection, and limited to 63 characters.

B In a ConfigMap

Feeds apps; does not describe the object.

C In an annotation

Descriptive metadata; long values allowed.

D In the namespace's name

Namespaces organise; they do not record builds.

THE SIGNAL

Describe, never select: annotation.

> The three bookings database replicas must discover each other's individual Pod addresses through DNS.

What should their Service set?

A clusterIP: None, a headless Service

B type: NodePort, one port per node

C type: ExternalName, a DNS alias

D type: LoadBalancer, one cloud IP

◆ PAUSE AND DECIDE

What should their Service set?

A clusterIP: None, a headless Service
DNS then returns each Pod's own IP.

B type: NodePort, one port per node
Opens node ports; still one virtual IP.

C type: ExternalName, a DNS alias
An alias pointing outside.

D type: LoadBalancer, one cloud IP
External, still one balanced address.

THE SIGNAL

Each Pod's own address through DNS: headless.

> Before kernel patching, every Pod on node-7 must move elsewhere.
>
> The booking API's disruption budget must be respected.

Which command does this?

A kubectl cordon node-7

B kubectl drain node-7 --ignore-daemonsets

C kubectl delete node node-7

D kubectl taint nodes node-7 x=y:PreferNoSchedule

◆ PAUSE AND DECIDE

Which command does this?

A kubectl cordon node-7

Stops new Pods; existing ones stay.

B kubectl drain node-7 --ignore-daemonsets

Cordons, then evicts through the Eviction API.

C kubectl delete node node-7

Removes the node object; no graceful eviction.

D kubectl taint nodes node-7 x=y:PreferNoSchedule

A soft preference; evicts nothing.

THE SIGNAL

Move the Pods out, respecting budgets: drain.

> Upgrade 5 of the booking Helm release broke checkout. Revision 3 was the last good one.

Which command returns the release to revision 3?

A helm upgrade booking --revision 3

B kubectl rollout undo release/booking

C helm rollback booking 3

D helm history booking --restore 3

◆ PAUSE AND DECIDE

Which command returns the release to revision 3?

A helm upgrade booking --revision 3
No such flag.

B kubectl rollout undo release/booking
Helm releases are not kubectl rollouts.

C helm rollback booking 3
Rolls back to revision 3, as a new revision.

D helm history booking --restore 3
history lists; it restores nothing.

THE SIGNAL

Helm rollback: helm rollback RELEASE REVISION.

> In the payments namespace, containers that set no resource requests should automatically get sensible default requests.

Which object provides that?

A A ResourceQuota

B A PodDisruptionBudget

C A PriorityClass

D A LimitRange

◆ PAUSE AND DECIDE

Which object provides that?

A A ResourceQuota
Caps totals; sets no defaults.

B A PodDisruptionBudget
Governs voluntary evictions.

C A PriorityClass
Sets scheduling priority.

D A LimitRange
Fills in default requests and limits.

THE SIGNAL

Per-container defaults: LimitRange.

> A developer's kubectl command returns 403 Forbidden. Their identity-provider login worked normally.

Which stage rejected the request?

A Authorisation: known user, no permission

B Authentication: identity not recognised

C An admission webhook that is down

D etcd being unavailable

◆ PAUSE AND DECIDE

Which stage rejected the request?

A Authorisation: known user, no permission

403 means authenticated but not allowed.

B Authentication: identity not recognised

That returns 401.

C An admission webhook that is down

A down webhook gives a different error.

D etcd being unavailable

Storage errors are not 403.

THE SIGNAL

403: I know you, and no.

> A Pod with restartPolicy: Never runs a report script. The container exits with code 0.

What is the Pod's phase afterwards?

A Running

B Failed

C Succeeded

D Pending

◆ PAUSE AND DECIDE

What is the Pod's phase afterwards?

A Running

Nothing is running any more.

B Failed

Exit code 0 is success.

C Succeeded

All containers exited successfully.

D Pending

It already ran.

THE SIGNAL

All containers exited 0, no restart: Succeeded.

> Quayside wants HTTP services that scale to zero when idle, with revisions and traffic splitting between them, on Kubernetes.

Which CNCF project provides that?

A Knative

B KEDA

C Karpenter

D Helm

◆ PAUSE AND DECIDE

Which CNCF project provides that?

A Knative
Serverless Serving: scale to zero, revisions, splits.

B KEDA
Event-driven scaling; no revisions or HTTP routing.

C Karpenter
Adds nodes, not serverless apps.

D Helm
Packages apps; does not scale them.

THE SIGNAL

Serverless HTTP, revisions, scale to zero: Knative.

> An engineer wants the built-in documentation for the strategy field of a Deployment's spec.

Which command shows it?

A kubectl describe deployment --field strategy

B kubectl get deployment --docs

C kubectl help deployment.strategy

D kubectl explain deployment.spec.strategy

◆ PAUSE AND DECIDE

Which command shows it?

A kubectl describe deployment --field strategy
Describes objects, not field docs.

B kubectl get deployment --docs
No such flag.

C kubectl help deployment.strategy
Not how kubectl help works.

D **kubectl explain deployment.spec.strategy**
Documents any field, from the API schema.

THE SIGNAL

Field documentation: kubectl explain.

> Quayside must block privileged containers and host namespaces in app namespaces, while most apps keep working unchanged.

Which Pod Security Standard should be enforced?

A Privileged

B Baseline

C Restricted

D Default

◆ PAUSE AND DECIDE

Which Pod Security Standard should be enforced?

A Privileged

Allows everything.

B Baseline

Blocks known escalations; most apps pass.

C Restricted

Also requires non-root and more.

D Default

Not a Pod Security Standard.

THE SIGNAL

Block the dangerous, keep apps working: Baseline.

> Quayside wants exactly 5% of requests to reach a canary.
>
> The share must not depend on how many Pods each version runs.

What achieves that?

A Weighted backendRefs in an HTTPRoute

B One canary Pod among twenty stable Pods

C Two Deployments, switched with Recreate

D A PodDisruptionBudget on the canary

◆ PAUSE AND DECIDE

What achieves that?

A Weighted backendRefs in an HTTPRoute
Exact weights, independent of Pod count.

B One canary Pod among twenty stable Pods
About 5%, but tied to Pod count.

C Two Deployments, switched with Recreate
Recreate swaps everything at once.

D A PodDisruptionBudget on the canary
Governs evictions, not traffic.

THE SIGNAL

Exact share, independent of Pods: traffic weights.

> On a kubeadm control plane node, the API server runs as a Pod.
>
> It must start before any API server exists to accept Pod objects.

How does the kubelet run it?

A As a DaemonSet in kube-system

B As a Deployment in kube-system

C Only as a systemd service, never a Pod

D As a static Pod, from a manifest on disk

◆ PAUSE AND DECIDE

How does the kubelet run it?

A As a DaemonSet in kube-system
DaemonSets need a running API server.

B As a Deployment in kube-system
Deployments need a running API server.

C Only as a systemd service, never a Pod
kubeadm runs it as a Pod.

D As a static Pod, from a manifest on disk
The kubelet reads the manifest directly.

THE SIGNAL

Control plane Pods before the API exists: static Pods.

> The bookings database's data must survive even if its PersistentVolumeClaim is deleted by mistake.

Which reclaim policy should its StorageClass use?

A Delete

B Retain

C Recycle

D Immediate

◆ PAUSE AND DECIDE

Which reclaim policy should its StorageClass use?

A Delete

Deletes the disk with the claim.

B Retain

Keeps the volume and its data.

C Recycle

Deprecated.

D Immediate

A binding mode, not a reclaim policy.

THE SIGNAL

Data outlives the claim: Retain.

> The control plane runs Kubernetes 1.37. The team checks which kubelet versions are supported with it.

Which kubelet version is supported?

A 1.38

B 1.33

C 1.35

D 1.32

◆ PAUSE AND DECIDE

Which kubelet version is supported?

A 1.38

Newer than the API server: never allowed.

B 1.33

Four minors older: too old.

C 1.35

Two minors older: allowed.

D 1.32

Five minors older: too old.

THE SIGNAL

Kubelets: up to three minors older, never newer.

> Quayside's traces are produced with OpenTelemetry. The team needs a CNCF backend to store and search them.

Which project fits?

A Jaeger

B Prometheus

C Fluentd

D Grafana Loki

◆ PAUSE AND DECIDE

Which project fits?

A **Jaeger**
A graduated tracing backend.

B **Prometheus**
Stores metrics, not traces.

C **Fluentd**
Collects and routes logs.

D **Grafana Loki**
A log store, from Grafana Labs.

THE SIGNAL

Store and search traces: Jaeger.

> A new fare feature is deployed with the code switched off.
>
> Product managers want to turn it on for staff first, then everyone, without redeploying.

What makes that possible?

A A canary release

B A blue-green switch

C Feature flags

D The Recreate strategy

◆ PAUSE AND DECIDE

What makes that possible?

A A canary release
Splits by version, and needs deployments.

B A blue-green switch
Switches whole versions.

C Feature flags
Turn features on per audience, no redeploy.

D The Recreate strategy
Redeploys everything.

THE SIGNAL

Release without redeploying: feature flags.

> The timetable renderer never calls the Kubernetes API. Security wants no API token inside its Pods at all.

What should the team set?

A Delete the namespace's default ServiceAccount

B automountServiceAccountToken: false

C A NetworkPolicy blocking the API server

D An RBAC rule denying its ServiceAccount

◆ PAUSE AND DECIDE

What should the team set?

A Delete the namespace's default ServiceAccount
It is recreated automatically.

B automountServiceAccountToken: false
No token is mounted into the Pods.

C A NetworkPolicy blocking the API server
Restricts traffic; the token is still there.

D An RBAC rule denying its ServiceAccount
RBAC has no deny rules.

THE SIGNAL

No token at all: automountServiceAccountToken: false.

> Every container in the payment worker sets CPU and memory requests exactly equal to its limits.

Which QoS class does the Pod get?

A BestEffort

B Burstable

C Critical

D Guaranteed

◆ PAUSE AND DECIDE

Which QoS class does the Pod get?

A BestEffort
That needs no requests or limits at all.

B Burstable
Some requests or limits, but not equal.

C Critical
Not a QoS class.

D Guaranteed
Requests equal limits, for CPU and memory.

THE SIGNAL

Requests equal limits everywhere: Guaranteed.

Score yourself by domain

DOMAIN	QUESTIONS	YOUR SCORE
Kubernetes Fundamentals (44%)	1, 3, 5, 7, 9, 11, 14, 16, 20	of 9
Container Orchestration (28%)	4, 8, 12, 15, 19	of 5
Application Delivery (16%)	2, 6, 13, 18	of 4
Cloud Native Architecture (12%)	10, 17	of 2

ON THE EXAM

Three papers in: the same weak domain each time means rewatch that domain's modules now.

The final paper: practice exam 4, then a trend review across all four.

PRACTICE EXAM 4 OF 4 · EXAM STRATEGY - ALL DOMAINS

Practice exam 4

The final twenty questions, then a trend review across all four papers.

QUESTIONS 20 TIME About 30 minutes MODULE 35 of 35

admin@sthithapragna.com

> An auditor asks which control plane component is allowed to read and write etcd directly.

Which component is it?

A The kube-apiserver

B The kube-scheduler

C The kubelet

D The kube-controller-manager

◆ PAUSE AND DECIDE

Which component is it?

A The kube-apiserver
The only component that talks to etcd.

B The kube-scheduler
Works through the API server.

C The kubelet
Works through the API server.

D The kube-controller-manager
Works through the API server.

THE SIGNAL

Only the API server talks to etcd.

> Apps must reach a legacy database at db.legacy.example.com through an in-cluster name, so moving it later only changes one object.

What should the team create?

A A headless Service with a selector

B A NodePort Service

C An ExternalName Service

D A ClusterIP Service with a selector

◆ PAUSE AND DECIDE

What should the team create?

A A headless Service with a selector

Selects in-cluster Pods; there are none.

B A NodePort Service

Brings outside traffic in; wrong direction.

C An ExternalName Service

A DNS alias for the outside hostname.

D A ClusterIP Service with a selector

Needs Pods in the cluster to select.

THE SIGNAL

In-cluster name for an outside host: ExternalName.

> Quayside's payment worker processes a message queue. It must scale on queue length, and down to zero when the queue is empty.

Which tool fits?

A An HPA on the workers' CPU

B KEDA

C A VerticalPodAutoscaler

D Knative Serving

◆ PAUSE AND DECIDE

Which tool fits?

A An HPA on the workers' CPU

Cannot reach zero alone; CPU lags the queue.

B KEDA

Event-driven scaling, including to zero.

C A VerticalPodAutoscaler

Resizes Pods; never removes them.

D Knative Serving

Built for HTTP services, with revisions.

THE SIGNAL

Queue length, down to zero: KEDA.

> An engineer lists which kinds of object do not belong to any namespace.

Which of these is cluster-scoped?

A A Pod

B A ConfigMap

C A Role

D A Node

◆ PAUSE AND DECIDE

Which of these is cluster-scoped?

A A Pod
Namespaced.

B A ConfigMap
Namespaced.

C A Role
Namespaced; ClusterRole is the cluster-scoped one.

D A Node
Nodes belong to the whole cluster.

THE SIGNAL

Nodes, PVs, Namespaces, ClusterRoles: cluster-scoped.

> Security requires every call between Quayside's internal services to be encrypted, with both sides authenticated, and no code changes.

What meets the requirement?

A NetworkPolicies between services

B TLS termination at the Ingress

C Headless Services for every app

D A service mesh enforcing mutual TLS

◆ PAUSE AND DECIDE

What meets the requirement?

A NetworkPolicies between services
Restricts traffic; encrypts nothing.

B TLS termination at the Ingress
Covers traffic coming in only.

C Headless Services for every app
Changes DNS; no encryption.

D A service mesh enforcing mutual TLS
Encrypts and authenticates each call.

THE SIGNAL

Encrypt and authenticate internal calls: mesh mTLS.

> A CronJob's manifest was deleted from Git. Argo CD syncs automatically, but the CronJob still exists in the cluster.

Which setting makes Argo CD delete it?

A selfHeal: true

B prune: true

C A new targetRevision

D A Replace=true sync option

◆ PAUSE AND DECIDE

Which setting makes Argo CD delete it?

A selfHeal: true

Reverts edits; does not delete removed objects.

B prune: true

Deletes objects removed from Git.

C A new targetRevision

Changes the tracked commit.

D A Replace=true sync option

Changes how objects are applied.

THE SIGNAL

Removed from Git, still in the cluster: prune.

> A Deployment has 4 replicas and default rolling update settings. A new image is rolled out.

What limits hold throughout the rollout?

A At most 5 Pods; at least 3 available

B At most 4 Pods; at least 4 available

C At most 6 Pods; at least 2 available

D At most 8 Pods; none need be available

◆ PAUSE AND DECIDE

What limits hold throughout the rollout?

A At most 5 Pods; at least 3 available
25% of 4 is 1 either way.

B At most 4 Pods; at least 4 available
Would need both settings at zero.

C At most 6 Pods; at least 2 available
That is 50%, not the default.

D At most 8 Pods; none need be available
Nothing like the defaults.

THE SIGNAL

Defaults: 25% surge, 25% unavailable.

> Quayside's cluster still uses the community Ingress NGINX controller, which is now retired. The team wants the recommended way forward for HTTP traffic into the cluster.

What should they move to?

A Stay on it; it still gets patches

B A NodePort Service for every app

C A Gateway API implementation

D ExternalName Services

◆ PAUSE AND DECIDE

What should they move to?

A Stay on it; it still gets patches
No patches since March 2026.

B A NodePort Service for every app
No routing, and awkward for users.

C A Gateway API implementation
The recommended successor.

D ExternalName Services
Aliases point outwards, not inwards.

THE SIGNAL

Ingress NGINX retired: Gateway API.

> An engineer runs kubectl run debug --image=busybox:1.37 on a current cluster.

What does the command create?

A A Deployment

B A single Pod

C A Job

D A ReplicaSet

◆ PAUSE AND DECIDE

What does the command create?

A A Deployment
Only in very old kubectl versions.

B A single Pod
kubectl run creates one bare Pod.

C A Job
Needs kubectl create job.

D A ReplicaSet
Deployments create those.

THE SIGNAL

kubectl run: one bare Pod.

> A CNCF incubating project has met every graduation requirement and applies to graduate.

Which body approves the move?

A The CNCF Governing Board

B The Linux Foundation's executive team

C The Kubernetes Steering Committee

D The CNCF Technical Oversight Committee

◆ PAUSE AND DECIDE

Which body approves the move?

A The CNCF Governing Board
Business and budget, not maturity.

B The Linux Foundation's executive team
Not responsible for CNCF maturity.

C The Kubernetes Steering Committee
Governs Kubernetes only.

D The CNCF Technical Oversight Committee
The TOC decides every maturity move.

THE SIGNAL

Project maturity: the TOC.

> Machine-learning Pods must run only on the GPU nodes, and every other Pod must stay off those nodes.

What configuration achieves both?

A A toleration on ML Pods, with no taint added

B A nodeSelector on ML Pods, with no taint anywhere

C Taint, plus toleration and affinity on ML Pods

D A high PriorityClass for the ML Pods

◆ PAUSE AND DECIDE

What configuration achieves both?

A A toleration on ML Pods, with no taint added

A toleration without a taint does nothing.

B A nodeSelector on ML Pods, with no taint anywhere

Others can still land on GPU nodes.

C Taint, plus toleration and affinity on ML Pods

Taint keeps others out; affinity pulls ML in.

D A high PriorityClass for the ML Pods

Changes order, not placement.

THE SIGNAL

Others out and ours in: taint plus affinity.

> A Pod stays Pending with: 0/4 nodes are available: 4 node(s) didn't match Pod's node affinity/selector.

What is the cause?

A No node has the labels the Pod requires

B The nodes lack enough free memory

C The Pod's image cannot be pulled

D The scheduler has stopped running

◆ PAUSE AND DECIDE

What is the cause?

A No node has the labels the Pod requires

Exactly what the event says.

B The nodes lack enough free memory

That would say Insufficient memory.

C The Pod's image cannot be pulled

Pulls happen after scheduling.

D The scheduler has stopped running

A dead scheduler writes no events.

THE SIGNAL

Read the FailedScheduling reason literally.

> A chart's values.yaml sets replicaCount: 2. The team installs it with -f prod.yaml, which sets 4. The install command also passes --set replicaCount=6.

How many replicas are created?

A 2

B 4

C 6

D None: Helm rejects the conflict

◆ PAUSE AND DECIDE

How many replicas are created?

A 2

Defaults have the lowest priority.

B 4

Files beat defaults, but not --set.

C 6

--set has the highest priority.

D None: Helm rejects the conflict

Overrides are normal.

THE SIGNAL

Defaults, then -f files, then --set.

> A helper container in the booking API Pod converts the app's custom metrics output into Prometheus format.

Which multi-container pattern is that?

A Adapter

B Ambassador

C Sidecar, in the pattern sense

D Init

◆ PAUSE AND DECIDE

Which multi-container pattern is that?

A **Adapter**
Reshapes the app's output for others.

B **Ambassador**
Proxies the app's outgoing calls.

C **Sidecar, in the pattern sense**
Extends or enhances the app generally.

D **Init**
Runs before the app, then exits.

THE SIGNAL

Reshaping output: adapter.

> On a brand-new kubeadm cluster, every node is NotReady and the CoreDNS Pods are Pending.

What is most likely missing?

A kube-proxy

B etcd

C metrics-server

D A CNI network plugin

◆ PAUSE AND DECIDE

What is most likely missing?

A kube-proxy
kubeadm installs it.

B etcd
The control plane is up.

C metrics-server
Affects top and autoscaling only.

D A CNI network plugin
kubeadm installs no network plugin.

THE SIGNAL

Fresh kubeadm, NotReady, CoreDNS Pending: no CNI.

> A new version of the payment worker must never run at the same time as the old one, even briefly.
>
> A short outage during each release is acceptable.

Which Deployment strategy fits?

A RollingUpdate with maxUnavailable: 0

B Recreate

C A canary at 5%

D RollingUpdate with default settings

◆ PAUSE AND DECIDE

Which Deployment strategy fits?

A RollingUpdate with maxUnavailable: 0

Guarantees overlap.

B **Recreate**

Stops all old Pods before starting new ones.

C A canary at 5%

Overlap by design.

D RollingUpdate with default settings

Still overlaps.

THE SIGNAL

Must never overlap: Recreate.

> Quayside's contract with a partner promises 99.5% monthly availability, with refunds if it is missed.

What is this promise?

A An SLA

B An SLO

C An SLI

D An error budget

◆ PAUSE AND DECIDE

What is this promise?

A **An SLA**
A contract with consequences.

B **An SLO**
An internal target.

C **An SLI**
A measurement.

D **An error budget**
What an SLO leaves over.

THE SIGNAL

A contract with consequences: SLA.

> Quayside's GitOps repository must include the database password Secret for the bookings service.
>
> Nothing readable may be committed.

What is the safest approach?

A Commit the Secret manifest; values are base64

B Seal or encrypt it, or reference an external store

C Put the password in a ConfigMap instead

D Store it in an annotation on the Deployment

◆ PAUSE AND DECIDE

What is the safest approach?

A Commit the Secret manifest; values are base64
Base64 is readable by anyone.

B Seal or encrypt it, or reference an external store
Sealed Secrets, SOPS or External Secrets.

C Put the password in a ConfigMap instead
Worse: plain text.

D Store it in an annotation on the Deployment
Worse: plain text, and visible.

THE SIGNAL

Secrets in GitOps: encrypted or external, never plain.

> A container spec requests cpu: 500m.

How much CPU is that?

A 500 CPU cores

B 500 megabytes

C 5% of one CPU

D Half of one CPU

◆ PAUSE AND DECIDE

How much CPU is that?

A 500 CPU cores

m means thousandths, not whole cores.

B 500 megabytes

CPU is not measured in bytes.

C 5% of one CPU

That would be 50m.

D Half of one CPU

500 millicores: half a core.

THE SIGNAL

1000m is one CPU.

> A NetworkPolicy must allow traffic only from worker Pods that are also in namespaces labelled team=payments.

How should its from section be written?

A Two entries: a namespaceSelector, then a podSelector

B A podSelector entry on its own

C One entry holding both selectors together

D A namespaceSelector entry on its own

◆ PAUSE AND DECIDE

How should its from section be written?

A Two entries: a namespaceSelector, then a podSelector

Two entries combine with OR: far too wide.

B A podSelector entry on its own

Only Pods in the policy's own namespace.

C One entry holding both selectors together

Selectors in one entry combine with AND.

D A namespaceSelector entry on its own

Every Pod in those namespaces.

THE SIGNAL

Both conditions at once: one entry, both selectors.

Score yourself by domain

DOMAIN	QUESTIONS	YOUR SCORE
Kubernetes Fundamentals (44%)	1, 4, 7, 9, 11, 14, 16, 19	of 8
Container Orchestration (28%)	2, 5, 8, 12, 15, 20	of 6
Application Delivery (16%)	6, 13, 18	of 3
Cloud Native Architecture (12%)	3, 10, 17	of 3

ON THE EXAM

Now add this to your first three papers: the trend is next.

Your trend across all four papers

DOMAIN	OUT OF	YOUR TOTAL	READY AT
Kubernetes Fundamentals	35		30 or more
Container Orchestration	22		19 or more
Application Delivery	13		11 or more
Cloud Native Architecture	10		8 or more
All four papers	80		68 or more (85%)

ON THE EXAM

A domain below its 'ready' line is your study plan. The total is only the headline.

Where to go back to

WEAK DOMAIN	REWATCH	THEN
Kubernetes Fundamentals	Modules 1 to 12, scenarios first	Module 30's first three tables
Container Orchestration	Modules 13 to 21, especially 14 and 21	Module 30's orchestration tables
Application Delivery	Modules 22 to 25, and the Domain 3 review	Module 30's delivery tables
Cloud Native Architecture	Modules 26 to 28	Module 29, what changed

ON THE EXAM

Rewatch the worked scenarios before the teaching: they show the decisions the exam tests.

Before you book

- **Every domain at or above its ready line,** and 68 or more of 80 overall.
- **Replay modules 29 and 30** in the final week.
- **Check cncf.io/projects** and the Linux Foundation exam page for anything new.
- **Run the secure-browser system check** days before, not minutes.

> ON THE EXAM
>
> You have now worked through 240 exam-style decisions. The real exam asks about 60.

That is the whole course. Good luck in the exam, and welcome to the cloud native community.